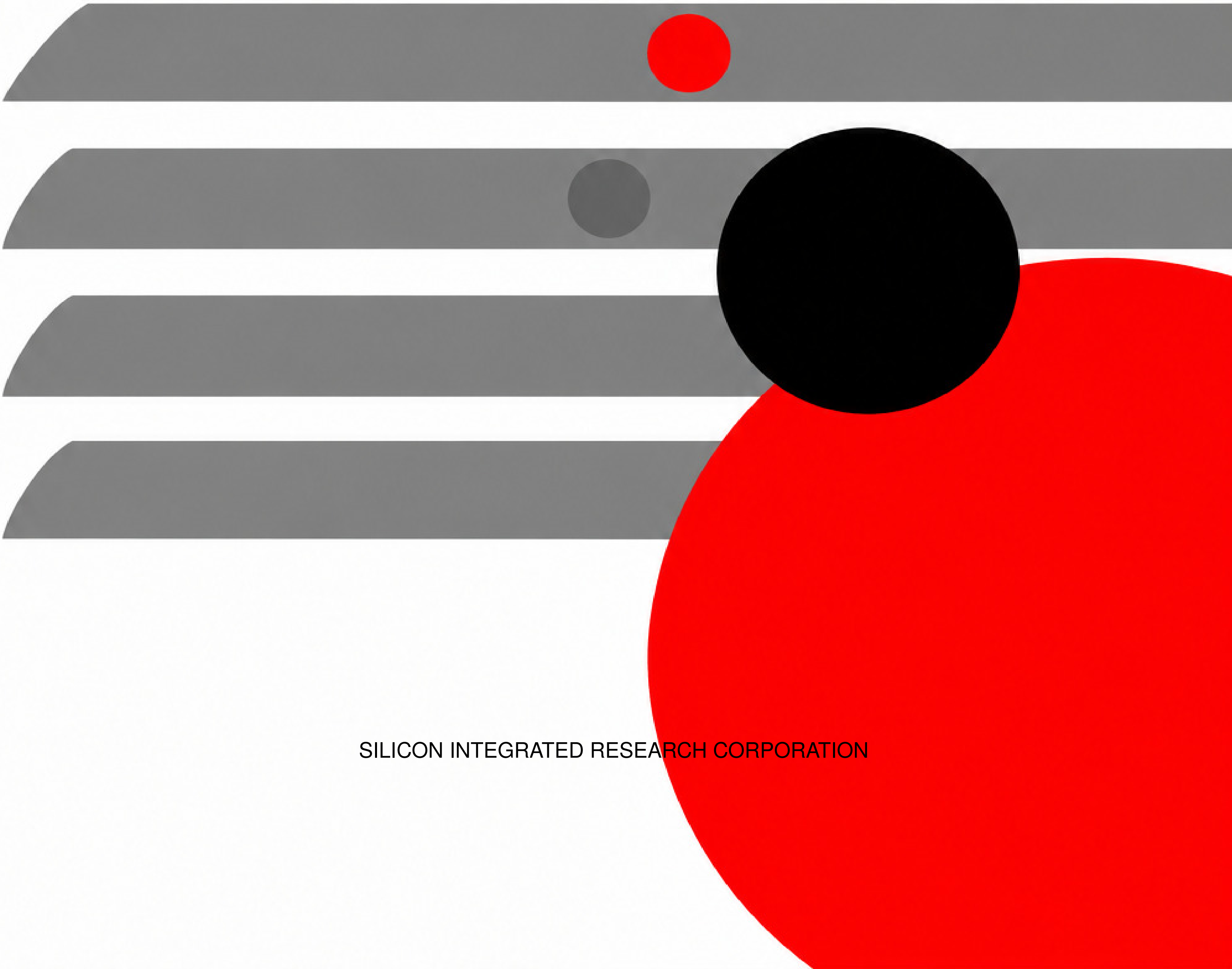
SIRC

SIRC-1

Architecture Reference Manual

Featuring the SIRCIS Instruction Set



SILICON INTEGRATED RESEARCH CORPORATION

Preface

About This Manual

This manual documents the SIRC-1 CPU architecture and the SIRCIS (SIRC Instruction Set) instruction set completely enough that an assembler, compiler backend, simulator, or hardware implementation can be built.

It is written for:

- Assembly language programmers
- Compiler and toolchain developers
- Hardware designers and implementers
- Simulator authors
- System software developers

Assumed Knowledge

This manual assumes the reader is already familiar with general computer architecture concepts: registers, memory addressing, binary and hexadecimal number representation, and assembly-language programming in general. It teaches the SIRC-1 architecture specifically; it does not teach these prerequisites from first principles.

How to Read This Manual

Part I introduces the architecture as a whole: registers, data representation, the status register, and exception handling. Part II covers the instruction encoding formats, addressing modes, shift operations, and condition codes that every instruction is built from. Part III is the instruction reference itself, one dedicated entry per instruction; Chapter 11 explains the fields every entry uses (Opcodes, Syntax, Operands, Operation, Flags, Exceptions, Timing, Privilege, and so on) before the entries begin, and is worth reading first even for a reader who intends to use the rest of the manual only as a lookup reference. The appendices collect quick lookups (the complete opcode map, timing figures, undocumented-opcode notes, worked examples, and an alphabetical mnemonic index) that restate information from the main chapters in a faster-to-scan form. Notation used throughout, including the canonical assembly syntax and a glossary, is collected in Chapter ; its Assembly Language Summary is enough to read the listings in Parts I and II before the instruction entries are reached.

Scope

This manual defines the SIRC-1 architecture: the instruction set and its encodings, the canonical assembly notation, the register model and other programmer-visible state, exceptions and interrupts, and the coprocessor interface. It does not cover the physical chip, which is the subject of the *SIRC-1 Hardware Reference Manual*, nor software conventions built on top of the architecture — register usage conventions, calling conventions, and stack frame layout — which are the concern of the (forthcoming) SIRC-1 ABI documentation. The toolchain and the object file format are documented in the *SIRC-1 Assembly Programmer's Manual*.

The SIRC-1 documentation is divided into three manuals:

Manual	**Scope**
SIRC-1 Architecture Reference Manual	ISA, instruction encodings, canonical assembly notation, exceptions, programmer-visible state
SIRC-1 Assembly Programmer's Manual	Complete assembly syntax, directives, symbols, meta-instructions, and toolchain usage
SIRC-1 Hardware Reference Manual	Pinout, bus interface, timing, electrical characteristics

Table 1: The SIRC Manual Set (This Manual Is the First)

Revision Status

This is Version 1.0 of the *SIRC-1 Architecture Reference Manual*, issued in draft for internal review and not for external distribution. Except where an entry explicitly notes otherwise, every instruction, register, and behavior described here applies to *SIRC-1, all revisions*.

Contents

Preface iii

Notation and Glossary xix

I Architecture Overview 1

1 Introduction 3
- 1.1 Overview . . . 3
- 1.2 Design Philosophy . . . 3
  - 1.2.1 Design Principles . . . 3
  - 1.2.2 Design Goals . . . 3
  - 1.2.3 Applications . . . 3
- 1.3 Key Features . . . 4
  - 1.3.1 Architecture Characteristics . . . 4
  - 1.3.2 Execution Model . . . 4
  - 1.3.3 Coprocessors . . . 4
  - 1.3.4 Privilege Levels . . . 5
- 1.4 Memory Organization . . . 5
  - 1.4.1 Address Space . . . 5
  - 1.4.2 Memory Map . . . 5
  - 1.4.3 Alignment . . . 6
- 1.5 Document Organization . . . 6

2 CPU Architecture 7
- 2.1 Overview . . . 7
- 2.2 Six-Stage Execution Phases . . . 7
- 2.3 Architectural Components . . . 8
  - 2.3.1 Control Unit (CU) . . . 8
  - 2.3.2 Coprocessors and the Coprocessor Select Register . . . 8
  - 2.3.3 Arithmetic Logic Unit (ALU) . . . 10
  - 2.3.4 Addressing Unit (AU) . . . 10
  - 2.3.5 Register File . . . 10
  - 2.3.6 Barrel Shifter . . . 10
  - 2.3.7 Exception Coprocessor . . . 11
- 2.4 Design Characteristics . . . 11
  - 2.4.1 Hardwired Control Implementation . . . 11
  - 2.4.2 Fixed Execution Sequence . . . 11
  - 2.4.3 Parallel Functional Units . . . 11
- 2.5 External Interface . . . 12

3 Register Model 13
- 3.1 Overview . . . 13
- 3.2 Programming Model . . . 13
- 3.3 General-Purpose Registers . . . 14
  - 3.3.1 Register Set . . . 14
  - 3.3.2 Usage Conventions . . . 15

3.4 Address Register Pairs . . . 15
3.4.1 Overview . . . 15
3.4.2 Link Register (l) . . . 15
3.4.3 Address Register (a) . . . 16
3.4.4 Stack Pointer (s) . . . 16
3.4.5 Program Counter (p) . . . 16
3.5 Status Register . . . 16
3.6 Privilege Restrictions . . . 17
3.6.1 High Address Register Restrictions . . . 17
3.6.2 Summary of Privilege Checks . . . 17
3.7 Register Addressing . . . 18
3.8 Usage Examples . . . 18

4 Data Representation and Memory Model 21
4.1 Overview . . . 21
4.2 Word and Byte Ordering . . . 21
4.3 Address Values . . . 21
4.4 Instruction Storage . . . 22
4.5 Exception Vector Storage . . . 22
4.6 Immediate Values . . . 22
4.7 Software Layout Conventions . . . 23
4.8 Arithmetic and Address Wraparound . . . 23
4.9 Memory Ordering . . . 24

5 Status Register 25
5.1 Overview . . . 25
5.2 Status Register Layout . . . 25
5.3 Condition Flags (Non-Privileged) . . . 25
5.3.1 Zero Flag (Z) – Bit 0 . . . 25
5.3.2 Negative Flag (N) – Bit 1 . . . 25
5.3.3 Carry Flag (C) – Bit 2 . . . 25
5.3.4 Overflow Flag (V) – Bit 3 . . . 26
5.3.5 Reserved Flags – Bits 4–7 . . . 26
5.4 Control Flags (Privileged) . . . 26
5.4.1 Protected Mode (P) – Bit 8 . . . 26
5.4.2 Hardware Interrupt Enable Bits (E1–E4) – Bits 9–12 . . . 26
5.4.3 Exception Active (EA) – Bit 13 . . . 27
5.4.4 Trap on Address Overflow (A) – Bit 14 . . . 28
5.4.5 Trace Mode (T) – Bit 15 . . . 28
5.5 Status Register Updates . . . 28
5.5.1 Automatic Updates . . . 28
5.5.2 Manual Updates . . . 29
5.6 Reading the Status Register . . . 29
5.7 Exception Handling Effects . . . 29
5.8 Usage Examples . . . 30

6 Exception and Interrupt Handling 31
6.1 Overview . . . 31
6.1.1 Exception Levels and Priority . . . 31
6.1.2 Vector Table . . . 33
6.1.3 Exception Summary . . . 33
6.1.4 Instructions Used in This Chapter . . . 34
6.2 Faults . . . 35

6.2.1 Fault Categories . . . 35
6.2.2 Bus Fault . . . 36
6.2.3 Bus Protection Fault . . . 36
6.2.4 Alignment Fault . . . 36
6.2.5 Segment Overflow Fault . . . 37
6.2.6 Invalid Opcode Fault . . . 37
6.2.7 Privilege Violation Fault . . . 37
6.2.8 Instruction Trace Fault . . . 39
6.2.9 Level Five Hardware Exception Conflict . . . 40
6.2.10 Double Fault . . . 40
6.3 Hardware Exceptions (Interrupts) . . . 42
6.3.1 Hardware Interrupt Enable Bits . . . 42
6.3.2 Pending Hardware Interrupts . . . 42
6.3.3 Interrupt Recognition and Latency . . . 43
6.3.4 Level 5 NMI-Like Behavior . . . 43
6.4 Software Exceptions (Traps) . . . 45
6.5 Exception Registers . . . 45
6.5.1 Link Registers . . . 45
6.5.2 Fault Metadata Register Format . . . 46
6.6 Exception Processing . . . 47
6.6.1 Exception Entry . . . 49
6.7 Return from Exception . . . 49
6.7.1 Accessing Link Registers . . . 50
6.8 Halt Request . . . 50
6.9 Reset and Startup . . . 51
6.9.1 Reset State . . . 51
6.9.2 Reset Output Hold . . . 52
6.9.3 Reset Vector Fetch . . . 52
6.10 Exception Coprocessor Instructions . . . 52
6.10.1 Internal Instructions . . . 53
6.10.2 Coprocessor Select Register (CSR) . . . 53

II Instruction Set Architecture 55

7 Instruction Formats 57
7.1 Overview . . . 57
7.2 Format Overview . . . 57
7.3 Common Fields . . . 57
7.4 Immediate Format . . . 57
7.4.1 Structure . . . 57
7.4.2 Encoding Examples . . . 59
7.5 Short Immediate Format (with Shift) . . . 60
7.5.1 Structure . . . 60
7.5.2 Encoding Examples . . . 61
7.6 Register Format . . . 61
7.6.1 Structure . . . 61
7.6.2 Register Operand Semantics . . . 62
7.6.3 Encoding Examples . . . 63
7.7 Operand Encoding Details . . . 64
7.7.1 Condition Codes . . . 64
7.7.2 Shift Types . . . 64

7.8 Opcode Encoding Patterns . . . 64
7.8.1 Opcode Groups . . . 64
7.8.2 Test vs. Save . . . 64
7.9 Decode Trees . . . 65
7.10 Instruction Fetch and Alignment . . . 69
7.10.1 Fetch Process . . . 69
7.10.2 Alignment Requirements . . . 70
7.11 Format Selection Guidelines . . . 70

8 Addressing Modes **71**
8.1 Overview . . . 71
8.2 Addressing Mode Matrix . . . 72
8.3 Legal Modes by Instruction Family . . . 72
8.4 Zero-Displacement Shorthand . . . 73
8.5 Operand Categories . . . 73
8.6 Common Address Calculation Rules . . . 74
8.7 Per-Mode Encoding Reference . . . 74
8.8 Address Register Selection . . . 77
8.9 Addressing Mode Examples . . . 77
8.10 Addressing Mode Restrictions . . . 78
8.10.1 Privilege Mode Restrictions . . . 78
8.10.2 Alignment . . . 78

9 Shift Operations **79**
9.1 Overview . . . 79
9.2 Shift Types . . . 79
9.2.1 Shift Type Encoding . . . 79
9.3 Shift Syntax . . . 79
9.4 Shift Type Details . . . 80
9.4.1 Logical Shift Left (LSL) . . . 80
9.4.2 Logical Shift Right (LSR) . . . 80
9.4.3 Arithmetic Shift Left (ASL) . . . 81
9.4.4 Arithmetic Shift Right (ASR) . . . 81
9.4.5 Rotate Left (RTL) . . . 82
9.4.6 Rotate Right (RTR) . . . 82
9.5 Shift Operands . . . 83
9.5.1 Mode 0: Shift by Immediate . . . 83
9.5.2 Mode 1: Shift by Register Count . . . 83
9.6 Shift Count . . . 83
9.7 Status Flag Effects . . . 84
9.7.1 Default Behavior . . . 84
9.7.2 Carry Flag . . . 84
9.7.3 Zero Flag . . . 84
9.7.4 Negative Flag . . . 84
9.7.5 Overflow Flag . . . 84
9.8 Common Use Cases . . . 84
9.9 Multi-Word Shifts . . . 85
9.10 Performance Considerations . . . 86
9.11 Limitations . . . 86
9.12 Status Flag Update Control . . . 86
9.12.1 Status Override Syntax . . . 86
9.12.2 Examples . . . 87

9.12.3 Use Cases 87
9.12.4 Assembler Syntax Notes 88

10 Condition Codes 89
10.1 Overview 89
10.2 Condition Code Encoding 89
10.3 Condition Code Categories 89
10.3.1 Simple Conditions 89
10.3.2 Unsigned Comparisons 90
10.3.3 Signed Comparisons 90
10.3.4 Special Conditions 90
10.4 Assembly Syntax 91
10.5 Usage with Compare Instructions 91
10.6 Condition Code Truth Tables 92
10.6.1 Unsigned Comparison (CMPR rA, rB) 92
10.6.2 Signed Comparison (CMPR rA, rB) 92
10.7 Advanced Usage 93
10.8 Condition Code Selection Guide 94
10.9 Performance Considerations 94
10.10Common Pitfalls 94

III SIRCIS Instruction Reference 97

11 Reading Instruction Descriptions 99
11.1 Instruction Entry Fields 99
11.2 Notation 101
11.3 Status Flag Effect Symbols 101
11.4 Conditional Execution 102
11.5 Status Update Overrides 102

12 Instruction Summary 103
12.1 Overview 103
12.2 Complete Instruction List 104
12.3 Instruction Grouping Patterns 105
12.3.1 Opcode Organization 105
12.3.2 Save vs. Test Variants 105
12.4 Meta-Instructions 105
12.5 Instruction Timing 106
12.6 Next Chapters 106

13 ALU Instructions 109
13.1 Overview 109
13.2 ALU Instruction Families 109
13.2.1 Arithmetic Instructions 109
13.2.2 Logical Instructions 109
13.2.3 Data Movement 109
13.2.4 Test and Compare 109
13.3 Legal Forms 110
13.4 Common ALU Semantics 111
13.5 Status Flag Updates 111
13.5.1 Flag Computation Formulas 112
13.5.2 Manual Flag Update Control 113

SHFT – Shift with Status Update Meta-Instruction . . . 114
ADDI / ADDR – Add Immediate / Add Register . . . 116
ADCI / ADCR – Add with Carry . . . 118
SUBI / SUBR – Subtract Immediate / Subtract Register . . . 120
SBCI / SBCR – Subtract with Carry . . . 122
ANDI / ANDR – AND Immediate / AND Register . . . 124
ORRI / ORRR – OR Immediate / OR Register . . . 126
XORI / XORR – XOR Immediate / XOR Register . . . 128
MOVI / MOVR – Move Immediate / Move Register . . . 130
CMNI / CMNR – Compare Negative Immediate / Compare Negative Register . . . 132
CNCI / CNCR – Compare Negative with Carry Immediate / Register . . . 134
CMPI / CMPR – Compare Immediate / Compare Register . . . 136
CPCI / CPCR – Compare with Carry Immediate / Register . . . 138
TSAI / TSAR – Test AND Immediate / Test AND Register . . . 140
TSOI / TSOR – Test OR Immediate / Test OR Register . . . 142
TSXI / TSXR – Test XOR Immediate / Test XOR Register . . . 144
13.6 Summary . . . 145

14 Memory Access Instructions **147**
14.1 Overview . . . 147
14.2 Effective Address Calculation . . . 147
14.3 Legal Forms . . . 148
14.4 Common Memory Semantics . . . 148
14.5 Status Flag Effects . . . 149
LOAD – Load from Memory . . . 150
STOR – Store to Memory . . . 152
14.6 Common Memory Access Patterns . . . 154
  14.6.1 Structure Field Access . . . 154
  14.6.2 Array Iteration . . . 154
  14.6.3 Stack Operations . . . 154
14.7 Shift Operations with Memory Instructions . . . 154
  14.7.1 STOR with Shift . . . 154
  14.7.2 Supported Shift Types . . . 154
  14.7.3 Shift Limitations . . . 155
  14.7.4 Use Cases . . . 155
  14.7.5 Status Flags . . . 155

15 Control Flow Instructions **157**
15.1 Overview . . . 157
15.2 Legal Forms . . . 158
15.3 Common Semantics . . . 158
15.4 Status Flag Effects . . . 159
15.5 Branch Meta-Instructions . . . 159
BRAN – Branch Meta-Instruction . . . 160
BRSR – Branch to Subroutine Meta-Instruction . . . 162
RETS – Return from Subroutine Meta-Instruction . . . 164
15.6 Load Effective Address . . . 164
LDEA – Load Effective Address . . . 165
LDEL – Load Effective Address and Link . . . 167
15.7 Long Jump Meta-Instructions . . . 168
LJMP – Long Jump Meta-Instruction . . . 169
LJSR – Long Jump to Subroutine Meta-Instruction . . . 171

15.8 Control Flow Patterns . . . 172
15.8.1 If-Then-Else . . . 172
15.8.2 While Loop . . . 172
15.8.3 Switch/Jump Table . . . 173

16 Coprocessor Instructions 175
16.1 Overview . . . 175
16.2 Legal Forms . . . 175
16.3 Common Semantics . . . 175
COPI/COPR – Coprocessor Call . . . 177
16.4 Coprocessor Compatibility . . . 178
16.4.1 Model and Revision Policy . . . 179
16.5 Exception Unit (Coprocessor 1) . . . 179
16.5.1 Common Operations . . . 179
16.6 Exception Unit Meta-Instructions . . . 180
EXCP – Software Exception Meta-Instruction . . . 181
WAIT – Wait for Interrupt Meta-Instruction . . . 183
RETE – Return from Exception Meta-Instruction . . . 184
RSET – System Reset Meta-Instruction . . . 185
ETFR/ETTR – Exception Link Transfer Meta-Instructions . . . 186
16.7 DMA Unit (Coprocessor 2) . . . 187
16.7.1 DMA Operations . . . 187
16.7.2 Segment Overflow . . . 188
16.7.3 Retrying After a Fault . . . 189
DMAR – DMA Read to Registers Meta-Instruction . . . 192
DMAW – DMA Write from Registers Meta-Instruction . . . 194
DMAT – DMA Memory Transfer Meta-Instruction . . . 196
16.8 Integer Maths Unit (Coprocessor 3) . . . 197
16.8.1 Maths Operations . . . 198
MULU/MULS – Integer Multiply Meta-Instructions . . . 199
DIVU/DIVS – Integer Divide Meta-Instructions . . . 201

17 Meta-Instructions 203
17.1 Overview . . . 203
NOOP – No Operation . . . 204

A Opcode Map 205
A.1 Complete Opcode Reference . . . 205
A.2 Opcode Patterns . . . 206
A.2.1 Format Identification . . . 206
A.2.2 Save vs. Test . . . 206
A.2.3 Memory Operations Decoding . . . 207
A.3 Undocumented Instructions . . . 207
A.4 Quick Lookup Table . . . 207

B Instruction Timing 209
B.1 Overview . . . 209
B.2 Six-Stage Execution Phases . . . 209
B.3 Timing Characteristics . . . 210
B.3.1 Coprocessor Call Timing . . . 210
B.3.2 Exception and Fault Timing . . . 211
B.4 Conditional Execution . . . 211
B.5 Program Timing Examples . . . 211

B.5.1 Simple Loop . . . 211
B.5.2 Function Call Overhead . . . 212
B.5.3 Conditional Block . . . 212
B.6 Performance Considerations . . . 212
B.6.1 No Pipeline Stalls . . . 212
B.6.2 Memory Access . . . 212
B.6.3 Optimization Guidelines . . . 212
B.7 Theoretical Performance . . . 213

C Undocumented Instructions 215
C.1 Overview . . . 215
C.2 Undefined Operand Combinations . . . 215
C.2.1 Aliased Address-Register Writes . . . 215
C.3 Undocumented Opcode List . . . 215
C.4 Implementation Notes . . . 216
C.4.1 Implementation Behavior . . . 216
C.5 Why Undocumented? . . . 216
C.6 Recommendations . . . 216
C.6.1 For Application Developers . . . 216
C.6.2 For Compiler Writers . . . 216
C.6.3 For Hardware Implementers . . . 216
C.7 Future Standardization . . . 216
C.8 Opcode Space for Expansion . . . 216

D Example Programs 217
D.1 Program Conventions . . . 217
D.2 Byte Sieve . . . 217
D.3 Faults . . . 218
D.4 Hardware Exception . . . 225
D.5 Software Exception . . . 229
D.6 Store Load . . . 230

E Quick Reference 235
E.1 Instruction Mnemonic Index . . . 235
E.2 Status Register Bit Reference . . . 237
E.3 Bus Access Type (BAT) Reference . . . 238
E.4 Where to Find Other Tables . . . 239

List of Tables

1 The SIRC Manual Set (This Manual Is the First) . . . iv
2 Manual-Wide Typographic Conventions . . . xix
3 Operand Forms . . . xx

1.1 SIRC-1 CPU Specifications . . . 4
1.2 List of Coprocessors . . . 5
1.3 Exception Vector Table Regions . . . 6

3.1 Register Encoding . . . 18

4.1 External Byte Order for 16-bit Words . . . 21
4.2 Address Register Pair Interpretation . . . 22
4.3 Immediate Field Interpretation . . . 23

6.1 Exception Quick Reference . . . 34
6.2 Fault-Inside-L5-Handler Timeline . . . 44
6.3 Exception Link Register Assignment . . . 46
6.4 Reset State . . . 51
6.5 Exception Coprocessor Instruction Summary . . . 52

7.1 Instruction Format Summary . . . 57
7.2 Immediate Format Encoding Examples . . . 59
7.3 Short Immediate Format Encoding Examples . . . 61
7.4 Register Format Encoding Examples . . . 63
7.5 Save and Test-Only ALU Instructions . . . 65
7.6 Format Selection Examples . . . 70

8.1 Addressing Modes Summary . . . 71
8.2 Assembler Operand Forms Mapped to Addressing Modes . . . 71
8.3 Addressing Mode Side Effects and Legal Instruction Families . . . 72
8.4 Addressing Mode Legality by Instruction Family . . . 73
8.5 Zero-Displacement Shorthand . . . 73
8.6 Address Register Encoding . . . 77

9.1 Shift Type Encoding . . . 79
9.2 Examples of First Source Operands . . . 80
9.3 Shift Operand Modes . . . 83
9.4 Status Flag Override Modifiers . . . 87

10.1 Condition Code Encoding . . . 89
10.2 Unsigned Comparison Results . . . 92
10.3 Signed Comparison Results . . . 92
10.4 Condition Code Selection Guide . . . 94

11.1 Instruction Description Notation . . . 101
11.2 Status Flag Effect Symbols . . . 102
11.3 Status Update Source Encoding . . . 102

12.1 Complete SIRCIS Instruction Set . . . 104
12.2 Meta-Instructions (Summary) . . . 106

13.1 Legal ALU Instruction Forms . . . 110
13.2 ALU Status Flag Effects . . . 112
13.3 Carry and Overflow Flag Formulas . . . 113

14.1 Legal Memory Instruction Forms . . . 148
14.2 Memory Instruction Status Flag Effects . . . 149

15.1 Legal Control-Flow and Effective-Address Forms . . . 158
15.2 Control-Flow Instruction Status Flag Effects . . . 159

16.1 Legal Coprocessor Instruction Forms . . . 175
16.2 Standard Coprocessor IDs . . . 178
16.3 Exception Unit Operations . . . 179
16.4 DMA Unit Operations . . . 187
16.5 Integer Maths Unit Operations . . . 198
16.6 Maths Status Word in `r3` . . . 198

17.1 Meta-Instruction Cross-Reference . . . 203

A.1 Complete SIRCIS Opcode Map . . . 205
A.2 Instruction Variant Quick Reference . . . 207

B.1 Instruction Timing . . . 210

C.1 Undocumented Instruction Opcodes . . . 215

E.1 Alphabetical Mnemonic Index . . . 235
E.2 Status Register Bit Summary . . . 238
E.3 Bus Access Type (BAT) Encoding . . . 238
E.4 Reference Table Locations . . . 239

List of Figures

2.1 SIRC-1 CPU Execution Phase Architecture 9

3.1 Register Programming Model 14
3.2 Address Register Pair Formation 15

5.1 Status Register Bit Layout 25

6.1 SIRC-1 Exception Handling Overview 32
6.2 Privilege Checks in Protected Mode 39
6.3 Exception Link Register Saved-State Layout 46
6.4 Fault Metadata Register Status Field Layout 46
6.5 SIRC-1 Exception Entry and Return 48

7.1 Immediate Instruction Format 58
7.2 Short Immediate with Shift Format 60
7.3 Register Instruction Format 61
7.4 Decode tree for the immediate ALU group (`0x0_`) 66
7.5 Decode tree for the short immediate ALU group (`0x2_`) 67
7.6 Decode tree for the register ALU group (`0x3_`) 68
7.7 Decode tree for the memory and control group (`0x1_`) 69

9.1 Logical Shift Left 80
9.2 Logical Shift Right 80
9.3 Arithmetic Shift Left 81
9.4 Arithmetic Shift Right 81
9.5 Rotate Left 82
9.6 Rotate Right 82

List of Examples

3-1 Basic Register Operations 18
3-2 Address Register Usage 19
3-3 Subroutine Call 19
5-1 Enabling Protected Mode 30
5-2 Saving and Restoring Status 30
8-1 Structure Access 77
8-2 Array Iteration 78
8-3 Stack Frame 78
9-1 Constant Multiply and Bit Set (LSL) 80
9-2 Unsigned Divide and Byte Extraction (LSR) 81
9-3 Signed Multiply by 2 (ASL) 81
9-4 Signed Divide and Averaging (ASR) 81
9-5 Rotating a Value Left (RTL) 82
9-6 Rotating a Value Right and Testing a Bit (RTR) 82
9-7 Multiplication by Constants 84
9-8 Division by Powers of 2 85
9-9 Bit Field Extraction 85
9-10 Bit Manipulation 85
9-11 32-bit Left Shift 85
9-12 32-bit Right Shift (Unsigned) 86
9-13 32-bit Right Shift (Signed) 86
9-14 Default Behavior (ALU Flags) 87
9-15 Shift Flag Override 87
9-16 No Flag Update 87
10-1 Unsigned Comparison 91
10-2 Signed Comparison 91
10-3 Equality Testing 91
10-4 Conditional Assignment 93
10-5 Conditional Increment 93
10-6 Conditional Function Call 93
10-7 Loop Construction 93
13-1 Manual Status Flag Update Control 113
13-2 Shifting and Testing with SHFT 115
13-3 Basic and Scaled Addition 117
13-4 32-bit Addition with Carry Propagation 119
13-5 Basic Subtraction and Loop Counting 121
13-6 32-bit Subtraction with Borrow Propagation 123
13-7 Bit Masking and Clearing with AND 125
13-8 Setting Bits with OR 127
13-9 Bit Toggling, Equality Testing, and XOR Swap 129
13-10Moving Constants and Copying Registers 131
13-11Testing Sums Without Writing Them 133
13-12Testing 32-bit Sums Without Writing Them 135
13-13Comparisons and Range Checking 137
13-1432-bit Comparison with Borrow Propagation 139
13-15Bit Testing with TSA 141
13-16Testing Two Values at Once with TSO 143

13-17 Equality and Pattern Testing with TSX . . . 145
14-1 Indirect, Post-Increment, and Register-Offset Loads . . . 151
14-2 Indirect, Shifted, and Pre-Decrement Stores . . . 153
15-1 Unconditional and Conditional Branching . . . 161
15-2 Calling and Returning with BRSR/RETS . . . 163
15-3 Returning from a Subroutine . . . 164
15-4 Pointer Arithmetic and Computed Jumps with LDEA . . . 166
15-5 Calling Through a Register with LDEL . . . 168
15-6 Jumping Through a Register with LJMP . . . 170
15-7 Calling Through a Register with LJSR . . . 172
16-1 Direct Coprocessor Calls . . . 178
16-2 Triggering a Software Exception . . . 182
16-3 Idling Until an Interrupt . . . 183
16-4 Returning from an Interrupt Handler . . . 184
16-5 Requesting a Software Reset . . . 185
16-6 Correcting a Faulting Return Address . . . 187
16-7 A Retry-Safe DMA Copy Routine . . . 191
16-8 Reading a Memory Block into Registers . . . 193
16-9 Writing Registers to a Memory Block . . . 195
16-10 Copying a Memory Block with DMAT . . . 197
16-11 Unsigned 16x16 Multiply . . . 200
16-12 Unsigned Divide with Error Check . . . 202
17-1 Timing Delay with NOOP . . . 204
D-1 Byte sieve . . . 217
D-2 Fault handling . . . 218
D-3 High-address fault handling . . . 224
D-4 Hardware exception . . . 225
D-5 Hardware exception data . . . 229
D-6 Software exception . . . 229
D-7 Store/load . . . 230

Notation and Glossary

This section collects the typographic conventions, normative vocabulary, and architectural terminology used throughout the manual, in one place. Instruction-entry-specific notation (`rD`, `rS1`, `shift`, and similar placeholder names used only inside instruction descriptions) is defined separately in Chapter 11, Table 11.1, alongside the status-flag-effect symbols (Table 11.2); this section does not repeat those, only the conventions that apply manual-wide.

Typographic Conventions

Convention	Meaning
`r1`, `sr`, `ph`	Register names appear in typewriter font, whether general-purpose, address-pair, or status.
`0x1A`	A literal instruction-opcode byte value, written `0x` followed by two uppercase hexadecimal digits.
`#123`	A concrete immediate value written as an operand, prefixed with `#`. May be decimal or hexadecimal.
`0x0002`	A concrete memory address or vector target.
`ADDI`	An instruction mnemonic, always bold typewriter and always uppercase.
`SR.X`	A field or bit of the status register, named by its bit label (for example `SR.A`, `SR.T`); see Chapter 5 for the full bit layout.
`(#4, a)`	Parentheses denote indirection through an address-register pair; a leading − denotes pre-decrement and a trailing + denotes post-increment, for example `-(0, a)` and `(0, a)+`.

Table 2: Manual-Wide Typographic Conventions

Other conventions that do not have a dedicated style:

- **Hexadecimal literals** that aren't a register, opcode, immediate, or address value in one of the forms above (status-word values, encoded field values, bit patterns, coprocessor command words) are written `0x` followed by uppercase hexadecimal digits, for example `0x1A` rather than `0x1a`.
- **Binary literals** in tables are written as a bare digit group with no prefix, for example `00`, `10`, `111` rather than `0b10`.
- **Bit numbers** are written as plain text: "bit 15", "bits 4–7". Bit 15, or the highest-numbered bit of whatever field is under discussion, is always the most significant bit, and is always drawn on the left in every bit-field diagram.
- **Numeric ranges** use an en dash, for example `0x00–0x0F` or "bits 5–4", both in prose and in tables.
- **Word size**: "16-bit word" is stated once per chapter or major section on first use, then shortened to plain "word" for the rest of that section.

Assembly Language Summary

Instruction entries in this manual use the canonical SIRC assembly notation. This summary is enough to read and write the listings in the manual. The complete language, including every directive and the tools that process it, is described in the *SIRC-1 Assembly Programmer's Manual*.

Source Format

- A **comment** starts with `;` and runs to the end of the line.
- A **label** is written `:name` before the next instruction or directive, and referred to as `@name`. Names use letters, digits and underscores. The colon is written before the name, not after it.
- **Numbers** used as values are prefixed with `#`: decimal (`#42`, `#-3`), hexadecimal (`#0x2A`) or binary (`#0b0010_1010`). Underscores may be used to separate digits.
- **Mnemonics** and condition codes are uppercase; register names are lowercase.
- A named constant is defined with `.EQU $NAME #value` and used as `$NAME`, without a `#`.

Instruction Syntax

An instruction is written as a mnemonic, an optional status update source, an optional condition, and its operands:

```
MNEMONIC[S]|cond operand, operand, ...
```

Status update source `[N]`, `[A]` or `[S]` selects what updates the status flags (Chapter 11). It may be omitted.

Condition `|cond` makes the instruction conditional, with one of `AL` (always, the default), `==`, `!=`, `CS`, `CC`, `NS`, `NC`, `OS`, `OC`, `HI`, `LO`, `>=`, «, », `<=` or `NV` (Table 10.1).

Destination first In every operand list the destination is written first, on the left.

Operand	Written as	Example
Immediate value	`#n`, `@label` or `$NAME`	`MOVI r1, #0x20`
Register	`r1`–`r7`, `sr`, address halves	`MOVR r2, r1`
Address register pair	`a`, `s`, `p`, `l`	`LJMP a`
Indirect, immediate offset	`(#n, addr)`	`LOAD r1, (#4, a)`
Indirect, register offset	`(rO, addr)`	`LOAD r1, (r2, a)`
Post-increment	`(#n, addr)+` or `(rO, addr)+`	`LOAD r1, (#0, a)+`
Pre-decrement	`-(#n, addr)` or `-(rO, addr)`	`STOR -(#0, s), r1`
Shift	`type #count` or `type rC`	`ADDR r1, r2, LSL #2`

Table 3: Operand Forms

The address register halves are `ah`, `al`, `sh`, `sl`, `ph`, `pl`, `lh` and `ll`. Only some instructions accept a label reference (`@label`); see the `@label` entry below. A shift type is one of `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL` or `RTR` (Table 9.1), and an immediate count is `0`–`15`. Chapter 11 defines the placeholder names (`rD`, `rS1`, `#imm16` and so on) used in instruction entries.

Directives and Symbol References

`.ORG addr` Sets the word address at which subsequent instructions and data are assembled. The operand syntax and alignment rules are defined in the Directives and Symbols chapter of the *SIRC-1 Assembly Programmer's Manual*. In a program linked from several objects the address is relative to the start of its own object, so the reset and exception vector table (Chapter 6) must be in the first object.

`.DB #value`, `.DW #value`, `.DQ #value` Reserve one program slot and initialize it with a numeric value, or, written as `.DB @label`, `.DW @label` or `.DQ @label`, with a label's full 24-bit address. *Despite their names, all three directives occupy exactly one 4-byte (2-word) program slot, the same size as an instruction.* `.DB #0xAB` assembles to the bytes `00 00 00 AB` (value in the low byte), `.DW #0xABCD` to `00 00 AB CD` (value in the low word), and `.DQ #0xABCD1234` to `AB CD 12 34` (value fills the slot). A label reference always fills the whole slot, so any of the three directives can hold an address anywhere in the 24-bit address space.

`.EQU $NAME #value` Defines a named constant, usable anywhere a `#immediate` operand is accepted by writing `$NAME` in its place. Unlike a label, a constant must be defined before its first use in the file; it cannot be the address of `.ORG`.

`@label` References a label declared anywhere in the program, before or after the reference; references are resolved at link time. In **`BRAN`** or **`BRSR`** it resolves to a 16-bit signed word displacement from the instruction to the label. In a **`MOVI`**, **`LOAD`**, **`STOR`**, **`LDEA`**, **`LDEL`**, **`LJMP`** or **`LJSR`** operand it resolves to the label's low word only. Other instructions (ALU immediates, **`COPI`** and the coprocessor meta-instructions) do not accept a label. In a `.DB`, `.DW` or `.DQ` directive it resolves to the label's full 24-bit address. An explicit suffix overrides this default: `@label.r` forces the PC-relative displacement form, `@label.l` the low word, and `@label.u` the high word.

```
    .EQU $ANSWER #42

    .ORG 0x0000
    .DQ @start ; reset vector: CPU begins by loading this pointer

    .ORG 0x0080
    .DQ @level2_handler ; vector 0x40: level 2 hardware exception vector

:start
    MOVI ah, @lookup.u ; @label.u selects the high word
    MOVI al, @lookup ; @label defaults to the low word for MOVI
    LOAD r1, (#0, a) ; r1 = lookup[0]
    CMPI r1, $ANSWER ; constants defined with .EQU are used without '#'
    BRAN|== @start ; @label defaults to a PC-relative offset for BRAN/BRSR

:done
    BRAN @done ; stop here

:level2_handler
    RETE

:lookup
    .DW #10
    .DW #20
    .DW #30
```

The reset vector in the listing is not optional: per Chapter 6, the CPU begins execution by reading a stored pointer from word addresses `0x000000`–`0x000001`, not by executing whatever instruction happens to be placed there.

Normative Vocabulary

The following words carry a specific technical meaning whenever they appear in this manual, distinct from their everyday English sense:

must / must not A mandatory architectural requirement. Hardware or software that does not comply is not a conformant SIRC-1 implementation.

may A permitted option. The CPU, the assembler, or the programmer is free to choose either way.

should / should not A recommendation, not a conformance requirement.

reserved A field, opcode, or coprocessor ID that is not currently assigned a meaning. Software must not assume any particular value when reading a reserved field, and must not rely on any behavior it may have, except where this manual defines a fault for it (for example, a reserved coprocessor operation raises an invalid-opcode fault). A future CPU revision may assign a reserved field a meaning.

architecturally undefined The architecture does not specify the result. It may vary by CPU implementation, by revision, or even between two executions of the identical instruction on the identical CPU; software must never depend on any particular outcome.

implementation-defined Behavior that is fixed and consistent within one particular CPU implementation or model, but that may differ between models – for example, the extra cycles an optional maths coprocessor takes to complete a multiply. This is a stronger guarantee than *architecturally undefined*: a given implementation's behavior is at least self-consistent, even though the manual does not pin down what that behavior is.

Signals, Requests and Pins

This manual describes the architecture, not a particular chip. It uses three terms for the CPU's external interface:

Signal A logical input or output of the CPU, such as $\overline{\text{BERR}}$, $\overline{\text{RSTI}}$, $\overline{\text{HALT}}$ or $\overline{\text{TRCE}}$. The manual names a signal only where the architectural behavior is defined by it. Active-low signal names are written with an overbar.

Request / event What a signal means architecturally: an *interrupt request*, a *reset request*, a bus error. A request is raised, recognized and withdrawn; the manual describes what the CPU does when that happens, and when requests are recognized, but not how the request physically reaches the CPU.

Pin A physical connection on a particular package. Pins, electrical levels and timing are implementation details and are documented in the *SIRC-1 Hardware Reference Manual*, not here.

A simulator or other implementation may carry the same signals by other means (a software call, an interrupt controller) and still conform to this manual.

Register Name Conventions

SIRC-1 has sixteen 16-bit registers: seven general-purpose registers `r1`–`r7`, four address register pairs (`l`, `a`, `s`, `p`), and the status register `sr`. Each address register pair splits into a high and low half, named by appending `h`/`l` to the pair name – `lh`/`ll`, `ah`/`al`, `sh`/`sl`, `ph`/`pl` – so that either half can be addressed as an ordinary 16-bit register on its own, or the pair addressed together as a 24-bit address. See Chapter 3 for the full register model, encoding, and privilege rules.

Glossary of Architectural Terms

Coprocessor One of the specialized execution units a **`COPI`**/**`COPR`** call can dispatch to – the processing unit itself (coprocessor `0x0`), the exception unit (`0x1`), the optional DMA unit (`0x2`), and the optional integer maths unit (`0x3`). See Chapter 16.

Coprocessor select register (CSR) The 16-bit internal register that holds the next coprocessor command: coprocessor ID, operation and operand. The CU dispatches to the coprocessor it selects. See Section 2.3.2.

Displacement A signed value added to an address register's low word to compute an effective address, as used by memory and effective-address instructions. Distinct from *offset*, which this manual reserves for memory/structure offsets.

Exception The general term for any event that diverts program flow to a vector: a *fault*, a *hardware exception*, or a *software exception*. See Chapter 6.

Exception level The priority level of the exception currently being serviced (0 when not handling any exception). A new exception can only preempt the current one if its priority is strictly higher, except that faults are dispatched at any level and a Level 5 request during a Level 5 handler raises a conflict fault (see Chapter 6).

Fault A priority-7 exception raised by the CPU itself in response to an error condition (for example, a bus error or a privilege violation) or a debug event (instruction trace). Faults cannot be masked, though two of them are conditional on whether they are raised at all: Segment Overflow requires `SR.A`, and Instruction Trace requires `SR.T` or the force-trace request ($\overline{\text{TRCE}}$) and is never raised at exception level 7.

General-purpose register Any of `r1`–`r7`. The CPU does not enforce a usage convention among them.

Hardware exception An exception signaled by an external interrupt. Also informally called an *interrupt* at the hardware level (interrupt requests, interrupt priority, interrupt enable) – the two terms are synonymous.

Meta-instruction An assembler-level mnemonic (for example **`BRAN`**, **`NOOP`**, **`LJSR`**) that assembles to a specific CPU instruction encoding rather than having an opcode of its own. See the relevant instruction-family chapter for each meta-instruction's lowering.

Protected mode The non-privileged CPU mode (`SR.P` set). Most privileged operations fault if attempted in protected mode; see Chapter 5. The inverse of supervisor mode.

Retryable fault A fault whose triggering operation does not take effect, so that returning with **`RETE`** safely retries it, except after a DMA burst (Section 16.7.3). See Chapter 6 for the full retryable set.

Segment The 64K-word range of addresses that share one high-word value (bits 7–0 of an address register pair's high word). Displacement arithmetic wraps within a segment; see Chapter 4.

Software exception An exception raised deliberately by executing `EXCP`. Also called a *trap*.

Supervisor mode The privileged CPU mode (`SR.P` clear). All operations are available. The inverse of protected mode.

Word address SIRC-1 addresses memory in units of 16-bit words, not 8-bit bytes; incrementing an address register by one advances by one word. See Chapter 4 for the boundary between this architectural word addressing and any external byte-level representation.

Part I

Architecture Overview

CHAPTER 1

Introduction

1.1 Overview

The SIRC-1 is a 16-bit RISC processor intended for general-purpose computing.
It implements the SIRCIS (SIRC Instruction Set) instruction set architecture, which provides a powerful set of operations while keeping the CPU implementation simple.

1.2 Design Philosophy

1.2.1 Design Principles

The SIRC-1 design was guided by Reduced Instruction Set Computer (RISC) design principles:

- **Load/Store Architecture:** All ALU operations work exclusively on registers. Memory access is performed only through dedicated load and store instructions.
- **Fixed Instruction Length:** All instructions are exactly 32 bits (2 words) in length, simplifying instruction fetch and decode logic.
- **Simple Instruction Formats:** Only three instruction formats are supported: Immediate, Short Immediate with Shift, and Register.
- **Register-Rich:** Sixteen addressable registers provide ample workspace for computations without frequent memory access.
- **Simple Execution Model:** The SIRC-1 uses a regular instruction set designed for straightforward hardware implementation without microcode. Most instructions execute in six clock cycles, corresponding to six fixed execution phases, excluding bus wait states. DMA commands require additional transfer cycles.

1.2.2 Design Goals

The SIRC-1 was designed with several key goals in order of importance:

Simplicity The CPU should be simple enough to be implemented in hardware without requiring complex microcode engines or multi-level instruction decoders.

Developer Ergonomics Powerful instruction encodings, including conditional execution and bit shifting, should reduce the number of instructions and branches a program must track.

Performance While simple, the design should allow for reasonable performance through efficient instruction encoding and the potential for future enhancements like pipelining.

Memory Protection Basic privilege levels and memory segmentation provide rudimentary protection for system software.

1.2.3 Applications

The SIRC-1 is designed for general-purpose computing, such as the core of a microcomputer.
It should not be used in memory-constrained environments, as fixed-length instructions and the lack of byte addressing can result in larger program sizes.

1.3 Key Features

1.3.1 Architecture Characteristics

Feature	**Specification**
Data Width	16-bit
Addressing	Word Addressing
Address Bus	24-bit word address
Instruction Size	32-bit fixed
General-Purpose Registers	7 (`r1`–`r7`)
Address Register Pairs	4 (l, a, s, p – link, address, stack pointer, program counter)
Total Addressable Registers	16
Instruction Formats	3
Addressing Modes	7 (see Table 8.1)
Condition Codes	16

Table 1.1: SIRC-1 CPU Specifications

1.3.2 Execution Model

The SIRC-1 executes every instruction through six sequential execution phases (normally one phase per clock cycle). For a full description of each phase, see Chapter 2.

This fixed six-stage model means that ordinary processing-unit instructions take 6 clock cycles to complete when bus operations do not require wait states, regardless of their complexity. An executed coprocessor call normally takes 12 cycles, because it adds a second 6-cycle dispatch slot; Appendix B (Section B.3.1) gives the rule and its exceptions. A bus device may hold the current phase by delaying bus acknowledgement, adding wait cycles without adding execution phases. While having a fixed 6-cycle instruction execution model may seem wasteful for simple register operations that do not require memory access, it dramatically simplifies the hardware design by eliminating the need for variable-length execution sequences or microcode.

1.3.3 Coprocessors

The SIRC-1 is made up of multiple coprocessors (also known as modules) that all share the same address/data bus and register file. Only one coprocessor can be active at a time.

The primary coprocessor is the “processing unit” (coprocessor `0x0`) which is active the majority of the time and executes SIRCIS instructions. At power on and after a reset, the exception unit first fetches the reset vector; control then passes to the processing unit, which also regains control whenever another coprocessor finishes.

Other coprocessors are usually activated using the **`COPI`** or **`COPR`** coprocessor-call instructions (see Chapter 16), except for the “exception unit,” which can also be activated when external interrupt requests are raised. Activating a coprocessor using the coprocessor-call instructions is also known as “dispatching” to a coprocessor.

All implementations of the SIRC-1 have the “processing unit” (coprocessor `0x0`) and the “exception unit” (coprocessor `0x1`). Every other coprocessor, including the DMA unit (coprocessor `0x2`), is optional and may not be present in some SIRC-1 models. The instruction set supports up to 16 coprocessors. The coprocessor IDs are stable between models, so that programs are cross-compatible. For example, coprocessor `0x3` is always the Integer Maths Unit, whether or not a given CPU model implements it. This means that if a CPU model is missing an optional coprocessor and a program requires it, the CPU may raise an invalid-opcode fault so that the behavior can be implemented in software where practical.

ID	Name	Optional	Purpose
`0x0`	Processing Unit	No	Executes instructions
`0x1`	Exception Unit	No	Handles interrupts and exceptions
`0x2`	DMA Unit	Yes	Transfers blocks of data via the bus
`0x3`	Integer Maths Unit	Yes	Multiplies and divides integer values
`0x4`–`0xF`	Reserved/model use	Yes	Defined by future or model-specific manuals

Table 1.2: List of Coprocessors

Each coprocessor can execute 16 opcodes: the first 8 (`0x0`–`0x7`) can be called in any mode, and the last 8 (`0x8`–`0xF`) can be called only in supervisor mode. A coprocessor may add further protected-mode restrictions on individual operations; for example, the exception unit's **`EXCP`** (operation `0x1`) faults in protected mode when its vector is below `0x60`.

Note: Software does not normally dispatch to the "processing unit" (coprocessor `0x0`) as it does to the other coprocessors, because it is the default coprocessor and automatically regains control when the other coprocessors are finished. Using **`COPI`** or **`COPR`** with coprocessor `0x0` is valid, but the CPU ignores any operand passed into **`COPI`** or **`COPR`**. The call is effectively a no-op, although the supervisor-only check on operations `0x8`–`0xF` still applies (see Chapter 16).

1.3.4 Privilege Levels

The SIRC-1 supports two privilege levels:

Supervisor Mode Full access to all registers and instructions. Can set the high address-register components, access privileged status register bits, and execute privileged coprocessor operations.

Protected Mode Restricted access for user programs. Cannot write to the high address-register components (providing memory segmentation/protection), cannot access privileged status register bits, and cannot execute privileged coprocessor instructions.

The privilege level is controlled by the protected mode bit in the status register.

1.4 Memory Organization

1.4.1 Address Space

The SIRC-1 uses word addressing, where each address refers to a 16-bit word. With a 24-bit address space, this allows access to 2^{24} = 16,777,216 words, or 32 megabytes of external byte storage. Addresses are formed by combining the low 8 bits of the high address-register component with the full 16-bit low address-register component:

$$\text{Address} = (\text{RegHigh}[7:0] \ll 16) \mid \text{RegLow}[15:0] \qquad (1.1)$$

The high 8 bits of the high register (bits 15–8) are not used for addressing but are available for storage, enabling techniques such as tagged pointers.

For normative byte order, multi-word storage, exception vector layout, immediate interpretation, and wraparound rules, see Chapter 4.

1.4.2 Memory Map

The CPU expects the exception vector table to begin at word address `0x000000`. Each vector-table entry occupies two consecutive 16-bit words: the target address high word followed by the target address low word.

Region	Vector IDs	Word addresses	Purpose
Reset	`0x00`	`0x000000`–`0x000001`	Initial program counter after reset
Faults	`0x01`–`0x0F`	`0x000002`–`0x00001F`	Fault and reserved privileged exception vectors
Hardware	`0x10`–`0x5F`	`0x000020`–`0x0000BF`	Hardware exception vectors at `0x10`, `0x20`, `0x30`, `0x40`, `0x50`; all other entries reserved
User traps	`0x60`–`0xFF`	`0x0000C0`–`0x0001FF`	User software exception vectors

Table 1.3: Exception Vector Table Regions

Beyond these reserved regions, the memory layout is flexible and can be defined by system software.

1.4.3 Alignment

- All memory addresses refer to 16-bit words.
- There are no alignment restrictions for general memory I/O.
- All instructions span 2 words and must begin at an even word address.
- Unaligned instruction fetches (odd word address) trigger an alignment fault.

1.5 Document Organization

This manual is organized into three main parts:

Part I: Architecture Overview Covers the register model, status register, and overall CPU architecture.

Part II: Instruction Set Architecture Details instruction formats, addressing modes, shift operations, and condition codes.

Part III: SIRCIS Instruction Reference Provides detailed documentation for each instruction, including encoding, operation, and examples.

Appendices provide quick reference materials including complete opcode maps, instruction timing, notes on undocumented instructions, and an alphabetical instruction index.

CHAPTER 2

CPU Architecture

2.1 Overview

The SIRC-1 CPU architecture takes advantage of the regularity of the SIRCIS instruction set to execute ordinary instructions through a common six-phase execution sequence.

Control signals are generated directly from the decoded instruction and current execution phase, eliminating the need for separate microprograms for individual instructions.

This approach is intended to support compact, hardwired control logic, potentially reducing transistor count and die area compared with microprogrammed implementations. Eliminating microinstruction fetch delays may also permit higher clock frequencies.

2.2 Six-Stage Execution Phases

Most SIRC-1 instructions follow a six-phase execution model and complete in 6 clock cycles when no bus wait states are required.

A phase that performs a bus operation may be held until the bus acknowledges it. DMA commands dispatch through the normal six-phase coprocessor-call path and then hold instruction fetch while the DMA unit completes its transfer.

Each address-register pair, including `p`, splits into a high half and a low half, written `Xh` and `Xl` (for example `ph` and `pl`); Chapter 3 describes this split in detail.

Phase 0: Instruction Fetch (High Word) – 1 cycle

- Fetch bits 31–16 from memory address in `p`
- `p` is unchanged

Phase 1: Instruction Fetch (Low Word) – 1 cycle

- Fetch bits 15–0 from memory address in `p` + 1
- Instruction is now fully available for decode
- `p` is unchanged

Phase 2: Decode and Register Fetch – 1 cycle

- Decode instruction opcode and operands
- Read source registers (optionally shifted)
- Evaluate condition codes
- Advance `pl` by 2 to the address of the next instruction (`ph` is unchanged)

Phase 3: Execute and Address Calculation – 1 cycle

- Perform ALU operation, and/or
- Calculate effective memory address

Phase 4: Memory Access – 1 cycle

- Read from memory (LOAD), or
- Write to memory (STOR), or

- No operation (register-only instructions)

Phase 5: Write Back – 1 cycle

- Write result to destination register
- Update status register flags if applicable
- Update address registers for post-increment/pre-decrement

2.3 Architectural Components

The architecture is composed of several key functional units that work together to fetch, decode, execute, and complete instructions.

Figure 2.1 illustrates the major functional units within the SIRC-1 CPU and their interconnections.

2.3.1 Control Unit (CU)

The Control Unit sits at the heart of the SIRC-1 CPU, coordinating all other functional units throughout the instruction execution phases. It decodes instructions and generates control signals that orchestrate data movement and operations across the CPU.

Key responsibilities include:

- Instruction fetch and decode
- Execution phase coordination
- Control signal generation for all functional units
- Condition code evaluation for conditional execution
- Fetching exception vectors on behalf of the exception coprocessor
- Dispatching coprocessor commands through the coprocessor select register (CSR, Section 2.3.2)

2.3.2 Coprocessors and the Coprocessor Select Register

Operations are carried out by coprocessors. The processing unit (coprocessor `0x0`) executes ordinary instructions using the ALU, AU and barrel shifter described below. Other coprocessors, such as the exception coprocessor (`0x1`), carry out specialized operations on request. Every coprocessor reaches the register file and the bus interface through the coprocessor interface, and only one is active at a time.

The coprocessor select register (CSR) holds the pending coprocessor command: it tells the CU which coprocessor runs next and what it should do. It is a 16-bit internal register: bits 15–12 hold the coprocessor ID, bits 11–8 the coprocessor operation, and bits 7–0 an operand or vector. A **`COPI`** or **`COPR`** instruction writes its command operand to the CSR during write-back. The CU consumes the CSR by dispatching the command to the selected coprocessor in a further six-phase slot, and that coprocessor stays active until the command completes. While nothing is pending the CSR is `0x0000`, which selects the processing unit, so the CPU carries on running the program. Chapter 16 describes the instructions and Section 6.10.2 the register's layout.

The exception coprocessor never overwrites a pending command. It keeps its own record of the exception it wants to service, and loads the CSR with a hardware interrupt, software exception or trace fault only when the CSR is empty and no coprocessor is active. A fault raised by the call or its command is the exception: it ends the command and is dispatched at once. A coprocessor call is therefore part of its instruction: its dispatch slot, and the command it starts, complete before any pending interrupt or instruction trace fault is taken, except that a traced **`EXCP`** takes its trace fault first (Section 6.3.3).

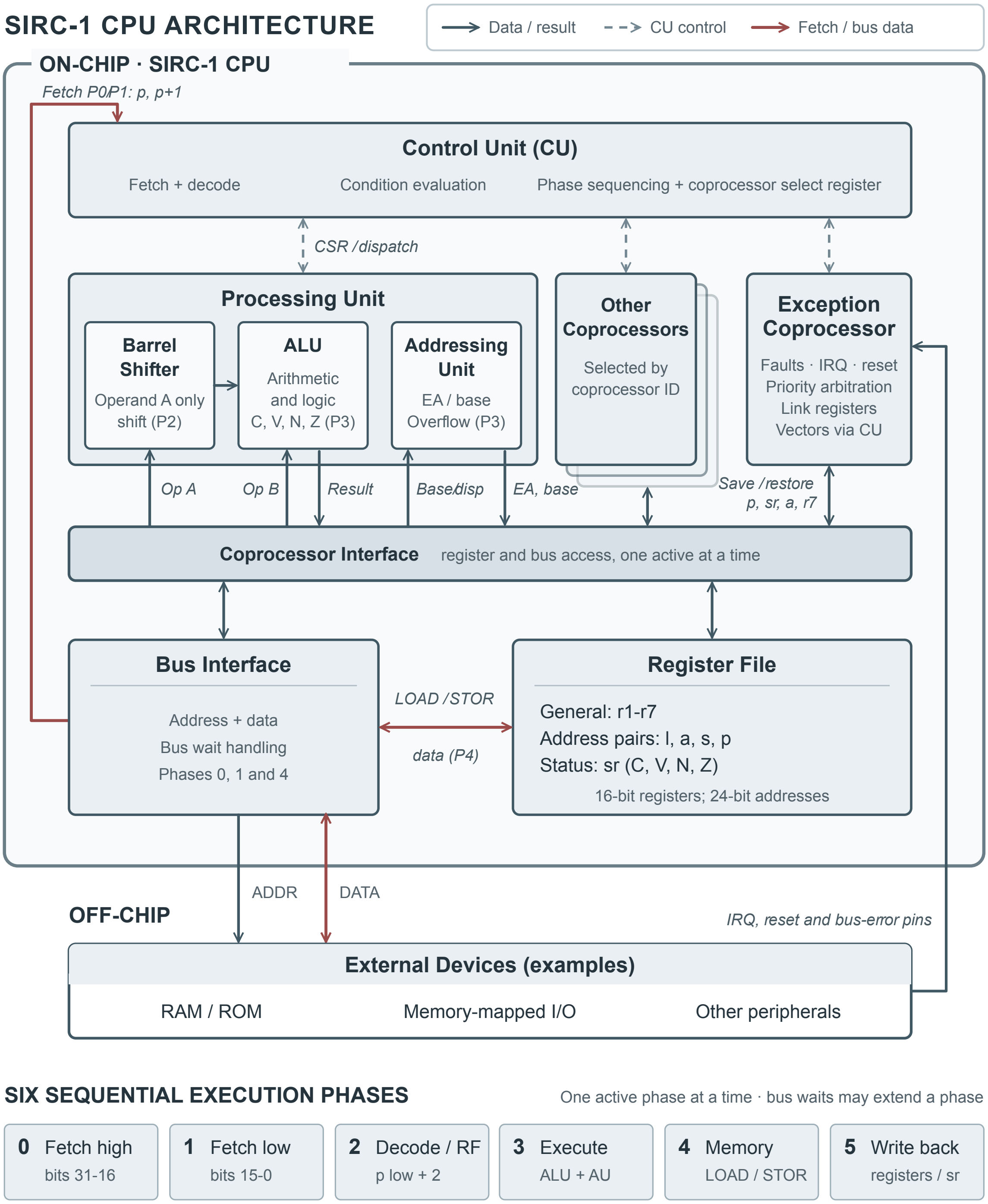

SIRC-1 CPU ARCHITECTURE
Data / result
CU control
Fetch / bus data
ON-CHIP · SIRC-1 CPU
Fetch P0/P1: p, p+1
Control Unit (CU)
Fetch + decode
Condition evaluation
Phase sequencing + coprocessor select register
CSR / dispatch
Processing Unit
Barrel Shifter
Operand A only shift (P2)
ALU
Arithmetic and logic C, V, N, Z (P3)
Addressing Unit
EA / base Overflow (P3)
Other Coprocessors
Selected by coprocessor ID
Exception Coprocessor
Faults · IRQ · reset
Priority arbitration
Link registers
Vectors via CU
Op A
Op B
Result
Base/disp
EA, base
Save / restore p, sr, a, r7
Coprocessor Interface
register and bus access, one active at a time
Bus Interface
Address + data
Bus wait handling
Phases 0, 1 and 4
LOAD / STOR
data (P4)
Register File
General: r1-r7
Address pairs: l, a, s, p
Status: sr (C, V, N, Z)
16-bit registers; 24-bit addresses
ADDR
DATA
OFF-CHIP
IRQ, reset and bus-error pins
External Devices (examples)
RAM / ROM
Memory-mapped I/O
Other peripherals
SIX SEQUENTIAL EXECUTION PHASES
One active phase at a time · bus waits may extend a phase
0 Fetch high bits 31-16
1 Fetch low bits 15-0
2 Decode / RF p low + 2
3 Execute ALU + AU
4 Memory LOAD / STOR
5 Write back registers / sr

Figure 2.1: SIRC-1 CPU Execution Phase Architecture

2.3.3 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations on register values. It supports the following operations:

- **Arithmetic:** Addition, Subtraction (with and without carry)
- **Logical:** AND, OR, XOR
- **Comparison:** Test and compare operations
- **Status Flag Generation:** Carry, Overflow, Negative, Zero

The ALU operates on data from the Register File (after optional shifting), along with immediate constants decoded from instructions and outputs results back to the Register File. It cannot operate on data stored in memory directly.

2.3.4 Addressing Unit (AU)

The Addressing Unit calculates memory addresses and manages address-related operations. It is responsible for:

- Computing effective addresses from base + displacement
- Computing the updated base address for pre-decrement and post-increment
- Detecting segment overflow in the address calculation, and when an instruction would use a program-counter advance that wraps the segment
- Presenting calculated addresses to the address bus

The AU works in parallel with the ALU during the execution phase, allowing address calculation to occur simultaneously with ALU operations.

2.3.5 Register File

The Register File contains all CPU registers organized into three categories:

- **General-Purpose Registers:** `r1–r7`
- **Address Register Pairs:** `l` (link), `a` (address), `s` (stack), `p` (program counter)
- **Status Register:** `sr`

The Register File allows the ALU, AU, and Control Unit to read and write registers as needed during each execution phase. All registers are 16 bits wide, though address register pairs can be combined to form 24-bit addresses.

2.3.6 Barrel Shifter

A barrel shifter sits between the Register File and the ALU, enabling efficient bit manipulation operations.

The barrel shifter supports logical shift left (**`LSL`**), logical shift right (**`LSR`**), arithmetic shift left (**`ASL`**), arithmetic shift right (**`ASR`**), rotate left (**`RTL`**), and rotate right (**`RTR`**). Shift amounts can be specified as immediate values or dynamically from register contents.

The shift is applied to the first source operand when fetching the source registers for an instruction. Shifts cannot be applied to the result or any other source operand.

2.3.7 Exception Coprocessor

The Exception Coprocessor (coprocessor ID `0x1`) manages faults, hardware interrupts, software exceptions and reset. When an exception is pending, it takes control of the processor and diverts execution to the handler through a vector. It is responsible for:

- Receiving faults, interrupt requests and reset
- Comparing each exception's priority with the current exception level
- Canceling an instruction that raises a retryable fault, so its remaining phases run as no-ops
- Holding the link registers (Chapter 6) that save the return address, status register and exception level
- Writing the exception to service into the coprocessor select register (CSR)

The Control Unit runs the resulting entry sequence through the normal execution phases. The exception instructions (**`EXCP`**, **`RETE`**, **`WAIT`**, **`RSET`**, **`ETFR`** and **`ETTR`**) reach the coprocessor as coprocessor calls. Chapter 6 describes exception handling in detail.

2.4 Design Characteristics

2.4.1 Hardwired Control Implementation

The SIRC-1 is designed to support hardwired control logic without requiring a microcode engine.

A six-phase state machine coordinates instruction execution, with combinational logic performing instruction decoding, arithmetic, address calculation and control signal generation. Clocked registers retain architectural and intermediate execution state between phases.

Combined with the regularity of the SIRCIS instruction set, this approach offers several potential advantages:

- A deterministic, fixed execution sequence
- Reduced control logic complexity and hardware requirements
- Predictable instruction behavior and execution timing

2.4.2 Fixed Execution Sequence

Ordinary instructions execute in exactly 6 phases, regardless of instruction type or operands, and complete in 6 clock cycles when no bus wait states are required. This design choice reduces the complexity of instruction sequencing and simplifies execution timing.

Once an instruction has started, the sequence is never cut short. If a retryable fault cancels an instruction, its remaining phases still run, but as no-ops that access no bus and commit no results, and the exception dispatch begins at the next phase 0. The alignment fault is the exception: it is detected at phase 0, before the instruction starts, and its dispatch begins at that same phase 0 (Section 6.2.4). Hardware interrupts, software exceptions and the instruction trace fault let the current instruction finish before they are dispatched.

Coprocessor calls are the exception: the processing-unit instruction takes six cycles and the coprocessor dispatch normally takes a further six. A call whose condition is false, or that names coprocessor `0x0`, has no dispatch slot and takes only six cycles. Some coprocessors, such as the DMA coprocessor, can additionally hold instruction fetch until the operation completes.

2.4.3 Parallel Functional Units

The ALU and AU operate in parallel during phase 3 (Execute and Address Calculation), allowing address calculation to proceed simultaneously with arithmetic operations. This parallelism enables efficient execution of memory reference instructions without additional clock cycles.

2.5 External Interface

The SIRC-1 CPU communicates with memory and peripherals over an external bus, and receives interrupt and reset requests from external hardware. How those signals are carried (the package, pinout, electrical characteristics and bus timing) is implementation-specific and is described in the *SIRC-1 Hardware Reference Manual*. When the reset request ($\overline{\mathrm{RSTI}}$) is raised, the CPU abandons the current instruction or bus wait, holds the reset output (Section 6.9.2), and then restarts the phase sequence at phase 0 with the reset-vector fetch (Section 6.9.3).

CHAPTER 3

Register Model

3.1 Overview

The SIRC-1 CPU provides sixteen 16-bit registers, classified into three categories:

- General-purpose registers (`r1`–`r7`)
- Address register pairs (`l`, `a`, `s`, `p`)
- Status register (`sr`)

All programmer-visible registers are 16 bits wide. Address registers can be used in pairs to form 24-bit addresses for memory operations.

Only `sr` (cleared to zero) and the `p` pair (loaded from the reset vector) have defined values after reset; all other registers are architecturally undefined. See Chapter 6.

3.2 Programming Model

Figure 3.1 shows the complete sixteen-register programming model: the seven general-purpose registers, both words of each of the four address register pairs, and `sr`. The high word of each address register pair, marked *, is read-only in protected mode (the restricted mode selected by the P bit of `sr`, Chapter 5): a protected-mode program can read its real value but a direct write faults. `sr`'s high byte, marked †, is stricter still: in protected mode it is hidden even from reads, which are redacted to zero, and writes to it are silently ignored rather than faulting. Its low byte, shown in italics, is the one truly unrestricted half of `sr` and remains fully accessible in both modes. See Section 3.6 below for the full read/write privilege rules.

<table>
<tr><th>15</th><th>0</th></tr>
<tr><td colspan="2">`r1`</td></tr>
<tr><td colspan="2">`r2`</td></tr>
<tr><td colspan="2">`r3`</td></tr>
<tr><td colspan="2">`r4`</td></tr>
<tr><td colspan="2">`r5`</td></tr>
<tr><td colspan="2">`r6`</td></tr>
<tr><td colspan="2">`r7`</td></tr>
<tr><td colspan="2">`ll` (link, low word)</td></tr>
<tr><td colspan="2">`lh` (link, high word)[*]</td></tr>
<tr><td colspan="2">`al` (address, low word)</td></tr>
<tr><td colspan="2">`ah` (address, high word)[*]</td></tr>
<tr><td colspan="2">`sl` (stack, low word)</td></tr>
<tr><td colspan="2">`sh` (stack, high word)[*]</td></tr>
<tr><td colspan="2">`pl` (program counter, low word)</td></tr>
<tr><td colspan="2">`ph` (program counter, high word)[*]</td></tr>
<tr><td>`sr` high byte[†]</td><td>*`sr` low byte*</td></tr>
</table>

Figure 3.1: Register Programming Model

[*] Read-only in protected mode: reads return the real value, but direct writes trigger a privilege violation fault (address-register-pair write-back is still allowed, provided it would leave this word unchanged). [†] Hidden in protected mode: reads are redacted to zero rather than the real value, and writes to it are silently ignored (no fault). The *italicized* `sr` low byte is the odd one out among these: despite sharing a row with a hidden half, it is unprivileged and fully accessible in both modes.

Table 3.1 below lists every register's encoding and states exactly which ones are privileged; see Section 3.6 for the full read/write privilege rules.

3.3 General-Purpose Registers

3.3.1 Register Set

The SIRC-1 provides seven general-purpose registers:

Register	**Description**
`r1`	General-purpose register 1
`r2`	General-purpose register 2
`r3`	General-purpose register 3
`r4`	General-purpose register 4
`r5`	General-purpose register 5
`r6`	General-purpose register 6
`r7`	General-purpose register 7

The programmer may use these registers for any purpose. They hold 16-bit values and may be used as source or destination operands for all ALU instructions.

3.3.2 Usage Conventions

The CPU does not enforce any register-usage convention among `r1`–`r7`; all seven registers are equally general purpose. Calling conventions and register-preservation rules are outside the scope of this document and are addressed in the forthcoming SIRC-1 ABI documentation. Toolchain and object-file topics are covered in the *SIRC-1 Assembly Programmer's Manual*.

3.4 Address Register Pairs

3.4.1 Overview

The SIRC-1 provides four address register pairs, each consisting of a high and low 16-bit register. When used together, these pairs form 24-bit addresses by combining the low 8 bits of the high register with the full 16 bits of the low register.

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

High register [15:8] (not used for addressing)	High register [7:0]	Low Register
bits 31–24	bits 23–16	bits 15–0

Figure 3.2: Address Register Pair Formation

The high 8 bits of the high register (bits 15–8, shown as pair bits 31–24 in the figure) are not used for addressing but remain available for storage, enabling techniques such as tagged pointers.

The exact address composition and stored-address layout are specified in Chapter 4.

Although each address register pair has a specific name and a suggested usage by convention, **`STOR`** and **`LOAD`** instructions treat all four pairs identically, and any pair may be used for indirect addressing.

Loading a value directly into the `p` pair performs a jump. This includes an ALU or **`LOAD`** instruction that names `pl` as its destination: its write-back replaces the advanced `pl`, so execution continues at the written address within the current segment. Such an instruction does not use the advanced address, so it is exempt from the program-counter wrap check in the same way as a taken branch (Section 6.2.5). An odd value raises an alignment fault at the next instruction fetch. A write to `ph` alone (supervisor mode only) changes the segment but not `pl`, so it still uses the advanced address and is still checked.

The link register pair updates automatically when **`LDEL`**, **`LJSR`**, or **`BRSR`** executes. The low word of the program counter, `pl`, advances by 2 during the decode phase of every instruction; `ph` is never incremented.

3.4.2 Link Register (l)

Component	**Register**
High Register	`lh` (privileged)
Low Register	`ll`

The link register pair stores return addresses for subroutine calls. When a subroutine is called using **`LDEL`**, **`LJSR`**, or **`BRSR`**, the return address is automatically saved to this register pair.

Typical Use:

- Storing return addresses from subroutine calls
- Alternate storage for general memory addressing

3.4.3 Address Register (a)

Component	Register
High Register	`ah` (privileged)
Low Register	`al`

This general-purpose address register pair is used for pointer operations and memory access.

Typical Use:

- Base pointer for data structures
- Array indexing
- General memory addressing

3.4.4 Stack Pointer (s)

Component	Register
High Register	`sh` (privileged)
Low Register	`sl`

The stack pointer register pair points to the current top of the stack. Pre-decrement and post-increment addressing modes are particularly useful with this register.

Typical Use:

- Stack operations (push/pop)
- Local variable allocation
- Function parameter passing

3.4.5 Program Counter (p)

Component	Register
High Register	`ph` (privileged)
Low Register	`pl`

The program counter register pair points to the instruction currently being fetched. It is not modified during either fetch phase; the low word `pl` advances by 2 during the Decode and Register Fetch phase, and `ph` is never incremented, so an overflow stays inside the segment.

Typical Use:

- Automatically updated during normal execution
- Modified by branch and jump instructions
- Used for position-independent code

3.5 Status Register

The status register (`sr`) is a special 16-bit register that contains CPU status flags and control bits. It is divided into two bytes:

- **Low Byte (bits 7–0):** Condition flags (bits 3–0) and reserved bits (bits 7–4), readable and writable in both modes; optionally updated as a side effect of ALU instructions in both modes.

- **High Byte (bits 15–8):** Control and privileged state

In protected mode, reads of `sr` are redacted to the low byte (high byte reads as zero), and direct writes to `sr` update only the low byte: bits 15–8 are left unchanged, silently and without a fault. In supervisor mode, a direct write to `sr` updates all 16 bits, except that it cannot change the Exception Active (EA) bit.

The status register is discussed in detail in Chapter 5.

3.6 Privilege Restrictions

3.6.1 High Address Register Restrictions

The high component of each address register pair (`lh`, `ah`, `sh`, `ph`) is privileged:

- In **supervisor mode**: Reading and writing are both allowed.
- In **protected mode**:
  - Reading is allowed.
  - Direct writes trigger a privilege violation fault.
  - Address-register-pair write-back is allowed only when the high word would remain unchanged; otherwise it triggers a privilege violation fault. The comparison covers all 16 bits, including tag bits 15–8.
- A privilege check is made only when the instruction's condition is true; a skipped instruction never faults on privilege (Chapter 10).

This restriction provides a form of memory segmentation and protection. Programs running in protected mode cannot modify the segment portion of addresses, preventing them from accessing memory outside their designated segment. Instructions such as **LDEA**, **LDEL**, **BRAN**, **BRSR**, **LJMP**, **LJSR**, and **RETS** remain usable in protected mode provided every high word they write is bit-for-bit equal to its current value, tag bits included. This applies to the destination pair's high word and, for the linking forms, to `lh`, which receives `ph`. Otherwise the CPU raises a privilege violation fault.

The status register differs slightly from the high address registers:

- High address registers (`lh`, `ah`, `sh`, `ph`) remain readable in protected mode, but direct writes trigger a privilege violation fault.
- The status register's privileged byte is hidden on reads in protected mode, and direct writes in protected mode silently leave it unchanged instead of faulting.

3.6.2 Summary of Privilege Checks

Each privilege check is made at a fixed phase of the instruction, and each violation raises the same retryable Privilege Violation Fault (vector `0x05`). Figure 6.2, in Section 6.2.7, shows where every check is made, together with the external bus protection check that the memory system can apply to any bus access.

The following are not violations in protected mode:

- Writing `sr`: only bits 7–0 are written, and bits 15–8 are ignored.
- Reading `sr`: bits 15–8 read as zero.
- Compare, test, and store instructions, which only read the register named by their register field.
- Reading any register, including `ah`, `lh`, `ph`, and `sh`.

3.7 Register Addressing

Registers are addressed using 4-bit register identifiers in instruction encoding:

ID	Register	Privileged
`0x0`	`r1`	No
`0x1`	`r2`	No
`0x2`	`r3`	No
`0x3`	`r4`	No
`0x4`	`r5`	No
`0x5`	`r6`	No
`0x6`	`r7`	No
`0x7`	`sr`	Reads: high byte redacted. Writes: high byte ignored
`0x8`	`lh`	Yes
`0x9`	`ll`	No
`0xA`	`ah`	Yes
`0xB`	`al`	No
`0xC`	`sh`	Yes
`0xD`	`sl`	No
`0xE`	`ph`	Yes
`0xF`	`pl`	No

Table 3.1: Register Encoding

3.8 Usage Examples

```
; Move immediate value into r1
MOVI r1, #100

; Add 2 to the value stored in r1 and store the result into r2
MOVR r2, r1
ADDI r2, #2

; Add r1 and r2, store in r3
ADDR r3, r1, r2
```

Example 3-1: Basic Register Operations

```
; Load value from memory at address in 'a' register pair
LOAD r1, (a)

; Store r1 to memory at address (a + 8)
STOR (#8, a), r1

; Push r1 onto stack (pre-decrement)
STOR -(s), r1

; Pop stack into r2 (post-increment)
LOAD r2, (s)+
```

Example 3-2: Address Register Usage

```
; Call subroutine at address in 'a' register
; Return address saved to 'l' register automatically
LJSR a

; ...subroutine code...

; Return from subroutine
; Copies 'l' register to 'p' register
RETS
```

Example 3-3: Subroutine Call

CHAPTER 4

Data Representation and Memory Model

4.1 Overview

This chapter defines the representation rules used by registers, memory, program images, exception vectors, and multi-word values. These rules are architectural: assemblers, loaders, simulators, hardware implementations, and debuggers must agree on them.

4.2 Word and Byte Ordering

The SIRC-1 is a word-addressed processor. Each architectural memory address names one 16-bit word. Byte addressing is not provided by the CPU instruction set.

When a word is represented outside the CPU as bytes, such as in a ROM image or debugger dump, the word is stored in big-endian byte order: the most significant byte appears first.

Value	**First byte**	**Second byte**
`0x1234`	`0x12`	`0x34`
`0xFF80`	`0xFF`	`0x80`

Table 4.1: External Byte Order for 16-bit Words

Multi-word quantities are stored with the most significant word at the lowest word address. This applies to instructions, exception vectors, saved addresses, and any software-defined multi-word integer convention that chooses to use the architectural order.

4.3 Address Values

Architectural addresses are 24-bit word addresses. The four address-register pairs (`l`, `a`, `s`, and `p`) represent an address by combining the low byte of the high register with the full low register:

$$\text{Address} = ((\text{HighRegister} \,\&\, \texttt{0x00FF}) \ll 16) \mid \text{LowRegister} \tag{4.1}$$

Bits 15–8 of the high register are not driven on the address bus. They are ignored by address calculation and are carried unchanged when an address pair is copied, loaded or stored. They are preserved unless a direct write to the high register of an address register pair overwrites them. They may be used to store metadata with addresses, which allows techniques such as tagged pointers. In protected mode, however, an address-register-pair write-back that would change any of the 16 bits of a high register raises a privilege-violation fault (Section 3.6), so a protected-mode program can only load pointers whose tag bits match the high register they replace.

Pair value	High register	Low register	Address bus value
`ah:al`	`0x0012`	`0x3456`	`0x123456`
`ah:al`	`0xAB12`	`0x3456`	`0x123456`
`ph:pl`	`0x0000`	`0x0100`	`0x000100`

Table 4.2: Address Register Pair Interpretation

When an address pair is stored in memory as a 32-bit quantity, the high word is stored first and the low word second.

4.4 Instruction Storage

Every instruction is 32 bits, stored as two consecutive 16-bit words:

- Word at `p`: instruction bits 31–16
- Word at `p+1`: instruction bits 15–0

Instruction fetch reads the high word from the address in `p` and the low word from that address plus one. `p` itself is not modified during the fetch; it advances by two words during instruction decode, so an instruction that reads or uses `p` as an address register sees the address of the instruction itself. Instruction addresses must be even word addresses. Fetching from an odd word address raises an alignment fault.

4.5 Exception Vector Storage

Each exception vector table entry is a stored 32-bit pointer that contains a 24-bit address and eight bits of metadata as described in Section 4.3.

The vector table begins at word address `0x000000`, so vector entry `V` starts at word address `V * 2`:

- Word at `V * 2`: target address high word
- Word at `V * 2 + 1`: target address low word

For example, vector `0x06` occupies word addresses `0x00000C` and `0x00000D`. To jump to `0x123456`, those two words must contain `0x0012` and `0x3456`.

4.6 Immediate Values

Immediate fields are encoded as raw two's-complement bit patterns. The consuming instruction defines whether the field is interpreted as signed, unsigned, or bitwise data.

Use	Width	Interpretation
ALU arithmetic immediate	16 bits	Two's-complement operand; result is truncated to 16 bits
ALU arithmetic short immediate	8 bits	Zero-extended operand in the range `0x00`–`0xFF`
ALU logical immediates	16 or 8 bits	Bit mask; no sign interpretation is applied by the logical operation
Memory displacements	16 bits	Two's-complement displacement added to the low address word; see Section 4.8 for the overflow trap
Coprocessor operands	16 bits	Coprocessor-defined command or parameter bits

Table 4.3: Immediate Field Interpretation

Short immediates occupy an 8-bit field and are zero-extended before use by ALU arithmetic. They are therefore suitable for compact non-negative constants and bit masks, not for encoding negative arithmetic operands.

Short immediates are not used for memory, effective-address, branch, jump, or subroutine-call displacements. Those forms use 16-bit signed immediate displacements (including assembler-resolved PC-relative branch offsets) or register displacements.

4.7 Software Layout Conventions

The architectural storage order for multi-word quantities is high word first. The *SIRC-1 Architecture Reference Manual* does not otherwise define a complete ABI, stack-frame layout, argument-passing convention, or software integer library format. Where examples show structures, arrays, saved registers, or stack use, offsets are word offsets and each push or pop moves the stack pointer by one 16-bit word.

Software standards may build on the architectural high-word-first rule for wider integers and stored addresses, but those conventions are non-normative unless a separate ABI or toolchain document says otherwise.

4.8 Arithmetic and Address Wraparound

General-purpose register arithmetic is performed modulo 2^{16}. Arithmetic condition flags describe the truncated 16-bit result: C records unsigned carry or borrow, and V records signed overflow.

Address calculation for memory, effective-address, branch, jump, and auto-update operations adds displacements to the low word of the selected address register pair. The high address-register word is not changed by ordinary displacement, pre-decrement, or post-increment calculation. If the low word overflows or underflows and `SR.A` (Trap on Address Overflow)[1] is set, the instruction raises a segment-overflow fault. If the bit is clear, the low word wraps modulo 2^{16}. Displacements are signed two's-complement, and the trap fires only when the low-word calculation wraps, whether the displacement is positive or negative. For pre-decrement and post-increment forms, the address-register auto-update and the effective-address calculation are checked separately: either one wrapping raises the fault, even if the other brings the result back into range.

The program counter follows the same low-word wrap rule during normal instruction fetch. Since instructions are two words long and must begin at even word addresses, a fetch that would begin

---

[1] `SR.`X denotes bit or field X of the status register; see Chapter 5 for the bit layout and Chapter 11 for the full notation convention.

from an odd `pl` value raises an alignment fault. If `SR.A` is set and advancing the program counter past an instruction would wrap the low word, that instruction raises a segment-overflow fault before it commits, unless it is a taken branch or jump that does not use the advanced address, or an ALU or **`LOAD`** instruction that writes `pl` (Section 6.2.5).

4.9 Memory Ordering

The SIRC-1 has no architectural cache, store buffer, memory reordering, or speculative memory access. Bus reads and writes issued by the CPU are externally visible in program order. A bus device may delay completion by withholding bus acknowledgement, but this stalls the current execution phase rather than allowing later bus accesses to pass it.

CHAPTER 5

Status Register

5.1 Overview

The Status Register (`sr`) is a 16-bit special-purpose register that contains condition flags and control bits for the CPU. It is divided into two bytes with different privilege levels:

- **Low Byte (bits 7–0):** Condition flags (bits 3–0) and reserved bits (bits 7–4) – readable and directly writable in both modes, and optionally updated as a side effect of ALU instructions in both modes
- **High Byte (bits 15–8):** Control and mode flags – accessible only in supervisor mode; in protected mode it reads as zero and direct writes to it are silently ignored, with no fault

5.2 Status Register Layout

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
T	A	EA	E4	E3	E2	E1	P					V	C	N	Z
Privileged								Non-Privileged							

Figure 5.1: Status Register Bit Layout

5.3 Condition Flags (Non-Privileged)

These flags reflect the results of ALU operations and comparisons. They can be read at any privilege level and are updated by ordinary instruction execution at any privilege level. A direct write to `sr` can also set them at any privilege level (see Section 5.5.2).

5.3.1 Zero Flag (Z) – Bit 0

Set when: The result of an operation is zero

Cleared when: The result of an operation is non-zero

Tested by: Equal (`==`), Not Equal (`!=`), and the compound conditions `HI`, `LO`, », `<=`

5.3.2 Negative Flag (N) – Bit 1

Set when: The result of an operation is negative (bit 15 = 1) when interpreted as a signed two's complement value

Cleared when: The result is positive or zero (bit 15 = 0)

Tested by: Negative Set (`NS`), Negative Clear (`NC`), and signed comparison condition codes

5.3.3 Carry Flag (C) – Bit 2

Set when: An addition produces a carry out of bit 15, or a subtraction requires a borrow

Cleared when: No carry/borrow occurs

Tested by: Carry Set (`CS`), Carry Clear (`CC`), Unsigned Higher (`HI`), Unsigned Lower or Same (`LO`) condition codes

Note: For unsigned comparisons, the carry flag acts as an unsigned overflow/borrow indicator

5.3.4 Overflow Flag (V) – Bit 3

Set when: A signed arithmetic operation produces a result that cannot be represented in 16 bits (signed overflow)

Cleared when: No signed overflow occurs

Tested by: Overflow Set (`OS`), Overflow Clear (`OC`), and signed comparison condition codes

Note: Overflow occurs when adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result. For subtraction (a - b), overflow occurs when a and b have different signs and the sign of the result differs from the sign of a. Shift operations define V separately (Chapter 9).

5.3.5 Reserved Flags – Bits 4–7

Bits 4 through 7 of the lower byte are reserved for future use; software must not rely on them. They can be read and written, but their value after any instruction that updates the condition flags is architecturally undefined.

Consequently, software should test specific bits with instructions such as **`TSAR`**/**`TSAI`**, rather than compare the whole status register word with an instruction such as **`CMPR`**/**`CMPI`**.

5.4 Control Flags (Privileged)

These flags control CPU behavior and can only be modified directly in supervisor mode. In protected mode, a direct write to `sr` leaves these bits unchanged: the write is silently ignored for bits 15–8 and does not raise a fault.

5.4.1 Protected Mode (P) – Bit 8

When Clear (0): CPU operates in supervisor mode with full privileges

When Set (1): CPU operates in protected mode with restricted access to:

- High address-register components (`lh`, `ah`, `sh`, `ph`)
- High byte of status register (reads as zero; writes are silently ignored)
- Privileged coprocessor operations (command-word operation nibble `0x8`–`0xF`, bits 11–8)
- Software exceptions targeting a vector below `0x60` (see Chapter 6)

Note: This bit provides basic memory protection by preventing protected-mode code from changing memory segments. Performing a restricted operation raises a privilege violation fault, with the exception of writes to the high byte of the status register, which are silently ignored. Figure 6.2 shows where each check is made.

5.4.2 Hardware Interrupt Enable Bits (E1–E4) – Bits 9–12

These four bits individually control whether each of the four maskable hardware interrupt lines is enabled:

- **E1 (Bit 9):** Enable Hardware Interrupt Line 1 (exception level 2, lowest maskable priority)
- **E2 (Bit 10):** Enable Hardware Interrupt Line 2 (exception level 3)
- **E3 (Bit 11):** Enable Hardware Interrupt Line 3 (exception level 4)
- **E4 (Bit 12):** Enable Hardware Interrupt Line 4 (exception level 5, highest maskable priority)

Hardware interrupt line n is dispatched at exception level n+1 (Chapter 6).

Note: Hardware Interrupt Line 5 (exception level 6) is a Non-Maskable Interrupt (NMI) and is always enabled. It cannot be disabled via the status register.

When a bit is set to 1, the corresponding hardware interrupt is enabled and can trigger an exception when requested. When cleared to 0, interrupts on that line are disabled and are not serviced.

Important behavior:

- Disabled interrupts are not queued while the enable bit is 0.
- A change to an enable bit takes effect at the next instruction boundary: setting it lets an already-active request in straight after the instruction that set it, and clearing it does not discard a request that was already latched (Section 6.3.2).
- Hardware interrupt lines 1–4 are level-sensitive, not edge-triggered. If an interrupt request remains active, it is serviced again as soon as the current handler is finished.
- Hardware interrupts can only interrupt execution if their exception level is higher than the currently executing exception level
- This prevents the same interrupt from re-triggering while its handler is executing
- A device requests service by holding its line until its handler services it. A lower-priority request held during a higher-priority handler is serviced as soon as the exception level drops to n or lower, where n is the line number. The line is not latched while its own handler or a higher-priority one is running, so a request withdrawn before then is not seen (Section 6.3.2)

These bits are typically managed by the operating system's interrupt handling code. At reset, all interrupt enable bits are cleared (interrupts disabled).

Interrupt handling is discussed in detail in Chapter 6.

5.4.3 Exception Active (EA) – Bit 13

When Set (1): The CPU is currently executing an exception handler (`current_exception_level != 0`)

When Clear (0): The CPU is executing normal code (`current_exception_level == 0`)

Automatically Set: When any exception is entered (software, hardware, or fault)

Set by RETE: **`RETE`** sets it if the restored exception level is non-zero and clears it when returning to exception level 0

Purpose: Allows software to detect whether it is running within an exception handler context

Typical Use: OS code can check this bit to determine appropriate behavior, for example avoiding re-enabling interrupts during nested exception handling

Note: This bit reflects the internal `current_exception_level` state. Programs can read this bit but cannot directly modify it – it is managed automatically by the exception handling hardware. This includes **`RETE`**, which sets `EA` from the restored exception level rather than from the saved status register, so editing bit 13 of a saved status register with **`ETTR`** has no effect.

5.4.4 Trap on Address Overflow (A) – Bit 14

When Set (1): An effective-address calculation, address-register auto-update, or program-counter advance that wraps the low word raises a segment overflow fault (vector `0x03`) before the instruction commits (Section 6.2.5)

When Clear (0): Address overflows wrap around without any side effects

Purpose: When set, helps detect pointer errors during development, or lets software increment or decrement the current segment if a data address goes out of bounds of the segment

Typical Use: Enabled during debugging, disabled in production for performance

5.4.5 Trace Mode (T) – Bit 15

When Set (1): An instruction trace fault (vector `0x06`) is generated after every instruction, except at exception level 7

When Clear (0): Normal execution

Purpose: Allows debuggers to single-step through code

Note: Used by SIRCIT (SIRC Instruction Tracer) for debugging

5.5 Status Register Updates

5.5.1 Automatic Updates

The condition flags (Z, N, C, V) are automatically updated by most ALU instructions, unless explicitly disabled:

- **Arithmetic Instructions** (**`ADDI`**, **`SUBI`**, **`ADCI`**, **`SBCI`**, **`ADDR`**, **`SUBR`**, **`ADCR`**, **`SBCR`**): Update all four condition flags
- **Logical Instructions** (**`ANDI`**, **`ORRI`**, **`XORI`**, **`ANDR`**, **`ORRR`**, **`XORR`**): Update Z and N flags; C and V flags are cleared
- **Compare Instructions** (**`CMNI`**, **`CMNR`**, **`CNCI`**, **`CNCR`**, **`CMPI`**, **`CMPR`**, **`CPCI`**, **`CPCR`**): Same as Arithmetic Instructions but does not update destination
- **Test Instructions** (**`TSAI`**, **`TSAR`**, **`TSOI`**, **`TSOR`**, **`TSXI`**, **`TSXR`**): Same as Logical Instructions but does not update destination
- **Move Instructions** (**`MOVI`**, **`MOVR`**): Same as Logical Instructions. They do not accept `[S]` (the shifter-result status suffix; `[N]` and `[A]` select no update and the ALU result, Chapter), since they have no shifted operand; an encoding of the opcode with the status-update field (AF, Chapter 7) set to `10`, not reachable from public assembly syntax, leaves all four flags architecturally undefined (`U U U U`).
- **Memory and Address Instructions** (**`LOAD`**, **`STOR`**, **`LDEA`**, **`LDEL`** and their aliases): Never update the condition flags as a side effect; AF selects the address register for these opcodes.
- **ALU Instructions with a Shift Operand**: The flags come from the ALU result by default. They come from the shift instead when the `[S]` status override modifier is used, or with the **`SHFT`** meta-instruction.

5.5.2 Manual Updates

In supervisor mode, the status register can be directly manipulated like any other register:

```
; Set protected mode bit (only in supervisor mode)
ORRI sr, #0x0100

; Enable all maskable hardware interrupts (set bits 9-12)
; Note: Hardware interrupt line 5 is an NMI and is always enabled
ORRI sr, #0x1E00

; Disable all maskable hardware interrupts (clear bits 9-12)
ANDI sr, #0xE1FF

; Restore status register saved to stack
LOAD sr, (s)+
```

An ALU instruction that targets `sr` writes its result to `sr` and then overwrites bits 7–0 with the flags produced by that same instruction, so only bits 15–8 can be set this way. To write the low byte, suppress the flag update with the `[N]` status source (for example **ORRI[N]** `sr`, `#0x0003`) or use **MOVR[N]** `sr`, `rN`. A direct write to `sr` also preserves the current value of bit 13 (`EA`); only exception entry and **RETE** change it, so the **LOAD** `sr`, (s)+ restore above does not reproduce the exception-active state saved at the time of the original write.

Important: When running in protected mode, a direct write to `sr` updates only the low byte (bits 7–0). Bits 15–8 – including the Protected Mode bit itself – are left unchanged, silently and without a fault, so a protected-mode program cannot leave protected mode or alter the interrupt-enable, trace, or address-overflow bits this way. Because bits 15–8 read as zero in protected mode, a read-modify-write of `sr` behaves consistently: the high byte is never disturbed. This applies to every instruction that writes `sr` as its destination. The flag-only comparison and test instructions (**CMNI**, **CMNR**, **CNCI**, **CNCR**, **CMPI**, **CMPR**, **CPCI**, **CPCR**, **TSAI**, **TSAR**, **TSOI**, **TSOR**, **TSXI**, **TSXR**) do not write their destination register, so naming `sr` there affects only the flags in bits 3–0.

5.6 Reading the Status Register

In supervisor mode, reading `sr` returns the full 16-bit value. In protected mode, reading `sr` returns the lower byte in bits 7–0, with bits 15–8 masked to zero (to hide supervisor-only state).

```
; In supervisor mode - gets full 16 bits
MOVR r1, sr ; r1 = 0xXXXX (all bits)

; In protected mode - high byte masked
MOVR r1, sr ; r1 = 0x00XX (high byte forced to 0)
```

5.7 Exception Handling Effects

When an exception occurs:

1. The current status register is saved
2. Protected mode bit (P) is cleared (entering supervisor mode)
3. Trace mode bit (T) is cleared (disabling single-stepping in exception handler)
4. Exception active bit (EA) is set
5. The hardware interrupt enable bits are not changed. A pending interrupt is serviced during a handler only if its priority level is higher than the current exception level

When returning from an exception with **`RETE`**, the saved status register is restored, except bit 13 (`EA`), which is set from the restored exception level.

5.8 Usage Examples

The SIRC-1 provides syntax to manually control how status flags are updated during ALU instructions, including those with a shift operand. This is particularly useful for advanced control flow and flag preservation. See Section 9.12 in Chapter 9 for complete details and worked examples.

Condition-code-driven branching and conditional execution (comparing registers, testing bits, and acting on the result) are a property of the condition flags this chapter defines, but the idioms themselves belong to Chapter 10 and the individual instruction entries (for example, **`TSAI`**/**`TSAR`** in Chapter 13), which cover them in more depth than a status-register-focused example could.

```
; Entering protected mode (from supervisor mode)
ORRI sr, #0x0100 ; Set protected mode bit
; Now running in protected mode

; Attempting to write privileged register will fault
; ADDI ah, #0x05 ; This would cause exception!
```

Example 5-1: Enabling Protected Mode

```
; Save status register (in protected mode only bits 7-0 are saved and restored)
MOVR r7, sr

; Do some operations that change flags
ADDR r1, r2, r3
CMPI r1, #100

; Restore original status ([N] stops the flag update overwriting the low byte)
MOVR[N] sr, r7
```

Example 5-2: Saving and Restoring Status

As with the stack restore in Section 5.5.2, this restore does not reproduce bit 13 (`EA`); only exception entry and **`RETE`** change it.

CHAPTER 6

Exception and Interrupt Handling

6.1 Overview

An exception is an event that causes the program flow to be diverted to an address stored in a predefined vector.

Some are directly caused by an instruction, some are a side effect of an instruction, and some are asynchronously caused by external hardware.

They are managed by the exception coprocessor, which takes control of the processor when an exception is pending. Figure 6.1 shows how the pieces fit together; each is described in the sections that follow.

6.1.1 Exception Levels and Priority

Exceptions are prioritized as follows (highest to lowest):

1. Priority 7: Faults
2. Priority 6: Level 5 (L5) hardware exceptions (NMI-like)
3. Priority 5: Level 4 hardware exceptions
4. Priority 4: Level 3 hardware exceptions
5. Priority 3: Level 2 hardware exceptions
6. Priority 2: Level 1 hardware exceptions
7. Priority 1: Software exceptions (traps)

Faults have priority over all hardware interrupts. If a fault is pending at the same instruction boundary as one or more hardware interrupts, the fault is dispatched first and the hardware interrupts remain pending. This includes instruction trace faults: when trace mode raises a post-instruction fault, that trace fault is serviced before pending hardware interrupts at the next boundary.

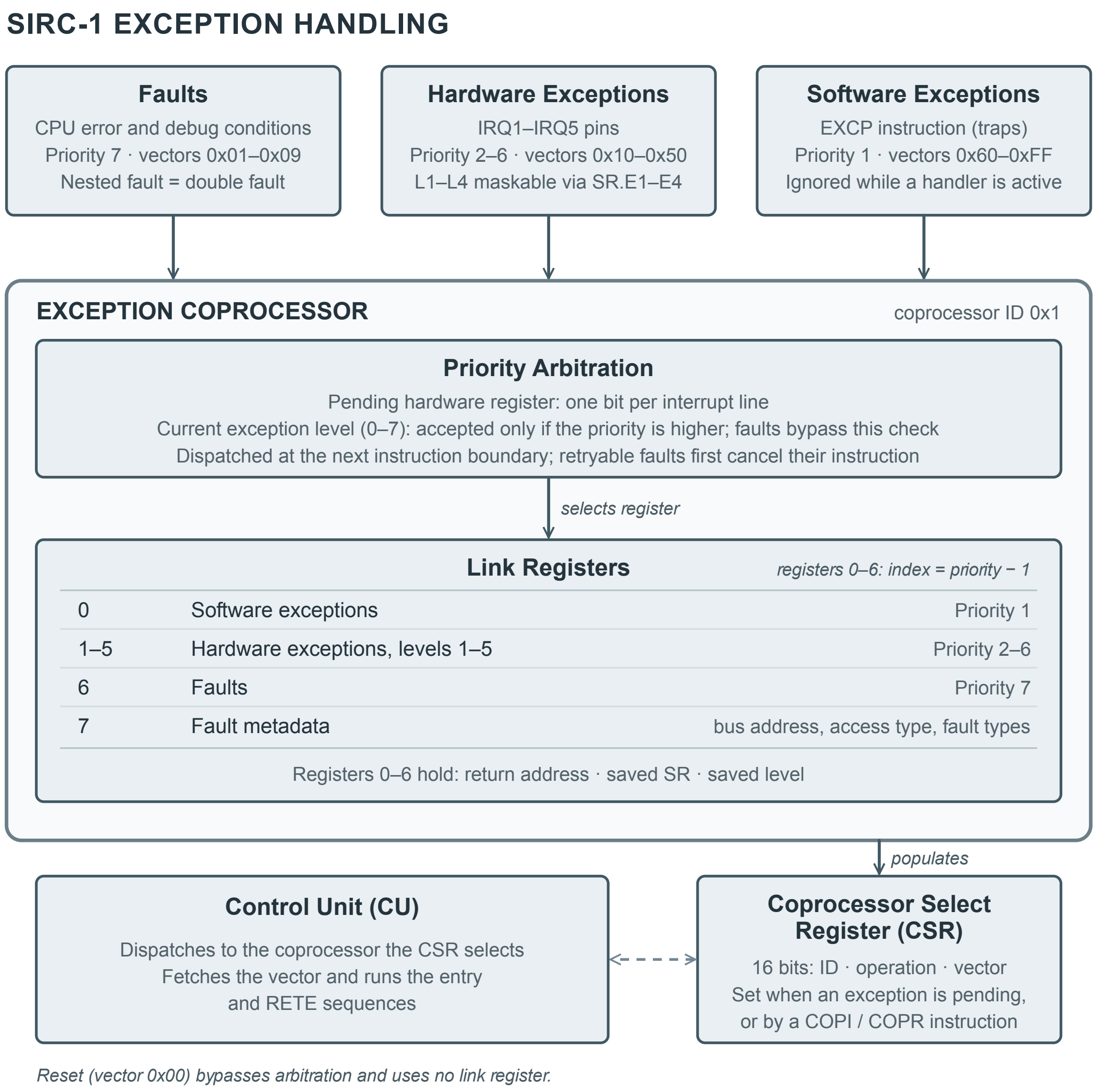
Hierarchical Exception Handling:

Exceptions in SIRC-1 are strictly hierarchical. A hardware interrupt can only interrupt the current execution if its priority level (exception level) is *higher* than the currently active exception level. This prevents:

- An interrupt handler from being interrupted by its own interrupt (same level)
- Lower priority interrupts from interrupting higher priority handlers

Software exceptions follow a separate rule. An **`EXCP`** executed inside any exception or fault handler is discarded rather than deferred, so a trap can never overwrite link register 0 (Section 6.4).

The exception unit maintains a `current_exception_level` field that tracks the active exception level (0–7). When not handling any exception, this is 0. When handling an exception at level N, only exceptions at level > N can interrupt. Two cases fall outside this rule. Faults are dispatched at any level: a fault raised inside a fault handler becomes a double fault. A Level 5 request that arrives while a Level 5 handler is running (level 6) is not deferred but raises a Level Five Hardware Exception Conflict fault (Section 6.2.9).



Faults

CPU error and debug conditions
Priority 7 · vectors 0x01–0x09
Nested fault = double fault

Hardware Exceptions

IRQ1–IRQ5 pins
Priority 2–6 · vectors 0x10–0x50
L1–L4 maskable via SR.E1–E4

Software Exceptions

EXCP instruction (traps)
Priority 1 · vectors 0x60–0xFF
Ignored while a handler is active

EXCEPTION COPROCESSOR coprocessor ID 0x1

Priority Arbitration

Pending hardware register: one bit per interrupt line
Current exception level (0–7): accepted only if the priority is higher; faults bypass this check
Dispatched at the next instruction boundary; retryable faults first cancel their instruction

Link Registers *registers 0–6: index = priority − 1*

0	Software exceptions	Priority 1
1–5	Hardware exceptions, levels 1–5	Priority 2–6
6	Faults	Priority 7
7	Fault metadata	bus address, access type, fault types

Registers 0–6 hold: return address · saved SR · saved level

Control Unit (CU)

Dispatches to the coprocessor the CSR selects
Fetches the vector and runs the entry
and RETE sequences

Coprocessor Select Register (CSR)

16 bits: ID · operation · vector
Set when an exception is pending,
or by a COPI / COPR instruction

Reset (vector 0x00) bypasses arbitration and uses no link register.

Figure 6.1: SIRC-1 Exception Handling Overview

6.1.2 Vector Table

The exception vectors are stored starting at address `0x0`. Each vector is a 32-bit pointer (a 24-bit target address and eight bits of metadata) stored as two consecutive 16-bit words in the address format described in Section 4.5. The high word is stored first at `vector * 2`, followed by the low word at `vector * 2 + 1`. The vector table has 256 entries. Vectors `0x00`–`0x5F` (96 entries) are reserved for the CPU; vectors `0x60`–`0xFF` (160 entries) are user trap vectors. Supervisor code can still name a CPU vector in **`EXCP`**; it is entered as an ordinary trap (see **`EXCP`** in Chapter 16). The vector table layout is:

- **`0x00`:** Reset vector
- **`0x01`:** Bus fault
- **`0x02`:** Alignment fault
- **`0x03`:** Segment overflow fault
- **`0x04`:** Invalid opcode fault
- **`0x05`:** Privilege violation fault
- **`0x06`:** Instruction trace fault
- **`0x07`:** Level five hardware exception conflict
- **`0x08`:** Double fault
- **`0x09`:** Bus protection fault
- **`0x0A`–`0x0F`:** Reserved for future privileged exceptions
- **`0x10`:** Level 5 hardware exception vector
- **`0x20`:** Level 4 hardware exception vector
- **`0x30`:** Level 3 hardware exception vector
- **`0x40`:** Level 2 hardware exception vector
- **`0x50`:** Level 1 hardware exception vector
- **`0x11`–`0x5F` (excluding `0x20`, `0x30`, `0x40`, `0x50`):** Reserved
- **`0x60`–`0xFF`:** Software exception vectors (160 user-accessible trap vectors)

6.1.3 Exception Summary

Table 6.1 gathers, for every exception source, the vector, the priority (the exception level its handler runs at, Section 6.1.1), the enable or masking rule, the link register used (Section 6.5.1), the return address saved, and whether the handler has to redirect the saved address with **`ETFR`** and **`ETTR`** before **`RETE`**. Retryable faults re-execute the faulting instruction on **`RETE`** (Section 6.2.1); a handler redirects only to skip it. The sources are described in the sections on faults (Section 6.2.1), hardware exceptions (Section 6.3.4) and software exceptions (Section 6.4).

Source	Vector	Priority	Maskability	Link Register	Return Address	Redirect Before **RETE**?
Reset	`0x00`	n/a	Not maskable	None	n/a	n/a: no **RETE**; execution restarts at the reset vector (Section 6.9.3)
Bus fault	`0x01`	7	Not maskable	6 (metadata in 7)	Faulting instruction	No: **RETE** retries; redirect only to skip it
Alignment fault	`0x02`	7	Not maskable	6 (metadata in 7)	Faulting instruction	No: **RETE** retries; redirect only to skip it
Segment overflow	`0x03`	7	SR.A gated	6 (metadata in 7)	Faulting instruction	No: **RETE** retries; redirect only to skip it
Invalid opcode	`0x04`	7	Not maskable	6 (metadata in 7)	Faulting instruction	Usually yes: a retry raises the fault again unless the opcode was emulated
Privilege violation	`0x05`	7	Not maskable	6 (metadata in 7)	Faulting instruction	No: **RETE** retries; redirect only to skip it
Instruction trace	`0x06`	7	SR.T or TRCE gated	6 (metadata in 7)	Next instruction	No: **RETE** continues past the traced instruction
L5 conflict	`0x07`	7	Not maskable	6 (metadata in 7)	Dispatch point (PC when detected)	No: not retryable; a lone **RETE** resumes the running code
Double fault, raised inside a fault handler	`0x08`	7	Not maskable	6 (metadata in 7)	PC when detected (for a retryable fault, the instruction that raised it)	Do not **RETE**: the CPU would stay at level 7; reset instead
Double fault, failed fault-vector fetch	`0x08`	7	Not maskable	6 (metadata in 7)	Original fault's return address	No: repair the vector, then **RETE** returns to the original fault's return address
Bus protection fault	`0x09`	7	Not maskable	6 (metadata in 7)	Faulting instruction	No: **RETE** retries; redirect only to skip it
Reserved faults	`0x0A–0x0F`	7	Reserved	Reserved	Reserved	Reserved
Level 5 hardware	`0x10`	6	Not maskable	5	Next instruction	No
Level 4 hardware	`0x20`	5	SR.E4	4	Next instruction	No
Level 3 hardware	`0x30`	4	SR.E3	3	Next instruction	No
Level 2 hardware	`0x40`	3	SR.E2	2	Next instruction	No
Level 1 hardware	`0x50`	2	SR.E1	1	Next instruction	No
Software trap	`0x60–0xFF`	1	Priority gated	0	Instruction after **EXCP**	No

Table 6.1: Exception Quick Reference

6.1.4 Instructions Used in This Chapter

This chapter refers to several exception-coprocessor instructions before their full reference entries appear. Briefly:

EXCP	Triggers a software exception (trap).
RETE	Returns from an exception handler, restoring the saved program counter, status register, and exception level.
WAIT	Stops instruction fetch until an interrupt that can be taken, or a reset, occurs.
RSET	Performs the reset sequence and jumps to the reset vector.
ETFR	Copies the saved return address and/or status register from a link register into `a` and/or `r7`.
ETTR	Copies `a` and/or `r7` into a link register's saved return address and/or status register.

EXCP, **RETE**, **WAIT**, **RSET**, **ETFR**, and **ETTR** are meta-instructions that assemble to **COPI** (coprocessor call) forms addressed to the exception coprocessor (coprocessor `0x1`); Section 6.10 in this chapter lists their opcodes, and Chapter 16 gives the full syntax, operation, flags, and legal forms for each.

6.2 Faults

Faults are priority 7 exceptions raised by the CPU itself in response to error conditions or debug events. They use vectors in the `0x01`–`0x0F` range. Unlike hardware and software exceptions, faults cannot be masked: once a fault condition is raised, the fault is dispatched at the next instruction boundary regardless of the current exception level. The one wait is for an instruction trace fault, which is taken after a coprocessor call's dispatch slot and command complete (Section 6.3.3). Two faults are conditional on whether they are raised at all: Segment Overflow requires `SR.A`, and Instruction Trace requires `SR.T` or the force-trace request ($\overline{\text{TRCE}}$) and is never raised at exception level 7 (Section 6.2.8).

6.2.1 Fault Categories

Not all faults behave the same way with respect to the faulting instruction and the **RETE** return address. There are three categories:

Retryable faults

The operation that triggered the fault does not take effect. For most faults this means the instruction takes no architectural effect (the exceptions to this are listed below). The link register stores the address of the faulting instruction. **RETE** returns to that instruction so it can be retried, typically after the fault handler has corrected the condition that caused the fault.

Examples of how a handler can correct the cause of a fault:

- Remapping or paging in a device for a bus fault
- Switching to supervisor mode for a bus protection fault or privilege violation
- Bringing an out-of-range address back into range for a segment overflow
- Emulating the missing operation for an invalid opcode fault

Every fault in this category follows the same pattern: a handler is always free to advance past the faulting instruction with **ETFR**/**ETTR** instead of retrying it. For Invalid Opcode Fault this is usually the right choice – simply retrying an unemulated invalid opcode raises the same fault again.

Two cases do not follow the pattern of the instruction being canceled before it commits:

- An alignment fault is raised at phase 0, before the instruction fetch reaches the bus, so there is no decoded instruction to cancel.
- A DMA burst (**DMAR**, **DMAW**, **DMAT**) performs several bus accesses per instruction; a fault partway through can leave some of those accesses already committed; see the individual fault descriptions below and Section 16.7.3.

The retryable faults are: *Alignment Fault; Bus Fault; Bus Protection Fault; Invalid Opcode Fault; Privilege Violation Fault; Segment Overflow Fault.*

Post-instruction faults

The instruction completes normally and its effects are committed before the fault fires. The link register stores the address of the *next* instruction. **RETE** advances to that instruction; the traced instruction is not re-executed.

The only post-instruction fault is the *Instruction Trace Fault*.

Exception dispatch faults

Raised during the exception handling machinery itself, not during normal instruction execution. The link register stores the program counter at the point the fault was detected, which may be inside an exception handler or, for an L5 overrun, in ordinary code. The one exception is a double fault caused by a failed fault-vector fetch, which keeps the original fault's return address (Section 6.2.10). For a double fault raised inside a fault handler by a retryable fault, the point the fault was detected is the instruction that raised it, so the link register stores that instruction's address, as the escalating fault would have done on its own. Recovery may require careful state management; these faults often indicate a hardware misconfiguration or a programming error in an exception handler.

The exception dispatch faults are: *Level Five Hardware Exception Conflict, Double Fault.*

6.2.2 Bus Fault

(Retryable) – Raised when the external memory system reports a bus cycle as failed by signaling a bus error ($\overline{\text{BERR}}$).

This happens, for example, if an unmapped address is presented by the CPU and another device detects this and raises an error. The instruction is canceled before it takes effect, and the link register stores the address of the *faulting* instruction, so a handler implementing virtual memory can map in the missing page and return with **`RETE`** to retry the same instruction.

Because ALU results and status-register updates are only committed during the write-back phase (Chapter 2), which runs after the memory-access phase where a bus fault is detected but does nothing for a canceled instruction, a bus fault can never leave a partial status-register or register update behind for an ordinary single-access instruction.

This atomicity does not extend to a DMA burst: **`DMAR`**, **`DMAW`**, and **`DMAT`** are each one instruction that performs multiple bus accesses in sequence, so a bus fault partway through a burst leaves the words already transferred in effect – retrying with **`RETE`** does not undo them. The fault metadata register's DMA read/write burst bus-access-type codes (Section 6.5.2) let a handler recognize this case. See Section 16.7.3 for what a handler can safely do about it.

6.2.3 Bus Protection Fault

(Retryable) – Raised when the external memory system reports a bus protection error ($\overline{\text{BPER}}$) in response to a bus cycle.

This indicates that the requested address and device are valid, but the operation is disallowed. Examples include writing to ROM or accessing a protected memory region while the CPU is in protected mode. The instruction is canceled before it takes effect, and the link register stores the address of the faulting instruction, so a handler can make the necessary corrections and retry with **`RETE`**.

As with Bus Fault, this applies cleanly to an ordinary single-access instruction; a DMA burst (**`DMAR`**, **`DMAW`**, **`DMAT`**) can raise this fault partway through, in which case words already transferred before the fault remain in effect. See Section 16.7.3 for what a handler can safely do about it.

6.2.4 Alignment Fault

(Retryable) – Raised when fetching instructions if the program counter contains an odd word address. The check is made at phase 0, before the fetch's bus cycle, so the odd address never reaches the bus. No instruction slot is used: the fault's dispatch slot begins at that same phase 0. The fault-metadata register records bus access type `0x1` (instruction fetch) and the odd address that would have been fetched.

This is to simplify fetching: the second word of an instruction is always at `pl | 0x1`, which ensures that instructions can never overflow a segment boundary. The saved return address is itself odd; a fault handler must correct the saved return address in link register 6 (for example, by clearing bit 0 using **`ETFR`** and **`ETTR`**) before returning with **`RETE`**, or the CPU raises the same fault again immediately.

6.2.5 Segment Overflow Fault

(Retryable) – Raised when a computed address would fall outside the current segment.

Only raised if `SR.A` (Trap on Address Overflow) is set, since some programs intentionally rely on address wrap-around. With `SR.A` set, there are two cases:

- An effective-address calculation wraps the low address word. The instruction is canceled before it takes effect, and the link register stores its address, so a handler can bring the address back into range and retry with **`RETE`**. The fault metadata register records the wrapped address and the bus access it was computed for: `0x2` (data read) or `0x3` (data write) for **`LOAD`** and **`STOR`**; `0x1` (instruction fetch) when the effective address is written to `p` (a branch or jump, including **`LDEA`**/**`LDEL`** with destination `p`); and `0x0` (none) when it is written to another address register, which makes no access with it. No bus cycle is performed in any of these cases. An address-register auto-update that wraps is recorded the same way, with the wrapped register value and the bus access type of the instruction's own access. If more than one calculation would wrap, only the first is recorded, in the order effective address, auto-update, program-counter advance.
- The instruction in the last slot of the segment would wrap the program counter's low word as it advances. The AU checks this in phase 3, before any memory access, for every instruction that uses the advanced address: as the next program counter (any instruction that does not branch, including a coprocessor call, and a branch whose condition is false), or as the return address saved in `l` (**`LDEL`**, **`BRSR`**, **`LJSR`**). A taken branch or jump that writes `p` without saving a return address does not use it and does not fault, and neither does an executed (condition true) ALU or **`LOAD`** instruction that writes `pl` (Section 3.4). The instruction is canceled and the link register stores its own address, as for any other retryable fault. No bus access is involved, so the fault metadata register records bus access type `0x0` (none). Retrying with **`RETE`** raises the same fault again, so a handler must emulate the instruction or redirect the saved return address with **`ETFR`**/**`ETTR`**. With `SR.A` set, the last instruction slot of a segment can therefore only usefully hold a taken branch or jump, or an executed (condition true) ALU or **`LOAD`** instruction that writes `pl`.

The same DMA-burst caveat as Bus Fault and Bus Protection Fault applies here: a segment-overflow fault raised partway through a **`DMAR`**, **`DMAW`**, or **`DMAT`** burst does not undo words already transferred before the fault. See Section 16.7.2 for where a DMA burst checks for overflow and Section 16.7.3 for what a handler can safely do about it.

6.2.6 Invalid Opcode Fault

(Retryable) – Raised when a coprocessor call targets a coprocessor ID not implemented by the selected CPU model, or an operation that the coprocessor it targets does not implement.

Can be used for forward compatibility: programs written for a future coprocessor (for example, a floating-point unit) will trap on older hardware, and the handler can emulate the operation in software. Because this fault is retryable, the link register stores the address of the faulting instruction; a handler that has emulated the operation must use **`ETFR`**/**`ETTR`** to redirect the return address to the instruction following the trapped call before returning with **`RETE`** – retrying the instruction unmodified simply raises the same fault again.

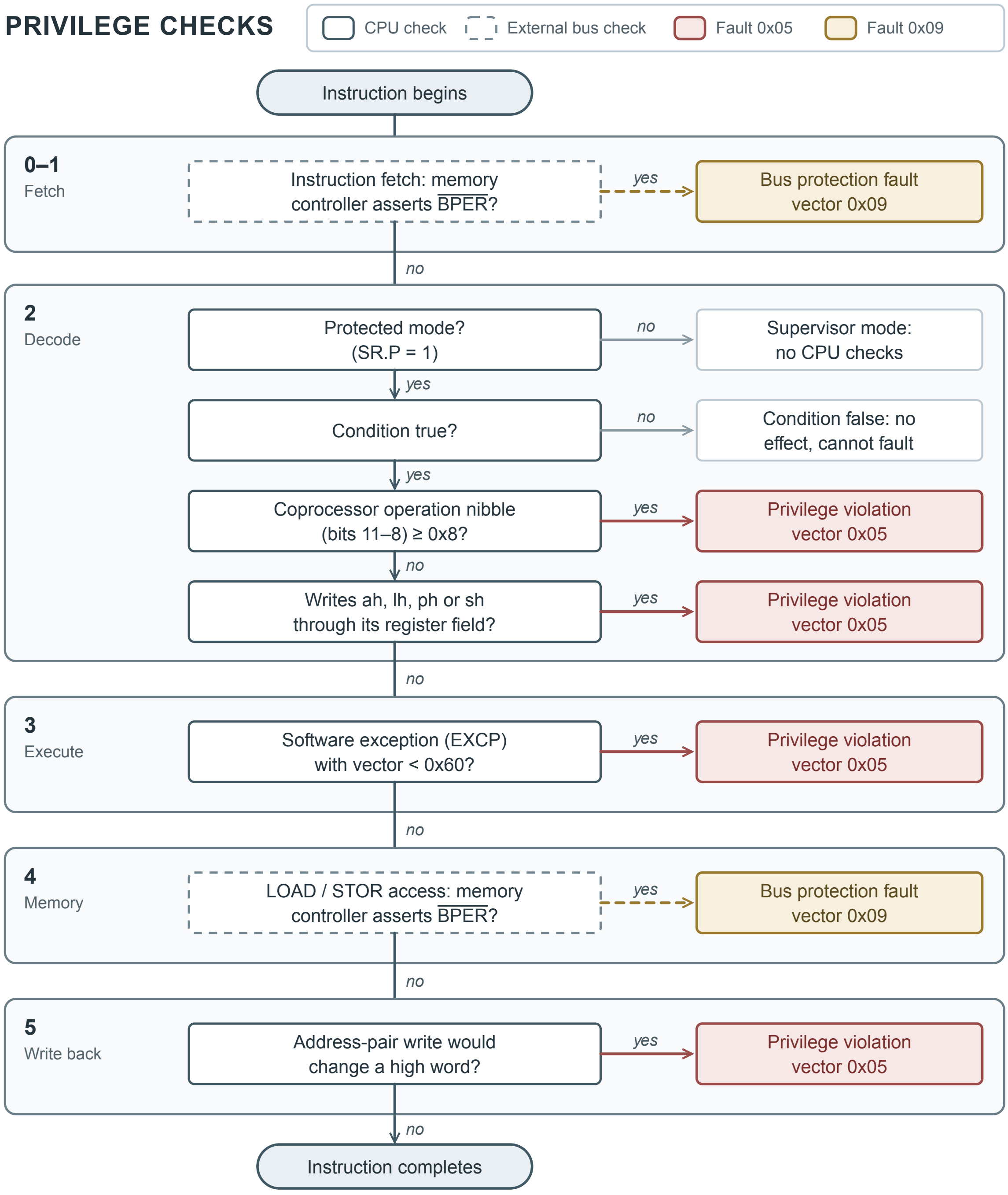
Note that this fault is specific to coprocessor instructions. Other instruction encodings outside the documented instruction set are reserved or architecturally undefined, and do not raise an invalid-opcode fault. They remain bound by the privilege rules: in protected mode they cannot change the privileged byte of `sr`, a high address-register word, or exception-unit state, and they may write only the registers named in their fields.

6.2.7 Privilege Violation Fault

(Retryable) – Raised when a privileged operation is attempted while not in supervisor mode:

1. Directly writing to the high word of any address register
2. Address-register-pair write-back that would change a high address-register word
3. Executing a supervisor-only coprocessor operation
4. Triggering a software exception with a vector below `0x60`

Figure 6.2 shows where each of these checks is made as an instruction moves through the execution phases: at phase 2 for a write to a privileged register field and for a supervisor-only coprocessor operation, at phase 3 for an **EXCP** vector below `0x60`, and at phase 5 for an address-register-pair write-back that would change a high address-register word. The checks apply to different kinds of instruction, so an instruction is only affected by the check for its own kind. The figure also shows the external check: on any bus access, in the instruction fetch (phases 0 and 1) or a **LOAD** or **STOR** (phase 4), the memory system can refuse the access by asserting $\overline{\text{BPER}}$, which raises a Bus Protection Fault (vector `0x09`, Section 6.2.3) instead. That check is made by external hardware, which sees the `PROT` output, so it can apply in supervisor mode too.


PRIVILEGE CHECKS
CPU check
External bus check
Fault 0x05
Fault 0x09
Instruction begins
0–1
Fetch
Instruction fetch: memory controller asserts BPER?
yes
Bus protection fault vector 0x09
no
2
Decode
Protected mode? (SR.P = 1)
no
Supervisor mode: no CPU checks
yes
Condition true?
no
Condition false: no effect, cannot fault
yes
Coprocessor operation nibble (bits 11–8) ≥ 0x8?
yes
Privilege violation vector 0x05
no
Writes ah, lh, ph or sh through its register field?
yes
Privilege violation vector 0x05
no
3
Execute
Software exception (EXCP) with vector < 0x60?
yes
Privilege violation vector 0x05
no
4
Memory
LOAD / STOR access: memory controller asserts BPER?
yes
Bus protection fault vector 0x09
no
5
Write back
Address-pair write would change a high word?
yes
Privilege violation vector 0x05
no
Instruction completes

External bus checks apply in both modes: the controller sees PROT, which follows SR.P. All faults are retryable.

Figure 6.2: Privilege Checks in Protected Mode

The illegal instruction never takes effect: it is canceled before it writes anything, and the link register stores its own address so **RETE** returns to it directly. This matters because a supervisor-mode handler needs to be able to inspect the exact instruction that faulted, decide how to respond (deny it, emulate its effect, or promote the caller), and then either retry it or use **ETFR**/**ETTR** to redirect the return address elsewhere. None of that is possible if the illegal instruction had already been allowed to run.

6.2.8 Instruction Trace Fault

(Post-instruction) – Raised after every instruction completes when `SR.T` is set, or when the force-trace request ($\overline{\text{TRCE}}$) is active at the instruction boundary (see the Force Trace Mode Input in the

SIRC-1 Hardware Reference Manual).

Used for single-step debugging. The fault fires after the instruction commits; for a coprocessor call, that includes the call's dispatch slot (Section 6.3.3). **`RETE`** returns to the instruction *following* the traced one. The T bit is cleared on entry to the fault handler (as with all exceptions), and no trace fault is raised at exception level 7, so the handler itself is not traced even while $\overline{\text{TRCE}}$ is asserted. **`RETE`** restores the saved SR, which re-enables tracing unless the handler explicitly clears T from the saved status register via **`ETTR`** before returning. `SR.T` is sampled once at the start of each instruction, before that instruction can modify it, so a write to `SR.T` takes effect starting with the next instruction boundary; exception-unit dispatches are never traced.

A traced **`EXCP`** is the one exception to the rule that a call's command completes before its trace fault. The trace fault is taken first, with the link register storing the address after the **`EXCP`**, and the exception unit records the trap as a pending software exception instead of entering it. When the trace handler returns with **`RETE`** to exception level 0, the recorded trap is entered at that instruction boundary as if the **`EXCP`** had just executed: link register 0 stores the address after the **`EXCP`** and the status register restored by the trace handler. The recorded trap is held by the exception unit, not in the CSR, so it is entered before any pending hardware interrupt. A debugger single-stepping through a system call therefore stops once before the trap handler runs (with the instruction after the **`EXCP`** as the return address), and again after that instruction has executed once the handler has returned.

Trace faults are not raised while a fault handler runs (exception level 7), whether `SR.T` is set or $\overline{\text{TRCE}}$ is active, because each one would otherwise escalate to a double fault. A handler at a lower level is traced if it sets `SR.T` itself or if $\overline{\text{TRCE}}$ is active.

6.2.9 Level Five Hardware Exception Conflict

(Exception dispatch) – Raised when a Level 5 hardware exception cannot be handled normally because it collides with another one. There are two cases:

- A Level 5 hardware exception is dispatched while one is already being serviced (exception level 6).
- The exception unit is about to dispatch a queued Level 5 hardware exception, but one or more further Level 5 requests arrived while it was queued and could not be queued as well (the *L5 overrun* described in Section 6.3.4).

In the first case the fault consumes the pending request: it is not dispatched after the handler returns. In the second case the request stays pending and is dispatched after the fault handler returns. If both cases apply at once, a single fault is raised, and it also clears the L5 overrun flag.

The exception unit has no spare link register above level 6, and the pending hardware exceptions register holds only one bit per line, so a second L5 cannot be handled normally. Rather than silently dropping it (which could mask a hardware misconfiguration), the CPU raises this fault. Level 5 interrupts should be treated as NMIs.

A program that does not care about conflicting L5 requests can install a handler for this fault that consists of a single **`RETE`**; the fault is not retryable, so **`RETE`** resumes the code that was running when the fault was raised. A program that does care can use the handler to log the overrun or take corrective action.

6.2.10 Double Fault

(Exception dispatch) – Raised when a fault occurs while another fault is already being handled, or when a fault's own vector cannot be fetched (Section 6.2.10).

When the CPU is already at fault level (priority 7) and another fault occurs, there is no additional banked link register available. The fault link register (register 6) is overwritten with the new return address, losing the original fault's return address. The fault metadata register (register 7) is also overwritten; it records the double fault flag, the original fault's type, and the type of the fault that caused the escalation (Section 6.5.2).

The CPU jumps to vector `0x08`. This is typically a last-chance handler that dumps registers and resets. If another fault occurs while the CPU is already at fault level, the CPU attempts to dispatch vector `0x08` again. There is no limit to the number of times vector `0x08` is dispatched. A double fault handler that causes a fault may cause an infinite loop without special handling from the programmer. If the double-fault vector fetch itself fails, the CPU resets. An instruction raises at most one fault: the first one detected cancels it, and the checks in its remaining phases are not made (Section B.3.2).

Link Register Preservation

There is no way to fully recover from a double fault raised inside a fault handler: the valid response is to declare the CPU state invalid, perform some last-chance error handling (for example, dumping the registers), and reset the CPU.

A fault handler that wants a better chance of capturing useful diagnostics before that reset should still save the link register contents early – before doing anything else that might fault again – using the **`ETFR`** instruction, so the original fault's return address and status register can be included in whatever gets dumped.

The values can be stored on the stack but since the stack pointer may be corrupted, a more robust implementation stores and loads from hardcoded memory addresses that are known to always be valid, instead of relying on the stack pointer.

For link registers 6 and 7, as for every link register, **`ETFR`** transfers only the return address and status register; the saved exception level is not accessible to software through it, nor can **`ETTR`** write it back. This is not just a missing diagnostic field: **`RETE`** always restores `current_exception_level` directly from whatever value is currently latched in the link register, and after a double fault raised inside a fault handler that value is the fault level (7), not the level that was active before the original fault. Even a handler that perfectly restores the return address and status register and attempts **`RETE`** anyway would leave the CPU stuck at exception level 7 – unable to service any hardware interrupt or software exception, including Level 5, since nothing outranks the highest priority level. New faults still dispatch normally regardless of the stuck level (faults bypass the priority check), so the CPU would keep running rather than visibly failing; it would simply be deaf to every interrupt and trap for the rest of the program's life. This is the underlying reason reset, not **`RETE`**, is the only valid response to a double fault raised inside a fault handler.

Fault Vector Fetch Failure

A double fault is also raised when the vector fetch for a fault fails with a bus or bus protection fault. This case is different from a fault raised inside a fault handler, because exception entry commits only after the vector fetch succeeds (Section 6.6.1). The original fault never entered its handler, so nothing has been overwritten yet:

- The CPU is still at the exception level from before the original fault.
- Link register 6 receives the original fault's return address, the status register from before the original fault, and that exception level, exactly as the original fault would have saved them.
- The fault metadata register has the double fault flag set and bus access type `0x4` (exception vector fetch). The Original Fault field holds the fault whose vector could not be fetched, the Escalating Fault field holds the bus or bus protection fault raised by the fetch, and the Return Address field holds the vector-table address that failed.

A double fault raised inside a fault handler never records an exception vector fetch, so a double-fault handler can identify this case from the double fault flag together with bus access type `0x4`.

Unlike a double fault raised inside a fault handler, this case is recoverable: **`RETE`** restores the state from before the original fault and returns to the original fault's return address. The handler must first repair whatever made the vector fetch fail, typically the vector-table entry or the memory mapping that holds it. For a retryable original fault, **`RETE`** retries the faulting instruction, which raises the original fault again; if its vector still cannot be fetched, the CPU double faults again and loops. If the original fault was an Instruction Trace Fault or a Level Five Hardware Exception Conflict, **`RETE`** continues past it and that fault is not raised again.

6.3 Hardware Exceptions (Interrupts)

A hardware exception occurs when external hardware raises an interrupt request. These are also referred to as "interrupts". When a hardware exception occurs, the current instruction is completed, and control is transferred to different vectors based on the interrupt "priority". The vectors for hardware exceptions are in the `0x10`–`0x50` range.

The return address stored in the exception link register is the address of the instruction that would have executed next, not the interrupted one – so once the handler returns via **`RETE`**, the interrupted program resumes normally as if nothing happened.

6.3.1 Hardware Interrupt Enable Bits

The status register contains 4 individual enable bits (bits 9–12 in the privileged byte) that control whether each maskable hardware interrupt line can trigger an exception:

- Bit 9 (E1): Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- Bit 10 (E2): Hardware Interrupt Line 2 (Level 2)
- Bit 11 (E3): Hardware Interrupt Line 3 (Level 3)
- Bit 12 (E4): Hardware Interrupt Line 4 (Level 4, highest maskable priority)

Note: Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and cannot be disabled. It always triggers an exception when requested, regardless of the enable bit settings. Bit 13 is used for the Exception Active flag instead.

When an interrupt line is disabled (bit = 0), interrupts on that line are completely ignored and not queued. When enabled (bit = 1), the interrupt can trigger an exception if its priority is higher than the currently executing exception level.

6.3.2 Pending Hardware Interrupts

Interrupt requests at levels 1–4 are level-sensitive; Level 5 is edge-triggered and is described separately in Section 6.3.4. When an enabled level-sensitive interrupt request is observed active, the corresponding bit is set in the pending hardware-exceptions register. The request lines are sampled on every clock cycle against the current enable bits. Requests at disabled maskable levels are ignored and are not latched for later service. A line is also not latched while the current exception level is at or above the level its own handler would run at, that is, while its own handler or a higher-priority one is running. Without this rule, a request that is still active when its handler starts would be latched again before the handler could acknowledge the device, and the handler would be dispatched a second time after **`RETE`** with no request behind it. A device must therefore hold its request until it is serviced: a request withdrawn while a handler at or above its level runs is not seen. A write to `sr` that sets an enable bit therefore lets an already-active request on that line be dispatched at the next instruction boundary, straight after the instruction that set it. A write that clears an enable bit does not discard a request already latched on that line; see below.

For the level-sensitive lines, the pending register stores one bit per hardware interrupt line, so repeated requests at the same level coalesce into a single pending interrupt. If the request is still active after its handler returns with **`RETE`**, it is sampled again and becomes pending again.

At an instruction boundary, the exception unit selects the highest-priority pending hardware interrupt whose priority is higher than the current exception level. The selected bit is cleared as the handler is dispatched, and set again if the handler's vector fetch fails (Section 6.6.1). Lower-priority interrupts latched before the handler was entered remain latched and are considered again after the current handler returns with **RETE**. If the highest-priority pending interrupt's enable bit has been cleared since it was latched, it does not block dispatch of any other pending, enabled interrupt of lower priority. The disabled interrupt stays pending, and is dispatched if its line is re-enabled, even if the device has since withdrawn its request.

6.3.3 Interrupt Recognition and Latency

Hardware interrupts, software exceptions and instruction trace faults are recognized only at an instruction boundary, and only when the CSR is empty and no coprocessor is active (Section 2.3.2). A coprocessor call (**COPI**, **COPR**, or a meta-instruction that assembles to one) is part of its instruction: its dispatch slot, and the command it starts, complete before a pending hardware interrupt or an instruction trace fault is taken. The one exception is a traced **EXCP**, whose trace fault is taken before the trap is entered (Section 6.2.8). Nothing is lost by the wait. Level-sensitive requests stay asserted, a Level 5 request stays in its pending bit, and a trace fault is still raised. A fault raised by the call itself, such as a privilege violation or a failed DMA bus access, is serviced as soon as it occurs.

The worst-case wait for an interrupt is therefore the remainder of the current instruction, plus the dispatch slot of a pending coprocessor call, plus the time a running coprocessor command takes, plus the dispatch slot of the interrupt itself. A DMA transfer cannot be interrupted, including by Level 5 (Section 16.7). Appendix B gives the cycle counts.

6.3.4 Level 5 NMI-Like Behavior

Level 5 hardware exceptions behave similarly to a Non-Maskable Interrupt (NMI). Unlike interrupt levels 1–4, which are level-sensitive (recognized for as long as the request remains active), Level 5 is edge-triggered: it fires once when the request becomes active, regardless of how long it is held, and must be withdrawn and raised again to fire again. This avoids spurious Level Five Hardware Exception Conflict faults from a request that simply stays active. The edge is latched in every CPU state except reset, including bus wait states, a halt (Section 6.8), and **WAIT**, so a request that is active for at least one clock cycle is never lost. Level 5 cannot be disabled via the interrupt enable bits in the status register, and will preempt any lower-priority exception handler. However, Level 5 is *not* fully non-maskable. Because faults carry priority 7 (higher than Level 5's priority 6), an L5 interrupt that arrives while a fault is being handled is *queued* in the pending hardware exceptions register rather than serviced immediately.

Once the fault handler returns via **RETE** and the exception level drops back below 6, the pending L5 interrupt fires normally.

The pending hardware exceptions register holds a single bit for L5, so only one L5 request can be queued. If a further L5 request arrives while the L5 pending bit is already set (for example, during a long fault handler), the request is lost, and the exception unit records the loss in an internal *L5 overrun* flag. This flag is not part of the status register or any other software-visible register. When the exception unit next selects L5 for dispatch and the L5 overrun flag is set, it clears the flag and raises a Level Five Hardware Exception Conflict fault (Section 6.2.9) instead of dispatching L5. The L5 pending bit remains set. Because that fault is not retryable, **RETE** from its handler resumes the code that was running, and the still-pending L5 interrupt is then dispatched normally. Any number of lost L5 requests while one is queued are reported by a single conflict fault. The L5 pending bit and the L5 overrun flag are both cleared by reset.

The L5 pending bit is cleared at the moment L5 is selected for dispatch, and set again only if its vector fetch fails. An edge that arrives after selection, even during the dispatch slot, is therefore a new request: it sets the pending bit again, and once the L5 handler is running it raises a Level Five

Hardware Exception Conflict fault. If the L5 vector fetch fails while a new edge has already set the L5 pending bit again, the exception unit sets the L5 overrun flag.

This means Level 5 response latency is not deterministic when a fault is in progress. For timing-critical use of the NMI line, fault handlers should be kept as short as possible.

If a second L5 interrupt arrives *while the CPU is already executing an L5 handler* (exception level 6), it is not dispatched as a second L5. It sets the L5 pending bit, and at the next instruction boundary it raises a Level Five Hardware Exception Conflict fault, which consumes that pending bit. See Section 6.2.9.

These two rules compose in a non-obvious way when a fault is raised *inside* an L5 handler while a second L5 interrupt is pending. Table 6.2 walks through the sequence step by step.

Step	Event	Exception Level	Link Register Used
1	CPU is executing inside the L5 handler.	6	Link register 5 (holds the L5 handler's return state)
2	A fault condition is raised inside the L5 handler. Priority 7 exceeds the current level (6), so the fault preempts it.	6 → 7	Link register 6 receives the L5 handler's return address, status register, and saved level (6)
3	A second L5 interrupt is requested while the fault handler runs. Priority 6 does not exceed the current level (7), so it is not serviced.	7	Not dispatched; latched as pending in the pending hardware-exceptions register
4	The fault handler executes `RETE`, restoring the return address, status register, and saved level (6) from link register 6.	7 → 6	Link register 6 (restored, not overwritten)
5	Back at level 6, the exception unit reconsiders the pending L5 interrupt. Its priority (6) is not *higher* than the current level (6), so it is not dispatched normally.	6	—
6	The CPU raises a Level Five Hardware Exception Conflict fault instead, because the pending L5 collides with the still-active L5 handler.	6 → 7	Link register 6 receives the dispatch point as the return address

Table 6.2: Fault-Inside-L5-Handler Timeline

In short: the fault (priority 7) blocks the second L5 (priority 6) from firing while the fault handler runs. Once **RETE** returns to level 6, the queued L5 collides with the still-active outer L5 handler, and a Level Five Hardware Exception Conflict fault is raised then, not during the inner fault handler.

6.4 Software Exceptions (Traps)

A software exception occurs when the program executes an **EXCP** instruction that signals the exception coprocessor that it should trigger an exception. They are usually used by programs running in protected mode to switch to supervisor mode in a controlled way. They are also referred to as “traps”. The vectors for software exceptions are in the `0x60`–`0xFF` range. Supervisor code may also name a vector below `0x60`; the trap still enters at priority 1 through link register 0, so the handler it reaches does not find its own link register filled in (see **EXCP** in Chapter 16).

As with hardware exceptions, the return address stored in the link register is the instruction following the **EXCP** call, so **RETE** resumes the calling program immediately after the trap.

Software exceptions are only accepted from normal execution. If an **EXCP** instruction is executed while any exception or fault handler is already active, the software exception is ignored, is not queued, and does not fetch its vector. The privilege check is made first: a protected-mode **EXCP** with a vector below `0x60` raises a privilege-violation fault even inside a handler.

6.5 Exception Registers

The SIRC-1 exception coprocessor maintains several special-purpose registers for exception handling.

6.5.1 Link Registers

The exception unit uses banked link registers to store the processor state when an exception occurs. There are eight link registers in total, indexed 0–7:

- Register 0: Software exceptions (traps)
- Registers 1–5: Hardware exceptions (interrupts), one for each priority level
- Register 6: Faults
- Register 7: Fault metadata (stores bus address and bus access type information, see Section 6.5.2 for the format)

For registers 0–6 the link register index is the exception’s priority minus one, so each priority level has its own link register. This is what lets a higher-priority exception interrupt a lower-priority handler without software saving the return address first: the new exception saves its state in its own link register and leaves the interrupted handler’s untouched.

Link registers 0–6 each store three pieces of information; register 7 uses the same layout to hold fault metadata (Section 6.5.2):

- **Return Address:** The 32-bit program-counter value to resume at: the address of the faulting instruction for a retryable fault, or the address of the next instruction otherwise (see below), with bits 31–24 carried unchanged from `p`.
- **Return Status Register:** The 16-bit status register value at the time of the exception
- **Saved Exception Level:** The exception level that was active when the exception was taken; **RETE** restores it into `current_exception_level`

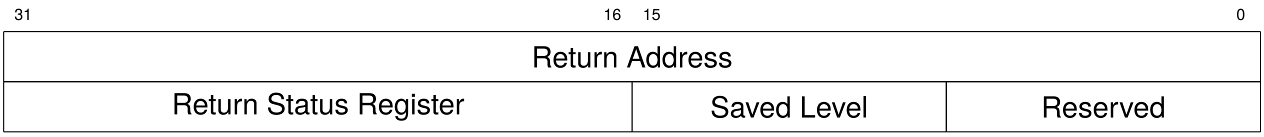


Figure 6.3: Exception Link Register Saved-State Layout

The Saved Level and Reserved fields are shown for illustration only: `ETFR` and `ETTR` transfer only the Return Address and the Return Status Register between a link register and `a`/`r7`. The saved exception level is not accessible to software through these instructions.

When an exception occurs, the appropriate link register (determined by the exception priority level) is populated with the current program counter and status register values. For the retryable faults (Section 6.2.1), the return address is the address of the faulting instruction, so it can be retried. For regular exceptions (hardware and software exceptions) and Instruction Trace Fault, the return address is the address after the current instruction, except as described in Section 6.2.1 for the exception-dispatch faults (Level Five Hardware Exception Conflict, Double Fault), which instead store the program counter at the point the fault was detected.

Register	Exception Source	Priority Level
0	Software exceptions	1
1	Level 1 hardware	2
2	Level 2 hardware	3
3	Level 3 hardware	4
4	Level 4 hardware	5
5	Level 5 hardware	6
6	Faults	7
7	Fault metadata	n/a

Table 6.3: Exception Link Register Assignment

6.5.2 Fault Metadata Register Format

The fault metadata register (register 7) stores additional information about faults beyond just the return address. The Return Status Register field is used to encode fault-specific metadata in the following format:

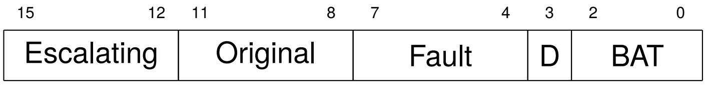


Figure 6.4: Fault Metadata Register Status Field Layout

- **Bits 0–2:** Bus access type (BAT) captured when the fault was raised (3 bits)
  - `0x0`: None
  - `0x1`: Instruction fetch
  - `0x2`: Data read
  - `0x3`: Data write
  - `0x4`: Exception vector fetch
  - `0x5`: DMA read burst
  - `0x6`: DMA write burst
  - `0x7`: Reserved

- **Bit 3:** Double fault flag (1 bit) – set to 1 if this is a double fault
- **Bits 4–7:** Fault type (4 bits) – the fault being dispatched; always `0x8` when the double fault flag is set
- **Bits 8–11:** Original fault type (4 bits) – for a double fault, the fault that was already being handled, or whose vector could not be fetched (Section 6.2.10). If a double-fault handler itself faults, this is `0x8`. Reserved and read as zero when the double fault flag is clear
- **Bits 12–15:** Escalating fault type (4 bits) – for a double fault, the fault that caused the escalation. Reserved and read as zero when the double fault flag is clear

For a double fault, the bus access type and the Return Address field describe the escalating fault. The original fault's bus access type and address are lost.

Fault types use the following values:

- **`0x1`:** Bus fault
- **`0x2`:** Alignment fault
- **`0x3`:** Segment overflow fault
- **`0x4`:** Invalid opcode fault
- **`0x5`:** Privilege violation fault
- **`0x6`:** Instruction trace fault
- **`0x7`:** Level five hardware exception conflict
- **`0x8`:** Double fault
- **`0x9`:** Bus protection fault

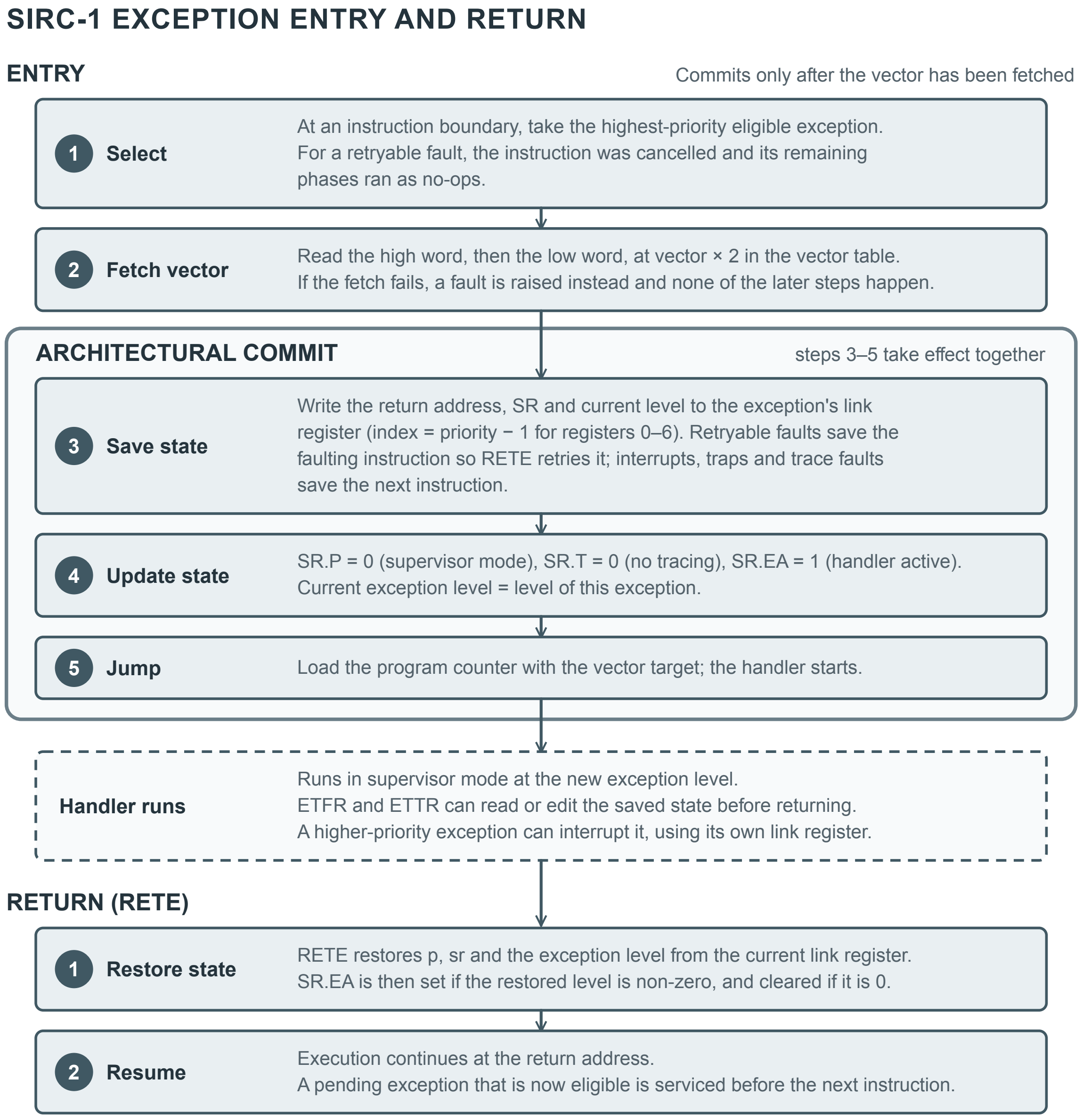
The Return Address field of register 7 stores the bus address that was being accessed when the fault occurred. For a fault that does not access the bus, this field is architecturally undefined, with four exceptions:

- An invalid-opcode fault raised during coprocessor dispatch, before any bus access, stores the 16-bit coprocessor command that could not be dispatched in the low word, and the high word is zero. This lets an invalid-opcode handler emulate missing optional coprocessor operations.
- An alignment fault stores the odd instruction address.
- An effective-address segment overflow stores the wrapped address (Section 6.2.5).
- An address-register auto-update segment overflow stores the wrapped register value (Section 6.2.5).

Exception handlers can use the **`ETFR`** instruction to read the fault metadata register and decode this information to determine how to handle the fault appropriately.

6.6 Exception Processing

Figure 6.5 shows how an exception is entered and returned from. This section gives the rules the figure does not: which exceptions are accepted, what is preserved, and what happens when entry fails. Section 6.7 describes the return.



SIRC-1 EXCEPTION ENTRY AND RETURN

ENTRY

Commits only after the vector has been fetched

1 **Select** At an instruction boundary, take the highest-priority eligible exception. For a retryable fault, the instruction was cancelled and its remaining phases ran as no-ops.

2 **Fetch vector** Read the high word, then the low word, at vector × 2 in the vector table. If the fetch fails, a fault is raised instead and none of the later steps happen.

ARCHITECTURAL COMMIT

steps 3–5 take effect together

3 **Save state** Write the return address, SR and current level to the exception's link register (index = priority − 1 for registers 0–6). Retryable faults save the faulting instruction so RETE retries it; interrupts, traps and trace faults save the next instruction.

4 **Update state** SR.P = 0 (supervisor mode), SR.T = 0 (no tracing), SR.EA = 1 (handler active). Current exception level = level of this exception.

5 **Jump** Load the program counter with the vector target; the handler starts.

Handler runs Runs in supervisor mode at the new exception level. ETFR and ETTR can read or edit the saved state before returning. A higher-priority exception can interrupt it, using its own link register.

RETURN (RETE)

1 **Restore state** RETE restores p, sr and the exception level from the current link register. SR.EA is then set if the restored level is non-zero, and cleared if it is 0.

2 **Resume** Execution continues at the return address. A pending exception that is now eligible is serviced before the next instruction.

Figure 6.5: SIRC-1 Exception Entry and Return

6.6.1 Exception Entry

Acceptance. Acceptance is decided before the exception-vector fetch, so an exception that is not accepted has no side effects.

- A hardware interrupt is accepted only if its line is enabled in the status register (Level 5 is always enabled) and its priority is higher than the current exception level. Otherwise it does not fetch its vector. A disabled interrupt is ignored and not queued; one that is only outranked remains pending, according to the rules in Section 6.3.2.
- A Level 5 hardware exception that collides with another one raises a Level Five Hardware Exception Conflict fault instead (Section 6.2.9).
- A software exception is accepted only when no exception handler is active. The privilege check is made first: a protected-mode **`EXCP`** with a vector below `0x60` raises a privilege-violation fault even inside a handler.

Selection and commit. A hardware interrupt, software exception or instruction trace fault is selected only when the CSR is empty and no coprocessor is active (Section 6.3.3). A fault raised by a coprocessor call or by the command it started is not held back: it ends the command, clears the CSR, and is dispatched at the next phase 0. The exception unit then fetches the high word and then the low word of the handler address from the vector table. The state updates shown in the figure commit as one architectural action, and only after the vector fetch has succeeded. The condition flags, reserved lower status bits, interrupt-enable bits, and `SR.A` are preserved in the live status register.

The status register saved in the link register is the value from before entry, with `SR.P`, `SR.T`, and `SR.EA` unmodified. **`RETE`** restores it, so a handler can inspect or edit the saved value with **`ETFR`** and **`ETTR`** before returning. `SR.T` is cleared on entry to every handler, including fault handlers, so software tracing does not continue into a handler; if it was set before the exception, tracing resumes after **`RETE`**. A handler that sets `SR.T` itself, or runs while $\overline{\text{TRCE}}$ is active, is traced, except that no trace fault is raised at exception level 7 (Section 6.2.8).

Vector fetch failure. Except for the reset vector (Section 6.9.3), if a bus or protection fault occurs during an exception-vector fetch, the original exception does not enter and the remaining phases of its dispatch slot run as no-ops (Section B.3.2). The CPU raises the corresponding fault with the bus access type set to exception vector fetch. If a fault-vector fetch itself fails, the CPU escalates to a double fault, which keeps the original fault's saved state so it can be recovered (Section 6.2.10). If the double-fault vector fetch fails, the CPU resets.

When the failed fetch belongs to a hardware interrupt, a Level 5 request or a software exception, that exception is not lost. Link register 6 stores the state the exception would have saved: its return address, and the status register and exception level from before entry. The fault-metadata register records the vector address with bus access type `0x4`. The pending bit of an interrupt, including the Level 5 bit, is set again, so the interrupt is selected again once the fault handler returns with **`RETE`**. A software exception is retryable: its return address is the address of the **`EXCP`** itself, so **`RETE`** executes the **`EXCP`** again.

Every exception-vector fetch is made as a supervisor access. `PROT` is deasserted for it, even for an exception taken from protected mode, although `SR.P` is not cleared until entry commits. A memory controller that blocks protected-mode access to the vector table therefore does not block exception entry.

6.7 Return from Exception

To return from an exception handler, the **`RETE`** (Return from Exception) instruction restores the saved state from the current link register:

1. The return address is loaded from the link register into the program counter
2. The saved status register value is restored (including interrupt enable bits)
3. The saved exception level is restored to `current_exception_level`
4. `SR.EA` is set if the restored exception level is non-zero and cleared if it is 0, whatever the saved status register holds, so `SR.EA` always agrees with the exception level
5. Execution resumes at the return address

Because **RETE** replaces the whole status register with the saved copy, any change a handler makes to `sr` – including the interrupt enable bits – is undone when it returns. To change the status register that the interrupted code resumes with, for example to leave an interrupt line disabled, a handler edits the saved copy with **ETFR** and **ETTR** before **RETE**.

After **RETE**, any pending interrupts that are now eligible to run (higher priority than the restored exception level and enabled) are serviced before the next instruction executes.

Executing **RETE** while the current exception level is 0 is architecturally undefined.

Where **RETE** returns depends on the kind of exception:

- For the retryable faults (Section 6.2.1), it returns to the faulting instruction, allowing it to be retried. This retry is not always a clean undo for a DMA burst (**DMAR**, **DMAW**, **DMAT**), since it performs several bus accesses per instruction; see Section 16.7.3.
- For the Instruction Trace Fault and for regular exceptions (hardware and software), it returns to the instruction following the one that was executing when the exception occurred.
- For the exception dispatch faults (Level Five Hardware Exception Conflict and Double Fault), it returns to the program counter at the point the fault was detected. A double fault caused by a failed fault-vector fetch instead returns to the original fault's return address (Section 6.2.10).

6.7.1 Accessing Link Registers

The exception coprocessor provides instructions to transfer data to and from the link registers:

- **ETFR (Exception Transfer From Register):** Copies the return address and/or status register from a link register to the address register and/or `r7`
- **ETTR (Exception Transfer To Register):** Copies the address register and/or `r7` to a link register's return address and/or status register

These instructions allow exception handlers to examine or modify the saved processor state before returning, enabling features like system calls, context switching, and debuggers.

6.8 Halt Request

The halt request ($\overline{\text{HALT}}$) lets external hardware stop the CPU, for example so that a debugger can examine the system. It is sampled at every phase 0. If it is active there, the CPU stops at that phase 0 with the bus idle. This can pause the CPU between an instruction and its coprocessor dispatch slot, between DMA slots, or before a fault, interrupt, or reset dispatch. Halt latency is at most 6 cycles. While halted the CPU issues no bus cycles and executes nothing. When the request is withdrawn, the CPU resumes exactly where it stopped. The halt is the state of this request and nothing else: no instruction enters or leaves it, and it is not saved by exception entry.

While halted, interrupt lines are still sampled under the normal rules, and a Level 5 edge is latched (Section 6.3.4). Nothing is dispatched until the CPU resumes: requests latched during the halt are taken under the normal rules once the CPU resumes. A reset ends a halt at once; if $\overline{\text{HALT}}$ is still active after the reset, the CPU completes the six-cycle $\overline{\text{RSTO}}$ hold and then halts at the first phase 0, before the reset-vector fetch. The *SIRC-1 Hardware Reference Manual* describes the pin that carries the request.

6.9 Reset and Startup

Reset can be initiated externally by raising the reset request ($\overline{\text{RSTI}}$), or in software by executing the privileged **`RSET`** meta-instruction. Both paths converge on the same reset-vector sequence: the CPU holds reset output active, then the exception unit fetches vector `0x00` and loads the fetched target address into the program counter.

6.9.1 Reset State

State	Reset Behavior
Instruction phase	Set to phase 0.
Pending bus request	Cleared; an in-progress stalled bus request is abandoned.
Halt state	Not stored. If the halt request is still active, the CPU halts again at the first phase 0 after reset, before the reset-vector fetch.
Wait-for-exception state	Cleared.
Current exception level	Cleared to 0. The CPU leaves reset outside any exception handler.
Status register	Cleared to `0x0000` when reset is requested.
Program counter	Loaded from reset vector `0x00`.
General registers	Not cleared by reset. Software must treat their contents as architecturally undefined.
Address register pairs	Not cleared by reset. Software must treat their contents as architecturally undefined.
Exception link registers	Not cleared by reset. Software must treat their contents as architecturally undefined.
Pending hardware exceptions and faults	Cleared when reset is requested. Interrupt lines, including Level 5, are not sampled until the reset-vector fetch completes; requests that are still active then are sampled according to the normal interrupt rules. A Level 5 line held asserted across reset does not fire afterwards: reset does not clear the edge detector's last sampled level, so there is no transition, and the line must be released and reasserted.
Pending software exception	Cleared when reset is requested. This includes the trap recorded by a traced **`EXCP`**, which is held by the exception unit.
Coprocessor select register (CSR)	Set internally to the reset operation until the reset vector sequence completes.

Table 6.4: Reset State

Because the status register is cleared to `0x0000` as part of the reset sequence, hardware interrupt lines 1–4 are masked immediately after reset – their enable bits, `SR.E1`–`SR.E4` (Section 6.3.1), are all cleared to 0. The Level 5 (NMI) line is architecturally non-maskable and remains enabled regardless of `sr`'s contents, but no interrupt line is sampled during the reset hold or the reset-vector fetch. The status register, the exception level and the pending hardware exceptions are all cleared at the moment reset is requested, so nothing latched before the reset survives it and the old enable bits never admit a request during the hold.

6.9.2 Reset Output Hold

While reset is active, the CPU asserts the reset output ($\overline{\text{RSTO}}$) and performs no bus cycles. This gives external devices that share the reset signal time to complete their own reset sequence before the CPU resumes.

For hardware reset, raising $\overline{\text{RSTI}}$ immediately aborts any current instruction, exception handler, exception dispatch, or pending bus wait, then asserts $\overline{\text{RSTO}}$. If $\overline{\text{RSTI}}$ remains active, $\overline{\text{RSTO}}$ remains asserted and the CPU does not advance. After $\overline{\text{RSTI}}$ is withdrawn, the reset hold completes before the CPU resumes. The reset assertion cycle counts as the first reset-output cycle; after release, five additional countdown cycles occur before the exception unit starts the reset-vector fetch.

For software reset, **`RSET`** completes as a normal coprocessor meta-instruction. When its write-back phase commits the reset command, the CPU requests reset and $\overline{\text{RSTO}}$ is asserted for six cycles before the reset-vector fetch begins. A reset caused by a failed double-fault vector fetch behaves as a software reset: a six-cycle $\overline{\text{RSTO}}$ hold, then the reset-vector fetch.

6.9.3 Reset Vector Fetch

The reset vector is vector `0x00` in the exception vector table. After the reset-output hold, the exception unit performs an exception-vector fetch with bus access type exception vector fetch:

1. Read the high word of the reset target address from `0x0000`.
2. Read the low word of the reset target address from `0x0001`.
3. Load the fetched target address into `p`.

Like every other exception-unit dispatch, the reset-vector fetch occupies one six-phase slot; step 3 happens in its execute phase. See Appendix B for the cycle counts, including the reset-output hold described in Section 6.9.2.

If a bus or protection fault occurs during the reset-vector fetch, the reset target is not loaded and no fault is raised: there is no valid state to save and no handler can be assumed to be reachable. The CPU re-enters reset instead, asserting $\overline{\text{RSTO}}$ for a further six-cycle hold before it tries the reset-vector fetch again, and keeps doing so until the fetch succeeds.

6.10 Exception Coprocessor Instructions

The exception coprocessor (coprocessor ID `0x1`) provides several specialized instructions for exception handling and system control.

Instruction	**Opcode**	**Privilege**	**Purpose**
`EXCP`	`0x1`	Protected, supervisor	Trigger a software exception (trap)
`WAIT`	`0x9`	Supervisor	Enter wait state until an interrupt can be taken or reset occurs
`RETE`	`0xA`	Supervisor	Return from an exception handler
`RSET`	`0xB`	Supervisor	Perform reset sequence and jump to reset vector
`ETFR`	`0xC`	Supervisor	Copy saved exception state from a link register
`ETTR`	`0xD`	Supervisor	Write CPU register state back to a link register

Table 6.5: Exception Coprocessor Instruction Summary

Exception coprocessor opcodes `0x2`–`0x8` are reserved; issuing one raises an invalid-opcode fault.

For full syntax, operation details, and worked examples, see Chapter 16.

6.10.1 Internal Instructions

The exception unit dispatches the following opcodes itself; no instruction is defined for them:

- **Fault (`0xE`):** Automatically invoked when a fault condition is detected
- **HardwareException (`0xF`):** Automatically invoked when a hardware interrupt is signaled
- **None (`0x0`):** No operation, used when no exception is pending

These internal operations follow the standard exception handling flow described in Section 6.6. Software must not issue opcode `0xE` or `0xF` directly, for example with a supervisor-mode **`COPI`** instruction: the result is architecturally undefined. Issuing `0x0` is a no-op.

6.10.2 Coprocessor Select Register (CSR)

The coprocessor select register (CSR) is a 16-bit internal register used to select the target coprocessor and coordinate exception handling. Section 2.3.2 describes how it is used when dispatching. It has the following structure:

- **Bits 15–12 (First Nibble):** Coprocessor ID (`0x1` for exception unit, `0x0` for processing unit)
- **Bits 11–8 (Second Nibble):** Coprocessor opcode (the operation to execute)
- **Bits 7–0 (Third and Fourth Nibbles):** Vector or parameter value

The exception unit loads the CSR when a pending exception needs to be serviced. For a hardware interrupt, software exception or trace fault it waits until the CSR is empty and no coprocessor is active, so such an exception never replaces a command written by **`COPI`** or **`COPR`** (Section 6.3.3). A fault raised by the call or its command replaces the command, which ends. For the commands the exception unit builds itself to dispatch a fault or a hardware exception, the priority is determined by the high nibble of the vector ID, calculated as 7 minus the value of that nibble (for example, fault vector `0x05` is priority 7 and `0x40` is priority 3). A software exception (**`EXCP`**) is always priority 1, whatever its vector.

A CSR value of `0x0000` – the value left in place once the reset sequence has completed, and whenever no exception is pending – selects coprocessor ID `0x0`, the processing unit, so the CPU simply continues executing the program normally in the absence of any exception activity.

Part II

Instruction Set Architecture

CHAPTER 7

Instruction Formats

7.1 Overview

All SIRCIS instructions are exactly 32 bits (4 bytes, or 2 words) in length. This fixed instruction size simplifies instruction fetch and decode logic, though it may result in larger code size compared to variable-length instruction sets.

The SIRC-1 supports three distinct instruction formats:

1. Immediate
2. Short Immediate (with Shift)
3. Register

Note: Operandless assembler meta-instructions such as **`NOOP`** still assemble to one of these three CPU instruction formats.

Each format is optimized for different types of operations and addressing modes.

7.2 Format Overview

Format	**Typical Use**
Immediate	Operations with a 16-bit constant value
Short Immediate	Operations with an 8-bit constant with optional shift
Register	Operations on multiple registers with optional shift

Table 7.1: Instruction Format Summary

7.3 Common Fields

All instruction formats share some common fields:

Opcode (6 bits) Identifies the operation to perform. With 6 bits, up to 64 distinct operations can be encoded.

Condition Flags (4 bits) Specifies the conditions under which the instruction executes. See Chapter 10 for details.

Additional Flags (2 bits) Used in memory operations to specify address register pairs (`l`, `a`, `s`, `p`) and in ALU operations to specify how the status register is updated.

7.4 Immediate Format

7.4.1 Structure

The immediate format encodes a full 16-bit immediate value within the instruction.

The format has a single register field. For ALU operations that save a result it is both the first source operand and the destination, so `ADDI r1, #5` computes `r1` = `r1` + 5; there is no separate destination field. Test operations read it as a source only, **MOVI** writes it without reading it, **LOAD** writes it, and **STOR** reads it.

Note: Due to the large 16-bit operand, shift is not supported in this format. AF = 10 is still encodable; with no shift fields the shifter result is treated as zero flags, so it clears N, Z, C and V (Chapter 13).

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">Opcode</td><td colspan="4">Reg</td><td colspan="16">Immediate Value</td><td colspan="2">AF</td><td colspan="4">Cond</td></tr>
</table>

Figure 7.1: Immediate Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit register identifier; its role depends on the instruction (see above). For **LDEA** and **LDEL** (opcodes `0x18`–`0x1F`), this field instead holds a 2-bit address-register-pair index in bits 23–22 (00 = `l`, 01 = `a`, 10 = `s`, 11 = `p`) naming the destination pair, using the same encoding as the AF field. Bits 25–24 are reserved for these opcodes and are written as zero; a non-zero value is architecturally undefined.

Immediate (bits 21–6) 16-bit signed or unsigned immediate value

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = `l` (link register)
- 01 = `a` (address register)
- 10 = `s` (stack pointer)
- 11 = `p` (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved; N, Z, C and V are architecturally undefined (the operation still completes and writes its result), and assemblers never emit this encoding

Condition (bits 3–0) 4-bit condition code

7.4.2 Encoding Examples

Instruction	Opcode	Reg	Immediate	AF	Cond	Hex
`ADDI r1, #100`	0x00 (000000)	0x0 (0000)	0x0064 (0000 0000 0110 0100)	01 (01)	0x0 (0000)	`0x00001910`
`CMPI r2, #0x8000`	0x0A (001010)	0x1 (0001)	0x8000 (1000 0000 0000 0000)	01 (01)	0x0 (0000)	`0x28600010`
`LOAD r3, (#16, a)`	0x14 (010100)	0x2 (0010)	0x0010 (0000 0000 0001 0000)	01 (01)	0x0 (0000)	`0x50800410`
`STOR (#-2, s), r4`	0x10 (010000)	0x3 (0011)	0xFFFE (1111 1111 1111 1110)	10 (10)	0x0 (0000)	`0x40FFFFA0`
`LDEA a, (#4, s)`	0x18 (011000)	0x1 (0001)	0x0004 (0000 0000 0000 0100)	10 (10)	0x0 (0000)	`0x60400120`
`LDEA a, -(#0, s)`	0x1A (011010)	0x1 (0001)	0x0000 (0000 0000 0000 0000)	10 (10)	0x0 (0000)	`0x68400020`
`LDEL p, (#0, a)`	0x1C (011100)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	01 (01)	0x0 (0000)	`0x70C00010`
`LDEL p, (#0, a)+`	0x1E (011110)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	01 (01)	0x0 (0000)	`0x78C00010`
`BRAN #4`	0x18 (011000)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	11 (11)	0x0 (0000)	`0x60C00130`
`BRSR #4`	0x1C (011100)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	11 (11)	0x0 (0000)	`0x70C00130`
`ADDI|== r5, #42`	0x00 (000000)	0x4 (0100)	0x002A (0000 0000 0010 1010)	01 (01)	0x1 (0001)	`0x01000A91`
`SUBI|>= r6, #10`	0x02 (000010)	0x5 (0101)	0x000A (0000 0000 0000 1010)	01 (01)	0xB (1011)	`0x0940029B`
`ADDI[N] r1, #100`	0x00 (000000)	0x0 (0000)	0x0064 (0000 0000 0110 0100)	00 (00)	0x0 (0000)	`0x00001900`
`CMPI[A] r2, #0x8000`	0x0A (001010)	0x1 (0001)	0x8000 (1000 0000 0000 0000)	01 (01)	0x0 (0000)	`0x28600010`

Table 7.2: Immediate Format Encoding Examples

Notes:

- Condition code examples: `0x0` (0000) = Always, `0x1` (0001) = Equal (==), `0xB` (1011) = Signed >=
- AF field for memory ops: 00=`l`, 01=`a`, 10=`s`, 11=`p`; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift, 11=reserved (N, Z, C and V architecturally undefined; the result is still written)
- Negative immediates use two's complement (`#-2` = `0xFFFE`)

7.5 Short Immediate Format (with Shift)

7.5.1 Structure

The short immediate format sacrifices immediate value width to include shift information, allowing for more powerful single-instruction operations.

This format is supported only for ALU instructions; it cannot be used with branching or load/store instructions.

As in the immediate format, the single register field is both the first source operand and the destination for ALU operations that save a result, so `ADDI r1, #5, LSL #2` computes `r1` = (`r1` shifted left by 2) + 5: the shift applies to the register, not the immediate (Chapter 9). Test operations read it as a source only.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">Op</td><td colspan="4">Reg</td><td colspan="8">Imm</td><td>SO</td><td colspan="3">ST</td><td colspan="4">SA</td><td colspan="2">AF</td><td colspan="4">Cond</td></tr>
</table>

Figure 7.2: Short Immediate with Shift Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit register identifier; for ALU operations that save a result it is both the first source and the destination (see above)

Immediate (bits 21–14) 8-bit unsigned immediate value, zero-extended to 16 bits before use

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 9.1 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit status register update source selector

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved; N, Z, C and V are architecturally undefined (the operation still completes and writes its result), and assemblers never emit this encoding

Condition (bits 3–0) 4-bit condition code

7.5.2 Encoding Examples

Instruction	Op	Reg	Imm	SO	ST	SA	AF	Cond	Hex
`ADDI r1, #2, LSL #3`	0x20 (100000)	0x0 (0000)	0x02 (00000010)	0 (0)	001 (001)	0x3 (0011)	01 (01)	0x0 (0000)	`0x800084D0`
`ORRI r2, #1, LSL #10`	0x25 (100101)	0x1 (0001)	0x01 (00000001)	0 (0)	001 (001)	0xA (1010)	01 (01)	0x0 (0000)	`0x94404690`
`ANDI r3, #7, ASR #2`	0x24 (100100)	0x2 (0010)	0x07 (00000111)	0 (0)	100 (100)	0x2 (0010)	01 (01)	0x0 (0000)	`0x9081D090`
`SUBI r4, #5, LSL r2`	0x22 (100010)	0x3 (0011)	0x05 (00000101)	1 (1)	001 (001)	0x1 (0001)	01 (01)	0x0 (0000)	`0x88C16450`
`XORI|!= r7, #15, LSR #4`	0x26 (100110)	0x6 (0110)	0x0F (00001111)	0 (0)	010 (010)	0x4 (0100)	01 (01)	0x2 (0010)	`0x9983C912`
`CMPI|NS r1, #8, RTL r3`	0x2A (101010)	0x0 (0000)	0x08 (00001000)	1 (1)	101 (101)	0x2 (0010)	01 (01)	0x5 (0101)	`0xA8023495`
`SHFT r3, RTR #5`	0x25 (100101)	0x2 (0010)	0x00 (00000000)	0 (0)	110 (110)	0x5 (0101)	10 (10)	0x0 (0000)	`0x94801960`
`ADDI[N] r1, #2, LSL #3`	0x20 (100000)	0x0 (0000)	0x02 (00000010)	0 (0)	001 (001)	0x3 (0011)	00 (00)	0x0 (0000)	`0x800084C0`
`ORRI[S] r2, #1, LSL #10`	0x25 (100101)	0x1 (0001)	0x01 (00000001)	0 (0)	001 (001)	0xA (1010)	10 (10)	0x0 (0000)	`0x944046A0`

Table 7.3: Short Immediate Format Encoding Examples

Notes:

- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 4 and 6)
- ST (Shift Type): see Table 9.1
- AF field for ALU: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 7), 11=reserved (N, Z, C and V architecturally undefined; the result is still written)
- Condition code examples: `0x0` = Always, `0x2` = Not Equal (!=), `0x5` = Negative Set (NS, N = 1)
- SHFT is a meta-instruction using ORRI opcode `0x25` with AF=10

7.6 Register Format

7.6.1 Structure

The register format allows operations on up to three registers with optional shift.

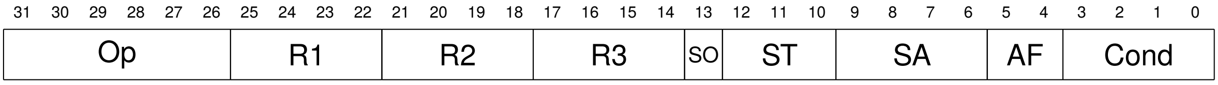


Figure 7.3: Register Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register 1 (bits 25–22) 4-bit destination register. For **`LDEA`** and **`LDEL`** (opcodes `0x18`–`0x1F`), this field instead holds a 2-bit address-register-pair index in bits 23–22 (00 = `l`, 01 = `a`, 10 = `s`, 11 = `p`) naming the destination pair, using the same encoding as the AF field. Bits 25–24 are reserved for these opcodes and are written as zero; a non-zero value is architecturally undefined.

Register 2 (bits 21–18) 4-bit source A register

Register 3 (bits 17–14) 4-bit source B register

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 9.1 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = `l` (link register)
- 01 = `a` (address register)
- 10 = `s` (stack pointer)
- 11 = `p` (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved; N, Z, C and V are architecturally undefined (the operation still completes and writes its result), and assemblers never emit this encoding

Condition (bits 3–0) 4-bit condition code

7.6.2 Register Operand Semantics

For most three-operand instructions:

- R1 = Destination
- R2 = First source operand
- R3 = Second source operand (optional)

Note: If only two register operands are given, the assembler uses R1 as both destination and first source operand when encoding the instruction:

```
ADDR r1, r2, r3 ; r1 = r2 + r3
ADDR r1, r2 ; r1 = r1 + r2 (R2 implicitly = R1)
```

7.6.3 Encoding Examples

Instruction	Op	R1	R2	R3	SO	ST	SA	AF	Cond	Hex
`ADDR r1, r2, r3`	0x30 (110000)	0x0 (0000)	0x1 (0001)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0xC0048010`
`ADDR r1, r2, r3, LSL #2`	0x30 (110000)	0x0 (0000)	0x1 (0001)	0x2 (0010)	0 (0)	001 (001)	0x2 (0010)	01 (01)	0x0 (0000)	`0xC0048490`
`LOAD r1, (r2, a)`	0x15 (010101)	0x0 (0000)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0x54004010`
`STOR -(r2, s), r1`	0x13 (010011)	0x0 (0000)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	10 (10)	0x0 (0000)	`0x4C004020`
`LDEA a, (r2, s)`	0x19 (011001)	0x1 (0001)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	10 (10)	0x0 (0000)	`0x64404020`
`LDEA a, -(r2, s)`	0x1B (011011)	0x1 (0001)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	10 (10)	0x0 (0000)	`0x6C404020`
`LDEL p, (r2, a)`	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0x74C04010`
`LDEL p, (r2, a)+`	0x1F (011111)	0x3 (0011)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0x7CC04010`
`LJMP a, r2`	0x19 (011001)	0x3 (0011)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0x64C04010`
`LJSR a, r2`	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x1 (0001)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0x74C04010`
`CMPR r4, r5`	0x3A (111010)	0x3 (0011)	0x3 (0011)	0x4 (0100)	0 (0)	000 (000)	0x0 (0000)	01 (01)	0x0 (0000)	`0xE8CD0010`
`XORR r2, r3, r4, RTR #1`	0x36 (110110)	0x1 (0001)	0x2 (0010)	0x3 (0011)	0 (0)	110 (110)	0x1 (0001)	01 (01)	0x0 (0000)	`0xD848D850`
`SUBR|» r1, r2, r3, ASR r4`	0x32 (110010)	0x0 (0000)	0x1 (0001)	0x2 (0010)	1 (1)	100 (100)	0x3 (0011)	01 (01)	0xD (1101)	`0xC804B0DD`
`ANDR|CS r5, r6, r7, LSL r1`	0x34 (110100)	0x4 (0100)	0x5 (0101)	0x6 (0110)	1 (1)	001 (001)	0x0 (0000)	01 (01)	0x3 (0011)	`0xD115A413`
`ADDR[N] r1, r2, r3`	0x30 (110000)	0x0 (0000)	0x1 (0001)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	00 (00)	0x0 (0000)	`0xC0048000`
`SUBR[S] r1, r2, r3, ASR r4`	0x32 (110010)	0x0 (0000)	0x1 (0001)	0x2 (0010)	1 (1)	100 (100)	0x3 (0011)	10 (10)	0x0 (0000)	`0xC804B0E0`

Table 7.4: Register Format Encoding Examples

Notes:

- When only two register operands are given, R2 is encoded as a copy of R1 and the second operand is encoded in R3, for example `ADDR r4, r5` uses R1=`0x3`, R2=`0x3`, R3=`0x4`
- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see the `SUBR|»` and `ANDR|CS` rows)
- ST (Shift Type): see Table 9.1

- AF for memory ops: 00=`l`, 01=`a`, 10=`s`, 11=`p`; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift, 11=reserved (N, Z, C and V architecturally undefined; the result is still written)
- Condition code examples: `0x0` = Always, `0xD` = Signed > (»), `0x3` = Carry Set (CS)

7.7 Operand Encoding Details

7.7.1 Condition Codes

The condition field occupies bits 3–0 of every instruction. The sixteen condition-code encodings and their meanings are tabulated once, in Table 10.1 of Chapter 10, which is the authoritative copy.

7.7.2 Shift Types

The shift-type field occupies bits 12–10 of the formats that carry one (see the field descriptions above). The shift-type encodings are tabulated once, in Table 9.1 of Chapter 9, which is the authoritative copy.

7.8 Opcode Encoding Patterns

The SIRCIS instruction set uses systematic opcode patterns to simplify decoding:

7.8.1 Opcode Groups

- **`0x0_`**: Immediate operand
- **`0x1_`**: Memory operations
- **`0x2_`**: Short immediate with shift
- **`0x3_`**: Register operand

7.8.2 Test vs. Save

This pattern applies to the ALU groups `0x0_`, `0x2_`, and `0x3_` only. Each ALU operation has both a "save result" and a "test only" variant:

- **`0x_0`–`0x_6`**: Save result to destination
- **`0x_8`–`0x_E`**: Test only (update flags, do not save the result)
- **`0x_F`**: Coprocessor call (**`COPI`**/**`COPR`**), not a test-only form

Adding `0x8` to a "save result" opcode in the range `0x_0`–`0x_6` gives its "test only" form. Opcode `0x_7` (**`MOVI`**/**`MOVR`**) has no test-only form. Opcodes `0x27` and `0x2F` are exceptions to this pattern: they are undocumented opcodes (see Appendix C), not a "save result" operation or a coprocessor call.

Operation	Save result	Test only	Test-only opcodes (Imm / Short / Reg)
Add	**`ADDI`** / **`ADDR`**	**`CMNI`** / **`CMNR`**	`0x08` / `0x28` / `0x38`
Add with carry	**`ADCI`** / **`ADCR`**	**`CNCI`** / **`CNCR`**	`0x09` / `0x29` / `0x39`
Subtract	**`SUBI`** / **`SUBR`**	**`CMPI`** / **`CMPR`**	`0x0A` / `0x2A` / `0x3A`
Subtract with carry	**`SBCI`** / **`SBCR`**	**`CPCI`** / **`CPCR`**	`0x0B` / `0x2B` / `0x3B`
AND	**`ANDI`** / **`ANDR`**	**`TSAI`** / **`TSAR`**	`0x0C` / `0x2C` / `0x3C`
OR	**`ORRI`** / **`ORRR`**	**`TSOI`** / **`TSOR`**	`0x0D` / `0x2D` / `0x3D`
XOR	**`XORI`** / **`XORR`**	**`TSXI`** / **`TSXR`**	`0x0E` / `0x2E` / `0x3E`

Table 7.5: Save and Test-Only ALU Instructions

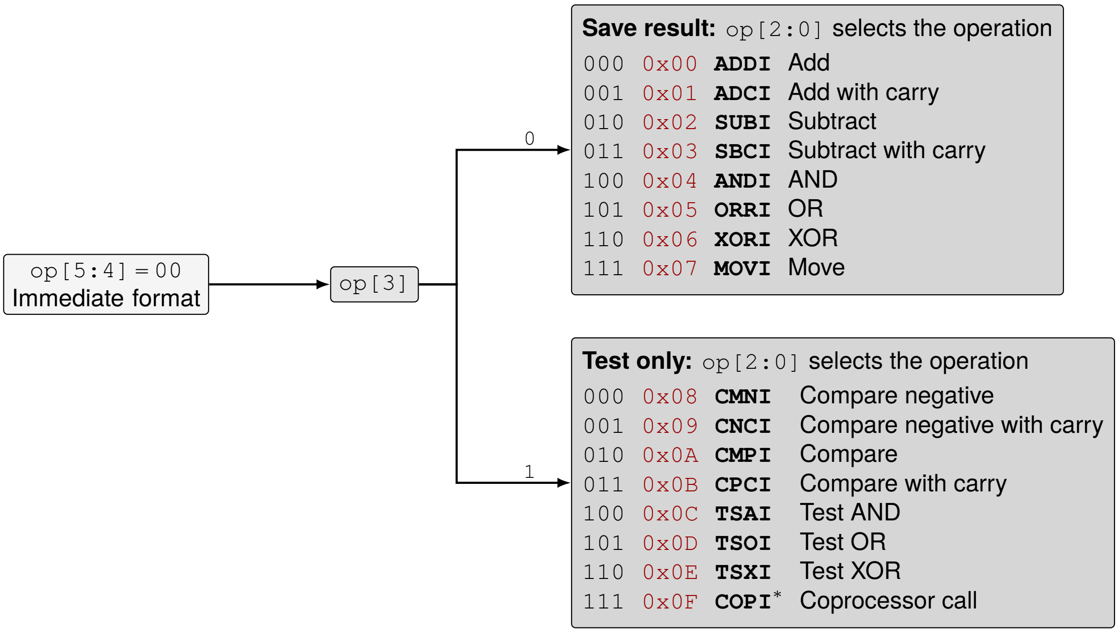
For example:

- `0x04`: **`ANDI`** – AND immediate, save result
- `0x0C`: **`TSAI`** – Test AND immediate (does not save the result)

7.9 Decode Trees

The opcode alone determines how the rest of a 32-bit instruction is decoded. The figures in this section show that process as a decision tree for each opcode group. In every figure the top strip is the instruction word, drawn to scale. The shaded boxes are the opcode bits consulted at each level of the tree, with the same shading on the matching tree node, and the unshaded boxes are operand fields, whose interpretation is given in the box below the tree. `op[n]` means opcode bit n, which is instruction bit n + 26. Appendix A lists every opcode in a single table.

5:4	3	2:0	Reg	Immediate	AF	Cond
31–30	29	28–26	25–22	21–6	5–4	3–0



Save result: `op[2:0]` selects the operation

000	0x00	**ADDI**	Add
001	0x01	**ADCI**	Add with carry
010	0x02	**SUBI**	Subtract
011	0x03	**SBCI**	Subtract with carry
100	0x04	**ANDI**	AND
101	0x05	**ORRI**	OR
110	0x06	**XORI**	XOR
111	0x07	**MOVI**	Move

Test only: `op[2:0]` selects the operation

000	0x08	**CMNI**	Compare negative
001	0x09	**CNCI**	Compare negative with carry
010	0x0A	**CMPI**	Compare
011	0x0B	**CPCI**	Compare with carry
100	0x0C	**TSAI**	Test AND
101	0x0D	**TSOI**	Test OR
110	0x0E	**TSXI**	Test XOR
111	0x0F	**COPI***	Coprocessor call

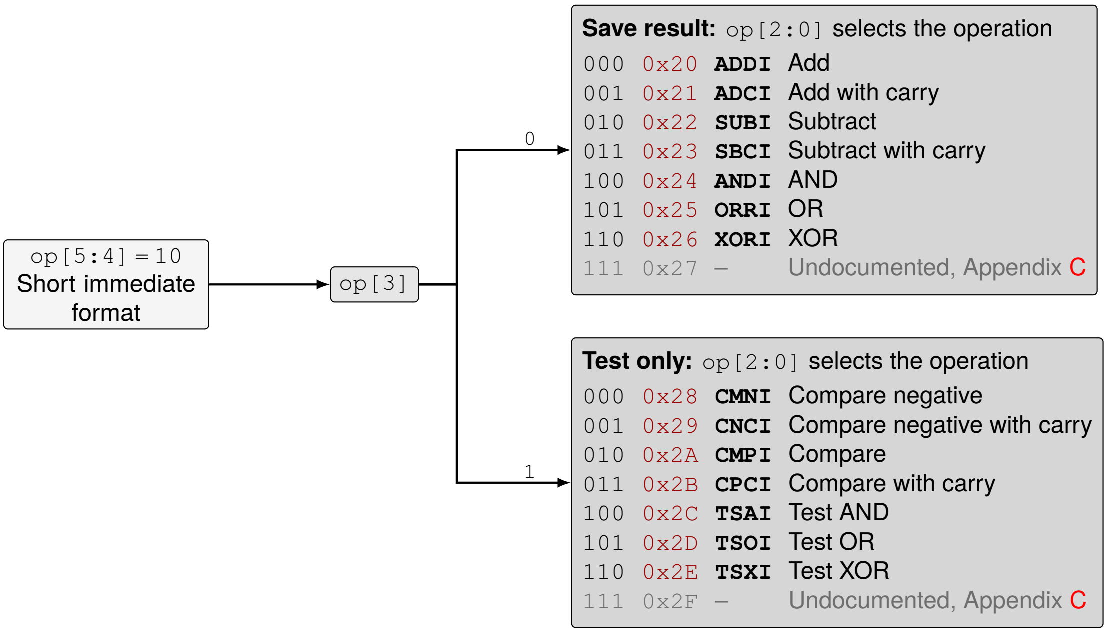
Operand fields (read the same way for every opcode above, unless noted)

Reg (25–22)	`rD`, the destination, for save forms; `rS`, the register tested, for test-only forms
Immediate (21–6)	16-bit value. This format has no shift fields
AF (5–4)	Status update source: `00` none, `01` ALU result, `10` shifter result, `11` reserved (N, Z, C and V architecturally undefined)
Cond (3–0)	One of 16 condition codes (Table 10.1)

*__COPI__ is a coprocessor call, not a test form; its fields are described in Chapter 16.

Figure 7.4: Decode tree for the immediate ALU group (`0x0_`)

5:4	3	2:0	Reg	Imm	SO	ST	SA	AF	Cond
31–30	29	28–26	25–22	21–14	13	12–10	9–6	5–4	3–0



Save result: `op[2:0]` selects the operation

000	0x20	**ADDI**	Add
001	0x21	**ADCI**	Add with carry
010	0x22	**SUBI**	Subtract
011	0x23	**SBCI**	Subtract with carry
100	0x24	**ANDI**	AND
101	0x25	**ORRI**	OR
110	0x26	**XORI**	XOR
111	0x27	–	Undocumented, Appendix C

Test only: `op[2:0]` selects the operation

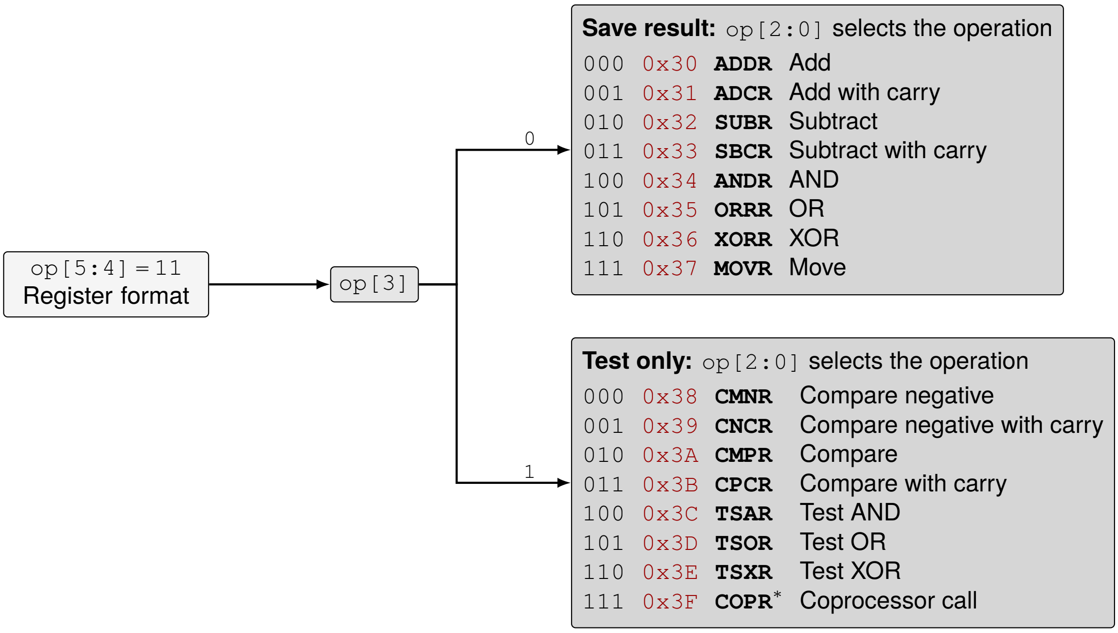
000	0x28	**CMNI**	Compare negative
001	0x29	**CNCI**	Compare negative with carry
010	0x2A	**CMPI**	Compare
011	0x2B	**CPCI**	Compare with carry
100	0x2C	**TSAI**	Test AND
101	0x2D	**TSOI**	Test OR
110	0x2E	**TSXI**	Test XOR
111	0x2F	–	Undocumented, Appendix C

Operand fields (read the same way for every opcode above, unless noted)

Reg (25–22)	`rD`, the destination, for save forms; `rS`, the register tested, for test-only forms
Imm (21–14)	8-bit unsigned value, zero-extended to 16 bits
SO, ST, SA (13–6)	`ST` selects the shift type (Table 9.1). If `SO` is 0, `SA` is the literal shift count; if 1, `SA` names the register holding the count. The shift applies to the register operand, never the immediate
AF (5–4)	Status update source: `00` none, `01` ALU result, `10` shifter result, `11` reserved (N, Z, C and V architecturally undefined)
Cond (3–0)	One of 16 condition codes (Table 10.1)

Figure 7.5: Decode tree for the short immediate ALU group (`0x2_`)

`5:4`	`3`	`2:0`	`R1`	`R2`	`R3`	`SO`	`ST`	`SA`	`AF`	`Cond`
31–30	29	28–26	25–22	21–18	17–14	13	12–10	9–6	5–4	3–0



Save result: `op[2:0]` selects the operation

`000`	`0x30`	**ADDR**	Add
`001`	`0x31`	**ADCR**	Add with carry
`010`	`0x32`	**SUBR**	Subtract
`011`	`0x33`	**SBCR**	Subtract with carry
`100`	`0x34`	**ANDR**	AND
`101`	`0x35`	**ORRR**	OR
`110`	`0x36`	**XORR**	XOR
`111`	`0x37`	**MOVR**	Move

Test only: `op[2:0]` selects the operation

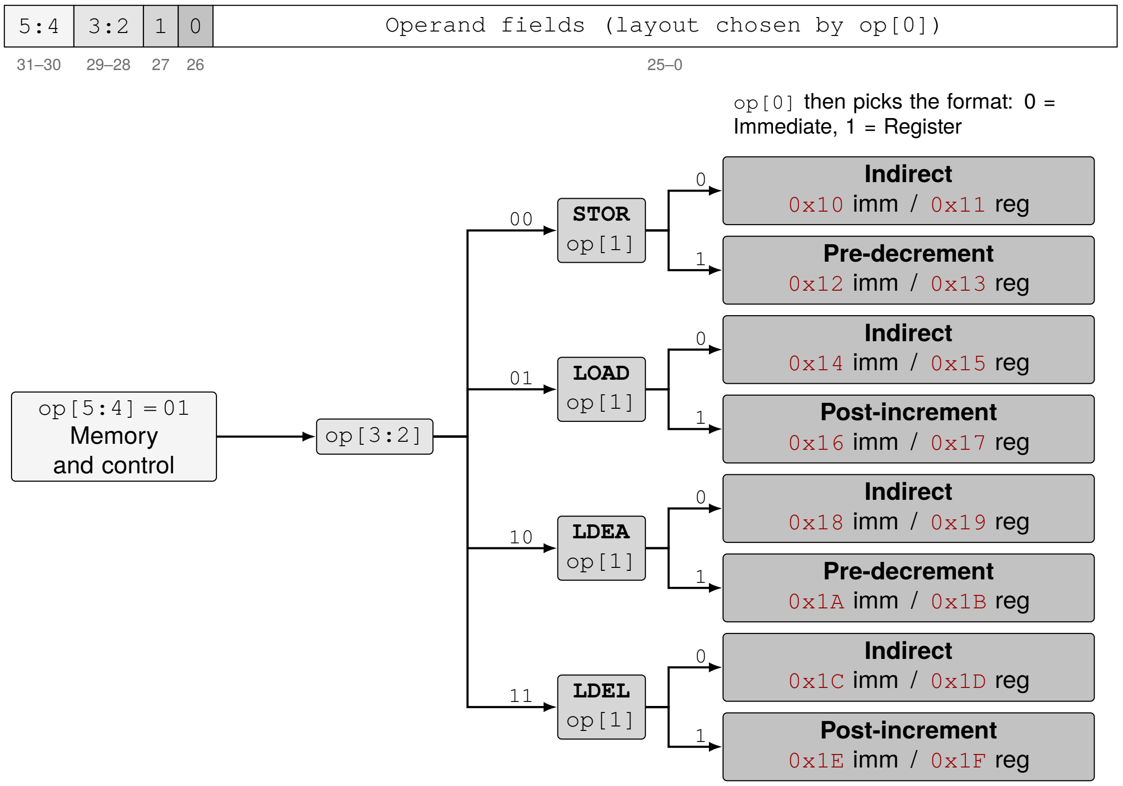
`000`	`0x38`	**CMNR**	Compare negative
`001`	`0x39`	**CNCR**	Compare negative with carry
`010`	`0x3A`	**CMPR**	Compare
`011`	`0x3B`	**CPCR**	Compare with carry
`100`	`0x3C`	**TSAR**	Test AND
`101`	`0x3D`	**TSOR**	Test OR
`110`	`0x3E`	**TSXR**	Test XOR
`111`	`0x3F`	**COPR***	Coprocessor call

Operand fields (read the same way for every opcode above, unless noted)

R1 (25–22)	`rD`, the destination, for save forms; ignored by test-only forms
R2 (21–18)	`rS1`, the left operand. **MOVR** does not use it
R3 (17–14)	`rS2`, the right operand (for **MOVR**, the source)
SO, ST, SA (13–6)	`ST` selects the shift type (Table 9.1). If `SO` is 0, `SA` is the literal shift count; if 1, `SA` names the register holding the count. The shift applies to `rS1`
AF (5–4)	Status update source: `00` none, `01` ALU result, `10` shifter result, `11` reserved (N, Z, C and V architecturally undefined)
Cond (3–0)	One of 16 condition codes (Table 10.1)

***COPR** is a coprocessor call, not a test form; its fields are described in Chapter 16.

Figure 7.6: Decode tree for the register ALU group (`0x3_`)


5:4
3:2
1
0
Operand fields (layout chosen by op[0])
31–30
29–28
27
26
25–0
op[0] then picks the format: 0 = Immediate, 1 = Register
op[5:4] = 01
Memory and control
op[3:2]
00
01
10
11
STOR op[1]
LOAD op[1]
LDEA op[1]
LDEL op[1]
0
1
Indirect
0x10 imm / 0x11 reg
Pre-decrement
0x12 imm / 0x13 reg
Indirect
0x14 imm / 0x15 reg
Post-increment
0x16 imm / 0x17 reg
Indirect
0x18 imm / 0x19 reg
Pre-decrement
0x1A imm / 0x1B reg
Indirect
0x1C imm / 0x1D reg
Post-increment
0x1E imm / 0x1F reg

Operand fields (chosen by `op[0]`)	**Immediate** (`op[0]` = 0)	**Register** (`op[0]` = 1)
25–22	`rD` for **LOAD**, `rS` for **STOR**. **LDEA** and **LDEL** put the destination pair in bits 23–22 (25–24 are zero)	As for the immediate format, except that **STOR** leaves them unused
21–6	16-bit displacement added to the address register pair	`rO`, the offset register (17–14). **STOR** also uses `rS` (21–18) and a shift of `rS` (`SO`, `ST`, `SA`); those fields are zero for the other three
5–4 (AF)	Address register pair used for the effective address: `00 l`, `01 a`, `10 s`, `11 p`	
3–0	Condition code	

Figure 7.7: Decode tree for the memory and control group (`0x1_`)

7.10 Instruction Fetch and Alignment

7.10.1 Fetch Process

Since instructions are 32 bits and the data bus is 16 bits wide, each instruction requires two fetch cycles:

1. Fetch the high word from the address in `pl`.
2. Fetch the low word from `pl` + 1.
3. After decode, `pl` advances by 2.

The address of the instruction is latched before `pl` advances. Every read of `p` and every address calculation that selects `p` uses that latched address, so they see the instruction's own address even though `pl` has already advanced by the time the address is calculated in phase 3. A link value is the latched address plus 2, the address of the next instruction.

7.10.2 Alignment Requirements

Instructions span two consecutive words and must begin at an even word address. The program counter (`pl`) must always be even.

Fetching an instruction from an odd word address raises an alignment fault.

7.11 Format Selection Guidelines

When writing assembly code, the assembler automatically selects the appropriate format based on operands:

Assembly	**Format Used**	**Notes**
`ADDI r1, #1000`	Immediate	Full 16-bit immediate
`ADDI r1, #10, LSL #2`	Short Immediate	8-bit immediate + shift
`ADDR r1, r2, r3`	Register	Three register operands

Table 7.6: Format Selection Examples

The choice of format affects instruction encoding but is generally transparent to the programmer, as the assembler handles the details.

CHAPTER 8

Addressing Modes

8.1 Overview

The SIRC-1 CPU supports seven addressing modes that specify how operands are accessed. These modes determine whether an operand is an immediate value, register contents, an address-register pair named directly, or data in memory at a computed address. The 8-bit short-immediate encoding used by ALU shift-and-add forms is an encoding-width variant of Immediate addressing, not a separate mode; Table 8.2 maps every operand form the assembler accepts onto one of the seven modes.

Mode	**Description**
Immediate	Operand is a constant value encoded in the instruction
Register Direct	Operand is the contents of a specified register
Address Register Direct	Operand names an address-register pair directly, with no indirection
Indirect Immediate	Memory address = address register + immediate displacement
Indirect Register	Memory address = address register + register displacement
Post-Increment	Indirect register, then increment address register
Pre-Decrement	Decrement address register, then use as indirect

Table 8.1: Addressing Modes Summary

Assembler operand form	**Example**	**Addressing mode**
Immediate	`#100`	Immediate
Immediate, 8-bit shift-qualified	`#10, LSL #2`	Immediate (encoding-width variant; see Short Immediate Addressing)
Register Direct	`r1`	Register Direct
Address Register Direct	`a`	Address Register Direct
Indirect Register Displacement	`(r2, a)`	Indirect Register
Indirect Immediate Displacement	`(#8, a)`, `(a)`	Indirect Immediate
Indirect Register Displacement, Post-Increment	`(r2, s)+`	Post-Increment
Indirect Immediate Displacement, Post-Increment	`(#2, s)+`, `(s)+`	Post-Increment
Indirect Register Displacement, Pre-Decrement	`-(r2, s)`	Pre-Decrement
Indirect Immediate Displacement, Pre-Decrement	`-(#2, s)`, `-(s)`	Pre-Decrement
Shift Definition	`LSL #2`	Not an addressing mode; modifies the register operand it follows (Chapter 9)

Table 8.2: Assembler Operand Forms Mapped to Addressing Modes

8.2 Addressing Mode Matrix

Mode	**Syntax**	**Effective value or address**	**Memory access**	**Legal families**
Immediate	`#imm16`, or `#imm8, shift` (short-immediate width variant)	16-bit encoded value; the 8-bit short-immediate form is zero-extended, and its shift applies to the register operand	None	ALU, MOVI, COPI, branch meta-instructions
Register Direct	`rN`	16-bit register value	None	ALU, MOVR, COPR, register-displacement forms
Address Register Direct	`addr`	`addr.high : addr.low`, used directly	None	LJMP/LJSR src, LDEA/LDEL dest
Indirect Immediate	`(#disp, addr)`, `(addr)`	`addr.high : (addr.low + disp)`	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Indirect Register	`(r0, addr)`	`addr.high : (addr.low + r0)`	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Post-Increment	`(#disp, addr)+`, `(addr)+`, `(r0, addr)+`	Address is calculated from original `addr`; `addr.low += 1` after access	LOAD reads; LDEL none	LOAD, LDEL, LJSR (assembles to LDEL)
Pre-Decrement	`-(#disp, addr)`, `-(addr)`, `-(r0, addr)`	`addr.low -= 1`; address then uses decremented `addr.low + disp`	STOR writes; LDEA none	STOR, LDEA (and LDEA-derived meta-instructions)

Table 8.3: Addressing Mode Side Effects and Legal Instruction Families

8.3 Legal Modes by Instruction Family

Table 8.3 answers "what does this addressing-mode syntax mean, and which instruction families can use it in general?" Table 8.4 answers the reverse question, "for this specific instruction, exactly which operand forms are legal?" The two tables must agree; consult the one that matches the question being asked.

Instruction family	Legal operand forms	Explicit exclusions and notes
ALU result instructions	`OPI rD, #imm16`; `OPI rD, #imm8, shift`; `OPR rD, rS1, rS2[, shift]`; register shorthand `OPR rD, rS2[, shift]` (encoded with R2 = rD, so the shift applies to rD).	No memory operands. Full 16-bit immediate forms do not encode shifts.
ALU test instructions	`CMNI/CNCI/CMPI/CPCI/TSAI/TSOI/TSXI rS, #imm16`; `CMNI/CNCI/CMPI/CPCI/TSAI/TSOI/TSXI rS, #imm8, shift`; `CMNR/CNCR/CMPR/CPCR/TSAR/TSOR/TSXR rS1, rS2[, shift]`.	No destination write-back. No memory operands.
ALU move instructions	`MOVI rD, #imm16`; `MOVR rD, rS`.	No public short-immediate **`MOVI`** syntax. Direct register-copy forms do not accept shift suffixes.
LOAD memory	`LOAD rD, (#disp, addr)`; `LOAD rD, (r0, addr)`; `LOAD rD, (#disp, addr)+`; `LOAD rD, (r0, addr)+`.	No pre-decrement LOAD form. No shift suffix on any **`LOAD`** form.
STOR memory	`STOR (#disp, addr), rS`; `STOR (r0, addr), rS[, shift]`; `STOR -(#disp, addr), rS`; `STOR -(r0, addr), rS[, shift]`.	No post-increment STOR form. Shift suffix is only available on register-displacement **`STOR`** forms.
LDEA	`LDEA dest, (#disp, src)`; `LDEA dest, (r0, src)`; `LDEA dest, -(#disp, src)`; `LDEA dest, -(r0, src)`.	No post-increment LDEA form. No shift suffixes.
LDEL	`LDEL dest, (#disp, src)`; `LDEL dest, (r0, src)`; `LDEL dest, (#disp, src)+`; `LDEL dest, (r0, src)+`.	No pre-decrement LDEL form. No shift suffixes.
Control-flow meta-instructions	`BRAN/BRSR #disp`; `BRAN/BRSR @label`; `RETS`; `LJMP src[, #disp|r0]`; `LJSR src[, #disp|r0]`; `LJSR (#disp, src)+`; `LJSR (r0, src)+`.	`LJMP (src)` and `LJSR (src)` are not public syntax; use `LJMP src` or `LJSR src` for zero displacement.
Coprocessor	`COPI #imm16`; `COPR rS`; standard coprocessor meta-instructions as documented in Chapter 16.	No hidden destination register syntax, no memory operands, no shifts, and no status-update override suffixes.

Table 8.4: Addressing Mode Legality by Instruction Family

8.4 Zero-Displacement Shorthand

Immediate-displacement indirect forms may omit the displacement when it is zero. The following forms are exact assembler-level equivalents:

Shorthand	Canonical form	Meaning
`(addr)`	`(#0, addr)`	Use the selected address register
`(addr)+`	`(#0, addr)+`	Use the address, then increment
`-(addr)`	`-(#0, addr)`	Decrement, then use the address

Table 8.5: Zero-Displacement Shorthand

The shorthand does not change which instruction families are legal. For example, `LOAD r1, (a)+` is legal because **`LOAD`** has a post-increment immediate-displacement form, but `STOR (a)+, r1` is not legal because **`STOR`** has no post-increment form. Likewise, `STOR -(s), r1` is legal because **`STOR`** has a pre-decrement form, but the corresponding pre-decrement **`LOAD`** form is not legal.

Register-displacement forms do not have a corresponding shorthand: `(r0, addr)`, `(r0, addr)+`, and `-(r0, addr)` always name the displacement register explicitly.

8.5 Operand Categories

Instruction descriptions use these operand categories:

Register value A 16-bit general-purpose, status, or address-component register operand used directly by an ALU, load, store, or coprocessor instruction.

Immediate value A constant encoded in the instruction. Full immediates are 16 bits; short immediates are 8 bits.

Memory source An effective address read by **`LOAD`**.

Memory destination An effective address written by **`STOR`**.

Branch target An effective address written to the program-counter pair `p`.

Address destination An address-register pair written by **`LDEA`** or **`LDEL`**.

Alterable destination Any destination register, address-register pair, or memory location that the instruction can modify when its condition is true.

8.6 Common Address Calculation Rules

For indirect, post-increment, pre-decrement, branch, jump, and effective-address forms, address arithmetic is performed on the low word of the selected address-register pair. The high word supplies the segment and is preserved unless the instruction explicitly writes an address-register pair destination.

Post-increment and pre-decrement change the selected address register by exactly one word. The displacement or index register participates only in the effective address calculation; it does not change the auto-update amount.

When `p` is used as the source address register, the value used for address calculation is the address of the instruction currently executing. PC-relative displacements are therefore relative to the address of the branch instruction itself, not to the next instruction; a displacement of 2 targets the following instruction and a displacement of 0 re-executes the branch. **`BRSR`** and **`LDEL`** still link to the instruction's own address + 2.

8.7 Per-Mode Encoding Reference

An addressing mode describes a single operand, not a whole instruction: `ADDR r1, r2, r3` has three Register Direct operands, and `ADDI r1, #100` has a Register Direct operand and an Immediate operand. Each box below gives the generation formula, assembler syntax, and field encoding for one operand of that mode, with examples shown in the context of a whole instruction. Which fields an instruction uses, and the role each operand plays (source, destination, or both), belong to the instruction format (Chapter 7) and to the instruction descriptions. Every SIRC-1 instruction is a fixed 32-bit (2-word) quantity regardless of the modes of its operands.

Immediate Addressing

Generation: Operand = the 16-bit value encoded directly in the instruction.
Assembler Syntax:

```
ADDI r1, #100 ; #100 is the immediate operand (16-bit)
ADDI r2, #-50 ; #-50 is the immediate operand (signed 16-bit)
```

Field Encoding: The Immediate field, bits 21–6 (16 bits), in the Immediate format; arithmetic instructions interpret this field according to Chapter 4. This mode uses only that field; the register named alongside it is a separate Register Direct operand.
Usage: Loading constant values; arithmetic with known values; bit manipulation with masks.

Register Direct Addressing

Generation: Operand = the 16-bit value currently stored in the specified register. This is the most common mode for ALU operations in a load/store architecture.
Assembler Syntax:

```
ADDR r1, r2, r3 ; three Register Direct operands: r1 = r2 + r3
ADDR r4, r5 ; two Register Direct operands: r4 = r4 + r5
```

Field Encoding: A 4-bit register number. In the Immediate and Short Immediate formats the single register operand is the Register field, bits 25–22. In the Register format the operands are Register 1, bits 25–22, Register 2, bits 21–18, and Register 3, bits 17–14. The role of each (source, destination, or both) is set by the instruction, not by the mode.
Usage: All ALU operations between registers; register-to-register data movement; testing register values.

Address Register Direct Addressing

Generation: Operand = the full 32-bit contents (high word and low word) of the named address-register pair (`l`, `a`, `s`, or `p`), with no indirection. **`LDEA`** and **`LDEL`** use this form to select the destination pair that receives the computed address; **`LJMP`** and **`LJSR`** use it to select the source pair whose contents become the new value of `p`.
Assembler Syntax:

```
LDEA a, (#4, s) ; a = s + 4 (destination named directly)
LJMP a ; p = a (source named directly)
```

Field Encoding: For **`LDEA`**/**`LDEL`** (opcodes `0x18`–`0x1F`), a 2-bit address-register-pair index in bits 23–22 of the Register (Immediate format) or R1 (Register format) field (00 = l, 01 = a, 10 = s, 11 = p). Bits 25–24 are reserved and always written as zero. **`LJMP`**/**`LJSR`** are meta-instructions that lower to this same encoding with `p` in bits 23–22; their source pair, also named directly, is encoded in AF, bits 5–4.
Usage: Selecting which address-register pair an effective-address instruction writes; jumping or calling through an address already held in a register pair.

Indirect Immediate Addressing

Generation:

$$\mathrm{EA} = \mathrm{AddressRegister.high} : (\mathrm{AddressRegister.low} + \mathrm{Displacement})$$

The address register can be `l` (link), `a` (address), `s` (stack), or `p` (program counter). Displacement is a signed two's-complement value. When `SR.A` (Trap on Address Overflow) is set, the Segment Overflow Fault (Chapter 6) fires only if the low-word addition wraps; a negative displacement that keeps the sum within the low word's range does not fault. For the post-increment and pre-decrement forms, the address-register auto-update and the effective address are checked separately, and a wrap of either raises the Segment Overflow Fault.
Assembler Syntax:

```
LOAD r1, (#8, a) ; Load from address (a + 8)
STOR (#-4, s), r2 ; Store to address (s - 4)
LDEA p, (#100, p) ; Jump relative (pc = pc + 100)
```

Field Encoding: The base address-register pair is a 2-bit index in AF, bits 5–4. The displacement is a 16-bit signed value in the Immediate field, bits 21–6, so this mode is carried in the Immediate format (even opcodes `0x10`–`0x1F`).

Usage: Accessing structure fields (fixed offset from base); array element access (constant index); stack frame access (local variables); position-independent code (PC-relative addressing).

Indirect Register Addressing

Generation:

$$\text{EA} = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{IndexRegister})$$

Identical to Indirect Immediate Addressing except the displacement is taken from a register at runtime instead of being encoded as an immediate value.
Assembler Syntax:

```
LOAD r1, (r2, a) ; Load from address (a + r2)
STOR (r4, s), r3 ; Store to address (s + r4)
```

Field Encoding: The base address-register pair is a 2-bit index in AF, bits 5–4. The index register is a 4-bit register number in R3, bits 17–14, so this mode is carried in the Register format (odd opcodes `0x10`–`0x1F`).
Usage: Array indexing with a variable index; table lookups; dynamic address calculations.

Post-Increment Addressing

Generation:

1. Compute EA = AddressRegister.high : (AddressRegister.low + Displacement)
2. Perform the memory operation (load)
3. AddressRegister.low = AddressRegister.low + 1

Particularly useful for iterating through arrays or implementing stack operations.
Assembler Syntax:

```
LOAD r1, (r2, s)+ ; Load from (s + r2), then s = s + 1
LOAD r1, (#2, s)+ ; Load from (s + 2), then s = s + 1
```

Field Encoding: Selected by the opcode itself, not a separate operand field: opcode bit 1 = 1 selects auto-update, and bit 0 selects an immediate (`0x16`) or register (`0x17`) displacement.
Usage: Stack pop operations; forward array traversal; buffer reading.

Pre-Decrement Addressing

Generation:

1. AddressRegister.low = AddressRegister.low - 1
2. Compute EA = AddressRegister.high : (AddressRegister.low + Displacement)
3. Perform the memory operation (store)

The complement of post-increment; useful for stack push operations.
Assembler Syntax:

```
STOR -(r2, s), r1 ; s = s - 1, then store to (s + r2)
STOR -(#2, s), r3 ; s = s - 1, then store to (s + 2)
```

Field Encoding: Selected by the opcode itself, not a separate operand field: opcode bit 1 = 1 selects auto-update, and bit 0 selects an immediate (`0x12`) or register (`0x13`) displacement.
Usage: Stack push operations; backward array traversal; buffer writing.

Short Immediate Addressing

Generation: Operand = an 8-bit value encoded in the instruction, zero-extended to 16 bits before ALU arithmetic. This is a width variant of Immediate addressing, not a separate addressing mode; see Table 8.2. The shift applies to the register operand, not to the immediate constant.
Assembler Syntax:

```
ADDI r1, #10, LSL #2 ; #10 is the short immediate: r1 = (r1 << 2) + 10
SUBI r2, #1, LSL #8 ; #1 is the short immediate: r2 = (r2 << 8) - 1
```

Field Encoding: The Immediate field, bits 21–14 (8 bits), in the Short Immediate with Shift format. The shift fields belong to the shift definition that qualifies the register operand, not to this operand.
Usage: Shift-and-add or shift-and-mask in one instruction, for example scaling a register and adding a small constant.

8.8 Address Register Selection

Many addressing modes require specifying which address register pair to use. The 2-bit address register field encodes:

Bits	Symbol	Register Pair
00	l	Link Register (lh, ll)
01	a	Address Register (ah, al)
10	s	Stack Pointer (sh, sl)
11	p	Program Counter (ph, pl)

Table 8.6: Address Register Encoding

8.9 Addressing Mode Examples

```
; Assume 'a' points to a struct:
; struct Point {
; word 0: int16_t x
; word 1: int16_t y
; }

LOAD r1, (#0, a) ; r1 = point.x
LOAD r2, (#1, a) ; r2 = point.y
```

Example 8-1: Structure Access

```
; Iterate array of 16-bit values
; Assume 'a' points to array start
; r7 = counter
:loop
  LOAD r1, (#0, a)+ ; Load element, advance by 1
  ; ... process r1 ...
  SUBI r7, #1
  BRAN|!= @loop
```

Example 8-2: Array Iteration

```
:begin_subroutine

; Function prologue - save registers
STOR -(#0, s), r1
STOR -(#0, s), r2
STOR -(#0, s), r3

; Access parameters (above saved regs)
LOAD r4, (#3, s) ; First parameter
LOAD r5, (#4, s) ; Second parameter

; Function epilogue - restore registers
LOAD r3, (#0, s)+
LOAD r2, (#0, s)+
LOAD r1, (#0, s)+
RETS
```

Example 8-3: Stack Frame

8.10 Addressing Mode Restrictions

8.10.1 Privilege Mode Restrictions

Protected mode restricts writes to the high word of an address register:

- **Direct high-register writes**: Trigger a privilege violation fault
- **Address-register-pair write-back**: Allowed only when the high word would remain unchanged; otherwise triggers a privilege violation fault
- **Post-increment/pre-decrement**: Always allowed in protected mode; the auto-update writes only the low word of the pair and never carries into the high word

In every mode, the auto-update forms must not name `p` as the address-register pair, because the update would compete with the program-counter advance. Such an encoding is architecturally undefined and the assembler rejects it (Appendix C).

8.10.2 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address
- Unaligned instruction fetches (odd word address) trigger an alignment fault exception

CHAPTER 9

Shift Operations

9.1 Overview

The SIRC-1 CPU supports shift operations that can be applied to the first source operand in many instructions. Shifts can multiply or divide values by powers of 2, extract bit fields, or perform bit rotations.

Shift operations are encoded within Short Immediate and Register format instructions, allowing a value to be shifted before being used in an ALU operation—all within a single instruction.

Shift operations are only applied to source operands. A shift cannot be applied to the result of an operation before register writeback.

Note: By default, when shifts are used with ALU instructions (apart from the `SHFT` meta-instruction), the status register flags are updated based on the ALU operation result, not the shift. This behavior can be overridden using the status override syntax (see Section 9.12).

9.2 Shift Types

9.2.1 Shift Type Encoding

Code	Mnemonic	Type	Description
000	NUL	–	No shift performed; any count is ignored
001	LSL	Logical Shift Left	Shift left, fill with zeros
010	LSR	Logical Shift Right	Shift right, fill with zeros
011	ASL	Arithmetic Shift Left	Same data result as LSL; sets V on signed overflow
100	ASR	Arithmetic Shift Right	Shift right, preserving sign bit
101	RTL	Rotate Left	Rotate left within the operand
110	RTR	Rotate Right	Rotate right within the operand
111	–	Reserved	Reserved for future use

Table 9.1: Shift Type Encoding

`NUL` is the no-shift form: it leaves the operand unchanged and is the default when no shift is specified.

9.3 Shift Syntax

A shift suffix may follow the ALU short-immediate and ALU register forms, the `SHFT` meta-instruction, and the register-displacement `STOR` forms. It is rejected on `MOVR rD, rS`, on `LDEA`/`LDEL`, and on coprocessor instructions.

For ALU instructions, the shift is applied in the "Decode and Register Fetch" phase, before the ALU operation. In register-format instructions the shifted operand is R2 (first source); in short-immediate instructions it is the register named by the Reg field (first source, since the short-immediate format has no separate source register). In register-displacement `STOR` forms it is applied to the source register before its value is written to memory. **LOAD** has no shift: its shift fields are reserved (see Chapter 14).

In the three-operand form of an instruction, the shifted operand is the first source operand. In the two-operand form, the first source operand is inferred to be the same as the destination.

See this table for some examples of which operand is shifted:

Instruction	How the CPU Interprets It	First Source Operand
ADDR r1, r2, r3, LSL #1	ADDR r1, r2, r3, LSL #1	r2
ADDR r1, r2, LSL #1	ADDR r1, r1, r2, LSL #1	r1
ADDI r1, #2, LSL #1	ADDI r1, r1, #2, LSL #1	r1

Table 9.2: Examples of First Source Operands

9.4 Shift Type Details

9.4.1 Logical Shift Left (LSL)

Operation: Shift bits to the left, filling vacated bits with zeros.

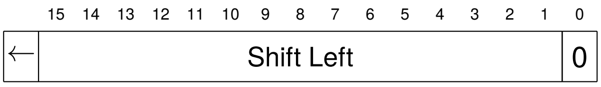


Figure 9.1: Logical Shift Left

- The last bit shifted out of bit 15 (bit 16-n for a count of n) is placed in the carry flag
- Bit 0 receives 0
- Equivalent to multiplying by 2^n where n is the shift count

```
; multiply r1 by 4
ADDI r1, #0, LSL #2 ; r1 = (r1 << 2) = r1 * 4

; set bit 10 (16-bit immediate form)
ORRI r3, #0x0400 ; r3 |= 0x0400
```

Example 9-1: Constant Multiply and Bit Set (LSL)

9.4.2 Logical Shift Right (LSR)

Operation: Shift bits to the right, filling vacated bits with zeros.

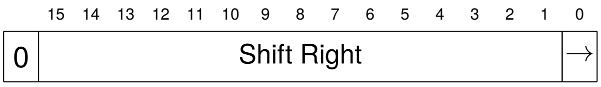


Figure 9.2: Logical Shift Right

- The last bit shifted out of bit 0 (bit n-1 for a count of n) is placed in the carry flag
- Bit 15 receives 0
- Equivalent to unsigned division by 2^n where n is the shift count
- Should be used for unsigned values only

```
; divide unsigned value by 8
ADDI r1, #0, LSR #3 ; r1 = (r1 >> 3) = r1 / 8 (unsigned)

; extract high byte
ADDI r3, #0, LSR #8 ; r3 = (r3 >> 8)
```

Example 9-2: Unsigned Divide and Byte Extraction (LSR)

9.4.3 Arithmetic Shift Left (ASL)

Operation: Produces the same data result as LSL, but sets the overflow flag if the sign bit changes.

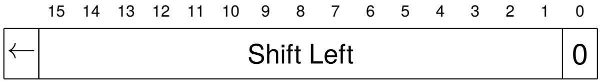


Figure 9.3: Arithmetic Shift Left

- Equivalent to signed multiplication by 2^n
- Can cause signed overflow if sign bit changes
- Overflow flag is set if sign bit changes during shift

```
; multiply signed value by 2
ADDI r1, #0, ASL #1 ; r1 = (r1 << 1) = r1 * 2 (signed)
```

Example 9-3: Signed Multiply by 2 (ASL)

9.4.4 Arithmetic Shift Right (ASR)

Operation: Shift bits to the right, preserving the sign bit (bit 15).

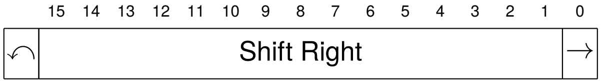


Figure 9.4: Arithmetic Shift Right

- The last bit shifted out of bit 0 (bit n-1 for a count of n) is placed in the carry flag
- Bit 15 (sign bit) is replicated into all bits vacated by the shift
- Equivalent to signed division by 2^n, except that the result is rounded down, not toward zero: -3 shifted right by 1 gives -2, whereas **`DIVS`** gives -1 for $-3/2$
- Should be used for signed values only

```
; divide signed value by 4
ADDI r1, #0, ASR #2 ; r1 = (r1 >> 2) = r1 / 4 (signed)

; average two signed values
ADDR r3, r4, r5
ADDI r3, #0, ASR #1 ; r3 = (r4 + r5) / 2, if r4 + r5 does not overflow
```

Example 9-4: Signed Divide and Averaging (ASR)

9.4.5 Rotate Left (RTL)

Operation: Rotate bits to the left within the 16-bit operand. The last bit to wrap from bit 15 to bit 0 (original bit 16-n for a count of n) is copied into the carry flag. The incoming carry flag is not used as an input.

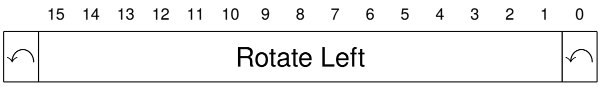


Figure 9.5: Rotate Left

- Bit 15 wraps into bit 0. The last bit to wrap from bit 15 to bit 0 (original bit 16-n for a count of n) is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost
- Useful for bit manipulation, circular buffers, and cyclic bit patterns

```
; rotate r1 left by 1
SHFT r1, RTL #1

; circular rotate of one word
SHFT r3, RTL #4
```

Example 9-5: Rotating a Value Left (RTL)

9.4.6 Rotate Right (RTR)

Operation: Rotate bits to the right within the 16-bit operand. The last bit to wrap from bit 0 to bit 15 (original bit n-1 for a count of n) is copied into the carry flag. The incoming carry flag is not used as an input.

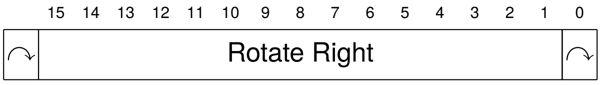


Figure 9.6: Rotate Right

- Bit 0 wraps into bit 15. The last bit to wrap from bit 0 to bit 15 (original bit n-1 for a count of n) is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost

```
; rotate r1 right by 1
SHFT r1, RTR #1

; check if bit 0 is set
SHFT r2, RTR #1 ; bit 0 -> carry
; now branch on carry to test the old bit 0
```

Example 9-6: Rotating a Value Right and Testing a Bit (RTR)

9.5 Shift Operands

The Shift Operand (SO) bit determines what value is shifted:

SO Bit	Mode	Description
0	Shift by Immediate	The first source operand (Register format: R2; Short Immediate format: the Reg field) is shifted by a literal amount
1	Shift Count in Register	The first source operand (Register format: R2; Short Immediate format: the Reg field) is shifted by the amount in the shift count register

Table 9.3: Shift Operand Modes

9.5.1 Mode 0: Shift by Immediate

The first source operand (R2) is shifted by a literal amount before the ALU operation.

```
; multiply r1 by 4 and add 1
ADDI r1, #1, LSL #2 ; r1 = (r1 << 2) + 1
                       ; SO=0: r1 is shifted by literal 2
```

9.5.2 Mode 1: Shift by Register Count

The shift count is taken from the shift amount field (interpreted as a register), allowing variable-length shifts. R2 (first source) is still what gets shifted.

```
; variable shift
; shift amount register contains shift count
ADDR r1, r2, r3, LSL r4 ; r1 = (r2 << r4) + r3
                       ; SO=1: shift r2 by value in r4

; barrel shifter emulation
ADDR r4, r5, r6, ASR r7 ; r4 = (r5 >> r7) + r6 (arithmetic)
```

9.6 Shift Count

The shift count is 0–15 when specified as an immediate (SO = 0); the field is 4 bits wide, and both the assembler and the encoder enforce this range, so a count greater than 15 can never occur through the immediate path. When the shift count is taken from a register (SO = 1), the full 16-bit register value is available as a count, and a count greater than 15 is well defined:

- Shift count of 0 means no shift is performed
- Maximum immediate shift count is 15
- For **`LSL`**, **`LSR`**, and **`ASL`**, a register-supplied count of 16 or more produces the same result as a count of exactly 16: every original bit is shifted out, giving `0x0000`. The carry flag reflects the last bit shifted out at count 16, as for any other count.
- For **`ASR`**, a register-supplied count of 16 or more sign-extends fully: `0xFFFF` if the original value was negative, `0x0000` if it was positive or zero. This is the same rule as any other count, just carried to its limit: every vacated bit is the sign bit. The carry flag is the original bit 15, since every bit shifted out at that point is a copy of the sign.

- **`RTL`** and **`RTR`** have no vacated bits to define, since a rotate never discards data; a register-supplied count is used modulo 16, so a count of 16 behaves as a count of 0, a count of 17 behaves as a count of 1, and so on.

9.7 Status Flag Effects

9.7.1 Default Behavior

When shift operations are used with ALU instructions, the status flags are **by default** updated based on the **ALU operation result**, not the shift operation. For example, in `CMPI r2, #1, LSL #2`, the flags reflect the comparison result, not the shift.

To update flags based on the shift operation instead, use the `[S]` status override modifier (see Section 9.12), or use the **`SHFT`** meta-instruction documented in Chapter 13.

9.7.2 Carry Flag

For all shifts and rotates:

- The last bit shifted out is placed in the carry flag
- For shifts of more than 1 position, only the final bit shifted out matters
- A shift count of 0 leaves the operand unchanged and clears the carry flag. `NUL` ignores its count, immediate or register, and always behaves as a count of 0. When the status register is updated from the shifter, all four condition flags are written, so no flag survives a shift-sourced update; to preserve flags across a shift, use the `[N]` status override.

9.7.3 Zero Flag

Set if the result after shifting is zero.

9.7.4 Negative Flag

Set if bit 15 of the result is 1 (result is negative when interpreted as signed).

9.7.5 Overflow Flag

- For **ASL**: Set if bit 15 changes at any point during the shift (signed overflow); that is, for a count n from 1 to 15, set unless the original bits 15 down to $15 - n$ are all equal. A count of 0 clears V. For a register-supplied count of 16 or more, V is set unless the original value was `0x0000`
- For other shifts: Cleared to 0

9.8 Common Use Cases

```
; multiply by 2
ADDI r1, #0, LSL #1 ; r1 = (r1 << 1) = r1 * 2

; multiply by 3
ADDR r1, r2, r2, LSL #1 ; r1 = (r2 << 1) + r2 = r2 * 3

; multiply by 5
ADDR r1, r2, r2, LSL #2 ; r1 = (r2 << 2) + r2 = r2 * 5
```

Example 9-7: Multiplication by Constants

```
; unsigned divide by 4
MOVR r1, r2 ; r1 = r2
ADDI r1, #0, LSR #2 ; r1 = (r1 >> 2) = r1 / 4 (unsigned)

; signed divide by 8
MOVR r1, r2 ; r1 = r2
ADDI r1, #0, ASR #3 ; r1 = (r1 >> 3) = r1 / 8 (signed)
```

Example 9-8: Division by Powers of 2

```
; extract bits 8-11 from r1
MOVR r2, r1 ; r2 = r1
ADDI r2, #0, LSR #8 ; shift down
ANDI r2, #0x0F ; mask to 4 bits

; extract and sign-extend bit 7
MOVR r2, r1 ; r2 = r1
ADDI r2, #0, LSL #8 ; shift bit 7 to bit 15
ADDI r2, #0, ASR #8 ; arithmetic shift to sign-extend
```

Example 9-9: Bit Field Extraction

```
; set bit N (where N is in r2)
MOVI r3, #1 ; r3 = 1
ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
ORRR r1, r3 ; r1 |= (1 << N)

; clear bit N
MOVI r3, #1
ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
MOVI r4, #0xFFFF
XORR r3, r4, r3 ; r3 = 0xFFFF ^ (1 << N) = ~(1 << N)
ANDR r1, r1, r3 ; r1 = r1 & ~(1 << N)

; test bit N
MOVI r3, #1
ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
TSAR r1, r3 ; test r1 & (1 << N)
BRAN|!= @bit_was_set ; branch if bit set
```

Example 9-10: Bit Manipulation

9.9 Multi-Word Shifts

For operations on values larger than 16 bits, shifts and rotates can be combined:

```
; shift 32-bit value in r1:r2 left by 1
; r1 = high word, r2 = low word
SHFT r2, LSL #1 ; shift low, MSB -> carry
ADCI r1, #0, LSL #1 ; shift high and add carry into bit 0
```

Example 9-11: 32-bit Left Shift

```
; shift 32-bit value in r1:r2 right by 1 (unsigned)
SHFT r1, LSR #1 ; shift high, LSB -> carry
ORRI[N] r2, #0, LSR #1 ; shift low while preserving carry from high word
ORRI|CS r2, #0x8000 ; if old high bit 0 was set, insert it into low bit 15
```

Example 9-12: 32-bit Right Shift (Unsigned)

```
; shift 32-bit value in r1:r2 right by 1 (signed)
SHFT r1, ASR #1 ; arithmetic shift high, LSB -> carry
ORRI[N] r2, #0, LSR #1 ; shift low while preserving carry from high word
ORRI|CS r2, #0x8000 ; if old high bit 0 was set, insert it into low bit 15
```

Example 9-13: 32-bit Right Shift (Signed)

9.10 Performance Considerations

- All shift operations complete in the same number of cycles regardless of shift count
- Shifts can be performed "for free" as part of another instruction (no extra cycles)
- Using shifts for multiplication or division by powers of 2 avoids a call to the optional maths coprocessor (**`MULU`**, **`DIVU`**, Chapter 16)
- Barrel shifter implementation in hardware allows any shift count in constant time

9.11 Limitations

- The immediate shift count is limited to 0–15 (4-bit field)
- For an immediate shift greater than 15 positions, use multiple shift instructions with immediate counts
- Rotates do not consume the carry flag; **`RTL`** and **`RTR`** rotate only within the 16-bit operand
- The reserved shift type (111) has architecturally undefined behavior

9.12 Status Flag Update Control

By default, ALU instructions with shift operations update the status register flags based on the **ALU operation result**, not the shift operation. However, the SIRC-1 provides manual control over which operation updates the flags.

9.12.1 Status Override Syntax

A status override modifier can be added immediately after an instruction mnemonic using square brackets:

Modifier	Meaning	Description
[A]	ALU	Update flags from ALU result (default except for the **`SHFT`** meta-instruction)
[S]	Shift	Update flags from shift result, not ALU (default for the **`SHFT`** meta-instruction)
[N]	None	Do not update status flags at all

Table 9.4: Status Flag Override Modifiers

9.12.2 Examples

```
; without override, flags reflect the ALU addition result
ADDI r2, #1, LSL #2 ; r2 = (r2 << 2) + 1
                           ; flags: based on addition result
```

Example 9-14: Default Behavior (ALU Flags)

```
; update flags based on shift result instead of ALU
ADDI[S] r2, #1, LSL #2 ; r2 = (r2 << 2) + 1
                           ; flags: based on (r2 << 2), not the +1

; check if shifted value is zero
CMPI[S] r3, #0, LSR #4 ; compare (r3 >> 4) with 0
                           ; flags: based on shift, not comparison
```

Example 9-15: Shift Flag Override

```
; perform operation without affecting status flags
ADDI[N] r1, #5, LSL #1 ; r1 = (r1 << 1) + 5
                           ; flags: unchanged

; useful when preserving flags from previous operation
CMPR r4, r5 ; set flags based on comparison
ADDI[N] r6, #1 ; increment r6 without changing flags
BRAN|== @equal_branch ; branch using flags from CMPR
```

Example 9-16: No Flag Update

9.12.3 Use Cases

Testing Shift Results

To determine properties of a shifted value without performing a subsequent comparison:

```
; check if r1 << 3 would be negative
TSOI[S] r1, #0, LSL #3 ; flags from r1 << 3, r1 unchanged
BRAN|NS @shifted_negative ; branch if sign bit set
```

Preserving Flags

When performing operations that should not affect condition codes:

```
; test condition
CMPR r1, r2 ; compare r1 and r2

; do some work without affecting flags
ADDI[N] r3, #1 ; increment counter (flags unchanged)
LOAD r4, (#0, a)+ ; load next value (LOAD never updates flags)

; branch using original comparison flags
BRAN|>= @r1_was_greater_or_equal
```

SHFT Meta-Instruction

The **SHFT** meta-instruction is the ALU-family shorthand for this common case:

```
SHFT r1, LSL #3 ; ORRI[S] r1, #0, LSL #3
```

See Chapter 13 for the full **SHFT** instruction entry.

9.12.4 Assembler Syntax Notes

- The status override modifier must appear immediately after the instruction mnemonic with no spaces: `ADDI[S]`
- The modifier is optional; omitting it defaults to [A] (ALU flag updates) for most instructions except **SHFT**.
- The override selects the flag source for the whole instruction. If the condition is false, the instruction has no effect and no flags change, whatever the override
- `AF = 10` is still encodable on the 16-bit immediate format, but with no shift fields the shifter result is treated as zero flags, so `[S]` clears N, Z, C and V (Chapter 13)
- Explicit [A] is rarely needed but can improve code readability

CHAPTER 10

Condition Codes

10.1 Overview

Every SIRC-1 instruction can be executed conditionally based on the state of the condition flags in the status register. This allows for efficient implementation of conditional logic without requiring explicit branch instructions.

The 4-bit condition code field in each instruction specifies the conditions under which the instruction executes. If the condition is not met, the instruction has no architectural effect beyond the normal two-word advance of `p`: no general-purpose register, flag, memory, address-register or coprocessor state changes, and no privilege check is performed. The advance itself is still checked: with `SR.A` set, a skipped instruction in the last instruction slot of a segment raises a segment-overflow fault (Section 6.2.5).

10.2 Condition Code Encoding

Code	Mnemonic	Condition (when instruction executes)
0000	AL (or none)	Always (unconditional)
0001	==	Equal (Z = 1)
0010	!=	Not Equal (Z = 0)
0011	CS	Carry Set (C = 1)
0100	CC	Carry Clear (C = 0)
0101	NS	Negative Set (N = 1)
0110	NC	Negative Clear (N = 0)
0111	OS	Overflow Set (V = 1)
1000	OC	Overflow Clear (V = 0)
1001	HI	Unsigned Higher (C = 0 AND Z = 0)
1010	LO	Unsigned Lower or Same (C = 1 OR Z = 1)
1011	>=	Signed Greater or Equal (N = V)
1100	«	Signed Less Than (N $\neq$ V)
1101	»	Signed Greater Than (Z = 0 AND N = V)
1110	<=	Signed Less or Equal (Z = 1 OR N $\neq$ V)
1111	NV	Never (never executes)

Table 10.1: Condition Code Encoding

10.3 Condition Code Categories

10.3.1 Simple Conditions

These conditions test a single status flag:

Always (default) No condition code specified; instruction always executes.

== (Equal) Executes when Z = 1. Typically used after **`CMPI`** or **`CMPR`** to test equality.

!= (Not Equal) Executes when Z = 0. Typical use: test inequality.

CS (Carry Set) Executes when C = 1. The Carry flag's meaning depends on which instruction last set it: after an add (**`ADDR`**, **`ADDI`**, and so on), C = 1 means an unsigned overflow occurred. After a subtraction or a compare (**`CMPI`**, **`CMPR`**), C = 1 means a *borrow* occurred – the first operand was unsigned lower than the second.

CC (Carry Clear) Executes when C = 0. After an add, C = 0 means no unsigned overflow occurred. After a subtraction or a compare (**`CMPI`**, **`CMPR`**), C = 0 means no borrow occurred: the first operand was unsigned higher than or equal to the second.

NS (Negative Set) Executes when N = 1. Tests if result is negative (signed).

NC (Negative Clear) Executes when N = 0. Tests if result is positive or zero (signed).

OS (Overflow Set) Executes when V = 1. Indicates signed overflow.

OC (Overflow Clear) Executes when V = 0. No signed overflow.

10.3.2 Unsigned Comparisons

These conditions are used after comparing unsigned integers:

HI (Unsigned Higher) Executes when C = 0 AND Z = 0. Tests if first operand $>$ second operand (unsigned).

LO (Unsigned Lower or Same) Executes when C = 1 OR Z = 1. Tests if first $\leq$ second (unsigned).

Note: For unsigned comparisons:

- Use `HI` and `LO` after **`CMPI`** or **`CMPR`**.
- `HI` tests if first $>$ second (unsigned).
- `LO` tests if first $\leq$ second (unsigned).
- `CS` is equivalent to unsigned $<$.
- `CC` is equivalent to unsigned $\geq$.
- `HI`/`LO` and `CS`/`CC` are not interchangeable: use `HI` for $>$, `LO` for $\leq$, `CS` for $<$ and `CC` for $\geq$.

10.3.3 Signed Comparisons

These conditions are used after comparing signed integers:

>= (Signed Greater or Equal) Executes when N = V. First operand $\geq$ second operand (signed).

« (Signed Less Than) Executes when N $\neq$ V. First operand $<$ second operand (signed).

» (Signed Greater Than) Executes when Z = 0 AND N = V. First operand $>$ second operand (signed).

<= (Signed Less or Equal) Executes when Z = 1 OR N $\neq$ V. First operand $\leq$ second operand (signed).

10.3.4 Special Conditions

Never (NV) Never executes: the instruction has no architectural effect beyond the normal two-word advance of `p`. No general-purpose register, flag, memory, address-register or coprocessor state changes, and no privilege check is performed. Can be used for instruction padding or to temporarily disable instructions during debugging.

10.4 Assembly Syntax

Condition codes are specified by appending a pipe character (`|`) followed by the condition mnemonic to the instruction:

```
; Unconditional
ADDI r1, #10

; Conditional - execute only if equal
ADDI|== r2, #20

; Conditional branch
BRAN|!= @loop_start

; Multiple conditions
ADDR|HI r3, r4, r5
STOR|NS (#0, s), r6
```

10.5 Usage with Compare Instructions

The most common use of condition codes is with the compare instructions (**`CMPI`** and **`CMPR`**):

```
; if (r1 > r2) { r3 = r4; } (unsigned)
CMPR r1, r2 ; Compare r1 and r2
MOVR|HI r3, r4 ; r3 = r4 if r1 > r2 (unsigned)

; if (r1 <= 100) { branch } (unsigned)
CMPI r1, #100
BRAN|LO @target
```

Example 10-1: Unsigned Comparison

```
; if (r1 >= r2) { r3 = r4; } (signed)
CMPR r1, r2 ; Compare r1 and r2
MOVR|>= r3, r4 ; r3 = r4 if r1 >= r2 (signed)

; if (r1 < -10) { branch } (signed)
CMPI r1, #-10
BRAN|<< @negative_case
```

Example 10-2: Signed Comparison

```
; if (r1 == r2) { branch }
CMPR r1, r2
BRAN|== @equal_case

; if (r1 != 0) { branch }
CMPI r1, #0
BRAN|!= @non_zero_case
```

Example 10-3: Equality Testing

10.6 Condition Code Truth Tables

10.6.1 Unsigned Comparison (CMPR rA, rB)

Relationship	Z	C	Condition	Code	Meaning
rA = rB	1	X	==	0001	Equal
rA $\neq$ rB	0	X	!=	0010	Not Equal
rA $>$ rB (unsigned)	0	0	HI	1001	Higher
rA $<$ rB (unsigned)	X	1	CS	0011	Lower
rA $\geq$ rB (unsigned)	X	0	CC	0100	Higher or Same
rA $\leq$ rB (unsigned)	C = 1 OR Z = 1*		LO	1010	Lower or Same

Table 10.2: Unsigned Comparison Results

`X` = the flag value does not affect this relationship.

*Unlike every other row, LO is not a single-corner condition: it is the OR of two flag values, so it cannot be expressed as one value per column with the other marked `X`. This is the only row in the table with that shape.

Note: after a subtraction or a compare (**`CMPI`**, **`CMPR`**), SIRC-1's Carry flag is set on a *borrow*, so `CS` (C = 1) means unsigned lower and `CC` (C = 0) means unsigned higher or same. `HI` and `LO` mean exactly what their names say.

10.6.2 Signed Comparison (CMPR rA, rB)

Relationship	Z	N	V	Condition	Code	Meaning
rA = rB	1	X	X	==	0001	Equal
rA $\neq$ rB	0	X	X	!=	0010	Not Equal
rA $>$ rB (signed)	0	0	0	»	1101	Greater
rA $>$ rB (signed)	0	1	1	»	1101	Greater
rA $<$ rB (signed)	X	0	1	«	1100	Less
rA $<$ rB (signed)	X	1	0	«	1100	Less
rA $\geq$ rB (signed)	X	0	0	>=	1011	Greater or Equal
rA $\geq$ rB (signed)	X	1	1	>=	1011	Greater or Equal
rA $\leq$ rB (signed)	1	X	X	<=	1110	Less or Equal
rA $\leq$ rB (signed)	0	0	1	<=	1110	Less or Equal
rA $\leq$ rB (signed)	0	1	0	<=	1110	Less or Equal

Table 10.3: Signed Comparison Results

`X` = the flag value does not affect this relationship.

10.7 Advanced Usage

```
; max = (a > b) ? a : b (unsigned)
CMPR r1, r2 ; Compare a and b
MOVR|HI r3, r1 ; r3 = a if a > b
MOVR|LO r3, r2 ; r3 = b if a <= b
```

Example 10-4: Conditional Assignment

```
; if (condition) count++
CMPI r1, #0
ADDI|!= r2, #1 ; Increment r2 if r1 != 0
```

Example 10-5: Conditional Increment

```
; if (r1 > 0) call_function()
CMPI r1, #0
LJSR|>> a ; Call function if r1 > 0 (signed)
```

Example 10-6: Conditional Function Call

```
; for (i = 10; i > 0; i--)
MOVI r1, #10 ; i = 10
:loop
    ; ... loop body ...
    SUBI r1, #1 ; i--
    BRAN|HI @loop ; Continue if i > 0 (unsigned)
```

Example 10-7: Loop Construction

10.8 Condition Code Selection Guide

Intent	After Compare	Condition Code
If r1 = r2	`CMPR r1, r2`	`==`
If r1 $\neq$ r2	`CMPR r1, r2`	`!=`
If r1 $>$ r2 (unsigned)	`CMPR r1, r2`	`HI`
If r1 $\geq$ r2 (unsigned)	`CMPR r1, r2`	`CC`
If r1 $<$ r2 (unsigned)	`CMPR r1, r2`	`CS`
If r1 $\leq$ r2 (unsigned)	`CMPR r1, r2`	`LO`
If r1 $>$ r2 (signed)	`CMPR r1, r2`	»
If r1 $\geq$ r2 (signed)	`CMPR r1, r2`	`>=`
If r1 $<$ r2 (signed)	`CMPR r1, r2`	«
If r1 $\leq$ r2 (signed)	`CMPR r1, r2`	`<=`
If r1 is negative	`CMPI r1, #0`	`NS`
If r1 is positive or zero	`CMPI r1, #0`	`NC`
If overflow occurred	After arithmetic	`OS`
If no overflow	After arithmetic	`OC`

Table 10.4: Condition Code Selection Guide

10.9 Performance Considerations

- A predicated instruction takes six cycles, plus instruction-fetch wait states, whether or not it executes. A skipped **`LOAD`** or **`STOR`** performs no data access, so it does not incur the data wait states it would incur when executed.

- An ordinary processing-unit instruction costs the same six cycles whether or not it executes. A coprocessor call whose condition is false skips its dispatch slot and takes 6 cycles instead of 12 or more (Chapter 16).

- The saving over a branch is the branch instruction itself, not a pipeline penalty, since the SIRC-1 has no overlapped pipeline to flush.

- For short conditional sequences, conditional execution can be faster than branching.

- For longer conditional blocks, explicit branches may be more efficient.

10.10 Common Pitfalls

- **Signed vs. Unsigned**: Using unsigned conditions (`HI`, `LO`) for signed values or vice versa produces incorrect results.

- **Flag Clobbering**: Flags are modified by most ALU instructions. Ensure that nothing modifies the condition flags between the comparison and the conditional instruction:

```
; WRONG - flags clobbered
CMPR r1, r2
ADDI r3, #100 ; Modifies flags!
BRAN|HI @target ; Tests wrong flags

; CORRECT - flags untouched
CMPR r1, r2
ADDI[N] r3, #100 ; [N]: flags untouched
BRAN|HI @target
```

- **Inverted Logic**: Remember that `CMPI` and `CMPR` compute the first operand minus the second, so condition codes test the first operand relative to the second, not the reverse.

Part III

SIRCIS Instruction Reference

CHAPTER 11

Reading Instruction Descriptions

This part of the manual describes each SIRCIS instruction in a consistent format. The goal is to make each instruction entry usable as an implementation contract, not only as programmer guidance.

11.1 Instruction Entry Fields

Each instruction entry begins with a title naming the mnemonic or mnemonic family, followed immediately by one applicability line: `SIRC-1, all revisions` for CPU instructions, or the coprocessor and revision required for coprocessor instructions, per the model and revision policy in Chapter 16. The fields below then follow, in this order, for every entry:

Mnemonic The assembly mnemonic or mnemonic family, given in the entry title. Immediate and register variants are listed together when they perform the same operation.

Syntax The accepted assembler forms. Meta-instructions are called out separately from real CPU instruction encodings.

Operands The permitted operand kinds and addressing modes for the instruction. Addressing-mode names come from the closed set in Table 8.1 (Chapter 8): Immediate, Register Direct, Address Register Direct, Indirect Immediate, Indirect Register, Post-Increment, and Pre-Decrement. An entry must not introduce a mode name outside that set; the 8-bit short-immediate encoding is a width variant of Immediate, not a mode.

Description Prose description of the instruction's behavior, supplementing the Operation pseudocode.

Operation Pseudocode for the architectural effect of the instruction. Arithmetic is performed on 16-bit word values unless otherwise stated.

Flags The effect on the condition flags in the status register.

Write-back Whether the computed result is written to a register, discarded, or used to update control state.

Exceptions Faults or traps that can be raised by the instruction. When an exception can occur, the entry must state whether partial side effects are possible.

Condition field The condition field values the instruction accepts for predicated execution. Every real instruction may be conditional. If the condition is false, the instruction has no architectural side effects unless a later instruction description explicitly states otherwise.

Timing The base execution time and any circumstances that add wait states.

Privilege Whether the instruction is available in protected mode, supervisor mode, or both.

Opcodes The 6-bit opcode values for each supported instruction format. Meta-instructions that assemble to another instruction's encoding use **Assembles to** instead of **Opcodes**.

Encoding diagrams A bit-field diagram for each encoding listed under Opcodes. A box labeled with an operand name (`rD`, `rS1`, `rO`, `imm16`, `disp`, `dest`, `src`, `addr` and so on) carries that operand; Table 11.1 defines the names. The shift fields (`SO`, `ST` and `SA`) appear as one box labeled `Shift`, the condition field `Cond`, and the status update source `SUS` (the AF field of ALU instructions) keep their field names. A box showing a literal value (for example `0x0`, `00` or `p`) is a field the instruction fixes to that value.

Instruction Fields One line per encoding field the instruction uses: register fields, the condition field, the status update source field and which of its encodings are legal for this instruction, shift type and count, and immediate width. Values come from Chapter 7; an entry does not introduce a value that is not defined there. Each line leads with the diagram label and gives the field's name in Chapter 7 and its bit range in parentheses.

Example / Examples One or more worked examples of the instruction in use. Each example is numbered `Example N-M:`, where N is the chapter number (or appendix letter) and M counts from 1 within the chapter; multiple examples in one entry take consecutive numbers.

Notes Additional informative remarks. A Notes block is informative and must not introduce a requirement that is not already stated in a normative field.

11.2 Notation

Notation	Meaning
`rD`	Destination general-purpose register
`rS`	Source general-purpose register
`rS1`	First source register, usually the left-hand operand
`rS2`	Second source register, usually the right-hand operand
`rO`	Offset or index general-purpose register supplying a register displacement in a memory or effective-address form
`#imm16`	16-bit immediate word encoded in immediate format
`#imm8`	8-bit immediate encoded in short immediate format
`shift`	Optional shift definition. It applies to the register operand being shifted (the first source register in register and short-immediate formats, or the source register for register-displacement **`STOR`** forms); it never applies to an immediate value, a second source register, or loaded data. **`LOAD`** has no shift. See Chapter 9 for the full encoding.
`SR.X`	Status register field `X`
`SUS`	Status update source: the AF field of ALU instructions, selecting `[N]`, `[A]` or `[S]`
`addr / src`	Source address-register pair (`l`, `a`, `s`, or `p`) supplying the base address for a memory-class or address-calculation instruction
`dest`	Destination address-register pair for an address-calculation instruction, encoded in the register field
`#disp`	Displacement added to the low word of an address-register pair
`@label`	Symbol reference resolved by the linker; PC-relative for **`BRAN`** and **`BRSR`**, a low-word address in the memory, effective-address and **`MOVI`** operands that accept one (see the Assembly Language Summary in the Notation and Glossary chapter for the full resolution rules, including data directives)
`|cond`	Condition-field suffix selecting the predicate for conditional execution

Table 11.1: Instruction Description Notation

11.3 Status Flag Effect Symbols

Flag tables use the following symbols:

Symbol	Meaning
`*`	Updated from the instruction result
`0`	Cleared to zero
`1`	Set to one
`-`	Preserved
`S`	Updated from the shifter result when `[S]` is used
`U`	Architecturally undefined; the implementation leaves the flag in an unspecified state

Table 11.2: Status Flag Effect Symbols

11.4 Conditional Execution

Every encoded instruction contains a condition field. If the condition evaluates true, the instruction executes normally. If the condition evaluates false, the instruction is skipped: it has no architectural effect beyond the normal two-word advance of `p` (set unconditionally at decode, before the condition is evaluated). No destination register, address register, memory, link register, coprocessor state, or status flag is modified by a skipped instruction. With `SR.A` set, a skipped instruction in the last instruction slot of a segment still raises a segment-overflow fault, because it uses the advanced address (Section 6.2.5).

11.5 Status Update Overrides

ALU instructions can select the source of status flag updates with the status update field:

Syntax	Encoded AF	Effect
`[N]`	`00`	Preserve status flags
`[A]`	`01`	Update flags from the ALU result
`[S]`	`10`	Update flags from the shifter result
Reserved	`11`	Reserved; N, Z, C and V architecturally undefined

Table 11.3: Status Update Source Encoding

When no override is written, ALU instructions use `[A]`. **`SHFT`** defaults to `[S]` instead. **`MOVI`** and **`MOVR`** do not accept `[S]`. Instructions that do not support status updates must not use status update override syntax.

CHAPTER 12

Instruction Summary

12.1 Overview

The SIRCIS instruction set consists of 64 possible opcodes (6-bit opcode field), of which 62 are documented and assigned; the remaining 2 are undocumented (see Appendix C). The instructions are organized into logical groups:

- **ALU Instructions (`0x00`–`0x0E`, `0x20`–`0x2E`, `0x30`–`0x3E`)**: Arithmetic, logical, and comparison operations
- **Memory Instructions (`0x10`–`0x17`)**: Load and store operations
- **Address and Control Flow (`0x18`–`0x1F`)**: Effective-address calculation, jumps, and calls
- **Coprocessor (`0x0F`, `0x3F`)**: Coprocessor interface

`0x27` and `0x2F` are undocumented (see Appendix C).

12.2 Complete Instruction List

Opcode	Mnemonic	Format	Flags	Operation
`0x00`	ADDI	Immediate	NZCV	Add immediate
`0x01`	ADCI	Immediate	NZCV	Add immediate with carry
`0x02`	SUBI	Immediate	NZCV	Subtract immediate
`0x03`	SBCI	Immediate	NZCV	Subtract immediate with carry
`0x04`	ANDI	Immediate	NZ (C,V=0)	AND immediate
`0x05`	ORRI	Immediate	NZ (C,V=0)	OR immediate
`0x06`	XORI	Immediate	NZ (C,V=0)	XOR immediate
`0x07`	MOVI	Immediate	NZ (C,V=0)	Move immediate
`0x08`	CMNI	Immediate	NZCV	Compare negative immediate
`0x09`	CNCI	Immediate	NZCV	Compare negative with carry immediate
`0x0A`	CMPI	Immediate	NZCV	Compare immediate
`0x0B`	CPCI	Immediate	NZCV	Compare with carry immediate
`0x0C`	TSAI	Immediate	NZ (C,V=0)	Test AND immediate
`0x0D`	TSOI	Immediate	NZ (C,V=0)	Test OR immediate
`0x0E`	TSXI	Immediate	NZ (C,V=0)	Test XOR immediate
`0x0F`	COPI	Immediate	–	Coprocessor call immediate
`0x10`	STOR	Indirect Imm	–	Store to (addr + imm)
`0x11`	STOR	Indirect Reg	–	Store to (addr + reg)
`0x12`	STOR	Pre-Dec	–	Store with pre-decrement
`0x13`	STOR	Pre-Dec Reg	–	Store with pre-dec (reg)
`0x14`	LOAD	Indirect Imm	–	Load from (addr + imm)
`0x15`	LOAD	Indirect Reg	–	Load from (addr + reg)
`0x16`	LOAD	Post-Inc	–	Load with post-increment
`0x17`	LOAD	Post-Inc Reg	–	Load with post-inc (reg)
`0x18`	LDEA	Indirect Imm	–	Load effective address
`0x19`	LDEA	Indirect Reg	–	Load effective address (reg)
`0x1A`	LDEA	Pre-Dec Imm	–	Load effective address, pre-dec
`0x1B`	LDEA	Pre-Dec Reg	–	Load effective address, pre-dec (reg)
`0x1C`	LDEL	Indirect Imm	–	Load effective address and link
`0x1D`	LDEL	Indirect Reg	–	Load effective address and link (reg)
`0x1E`	LDEL	Post-Inc Imm	–	Load effective address and link, post-inc
`0x1F`	LDEL	Post-Inc Reg	–	Load effective address and link, post-inc (reg)
`0x20`	ADDI	Short Imm+Shift	NZCV	Add short immediate
`0x21`	ADCI	Short Imm+Shift	NZCV	Add short imm with carry
`0x22`	SUBI	Short Imm+Shift	NZCV	Subtract short immediate
`0x23`	SBCI	Short Imm+Shift	NZCV	Subtract short imm with carry
`0x24`	ANDI	Short Imm+Shift	NZ (C,V=0)	AND short immediate
`0x25`	ORRI	Short Imm+Shift	NZ (C,V=0)	OR short immediate
`0x26`	XORI	Short Imm+Shift	NZ (C,V=0)	XOR short immediate
`0x27`	–	Short Imm+Shift	–	Undocumented
`0x28`	CMNI	Short Imm+Shift	NZCV	Compare negative short immediate
`0x29`	CNCI	Short Imm+Shift	NZCV	Compare negative with carry short immediate
`0x2A`	CMPI	Short Imm+Shift	NZCV	Compare short immediate
`0x2B`	CPCI	Short Imm+Shift	NZCV	Compare with carry short immediate
`0x2C`	TSAI	Short Imm+Shift	NZ (C,V=0)	Test AND short immediate
`0x2D`	TSOI	Short Imm+Shift	NZ (C,V=0)	Test OR short immediate
`0x2E`	TSXI	Short Imm+Shift	NZ (C,V=0)	Test XOR short immediate
`0x2F`	–	Short Imm+Shift	–	Undocumented
`0x30`	ADDR	Register	NZCV	Add register
`0x31`	ADCR	Register	NZCV	Add register with carry
`0x32`	SUBR	Register	NZCV	Subtract register
`0x33`	SBCR	Register	NZCV	Subtract register with carry
`0x34`	ANDR	Register	NZ (C,V=0)	AND register
`0x35`	ORRR	Register	NZ (C,V=0)	OR register
`0x36`	XORR	Register	NZ (C,V=0)	XOR register
`0x37`	MOVR	Register	NZ (C,V=0)	Move register
`0x38`	CMNR	Register	NZCV	Compare negative register
`0x39`	CNCR	Register	NZCV	Compare negative with carry register
`0x3A`	CMPR	Register	NZCV	Compare register
`0x3B`	CPCR	Register	NZCV	Compare with carry register
`0x3C`	TSAR	Register	NZ (C,V=0)	Test AND register
`0x3D`	TSOR	Register	NZ (C,V=0)	Test OR register
`0x3E`	TSXR	Register	NZ (C,V=0)	Test XOR register
`0x3F`	COPR	Register	–	Coprocessor call register

Table 12.1: Complete SIRCIS Instruction Set

The Flags column lists the condition flags each instruction updates. Table 13.2 gives the exact effect, using the symbols of Table 11.2.

12.3 Instruction Grouping Patterns

12.3.1 Opcode Organization

The opcode space is systematically organized:

- `0x0_`: Immediate operand format
- `0x1_`: Memory and control flow operations
- `0x2_`: Short immediate with shift format
- `0x3_`: Register operand format

12.3.2 Save vs. Test Variants

Many ALU instructions have two variants:

- **+`0x00` to +`0x06`**: Save result to destination register
- **+`0x07`**: **`MOVI`**/**`MOVR`** (`0x27` is undocumented)
- **+`0x08` to +`0x0E`**: Test only (update flags, discard result)
- **+`0x0F`**: Coprocessor call, `0x0F` and `0x3F` only (neither result nor flags are written; `0x2F` is undocumented)

The following pairs show a save instruction and its test-only counterpart:

- `0x00` = **`ADDI`** (save result), `0x08` = **`CMNI`** (test only)
- `0x04` = **`ANDI`** (save result), `0x0C` = **`TSAI`** (test only)
- `0x06` = **`XORI`** (save result), `0x0E` = **`TSXI`** (test only)

12.4 Meta-Instructions

Several commonly used operations are implemented as assembler meta-instructions that assemble to specific opcodes. This table is a summary; Chapter 17 is the canonical reference, and also gives the primary documentation chapter for each mnemonic:

Meta-Instruction	Assembles To	Purpose
NOOP	`ADDI[N] r1, #0`	No operation
RETS	`LDEA p, (#0, l)`	Return from subroutine
SHFT	`ORRI[S] rD, #0, shift`	Shift with status update
BRAN	`LDEA p, ...`	PC-relative branch
BRSR	`LDEL p, ...`	PC-relative call
LJMP	`LDEA p, ...`	Long jump
LJSR	`LDEL p, ...`	Long call
EXCP	`COPI #0x11vv`	Software exception
WAIT	`COPI #0x1900`	Wait for interrupt
RETE	`COPI #0x1A00`	Return from exception
RSET	`COPI #0x1B00`	System reset
ETFR	`COPI #0x1Cxy`	Read exception state
ETTR	`COPI #0x1Dxy`	Write exception state
DMAR	`COPI #0x28xx`	DMA read to registers
DMAW	`COPI #0x29xx`	DMA write from registers
DMAT	`COPI #0x2Ann`	DMA memory transfer
MULU	`COPI #0x3000`	Unsigned multiply
MULS	`COPI #0x3100`	Signed multiply
DIVU	`COPI #0x3200`	Unsigned divide
DIVS	`COPI #0x3300`	Signed divide

Table 12.2: Meta-Instructions (Summary)

12.5 Instruction Timing

Ordinary processing-unit instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required:

1. Instruction Fetch (High Word)
2. Instruction Fetch (Low Word)
3. Decode and Register Fetch
4. Execute and Address Calculation
5. Memory Access
6. Write Back

A coprocessor call whose condition is true takes a second 6-cycle dispatch slot, so most calls take 12 cycles. Section B.3.1 of Appendix B states this rule and its exceptions, and Chapter 16 gives the coprocessor timing.

This fixed sequence simplifies hardware design. Delayed bus acknowledgement may hold a phase and add wait cycles. DMA meta-instructions dispatch through the normal **`COPI`** path. A transfer of more than five bus accesses (more than five words for **`DMAR`** and **`DMAW`**, more than two for **`DMAT`**) runs on into further 6-cycle slots.

12.6 Next Chapters

The following chapters provide detailed documentation for each instruction group:

- **Chapter 13**: ALU instructions (arithmetic, logical, compare)
- **Chapter 14**: Load and store operations
- **Chapter 15**: Branches, jumps, and address operations
- **Chapter 16**: Coprocessor interface
- **Chapter 17**: Meta-instruction cross-reference

CHAPTER 13

ALU Instructions

13.1 Overview

The ALU (Arithmetic Logic Unit) instructions perform arithmetic and logical operations on register operands. All ALU instructions follow the load/store architecture principle: they operate only on registers, not directly on memory.

ALU instructions are available in three format variants:

- **Immediate**: Operation with 16-bit constant
- **Short Immediate with Shift**: Operation with 8-bit constant and optional shift
- **Register**: Operation between registers with optional shift

13.2 ALU Instruction Families

13.2.1 Arithmetic Instructions

- `ADD` – Addition
- `ADC` – Add with Carry
- `SUB` – Subtraction
- `SBC` – Subtract with Carry (Borrow)

13.2.2 Logical Instructions

- `AND` – Bitwise AND
- `ORR` – Bitwise OR
- `XOR` – Bitwise XOR (Exclusive OR)

13.2.3 Data Movement

- `MOV` – Move immediate or copy register

13.2.4 Test and Compare

- `CMN` – Compare Negative (add without saving result)
- `CNC` – Compare Negative with Carry (add with carry without saving result)
- `CMP` – Compare (subtract without saving result)
- `CPC` – Compare with Carry (subtract with carry without saving result)
- `TSA` – Test AND (AND without saving result)
- `TSO` – Test OR (OR without saving result)
- `TSX` – Test XOR (XOR without saving result)

13.3 Legal Forms

ALU instructions are register-only operations. Immediate operands are encoded in the instruction word; memory operands are not legal for this instruction family.

Instruction	Syntax	Encoding	Notes
ADD	`ADDI rD, #imm16`	`0x00`	Writes result
ADD	`ADDI rD, #imm8, shift`	`0x20`	Writes result
ADD	`ADDR rD, rS1, rS2[, shift]`	`0x30`	Writes result
ADD	`ADDR rD, rS2[, shift]`	`0x30`	Writes result (shorthand: rS1 = rD)
ADC	`ADCI rD, #imm16`	`0x01`	Writes result
ADC	`ADCI rD, #imm8, shift`	`0x21`	Writes result
ADC	`ADCR rD, rS1, rS2[, shift]`	`0x31`	Writes result
ADC	`ADCR rD, rS2[, shift]`	`0x31`	Writes result (shorthand: rS1 = rD)
SUB	`SUBI rD, #imm16`	`0x02`	Writes result
SUB	`SUBI rD, #imm8, shift`	`0x22`	Writes result
SUB	`SUBR rD, rS1, rS2[, shift]`	`0x32`	Writes result
SUB	`SUBR rD, rS2[, shift]`	`0x32`	Writes result (shorthand: rS1 = rD)
SBC	`SBCI rD, #imm16`	`0x03`	Writes result
SBC	`SBCI rD, #imm8, shift`	`0x23`	Writes result
SBC	`SBCR rD, rS1, rS2[, shift]`	`0x33`	Writes result
SBC	`SBCR rD, rS2[, shift]`	`0x33`	Writes result (shorthand: rS1 = rD)
AND	`ANDI rD, #imm16`	`0x04`	Writes result
AND	`ANDI rD, #imm8, shift`	`0x24`	Writes result
AND	`ANDR rD, rS1, rS2[, shift]`	`0x34`	Writes result
AND	`ANDR rD, rS2[, shift]`	`0x34`	Writes result (shorthand: rS1 = rD)
ORR	`ORRI rD, #imm16`	`0x05`	Writes result
ORR	`ORRI rD, #imm8, shift`	`0x25`	Writes result
ORR	`ORRR rD, rS1, rS2[, shift]`	`0x35`	Writes result
ORR	`ORRR rD, rS2[, shift]`	`0x35`	Writes result (shorthand: rS1 = rD)
XOR	`XORI rD, #imm16`	`0x06`	Writes result
XOR	`XORI rD, #imm8, shift`	`0x26`	Writes result
XOR	`XORR rD, rS1, rS2[, shift]`	`0x36`	Writes result
XOR	`XORR rD, rS2[, shift]`	`0x36`	Writes result (shorthand: rS1 = rD)
MOV	`MOVI rD, #imm16 | @sym | $name`	`0x07`	Writes result
MOV	`MOVR rD, rS`	`0x37`	Writes result
SHFT	`SHFT rD, shift`	`0x25`	Writes result; default `[S]`
CMN	`CMNI rS, #imm16`	`0x08`	Updates flags only
CMN	`CMNI rS, #imm8, shift`	`0x28`	Updates flags only
CMN	`CMNR rS1, rS2[, shift]`	`0x38`	Updates flags only
CNC	`CNCI rS, #imm16`	`0x09`	Updates flags only
CNC	`CNCI rS, #imm8, shift`	`0x29`	Updates flags only
CNC	`CNCR rS1, rS2[, shift]`	`0x39`	Updates flags only
CMP	`CMPI rS, #imm16`	`0x0A`	Updates flags only
CMP	`CMPI rS, #imm8, shift`	`0x2A`	Updates flags only
CMP	`CMPR rS1, rS2[, shift]`	`0x3A`	Updates flags only
CPC	`CPCI rS, #imm16`	`0x0B`	Updates flags only
CPC	`CPCI rS, #imm8, shift`	`0x2B`	Updates flags only
CPC	`CPCR rS1, rS2[, shift]`	`0x3B`	Updates flags only
TSA	`TSAI rS, #imm16`	`0x0C`	Updates flags only
TSA	`TSAI rS, #imm8, shift`	`0x2C`	Updates flags only
TSA	`TSAR rS1, rS2[, shift]`	`0x3C`	Updates flags only
TSO	`TSOI rS, #imm16`	`0x0D`	Updates flags only
TSO	`TSOI rS, #imm8, shift`	`0x2D`	Updates flags only
TSO	`TSOR rS1, rS2[, shift]`	`0x3D`	Updates flags only
TSX	`TSXI rS, #imm16`	`0x0E`	Updates flags only
TSX	`TSXI rS, #imm8, shift`	`0x2E`	Updates flags only
TSX	`TSXR rS1, rS2[, shift]`	`0x3E`	Updates flags only

Table 13.1: Legal ALU Instruction Forms

Register shorthand: When a register-form ALU instruction omits `rS1`, the assembler encodes `rD` as both the destination and first source operand. For example, `ADDR r1, r2` is encoded as `ADDR r1, r1, r2`.
Opcode `0x27`: Opcode `0x27` is undocumented and has no public assembly syntax; its behavior is architecturally undefined (see Appendix C).

13.4 Common ALU Semantics

Conditional execution, the status update source, and timing follow the rules in Chapter 11 and Appendix B. In the immediate format the left operand is the named register and the right operand is the encoded 16-bit immediate. In the short immediate format the left operand is the named register after the encoded shift is applied, and the right operand is the encoded 8-bit immediate. In the register format the left operand is `rS1` after the encoded shift is applied, and the right operand is `rS2`.

The test variants (**`CMN`**, **`CNC`**, **`CMP`**, **`CPC`**, **`TSA`**, **`TSO`**, and **`TSX`**) compute a result for flag updates but do not write it back. **`MOVI`** and **`MOVR`** write the selected operand to `rD`.
ALU instruction execution does not perform data-memory access and does not raise data bus, data bus-protection, data alignment, effective-address segment-overflow, or invalid-opcode faults. An ALU instruction in the last instruction slot of a segment still raises a segment-overflow fault when `SR.A` is set, because it advances the program counter, unless it writes `pl` (Section 6.2.5). In protected mode, an ALU instruction that writes `ah`, `lh`, `ph`, or `sh` through its register field raises a privilege-violation fault at decode and has no other effect; compare and test instructions only read that field and never fault. An ALU instruction whose register field names `sr` is permitted in protected mode, but only bits 7–0 of `sr` are written; bits 15–8 are left unchanged (Section 5.5.2). With the `[A]` or `[S]` status source, the instruction's own flag update then overwrites bits 7–0, so the written value survives only in bits 15–8 (supervisor mode) or not at all (protected mode). Use `[N]` to write the low byte (Section 5.5.2). Instruction fetch can still raise the normal fetch faults before an ALU instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits, except at exception level 7 (Section 6.2.8). Every 6-bit opcode decodes; no processing-unit encoding raises an invalid-opcode fault. Encodings outside the documented ALU forms execute with architecturally undefined results (see Appendix C).

13.5 Status Flag Updates

ALU instructions update the status register flags:

Arithmetic (ADD, ADC, SUB, SBC, CMN, CNC, CMP, CPC) Update all four flags: N, Z, C, V

Logical and move (AND, ORR, XOR, TSA, TSO, TSX, MOVI, MOVR) Update N and Z; clear C and V

Family	N	Z	C	V	Default AF	Meaning
ADD	*	*	*	*	`[A]`	C is carry out; V is signed overflow.
ADC	*	*	*	*	`[A]`	Adds incoming C; C is carry out.
SUB	*	*	*	*	`[A]`	C is set when a borrow occurs.
SBC	*	*	*	*	`[A]`	Subtracts incoming C as borrow input; C is set on outgoing borrow.
CMN	*	*	*	*	`[A]`	Same flags as ADD; result is discarded.
CNC	*	*	*	*	`[A]`	Same flags as ADC; result is discarded.
CMP	*	*	*	*	`[A]`	Same flags as SUB; result is discarded.
CPC	*	*	*	*	`[A]`	Same flags as SBC; result is discarded.
AND	*	*	0	0	`[A]`	C and V are cleared.
ORR	*	*	0	0	`[A]`	C and V are cleared.
XOR	*	*	0	0	`[A]`	C and V are cleared.
TSA	*	*	0	0	`[A]`	Same flags as AND; result is discarded.
TSO	*	*	0	0	`[A]`	Same flags as ORR; result is discarded.
TSX	*	*	0	0	`[A]`	Same flags as XOR; result is discarded.
MOV	*	*	0	0	`[A]`	C and V are cleared.

Table 13.2: ALU Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions. The table shows the effect when the instruction uses its default status update source. If `[N]` is encoded, all status flags are preserved. If `[A]` is encoded, flags are updated from the ALU result according to the instruction family. If `[S]` is encoded, flags are updated from the shifter result instead of the ALU result; see Chapter 9 for the shifter flag rules. The immediate format has no shift fields, so `[S]` on an immediate-format instruction clears N, Z, C and V; reserved bits 7–4 follow the rule in Chapter 5. `[S]` should be used only with short-immediate and register forms. An immediate written without a shift suffix always assembles to the immediate format, even when it fits in 8 bits, so `ADDI[S] r1, #10` clears the flags; write `ADDI[S] r1, #10, NUL #0` to get the short-immediate format and flags from the shifter. **`MOVI`** and **`MOVR`** have no shifted operand and do not accept `[S]`; an encoding of those opcodes with AF `10`, which is not reachable from public assembly syntax, leaves all four status flags architecturally undefined (`U U U U`). An ALU encoding with AF `11` completes the operation and writes its result, but leaves N, Z, C and V architecturally undefined (`U U U U`); assemblers never emit it.

13.5.1 Flag Computation Formulas

Table 13.2 states which flags each family updates; this section states exactly how, as a Boolean formula in terms of three sign bits (bit 15): $S1_m$ is that of the first source operand after any shift (`rS1`, or the named register in the immediate formats), $S2_m$ is that of the second source operand (`rS2`, imm16, or the zero-extended imm8), and R_m is that of the 16-bit result. For **`ADC`** and **`SBC`**,

R_m includes the incoming carry. For subtraction the result is $S1 - S2$.

Family	**C**	**V**
ADD, ADC, CMN, CNC	Carry out of bit 15 of the full addition	$S1_m \cdot S2_m \cdot \overline{R_m} + \overline{S1_m} \cdot \overline{S2_m} \cdot R_m$
SUB, SBC, CMP, CPC	$\overline{S1_m} \cdot S2_m + S2_m \cdot R_m + R_m \cdot \overline{S1_m}$	$S1_m \cdot \overline{S2_m} \cdot \overline{R_m} + \overline{S1_m} \cdot S2_m \cdot R_m$
AND, ORR, XOR, TSA, TSO, TSX	0 (always cleared)	0 (always cleared)

Table 13.3: Carry and Overflow Flag Formulas

N and Z are the same for every family: $N = R_m$ (the result's own sign bit), and Z is set when the full 16-bit result is zero. **ADC** and **SBC** use the same formulas as **ADD** and **SUB** respectively – the incoming carry/borrow is folded into the addition or subtraction before R_m (and the carry/borrow out) is evaluated, so no separate formula is needed for them. **CMN**/**CNC**/**CMP**/**CPC**/**TSA**/**TSO**/**TSX** compute the same flags as the corresponding **ADD**/**ADC**/**SUB**/**SBC**/**AND**/**ORR**/**XOR** family and simply discard the result.

13.5.2 Manual Flag Update Control

ALU instructions support manual control over how status flags are updated using the status override syntax. See Section 9.12 in Chapter 9 for complete details.

- `INSTR[A]` – Update flags from ALU result (default behavior)
- `INSTR[S]` – Update flags from shift result instead of ALU
- `INSTR[N]` – Do not update status flags
- **MOVI** and **MOVR** do not accept `[S]`.
- AF encoding `11` (binary) is reserved and has no assembly syntax; the operation completes and writes its result, and N, Z, C and V are architecturally undefined (`U U U U`).

```
; Update flags from shift operation
ADDI[S] r1, #0, LSL #3 ; Flags reflect (r1 << 3), not the ALU

; Preserve flags during operation
CMPR r2, r3 ; Set comparison flags
ADDI[N] r4, #1 ; Increment without changing flags
BRAN|>= @branch_if_r2_gte_r3 ; Use flags from CMPR
```

Example 13-1: Manual Status Flag Update Control

SHFT

Shift with Status Update Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
SHFT rD, shift ; rD = shifted rD; flags from shift
```

Operands: `rD` is both the destination and shifted source register. `shift` may use any standard immediate or register shift count supported by short-immediate ALU instructions.

Description:

Provides concise syntax for shifting a register value while taking status flags from the shifter result. It is not a separate CPU operation. The zero OR operation preserves the shifted source value, while `[S]` makes the status flags reflect the shift result instead of the OR result.

Operation:

```
shifted = shift(rD)
rD = shifted | 0
SR = flags updated per the status-update suffix:
    [S] shifter result (default for SHFT),
    [A] ALU result, [N] no update
```

Flags: Updated from the shifter result by default.

Write-back: Inherits **`ORRI`** write-back to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no shift result is written and status flags are preserved.

Timing: Same as **`ORRI`**: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Assembles to: `ORRI[S] rD, #0, shift`. See Table 13.1 for the full family of legal forms.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x25						rD				0x00								Shift								SUS		Cond			

Instruction Fields:

- **Register (bits 25–22):** `rD`, the destination and shifted source register; any of the 16 registers, `0x0`–`0xF`.
- **Immediate (bits 21–14):** fixed at `0x00` by the meta-instruction.
- **Shift (bits 13–6):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **AF (bits 5–4):** `10` (Shift) by default; an explicit `[A]` or `[N]` suffix encodes `01` or `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Shift r1 left by 3 and update flags
SHFT r1, LSL #3 ; ORRI[S] r1, #0, LSL #3

; Shift right and test result
SHFT r3, LSR #2 ; r3 = r3 >> 2
BRAN|== @zero_result ; Branch if shifted result is zero
```

Example 13-2: Shifting and Testing with SHFT

Notes:

- Immediate shift counts are 0–15; the assembler rejects a larger literal. A register shift count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`) – see Chapter 9.
- All six shift types are supported, plus **`NUL`**: NUL, LSL, LSR, ASL, ASR, RTL, and RTR. `SHFT rD, NUL #0` sets the flags from the unshifted value of `rD`.

ADDI / ADDR

Add Immediate / Add Register

Applicability: SIRC-1, all revisions

Syntax:

```
ADDI rD, #imm16 ; rD = rD + imm16
ADDI rD, #imm8, shift ; rD = shift(rD) + imm8
ADDR rD, rS1, rS2 ; rD = rS1 + rS2
ADDR rD, rS1, rS2, shift ; rD = shift(rS1) + rS2
```

Operands: `rD` is the destination and first source in immediate forms. Register forms use `rS1` as the first source and `rS2` as the second source; assembler shorthand may omit `rS1`. Short-immediate and register forms may apply a shift to the first source operand.

Description:

Adds two values and stores the result in the destination register. In short immediate format with shift, the first source register is shifted before adding the immediate. The carry flag is set if an unsigned overflow occurs (carry out of bit 15). The overflow flag is set if a signed overflow occurs (the sign of the result is incorrect for the operation).

Operation:

```
result = operand1 + operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = (operand1[15] == operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x00` (Imm), `0x20` (Short Imm), `0x30` (Reg). See Table 13.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
0x00	rD	imm16	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
0x20	rD	imm8	Shift	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18	17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
0x30	rD	rS1	rS2	Shift	SUS	Cond

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Simple addition
ADDI r1, #100 ; r1 = r1 + 100

; Add two registers
ADDR r3, r1, r2 ; r3 = r1 + r2

; Scaled addition (multiply by 3)
ADDR r4, r5, r5, LSL #1 ; r4 = r5 + (r5 * 2) = r5 * 3

; Build large constant
ADDI r1, #1, LSL #12 ; r1 = (r1 << 12) + 1
```

Example 13-3: Basic and Scaled Addition

Notes:

- For multi-precision arithmetic, use **`ADC`** to propagate carry
- Adding zero (`ADDI r1, #0`) sets N and Z from `r1` and clears C and V without changing it; use **`MOVR`** to copy a register
- Shift operations allow efficient constant multiplication

ADCI / ADCR

Add with Carry

Applicability: SIRC-1, all revisions

Syntax:

```
ADCI rD, #imm16 ; rD = rD + imm16 + C
ADCI rD, #imm8, shift ; rD = shift(rD) + imm8 + C
ADCR rD, rS1, rS2 ; rD = rS1 + rS2 + C
ADCR rD, rS1, rS2, shift ; rD = shift(rS1) + rS2 + C
```

Operands: Same operand forms as **`ADD`**. The incoming `C` flag is read as an additional least-significant carry input.

Description:

Adds two values plus the carry flag and stores the result. In short immediate format with shift, the first source register is shifted before adding the immediate. This instruction is used for multi-precision (32-bit, 48-bit, etc.) arithmetic to propagate the carry from lower-order additions to higher-order additions.

Operation:

```
result = operand1 + operand2 + SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = (operand1[15] == operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x01` (Imm), `0x21` (Short Imm), `0x31` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x01</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x21</td><td colspan="4">rD</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x31</td><td colspan="4">rD</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; 32-bit addition (r1:r2 = r3:r4 + r5:r6)
; Low words
ADDR r2, r4, r6 ; r2 = r4 + r6, set carry

; High words (with carry propagation)
ADCR r1, r3, r5 ; r1 = r3 + r5 + carry
```

Example 13-4: 32-bit Addition with Carry Propagation

Notes:

- Always use after **`ADD`** for multi-word arithmetic
- The carry flag must be in the correct state before **`ADC`**
- Can be used to add 1 conditionally based on carry flag

SUBI / SUBR

Subtract Immediate / Subtract Register

Applicability: SIRC-1, all revisions

Syntax:

```
SUBI rD, #imm16 ; rD = rD - imm16
SUBI rD, #imm8, shift ; rD = shift(rD) - imm8
SUBR rD, rS1, rS2 ; rD = rS1 - rS2
SUBR rD, rS1, rS2, shift ; rD = shift(rS1) - rS2
```

Operands: Same operand forms as **`ADD`**. The second operand is subtracted from the first operand.

Description:

Subtracts the second operand from the first and stores the result. In short immediate format with shift, the first source register is shifted before subtracting the immediate. The carry flag is set if a borrow occurs (operand2 > operand1 for unsigned). The overflow flag indicates signed overflow.

Operation:

```
result = operand1 - operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = (operand1[15] != operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x02` (Imm), `0x22` (Short Imm), `0x32` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x02</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x22</td><td colspan="4">rD</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x32</td><td colspan="4">rD</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Decrement by 1
SUBI r1, #1 ; r1 = r1 - 1

; Subtract registers
SUBR r3, r1, r2 ; r3 = r1 - r2

; Loop counter
SUBI r7, #1 ; Decrement counter
BRAN|!= @loop ; Branch if not zero
```

Example 13-5: Basic Subtraction and Loop Counting

Notes:

- Carry is set when the subtraction requires a borrow
- For multi-precision subtraction, use **`SBC`**
- Subtracting a value from itself (`SUBR r1, r1, r1`) clears the register

SBCI / SBCR

Subtract with Carry

Applicability: SIRC-1, all revisions

Syntax:

```
SBCI rD, #imm16 ; rD = rD - imm16 - C
SBCI rD, #imm8, shift ; rD = shift(rD) - imm8 - C
SBCR rD, rS1, rS2 ; rD = rS1 - rS2 - C
SBCR rD, rS1, rS2, shift ; rD = shift(rS1) - rS2 - C
```

Operands: Same operand forms as **SUB**. The incoming `C` flag is read as the borrow input.

Description:

Subtracts the second operand and borrow from the first operand. In short immediate format with shift, the first source register is shifted before subtracting the immediate. Used for multi-precision subtraction to propagate borrows from lower-order to higher-order subtractions.

Operation:

```
result = operand1 - operand2 - SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = (operand1[15] != operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x03` (Imm), `0x23` (Short Imm), `0x33` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x03</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x23</td><td colspan="4">rD</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x33</td><td colspan="4">rD</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; 32-bit subtraction (r1:r2 = r3:r4 - r5:r6)
; Low words
SUBR r2, r4, r6 ; r2 = r4 - r6, set/clear carry

; High words (with borrow propagation)
SBCR r1, r3, r5 ; r1 = r3 - r5 - borrow
```

Example 13-6: 32-bit Subtraction with Borrow Propagation

Notes:

- Always use after **`SUB`** for multi-word subtraction
- Carry is treated as the borrow input for multi-word subtraction

ANDI / ANDR

AND Immediate / AND Register

Applicability: SIRC-1, all revisions

Syntax:

```
ANDI rD, #imm16 ; rD = rD & imm16
ANDI rD, #imm8, shift ; rD = shift(rD) & imm8
ANDR rD, rS1, rS2 ; rD = rS1 & rS2
ANDR rD, rS1, rS2, shift ; rD = shift(rS1) & rS2
```

Operands: Same operand forms as `ADD`. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Description:

Performs bitwise AND operation. Each bit in the result is 1 only if the corresponding bits in both operands are 1. In short immediate format with shift, the first source register is shifted before ANDing with the immediate. Commonly used for bit masking and clearing specific bits.

Operation:

```
result = operand1 & operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to `rD`.

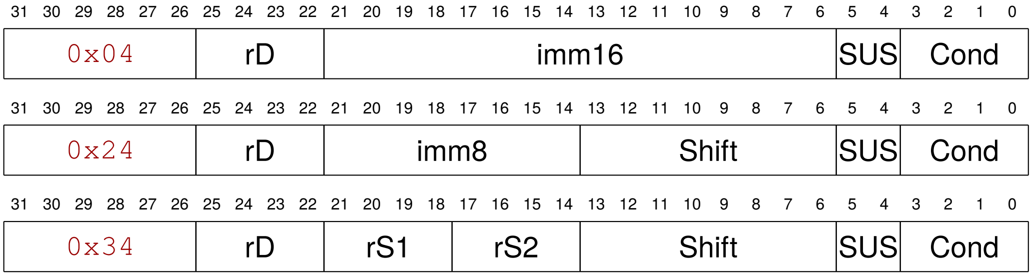
Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x04` (Imm), `0x24` (Short Imm), `0x34` (Reg). See Table 13.1 for the full family of legal forms.


31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x04
rD
imm16
SUS
Cond
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x24
rD
imm8
Shift
SUS
Cond
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x34
rD
rS1
rS2
Shift
SUS
Cond

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Mask lower byte
ANDI r1, #0x00FF ; r1 = r1 & 0xFF

; Clear bit 5
ANDI r2, #0xFFDF ; r2 = r2 & 0xFFDF (clear bit 5)

; Check if two registers share any set bits
ANDR r3, r1, r2 ; r3 = r1 & r2
CMPI r3, #0
BRAN|!= @common_bits ; Branch if any bits in common
```

Example 13-7: Bit Masking and Clearing with AND

Notes:

- ANDing with `0xFFFF` leaves value unchanged
- ANDing with `0x0000` clears the register
- Use **`TSA`** to test bits without modifying the register

ORRI / ORRR

OR Immediate / OR Register

Applicability: SIRC-1, all revisions

Syntax:

```
ORRI rD, #imm16 ; rD = rD | imm16
ORRI rD, #imm8, shift ; rD = shift(rD) | imm8
ORRR rD, rS1, rS2 ; rD = rS1 | rS2
ORRR rD, rS1, rS2, shift ; rD = shift(rS1) | rS2
```

Operands: Same operand forms as **`AND`**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Description:

Performs bitwise OR operation. Each bit in the result is 1 if either corresponding bit in the operands is 1. In short immediate format with shift, the first source register is shifted before ORing with the immediate. Commonly used for setting specific bits.

Operation:

```
result = operand1 | operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x05` (Imm), `0x25` (Short Imm), `0x35` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><th>31</th><th>30</th><th>29</th><th>28</th><th>27</th><th>26</th><th>25</th><th>24</th><th>23</th><th>22</th><th>21</th><th>20</th><th>19</th><th>18</th><th>17</th><th>16</th><th>15</th><th>14</th><th>13</th><th>12</th><th>11</th><th>10</th><th>9</th><th>8</th><th>7</th><th>6</th><th>5</th><th>4</th><th>3</th><th>2</th><th>1</th><th>0</th></tr>
<tr><td colspan="6">0x05</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><th>31</th><th>30</th><th>29</th><th>28</th><th>27</th><th>26</th><th>25</th><th>24</th><th>23</th><th>22</th><th>21</th><th>20</th><th>19</th><th>18</th><th>17</th><th>16</th><th>15</th><th>14</th><th>13</th><th>12</th><th>11</th><th>10</th><th>9</th><th>8</th><th>7</th><th>6</th><th>5</th><th>4</th><th>3</th><th>2</th><th>1</th><th>0</th></tr>
<tr><td colspan="6">0x25</td><td colspan="4">rD</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><th>31</th><th>30</th><th>29</th><th>28</th><th>27</th><th>26</th><th>25</th><th>24</th><th>23</th><th>22</th><th>21</th><th>20</th><th>19</th><th>18</th><th>17</th><th>16</th><th>15</th><th>14</th><th>13</th><th>12</th><th>11</th><th>10</th><th>9</th><th>8</th><th>7</th><th>6</th><th>5</th><th>4</th><th>3</th><th>2</th><th>1</th><th>0</th></tr>
<tr><td colspan="6">0x35</td><td colspan="4">rD</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Set bit 7
ORRI r1, #0x0080 ; r1 = r1 | 0x80

; Set multiple bits
ORRI r2, #0xF000 ; Set high nibble

; Combine two registers
ORRR r3, r1, r2 ; r3 = r1 | r2

; Set bit N (variable)
MOVI r4, #1 ; r4 = 1
SHFT r4, LSL r5 ; r4 = 1 << r5
ORRR r1, r4 ; r1 |= (1 << r5)
```

Example 13-8: Setting Bits with OR

Notes:

- ORing with `0x0000` leaves value unchanged
- ORing with `0xFFFF` sets all bits
- Use for enabling flag bits or combining bit fields

XORI / XORR

XOR Immediate / XOR Register

Applicability: SIRC-1, all revisions

Syntax:

```
XORI rD, #imm16 ; rD = rD ^ imm16
XORI rD, #imm8, shift ; rD = shift(rD) ^ imm8
XORR rD, rS1, rS2 ; rD = rS1 ^ rS2
XORR rD, rS1, rS2, shift ; rD = shift(rS1) ^ rS2
```

Operands: Same operand forms as `AND`. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Description:

Performs bitwise XOR (exclusive OR) operation. Each bit in the result is 1 if the corresponding bits in the operands are different. In short immediate format with shift, the first source register is shifted before XORing with the immediate. Used for toggling bits, bitwise comparison, and simple encryption.

Operation:

```
result = operand1 ^ operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x06` (Imm), `0x26` (Short Imm), `0x36` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x06</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x26</td><td colspan="4">rD</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x36</td><td colspan="4">rD</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).

- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Toggle bit 3
XORI r1, #0x0008 ; r1 = r1 ^ 0x08

; Invert all bits
XORI r2, #0xFFFF ; r2 = ~r2

; Check if registers are equal
XORR r3, r1, r2 ; r3 = r1 ^ r2
; If r3 == 0, then r1 == r2

; Swap using XOR (no temporary)
XORR r1, r1, r2 ; r1 = r1 ^ r2
XORR r2, r1, r2 ; r2 = r1 ^ r2 = (r1 ^ r2) ^ r2 = r1
XORR r1, r1, r2 ; r1 = (r1 ^ r2) ^ r1 = r2
```

Example 13-9: Bit Toggling, Equality Testing, and XOR Swap

Notes:

- XORing with `0x0000` leaves value unchanged
- XORing with `0xFFFF` inverts all bits (bitwise NOT)
- XORing a value with itself always gives 0
- Use **`TSX`** to check for differences without modifying register

MOVI / MOVR

Move Immediate / Move Register

Applicability: SIRC-1, all revisions

Syntax:

```
MOVI rD, #imm16 ; rD = imm16
MOVR rD, rS ; rD = rS
```

Operands: Immediate form writes a 16-bit immediate value to `rD`. Register form copies `rS` to `rD`. No short-immediate, memory, or shift forms are public assembly syntax; the register-to-register form does not accept a shift definition (use **`SHFT`**), and the `[S]` status update source is not accepted. Neither instruction accesses memory; the memory forms of **`LOAD`** are documented in Chapter 14.

Description:

The ALU passes the second operand straight through to the destination; the first operand is ignored. This is the primary way to load constants into registers or move data between them. Opcode `0x27` has no public assembly syntax and is undocumented.

Operation:

```
result = operand
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the loaded or copied value to `rD`.

Exceptions: Does not access memory in these forms and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when it writes a privileged register (`ah`, `lh`, `ph`, `sh`); see Chapter 6.

Opcodes: `0x07` (Imm), `0x37` (Reg). Opcode `0x27` is undocumented. See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x07</td><td colspan="4">rD</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x37</td><td colspan="4">rD</td><td colspan="4">0x0</td><td colspan="4">rS</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rD (Reg or R1, bits 25–22):** destination register; any of the 16 registers, `0x0`–`0xF`.
- **R2 (bits 21–18), register form only:** unused, encoded as 0.
- **rS (R3, bits 17–14), register form only:** source register.
- **imm16 (Immediate Value, bits 21–6), immediate form only:** 16-bit value. The register form has no immediate field.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined. `[S]` is not accepted by these instructions, which have no shifted operand; an encoding with AF `10`, not reachable from assembly syntax, leaves all four status flags architecturally undefined (`U U U U`).

- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Move a constant
MOVI r1, #1234 ; r1 = 1234

; Copy register
MOVR r2, r1 ; r2 = r1

; Move, then shift with an ALU instruction
MOVI r3, #1
ADDI r3, #0, LSL #8 ; r3 = (r3 << 8) + 0 = 256

; Zero a register
MOVI r4, #0 ; r4 = 0

; Build a large constant
MOVI r5, #0x12
ADDI r5, #0, LSL #8 ; r5 = (r5 << 8) + 0 = 0x1200
ORRI r5, #0x34 ; r5 = 0x1234
```

Example 13-10: Moving Constants and Copying Registers

CMNI / CMNR

Compare Negative Immediate / Compare Negative Register

Applicability: SIRC-1, all revisions

Syntax:

```
CMNI rS, #imm16 ; Flags from rS + imm16
CMNI rS, #imm8, shift ; Flags from shift(rS) + imm8
CMNR rS1, rS2 ; Flags from rS1 + rS2
CMNR rS1, rS2, shift ; Flags from shift(rS1) + rS2
```

Operands: Same source operand forms as **ADD**. The first operand is added to the second operand; no destination register is written.

Description:

Performs addition but discards the result, only updating the status flags. It is the flag-only counterpart of **ADD**, as **CMP** is of **SUB**. Adding two values and testing for zero is the same as comparing the first value with the negation of the second, so `CMNI r1, #5` sets Z when `r1` holds -5. In short immediate format with shift, the first source register is shifted before the immediate is added.

Operation:

```
result = operand1 + operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = (operand1[15] == operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: None. The sum is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `CMNI ah, #0`); see Chapter 6.

Opcodes: `0x08` (Imm), `0x28` (Short Imm), `0x38` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x08</td><td colspan="4">rS</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x28</td><td colspan="4">rS</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x38</td><td colspan="4">ignored</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Test if r1 is -1 (0xFFFF): r1 + 1 == 0
CMNI r1, #1
BRAN|== @is_minus_one

; Test if r1 is the negation of r2
CMNR r1, r2
BRAN|== @are_negations

; Would r3 + r4 overflow a signed 16-bit result?
; Neither register is changed
CMNR r3, r4
BRAN|OS @would_overflow
```

Example 13-11: Testing Sums Without Writing Them

Notes:

- Does not modify any register, only flags
- Z=1 means the operands sum to zero, so the first operand equals the negation of the second
- C is the carry out of the addition and V is signed overflow of the addition; the flags describe the sum, not an ordering, so the unsigned and signed ordering conditions that are meaningful after **`CMP`** are not meaningful after **`CMN`**
- Use **`CNC`** to continue the test across multiple words

CNCI / CNCR

Compare Negative with Carry Immediate / Register

Applicability: SIRC-1, all revisions

Syntax:

```
CNCI rS, #imm16 ; Flags from rS + imm16 + C
CNCI rS, #imm8, shift ; Flags from shift(rS) + imm8 + C
CNCR rS1, rS2 ; Flags from rS1 + rS2 + C
CNCR rS1, rS2, shift ; Flags from shift(rS1) + rS2 + C
```

Operands: Same source operand forms as **ADC**. The incoming `C` flag is read as an additional least-significant carry input; no destination register is written.

Description:

Adds two values plus the carry flag but discards the result, only updating the status flags. It is the flag-only counterpart of **ADC** and continues a **CMN** across multiple words: **CMN** on the low words followed by **CNC** on the high words leaves C, V and N describing the whole multi-word sum. In short immediate format with shift, the first source register is shifted before the immediate is added.

Operation:

```
result = operand1 + operand2 + SR.C
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = (operand1[15] == operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: None. The sum is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `CNCI ah, #0`); see Chapter 6.

Opcodes: `0x09` (Imm), `0x29` (Short Imm), `0x39` (Reg). See Table 13.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x09`	rS	imm16	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x29`	rS	imm8	Shift	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18	17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x39`	ignored	rS1	rS2	Shift	SUS	Cond

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Would the 32-bit signed sum r1:r2 + r3:r4 overflow?
; Low words first, then high words with the carry
CMNR r2, r4
CNCR r1, r3
BRAN|OS @overflowed ; V = signed overflow of the 32-bit sum

; Would the 32-bit unsigned sum carry out?
CMNR r2, r4
CNCR r1, r3
BRAN|CS @carried
```

Example 13-12: Testing 32-bit Sums Without Writing Them

Notes:

- Does not modify any register, only flags
- Use after **`CMN`** (or **`ADD`**) for multi-word arithmetic; the carry flag must be in the correct state before **`CNC`**
- Z reflects only the result of this instruction, not the whole multi-word sum, so use C, V and N rather than Z to describe the combined value

CMPI / CMPR

Compare Immediate / Compare Register

Applicability: SIRC-1, all revisions

Syntax:

```
CMPI rS, #imm16 ; Compare rS with imm16
CMPI rS, #imm8, shift ; Compare shift(rS) with imm8
CMPR rS1, rS2 ; Compare rS1 with rS2
CMPR rS1, rS2, shift ; Compare shift(rS1) with rS2
```

Operands: Same source operand forms as **SUB**. The first operand is compared with the second operand; no destination register is written.

Description:

Performs subtraction but discards the result, only updating the status flags. In short immediate format with shift, the first source register is shifted before comparing with the immediate. This is the primary instruction for implementing conditional execution and branches. The flags can be tested with condition codes to determine the relationship between the operands.

Operation:

```
result = operand1 - operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = (operand1[15] != operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: None. The subtraction result is discarded after status flags are updated.

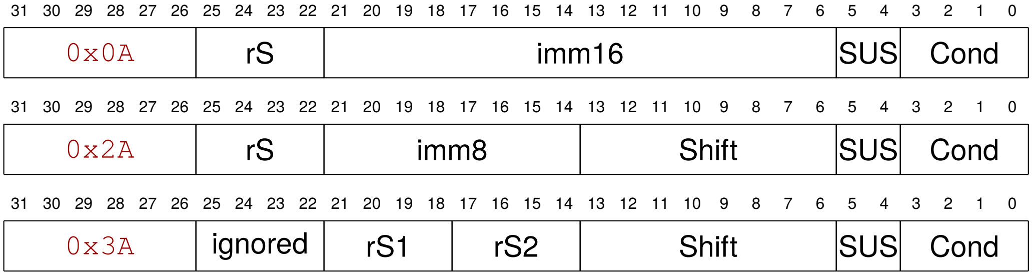
Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `CMPI ah, #0`); see Chapter 6.

Opcodes: `0x0A` (Imm), `0x2A` (Short Imm), `0x3A` (Reg). See Table 13.1 for the full family of legal forms.


31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x0A
rS
imm16
SUS
Cond
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x2A
rS
imm8
Shift
SUS
Cond
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0x3A
ignored
rS1
rS2
Shift
SUS
Cond

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Test if r1 is zero
CMPI r1, #0
BRAN|== @is_zero ; Branch if r1 == 0

; Test if r1 > r2 (unsigned)
CMPR r1, r2
BRAN|HI @r1_greater

; Test if r1 >= r2 (signed)
CMPR r1, r2
BRAN|>= @r1_greater_or_equal

; Range check: if (r1 >= 10 && r1 < 20)
CMPI r1, #10
BRAN|<< @out_of_range ; Branch if r1 < 10
CMPI r1, #20
BRAN|>= @out_of_range ; Branch if r1 >= 20
; r1 is in range [10, 20)
```

Example 13-13: Comparisons and Range Checking

Notes:

- Does not modify any register, only flags
- Use with conditional instructions or branches
- Remember: `CMP A, B` computes A - B
- For signed comparisons, use `>=`, «, », `<=`
- For unsigned comparisons, use `HI`, `LO`, `CS`, `CC`

CPCI / CPCR

Compare with Carry Immediate / Register

Applicability: SIRC-1, all revisions

Syntax:

```
CPCI rS, #imm16 ; Flags from rS - imm16 - C
CPCI rS, #imm8, shift ; Flags from shift(rS) - imm8 - C
CPCR rS1, rS2 ; Flags from rS1 - rS2 - C
CPCR rS1, rS2, shift ; Flags from shift(rS1) - rS2 - C
```

Operands: Same source operand forms as **`SBC`**. The incoming `C` flag is read as the borrow input; no destination register is written.

Description:

Subtracts the second operand and the borrow from the first operand but discards the result, only updating the status flags. It is the flag-only counterpart of **`SBC`** and continues a **`CMP`** across multiple words: **`CMP`** on the low words followed by **`CPC`** on the high words compares the whole multi-word values. In short immediate format with shift, the first source register is shifted before the immediate is subtracted.

Operation:

```
result = operand1 - operand2 - SR.C
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = (operand1[15] != operand2[15]) && (result[15] != operand1[15])
```

Flags: N Z C V

Write-back: None. The difference is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `CPCI ah, #0`); see Chapter 6.

Opcodes: `0x0B` (Imm), `0x2B` (Short Imm), `0x3B` (Reg). See Table 13.1 for the full family of legal forms.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0B						rS				imm16																SUS		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x2B						rS				imm8								Shift								SUS		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x3B						ignored				rS1				rS2				Shift								SUS		Cond			

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Unsigned 32-bit compare: is r1:r2 lower than r3:r4?
CMPR r2, r4 ; Low words
CPCR r1, r3 ; High words, borrowing from the low compare
BRAN|CS @first_is_lower ; C = borrow out of the whole 32-bit subtraction

; Signed 32-bit compare: is r1:r2 >= r3:r4?
CMPR r2, r4
CPCR r1, r3
BRAN|>= @first_is_gte ; N = V for the 32-bit result
```

Example 13-14: 32-bit Comparison with Borrow Propagation

Notes:

- Does not modify any register, only flags
- Always use after **CMP** (or **SUB**) for multi-word comparison; carry is treated as the borrow input
- After the final **CPC**, the `CS`, `CC`, `>=` and `«` conditions describe the whole multi-word comparison
- Z reflects only the result of this instruction, not the whole multi-word difference, so conditions that test Z (`==`, `!=`, `HI`, `LO`, `»`, `<=`) do not describe the combined value

TSAI / TSAR

Test AND Immediate / Test AND Register

Applicability: SIRC-1, all revisions

Syntax:

```
TSAI rS, #imm16 ; Test rS & imm16
TSAI rS, #imm8, shift ; Test shift(rS) & imm8
TSAR rS1, rS2 ; Test rS1 & rS2
TSAR rS1, rS2, shift ; Test shift(rS1) & rS2
```

Operands: Same source operand forms as **`AND`**. The result is used only for status flag updates; no destination register is written.

Description:

Performs bitwise AND but discards the result, only updating flags. In short immediate format with shift, the first source register is shifted before ANDing with the immediate. Used to test if specific bits are set without modifying the register. The zero flag indicates whether any tested bits were set.

Operation:

```
result = operand1 & operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: None. The AND result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `TSAI ah, #0`); see Chapter 6.

Opcodes: `0x0C` (Imm), `0x2C` (Short Imm), `0x3C` (Reg). See Table 13.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x0C`	rS	imm16	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x2C`	rS	imm8	Shift	SUS	Cond

31 30 29 28 27 26	25 24 23 22	21 20 19 18	17 16 15 14	13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x3C`	ignored	rS1	rS2	Shift	SUS	Cond

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Test if bit 5 is set
TSAI r1, #0x0020 ; Test r1 & 0x20
BRAN|!= @bit5_set ; Branch if bit 5 was set

; Test if any high byte bits are set
TSAI r2, #0xFF00
BRAN|!= @has_upper_bits

; Test if r1 and r2 have any bits in common
TSAR r1, r2
BRAN|== @no_common_bits ; Branch if no bits in common

; Test multiple bits
TSAI r3, #0xF000 ; Test high nibble
BRAN|== @upper_clear ; Branch if all tested bits clear
```

Example 13-15: Bit Testing with TSA

Notes:

- Does not modify any register, only flags
- Z=1 means all tested bits were 0
- Z=0 means at least one tested bit was 1
- Use for bit testing without side effects
- More efficient than AND + CMP for simple tests

TSOI / TSOR

Test OR Immediate / Test OR Register

Applicability: SIRC-1, all revisions

Syntax:

```
TSOI rS, #imm16 ; Test rS | imm16
TSOI rS, #imm8, shift ; Test shift(rS) | imm8
TSOR rS1, rS2 ; Test rS1 | rS2
TSOR rS1, rS2, shift ; Test shift(rS1) | rS2
```

Operands: Same source operand forms as **`ORR`**. The result is used only for status flag updates; no destination register is written.

Description:

Performs bitwise OR but discards the result, only updating flags. In short immediate format with shift, the first source register is shifted before ORing with the immediate. The zero flag is set only when every bit of both operands is zero, so a single **`TSOR`** tests whether two registers are both zero.

Operation:

```
result = operand1 | operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: None. The OR result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `TSOI ah, #0`); see Chapter 6.

Opcodes: `0x0D` (Imm), `0x2D` (Short Imm), `0x3D` (Reg). See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0D</td><td colspan="4">rS</td><td colspan="16">imm16</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x2D</td><td colspan="4">rS</td><td colspan="8">imm8</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x3D</td><td colspan="4">ignored</td><td colspan="4">rS1</td><td colspan="4">rS2</td><td colspan="8">Shift</td><td colspan="2">SUS</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Test if the 32-bit value in r1:r2 is zero
TSOR r1, r2
BRAN|== @is_zero ; Branch if both words are zero

; Test if either of two flag words has bit 15 set
TSOR r3, r4
BRAN|NS @flag_set
```

Example 13-16: Testing Two Values at Once with TSO

Notes:

- Does not modify any register, only flags
- Z=1 means every bit of both operands is 0
- Z=0 means at least one bit is 1 in one of the operands
- N=1 means bit 15 is set in at least one of the operands
- More efficient than ORR into a scratch register for testing whether two values are both zero

TSXI / TSXR

Test XOR Immediate / Test XOR Register

Applicability: SIRC-1, all revisions

Syntax:

```
TSXI rS, #imm16 ; Test rS ^ imm16
TSXI rS, #imm8, shift ; Test shift(rS) ^ imm8
TSXR rS1, rS2 ; Test rS1 ^ rS2
TSXR rS1, rS2, shift ; Test shift(rS1) ^ rS2
```

Operands: Same source operand forms as **`XOR`**. The result is used only for status flag updates; no destination register is written.

Description:

Performs bitwise XOR but discards the result, only updating flags. In short immediate format with shift, the first source register is shifted before XORing with the immediate. Used to test if bits match a specific pattern or to compare values for equality. Z=1 indicates the values are identical.

Operation:

```
result = operand1 ^ operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Flags: N Z (C and V cleared)

Write-back: None. The XOR result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode. This instruction only reads the register named by its register field and never writes it, so it does not raise a privilege-violation fault even when that field names a privileged register such as `ah` (for example, `TSXI ah, #0`); see Chapter 6.

Opcodes: `0x0E` (Imm), `0x2E` (Short Imm), `0x3E` (Reg). See Table 13.1 for the full family of legal forms.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`0x0E`						rS				imm16																SUS		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`0x2E`						rS				imm8								Shift								SUS		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`0x3E`						ignored				rS1				rS2				Shift								SUS		Cond			

Instruction Fields:

- **rS (Reg, bits 25–22), immediate and short-immediate forms:** source register; any of the 16 registers, `0x0`–`0xF`. No destination register is written.
- **ignored (R1, bits 25–22), register form only:** no destination register is written, so the CPU ignores this field. The assembler encodes a copy of `rS1`.

- **rS1 (R2, bits 21–18) and rS2 (R3, bits 17–14), register form only:** first and second source registers.
- **imm16 (Immediate Value, bits 21–6) or imm8 (Imm, bits 21–14):** 16-bit value (immediate form) or 8-bit value zero-extended to 16 bits (short-immediate form).
- **Shift (bits 13–6, short-immediate and register forms only):** the shift operand (SO, bit 13), type (ST, bits 12–10), and count (SA, bits 9–6). Types: `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is `0`–`15`; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **SUS (AF, bits 5–4):** status update source: `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved; N, Z, C and V are architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Test if r1 equals specific value
TSXI r1, #0x1234 ; Test r1 ^ 0x1234
BRAN|== @is_0x1234 ; Branch if r1 == 0x1234

; Test if two registers are equal
TSXR r1, r2 ; Test r1 ^ r2
BRAN|== @registers_equal ; Branch if r1 == r2

; Test if bit pattern matches
ANDI r3, #0x00FF ; Mask to low byte
TSXI r3, #0x00AA ; Test if low byte == 0xAA
BRAN|== @pattern_match
```

Example 13-17: Equality and Pattern Testing with TSX

Notes:

- Does not modify any register, only flags
- Z=1 means values are identical (all bits match)
- Can be used as equality test (alternative to CMP for == only)
- More efficient than XOR + CMP for simple equality tests

13.6 Summary

The ALU instructions form the computational core of the SIRCIS instruction set. Key points:

- All ALU operations work only on registers (load/store architecture)
- Three format variants provide flexibility (immediate, short+shift, register)
- Test variants (CMN, CNC, CMP, CPC, TSA, TSO, TSX) allow non-destructive testing
- Shift operations can be combined with ALU ops for powerful single-instruction operations
- Consistent flag behavior makes conditional execution predictable

CHAPTER 14

Memory Access Instructions

14.1 Overview

Memory access instructions transfer data between registers and memory. Following the load/store architecture, these are the only processing-unit instructions that access data memory; all arithmetic and logical operations must work on registers. The optional DMA coprocessor commands (Chapter 16) also transfer memory.

The SIRC-1 provides two primary memory operations:

- **`LOAD`** – Read from memory into a register
- **`STOR`** – Write from a register to memory

`LOAD` and **`STOR`** each support four memory addressing forms. Both support indirect addressing with an immediate displacement and indirect addressing with a register offset; **`LOAD`** additionally supports post-increment and **`STOR`** additionally supports pre-decrement:

- Indirect with immediate displacement: `(#disp, addr)`
- Indirect with register offset: `(r0, addr)`
- Post-increment (**`LOAD`** only): `(#disp, addr)+` or `(r0, addr)+`
- Pre-decrement (**`STOR`** only): `-(#disp, addr)` or `-(r0, addr)`

Important: **`STOR`**'s register-offset forms support an optional shift suffix, applied to the source register value before it is written to memory. **`LOAD`** does not support a shift suffix on any form: the offset register `r0` is always used unshifted. See Section 14.7 for details. Shifts are also **not** supported by **`MOVI`** and **`MOVR`**, which load immediate values and copy register values directly.

14.2 Effective Address Calculation

For all memory operations, the effective address (EA) is calculated as:

$$\text{EA} = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Displacement}) \tag{14.1}$$

Where:

- Address Register is one of: `l`, `a`, `s`, or `p`
- Displacement is either a 16-bit signed immediate value or the unshifted contents of the offset register `r0`

The high word supplies the segment and is not changed by the displacement; only the low word participates in the addition, modulo 2^{16}.

14.3 Legal Forms

Instruction	Syntax	Encoding	Notes
LOAD	`LOAD rD, (#disp, addr)`	`0x14`	Immediate displacement
LOAD	`LOAD rD, (r0, addr)`	`0x15`	Register offset; no shift suffix
LOAD	`LOAD rD, (#disp, addr)+`	`0x16`	Immediate displacement; post-increment
LOAD	`LOAD rD, (r0, addr)+`	`0x17`	Register offset; no shift suffix; post-increment
STOR	`STOR (#disp, addr), rS`	`0x10`	Immediate displacement
STOR	`STOR (r0, addr), rS[, shift]`	`0x11`	Register offset; optional source shift
STOR	`STOR -(#disp, addr), rS`	`0x12`	Immediate displacement; pre-decrement
STOR	`STOR -(r0, addr), rS[, shift]`	`0x13`	Register offset; optional shift; pre-decrement

Table 14.1: Legal Memory Instruction Forms

Zero displacement: Immediate-displacement memory forms may omit `#0`. Thus `LOAD rD, (addr)`, `LOAD rD, (addr)+`, `STOR (addr), rS`, and `STOR -(addr), rS` assemble as the matching `#0` forms. This shorthand does not add new addressing modes: **`LOAD`** has no pre-decrement form and **`STOR`** has no post-increment form.

Aliased register writes: In post-increment **`LOAD`** forms the destination register must not be either half of the auto-updated address-register pair; in pre-decrement **`STOR`** forms the source register must not be either half of the auto-updated pair. The auto-updated pair must not be `p` in either form, because the update would compete with the program-counter advance. The result of such an encoding is architecturally undefined and the assembler rejects it (Appendix C).

14.4 Common Memory Semantics

Conditional execution and timing follow the rules in Chapter 11 and Appendix B. Immediate-displacement forms add the 16-bit signed displacement to the low word of the selected address register. Register-offset forms add the unshifted `r0` to that low word; `r0` is never shifted, even on the register-offset forms of **`STOR`**, which shift the value being stored instead. The high word of the selected address register supplies the segment and is unchanged.

A post-increment **`LOAD`** calculates its memory address from the original address register value, then increments the address register low word by one word in write-back, after the memory access. A pre-decrement **`STOR`** uses the address register low word decremented by one word, plus the displacement, as its memory address, and the low word is decremented in write-back, after the memory access.

Destination register writes and address-register auto-updates occur in write-back after the memory-access phase, so a fault that cancels the instruction before write-back leaves them undone (write-back still runs, but as a no-op). Data bus, data bus-protection, segment-overflow, and privilege-violation faults are all retryable: the saved return address is the address of the faulting instruction, so returning from the fault handler with **`RETE`** retries the same memory access (see Chapter 6).

Memory instructions can raise data bus and data bus-protection faults during the memory-access phase. The 16-bit displacement is signed two's-complement; effective-address calculation can raise a segment-overflow fault when `SR.A` (Trap on Address Overflow) is set and the low-word address calculation wraps. For post-increment and pre-decrement forms the address-register auto-update is checked separately, and a wrap of either the effective address or the updated low word raises the fault (Section 4.8). With `SR.A` set, a memory instruction in the last instruction slot of a segment also raises a segment-overflow fault, in phase 3 and before its memory access, because it uses the wrapped next-instruction address; a **LOAD** whose destination is `pl` is the exception (Section 6.2.5). This segment-overflow fault is retryable: the saved return address is the address of the faulting instruction (see Chapter 6). A **LOAD** raises a privilege-violation fault in protected mode when its register field names a privileged high address register (`ah`, `lh`, `ph`, or `sh`); this applies to the **LOAD** destination register only: **STOR** merely reads its register field, so storing a high address register is permitted. Naming `sr` is permitted in protected mode: **LOAD** into `sr` updates only bits 7–0, and **STOR** of `sr` stores the redacted value (bits 15–8 read as zero). Post-increment and pre-decrement address-register auto-update writes only the low word of the pair, so it never changes the high word and never raises a privilege-violation fault. Documented memory forms do not raise invalid-opcode faults. Instruction fetch can still raise the normal fetch faults before a memory instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits. Processing-unit encodings outside the documented memory forms are reserved or architecturally undefined; they do not raise an invalid-opcode fault.

14.5 Status Flag Effects

Memory instructions do not update status flags as a side effect. All four condition flags are preserved regardless of addressing form, shift result, or auto-update side effect. The one exception is a **LOAD** whose destination is `sr`: it writes the loaded value to the status register (bits 7–0 only in protected mode; every bit except `SR.E` in supervisor mode), which replaces the flags. This applies only to the memory-access **LOAD** forms (`0x14`–`0x17`) and **STOR** (`0x10`–`0x13`); see Chapter 13 for the flag behavior of **MOVI** (`0x07`) and **MOVR** (`0x37`). The AF field (bits 5–4) is not a status-update source for these opcodes; it selects the address-register pair used to form the effective address.

Instruction	**N**	**Z**	**C**	**V**	**Meaning**
LOAD (`0x14`–`0x17`)	-	-	-	-	Flags are preserved, unless the destination is `sr`.
STOR (`0x10`–`0x13`)	-	-	-	-	Flags are preserved.

Table 14.2: Memory Instruction Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions.

LOAD

Load from Memory

Applicability: SIRC-1, all revisions

Syntax:

```
LOAD rD, (#disp, addr) ; rD = memory[addr + disp]
LOAD rD, (addr) ; equivalent to LOAD rD, (#0, addr)
LOAD rD, (r0, addr) ; rD = memory[addr + r0]
LOAD rD, (#disp, addr)+ ; rD = memory[addr + disp]; addr += 1
LOAD rD, (addr)+ ; equivalent to LOAD rD, (#0, addr)+
LOAD rD, (r0, addr)+ ; rD = memory[addr + r0]; addr += 1
```

Operands: `rD` is the destination register: any register, including `sr`, the address-register halves, and `pl`. A privileged high address register (`ah`, `lh`, `ph`, or `sh`) raises a privilege-violation fault in protected mode. A **LOAD** into `pl` is a jump within the current segment: the loaded value replaces the advanced program counter, and an odd value raises an alignment fault at the next fetch. A supervisor-mode **LOAD** into `ph` changes the segment of the next fetch. `addr` is the address-register pair that supplies the segment and base low word. Immediate-displacement forms use a 16-bit signed displacement. Register-offset forms use `r0` as the displacement register, always unshifted; **LOAD** does not accept a shift suffix.

Description:

Loads a 16-bit value from memory into the destination register. Post-increment forms increment the address register after the load in a single atomic operation.

Operation:

```
effective_address = addr + disp
destination = memory[effective_address]
if post_increment then
    addr.low = addr.low + 1
endif
; Flags are preserved
```

Flags: Preserved, unless the destination is `sr`, which replaces the flags with the loaded value.

Write-back: If the condition is true, writes the loaded value to `rD`. Post-increment forms also write the incremented address-register low word.

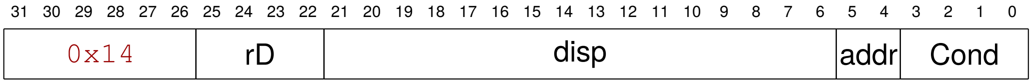
Exceptions: May raise data bus or data bus-protection faults during the memory read. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. For post-increment forms, a wrap of either the effective address or the updated low word raises the fault (Section 4.8). This fault is retryable (see Chapter 6). May raise a privilege-violation fault in protected mode if `rD` names a privileged high address register (`ah`, `lh`, `ph`, or `sh`).

Condition field: If the condition is false, no memory read occurs, no destination register is written, no address register is incremented, and status flags are preserved.

Timing: 6 cycles with no wait states. Each wait state on either an instruction fetch or the data access adds one cycle (3w cycles for w wait states per access).

Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection. A privilege-violation fault is raised in protected mode if `rD` names a privileged high address register (`ah`, `lh`, `ph`, or `sh`); a **LOAD** into `sr` is permitted but updates only bits 7–0.

Opcodes: `0x14` (Indirect Imm), `0x15` (Indirect Reg), `0x16` (Post-Inc Imm), `0x17` (Post-Inc Reg). See Table 14.1 for the full family of legal forms.



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x15						rD				0x0				rO				0	000			0x0				addr		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x16						rD				disp																addr		Cond			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x17						rD				0x0				rO				0	000			0x0				addr		Cond			

Instruction Fields:

- **rD (Reg, bits 25–22, Immediate format, `0x14`/`0x16`):** Destination register.
- **rD (R1, bits 25–22, Register format, `0x15`/`0x17`):** Destination register.
- **R2 (bits 21–18, Register format, `0x15`/`0x17`):** Unused, fixed at `0`. The assembler does not accept a shift suffix on any **`LOAD`** form: **`LOAD`** has no field that shifts the offset register or the loaded value. Use **`SHFT`** on `rO` before the **`LOAD`** if a scaled offset is needed.
- **Shift (bits 13–6, Register format, `0x15`/`0x17`):** Reserved, fixed at `0`. A non-zero value is architecturally undefined; implementations are expected to ignore it, since a shift would apply to the unused R2 field and **`LOAD`** takes its offset from R3, so portable software must not rely on that.
- **rO (R3, bits 17–14, Register format, `0x15`/`0x17`):** Offset register, used unshifted.
- **disp (Immediate Value, bits 21–6, Immediate format, `0x14`/`0x16`):** 16-bit signed displacement.
- **addr (AF, bits 5–4):** Address-register pair – `00` = `l`, `01` = `a`, `10` = `s`, `11` = `p`. Not a status-update source for these opcodes.
- **Condition field (bits 3–0):** All 16 condition-field encodings are legal; see **Condition field** below.

```
; Load from fixed offset
LOAD r1, (#4, a) ; r1 = memory[a + 4]

; Load using variable offset
LOAD r2, (r3, a) ; r2 = memory[a + r3]

; Array iteration
LOAD r4, (#0, a)+ ; r4 = *a; a += 1

; Stack pop
LOAD r5, (#0, s)+ ; r5 = *s; s += 1

; LOAD has no shift suffix: scale the offset register first with SHFT
SHFT r2, LSL #2 ; r2 = r2 << 2
LOAD r3, (r2, a) ; r3 = memory[a + r2]
```

Example 14-1: Indirect, Post-Increment, and Register-Offset Loads

STOR

Store to Memory

Applicability: SIRC-1, all revisions

Syntax:

```
STOR (#disp, addr), rS ; memory[addr + disp] = rS
STOR (addr), rS ; equivalent to STOR (#0, addr), rS
STOR (r0, addr), rS ; memory[addr + r0] = rS
STOR -(#disp, addr), rS ; addr -= 1; memory[addr + disp] = rS
STOR -(addr), rS ; equivalent to STOR -(#0, addr), rS
STOR -(r0, addr), rS ; addr -= 1; memory[addr + r0] = rS

; With optional shift on the source value (register-offset memory forms only)
STOR (r0, addr), rS, shift ; Shift rS, then store
STOR -(r0, addr), rS, shift ; Decrement, shift rS, then store
```

Operands: `rS` is the source register. **`STOR`** only reads it, so it may name a privileged high address register (`ah`, `lh`, `ph`, or `sh`) in either mode without a fault. `addr` is the address-register pair that supplies the segment and base low word. Immediate-displacement forms use a 16-bit signed displacement. Register-offset forms use `r0` as the displacement register and may apply a shift to the source value before storing it.

Description:

Stores a 16-bit value from a register into memory. Pre-decrement forms use the decremented address for the store and write the decremented address register value back after the memory access, useful for stack push operations.

Operation:

```
if pre_decrement then
    decremented_low = addr.low - 1
    effective_address = (addr.high : decremented_low) + offset
    memory[effective_address] = shift_if_encoded(source)
    addr.low = decremented_low
else
    effective_address = addr + disp
    memory[effective_address] = shift_if_encoded(source)
endif
; Source register and flags are preserved
```

Flags: Preserved.

Write-back: If the condition is true, writes the source value to memory. Pre-decrement forms also write the decremented address-register low word.

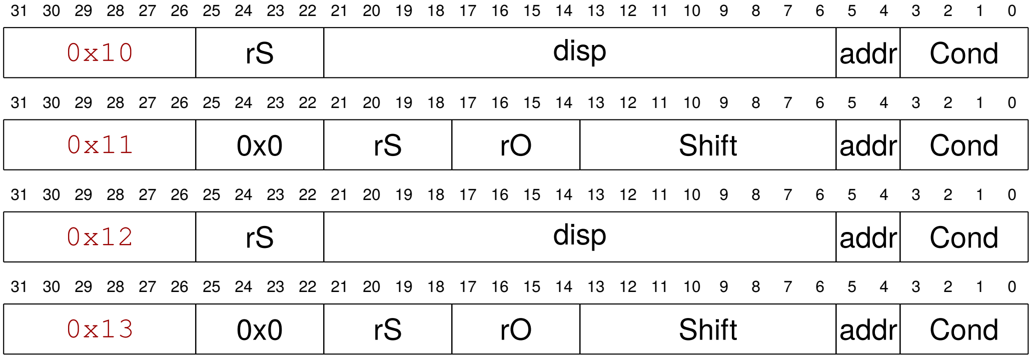
Exceptions: May raise data bus or data bus-protection faults during the memory write. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. For pre-decrement forms, a wrap of either the effective address or the updated low word raises the fault (Section 4.8). This fault is retryable (see Chapter 6).

Condition field: If the condition is false, no memory write occurs, no address register is decremented, and status flags are preserved.

Timing: 6 cycles with no wait states. Each wait state on either an instruction fetch or the data access adds one cycle (3w cycles for w wait states per access).

Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection. **`STOR`** only reads `rS`, so it may name any register, including the privileged ones; in protected mode `sr` is stored with bits 15–8 redacted to zero.

Opcodes: `0x10` (Indirect Imm), `0x11` (Indirect Reg), `0x12` (Pre-Dec Imm), `0x13` (Pre-Dec Reg). See Table 14.1 for the full family of legal forms.



Instruction Fields:

- **rS (Reg, bits 25–22, Immediate format, `0x10`/`0x12`):** Source register.
- **R1 (bits 25–22, Register format, `0x11`/`0x13`):** Reserved, written as zero; a non-zero value is architecturally undefined.
- **rS (R2, bits 21–18, Register format, `0x11`/`0x13`):** Source register.
- **rO (R3, bits 17–14, Register format, `0x11`/`0x13`):** Offset register.
- **disp (Immediate Value, bits 21–6, Immediate format, `0x10`/`0x12`):** 16-bit signed displacement.
- **Shift Type (bits 12–10, Register format, `0x11`/`0x13`):** `000` none, `001` LSL, `010` LSR, `011` ASL, `100` ASR, `101` RTL, `110` RTR, `111` reserved. Applies to `rS` before the value is written to memory.
- **Shift Amount (bits 9–6, Register format, `0x11`/`0x13`):** Immediate count 0–15, or (Shift Operand bit set) a register holding the count; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`; `NUL` ignores it) – see Chapter 9.
- **addr (AF, bits 5–4):** Address-register pair – `00` = `l`, `01` = `a`, `10` = `s`, `11` = `p`. Not a status-update source for these opcodes.
- **Condition field (bits 3–0):** All 16 condition-field encodings are legal; see **Condition field** below.

```
; Store to fixed offset
STOR (#8, a), r1 ; memory[a + 8] = r1

; Store using variable offset
STOR (r3, a), r2 ; memory[a + r3] = r2

; Store with shift (data shifted before storing)
STOR (r2, a), r1, LSL #1 ; memory[a + r2] = r1 << 1
STOR (r3, a), r1, LSR #3 ; memory[a + r3] = r1 >> 3

; Stack push
STOR -(#0, s), r4 ; s -= 1; memory[s] = r4
```

Example 14-2: Indirect, Shifted, and Pre-Decrement Stores

14.6 Common Memory Access Patterns

14.6.1 Structure Field Access

```
; struct { int16_t x, y, z; } point;
; 'a' points to point
LOAD r1, (#0, a) ; r1 = point.x
LOAD r2, (#1, a) ; r2 = point.y
LOAD r3, (#2, a) ; r3 = point.z
```

14.6.2 Array Iteration

```
; Process array of 16-bit values
; 'a' = array base, r7 = count
:loop
    LOAD r1, (#0, a)+ ; Load element, advance
    ; ... process r1 ...
    SUBI r7, #1
    BRAN|!= @loop
```

14.6.3 Stack Operations

```
; Push registers
STOR -(#0, s), r1
STOR -(#0, s), r2
STOR -(#0, s), r3

; Pop registers (reverse order)
LOAD r3, (#0, s)+
LOAD r2, (#0, s)+
LOAD r1, (#0, s)+
```

14.7 Shift Operations with Memory Instructions

Only **`STOR`**'s register-offset forms support an optional shift suffix, applied to the source register value before it is written to memory. **`LOAD`** does not support a shift suffix on any form – the offset register `rO` is always used unshifted, and there is no field that shifts the loaded value. This asymmetry is architectural, not an oversight: use **`SHFT`** on the offset register before a **`LOAD`** if a scaled index is needed.

14.7.1 STOR with Shift

When storing to memory, the source register value is shifted **before** being written to memory. The original register value is **not** modified:

```
STOR (r2, a), r1, LSL #1 ; memory[a + r2] = r1 << 1 (r1 unchanged)
STOR (r2, a), r1, ASL #2 ; memory[a + r2] = r1 << 2 (r1 unchanged)
STOR (r4, a), r1, ASR #4 ; memory[a + r4] = r1 >> 4 (r1 unchanged)
```

14.7.2 Supported Shift Types

All standard shift types are supported:

- **LSL** – Logical Shift Left (fill with zeros)
- **LSR** – Logical Shift Right (fill with zeros)

- **ASL** – Arithmetic Shift Left (same stored value as LSL; no flags are updated)
- **ASR** – Arithmetic Shift Right (sign-extend)
- **RTL** – Rotate Left within the operand
- **RTR** – Rotate Right within the operand

14.7.3 Shift Limitations

Important: The shift suffix is **only** supported on **`STOR`**'s register-offset forms. The following do **not** support a shift suffix:

- Any **`LOAD`** form, register-offset included – `LOAD r1, (r2, a)` – No shift suffix exists
- Loading immediate values: `MOVI r1, #0x1234` – No shift allowed
- Copying registers: `MOVR r1, r2` – No shift allowed
- Immediate-displacement memory load: `LOAD r1, (#0, a)` – No shift allowed
- Immediate-displacement memory store: `STOR (#0, a), r1` – No shift allowed

For any of these, load or store the unshifted value and use an ALU instruction or **`SHFT`** for the shift operation.

14.7.4 Use Cases

Scaled Indexing

`LOAD` has no shift suffix, so scaling an index for array access takes a separate **`SHFT`** before the **`LOAD`**:

```
; r2 holds an element index; each element occupies 4 words
SHFT r2, LSL #2 ; r2 = r2 << 2
LOAD r3, (r2, a) ; r3 = memory[a + r2]
```

Bit Manipulation

Shifting the source register before a **`STOR`** is useful for packing a scaled or shifted value into memory without a separate ALU instruction:

```
; Store a scaled value
STOR (r3, a), r2, LSL #2 ; memory[a + r3] = r2 * 4
```

Combined with Auto-Increment/Decrement

`STOR`'s shift suffix works with pre-decrement mode:

```
; Decrement, shift the source value, and store
STOR -(r2, a), r1, LSR #2 ; a -= 1; memory[a + r2] = r1 >> 2
```

14.7.5 Status Flags

Note: Unlike shifts in ALU instructions, a shift performed during a **`STOR`** cannot update the status register flags. For flag updates based on a shift result, use an ALU instruction or the **`SHFT`** meta-instruction (see Chapter 13).

CHAPTER 15

Control Flow Instructions

15.1 Overview

Control flow in SIRC-1 is built from effective-address instructions. The instructions compute an address and write them to an address-register pair. Jumps and calls are produced by targeting the program-counter pair `p`.

SIRC-1 provides these control-flow and address-calculation operations:

- **`LDEA`** – Load Effective Address into an address-register pair
- **`LDEL`** – Load Effective Address and Link; also saves the return address in `l`
- **`BRAN`** – PC-relative assembler meta-instruction for `LDEA p, ...`
- **`BRSR`** – PC-relative assembler meta-instruction for `LDEL p, ...`
- **`RETS`** – Return meta-instruction for `LDEA p, (#0, l)`
- **`LJMP`** – Absolute-jump assembler meta-instruction for `LDEA p, ...`
- **`LJSR`** – Absolute-call assembler meta-instruction for `LDEL p, ...`

All encoded control-flow instructions can be conditional. If the condition is false, the instruction is skipped and has no side effects (the program-counter advance is still checked for segment overflow; see Section 6.2.5), as described in Chapter 11.

15.2 Legal Forms

Instruction	Syntax	Encoding	Notes
LDEA	`LDEA dest, (#disp, src)`	`0x18`	Immediate-displacement effective address
LDEA	`LDEA dest, (r0, src)`	`0x19`	Register-offset effective address
LDEA	`LDEA dest, -(#disp, src)`	`0x1A`	Pre-decrement source, immediate displacement
LDEA	`LDEA dest, -(r0, src)`	`0x1B`	Pre-decrement source, register offset
LDEL	`LDEL dest, (#disp, src)`	`0x1C`	Immediate-displacement effective address and link; `dest` excludes `l`
LDEL	`LDEL dest, (r0, src)`	`0x1D`	Register-offset effective address and link; `dest` excludes `l`
LDEL	`LDEL dest, (#disp, src)+`	`0x1E`	Post-increment source, immediate displacement; `dest` excludes `l`
LDEL	`LDEL dest, (r0, src)+`	`0x1F`	Post-increment source, register offset; `dest` excludes `l`
BRAN	`BRAN #disp`	`LDEA` meta	Assembles to `LDEA p, (#disp, p)`
BRAN	`BRAN @label`	`LDEA` meta	PC-relative linker-computed displacement
BRAN	`BRAN $name`	`LDEA` meta	Displacement from an `.EQU` placeholder
BRSR	`BRSR #disp`	`LDEL` meta	Assembles to `LDEL p, (#disp, p)`
BRSR	`BRSR @label`	`LDEL` meta	PC-relative linker-computed call
BRSR	`BRSR $name`	`LDEL` meta	Displacement from an `.EQU` placeholder
RETS	`RETS`	`LDEA` meta	Assembles to `LDEA p, (#0, l)`
LJMP	`LJMP src`	`LDEA` meta	Assembles to `LDEA p, (#0, src)`
LJMP	`LJMP src, #disp`	`LDEA` meta	Assembles to `LDEA p, (#disp, src)`
LJMP	`LJMP src, r0`	`LDEA` meta	Assembles to `LDEA p, (r0, src)`
LJSR	`LJSR src`	`LDEL` meta	Assembles to `LDEL p, (#0, src)`
LJSR	`LJSR src, #disp`	`LDEL` meta	Assembles to `LDEL p, (#disp, src)`
LJSR	`LJSR src, r0`	`LDEL` meta	Assembles to `LDEL p, (r0, src)`
LJSR	`LJSR (#disp, src)+`	`LDEL` meta	Assembles to `LDEL p, (#disp, src)+`
LJSR	`LJSR (r0, src)+`	`LDEL` meta	Assembles to `LDEL p, (r0, src)+`

Table 15.1: Legal Control-Flow and Effective-Address Forms

Address registers: `src` and `dest` are address-register pairs: `l`, `a`, `s`, or `p`. The branch meta-instructions always use `p` as both the source and destination address-register pair. `l` is never a legal **`LDEL`** destination, because **`LDEL`** always writes the return address to `l`; the destination and the link write would alias. **`LDEL`** destinations are `a`, `s`, or `p`.

Zero displacement: Immediate-displacement forms may omit `#0`. Thus `LDEA dest, (src)`, `LDEA dest, -(src)`, `LDEL dest, (src)`, and `LDEL dest, (src)+` assemble as the matching `#0` forms. For non-auto-update absolute jumps and calls, the meta-instruction syntax is `LJMP src` or `LJSR src`; `LJMP (src)` and `LJSR (src)` are not public syntax. The post-increment call shorthand `LJSR (src)+` is legal and assembles as `LDEL p, (#0, src)+`.

Aliased register writes: Instructions that write the same address-register pair through more than one write-back path have architecturally undefined behavior. The assembler rejects exactly these forms: **`LDEA`** pre-decrement with destination equal to source; **`LDEL`** with destination `l` (any form); **`LDEL`** post-increment with source `l` or with destination equal to source; **`LJSR`** post-increment from `l` or `p`; **`LOAD`** post-increment whose destination is a half of the auto-updated pair; **`STOR`** pre-decrement whose source is a half of the auto-updated pair; and any auto-update of `p` (`(p)+` or `-(p)` in any instruction), because the update would compete with the program-counter advance.

Shift suffixes: Control-flow and effective-address displacements are not shifted. Register-offset forms do not accept shift suffixes in the assembler.

15.3 Common Semantics

Conditional execution and timing follow the rules in Chapter 11 and Appendix B. The high word of the selected source address register supplies the segment. The target low word is the selected source low word plus the immediate or register displacement. A pre-decrement **`LDEA`** decrements the source low word by one word before the displacement is applied and writes the decremented value back. A post-increment **LDEL** calculates the target address from the original source value and increments the source low word by one word in write-back. **LDEL**, **BRSR**, and **LJSR** save the normal next-instruction address in `l`.

In protected mode, address-register-pair write-back is allowed only when every written high word would remain unchanged. If any high word would change, the instruction raises a privilege-violation fault instead of completing write-back. Direct writes to high address registers also raise a privilege-violation fault.

Program-counter, destination address-register, source auto-update, and link-register updates occur after condition evaluation. If a fault cancels the instruction before write-back, those effects do not complete (write-back still runs, but as a no-op).

Control-flow and effective-address instructions do not perform data-memory access and do not raise data bus, data bus-protection, or invalid-opcode faults in documented forms. Effective-address calculation can raise a segment-overflow fault when `SR.A` (Trap on Address Overflow) is set and the 16-bit low-word address calculation wraps. For post-increment and pre-decrement forms, the address-register auto-update is checked separately, and a wrap of either the effective address or the updated low word raises the fault (Section 4.8). With `SR.A` set, an instruction in the last slot of a segment also raises a segment-overflow fault if it uses the wrapped next-instruction address, either as the next program counter (condition false) or as the return address saved by **LDEL**, **BRSR**, or **LJSR**; a taken branch or jump that saves no return address does not (Section 6.2.5). Instruction fetch can still raise the normal fetch faults before the instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits.

15.4 Status Flag Effects

Control-flow and effective-address instructions do not update status flags. All four condition flags are preserved regardless of whether the instruction branches, links, faults, or is skipped. Control-flow instructions do not support status update override syntax.

Instruction	N	Z	C	V	Meaning
LDEA	-	-	-	-	Flags are preserved.
LDEL	-	-	-	-	Flags are preserved.

Table 15.2: Control-Flow Instruction Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions. Meta-instructions (**BRAN**, **BRSR**, **RETS**, **LJMP**, **LJSR**) inherit the flag-preservation behavior of the **LDEA** or **LDEL** form they assemble to.

15.5 Branch Meta-Instructions

BRAN

Branch Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
BRAN #disp ; p = p + disp
BRAN @label ; p = p + label offset
BRAN|cond #disp ; Conditional PC-relative branch
```

Operands: `#disp` is a signed 16-bit word displacement (`−32768` to `+32767`) added to the low word of `p` modulo 2^{16}; a branch cannot change the program segment, so the displacement wraps within the current 64K segment, unless `SR.A` is set, in which case a wrap raises a segment-overflow fault. `@label` is resolved by the linker to a word displacement from the address of the branch instruction itself.

Description:

Branches relative to the current program counter. **`BRAN`** is an assembler meta-instruction, not a distinct CPU opcode. The displacement base is the branching instruction's own address, so `BRAN #0` branches to itself and `BRAN #2` falls through to the next instruction. Non-PC-relative computed jumps use **`LDEA p, (#disp, src)`** or **`LDEA p, (r0, src)`**.

Operation:

```
; Assembles to LDEA p, (#disp, p)
if condition:
    p = p + displacement
```

Flags: Preserved.

Write-back: Inherits **`LDEA`** write-back to `p`.

Exceptions: Same as **`LDEA`**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no branch occurs and status flags are preserved.

Timing: Same as **`LDEA`**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **`LDEA`**. Protected-mode branches must preserve the high word of `p`.

Assembles to: **`LDEA`** with `p` as the source and destination. See Table 15.1 for the full family of legal forms.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`0x18`						00		p		disp																p		Cond			

Instruction Fields:

- **Register (bits 25–22):** bits 23–22 are fixed at `0x3` (`p`) by the meta-instruction. Bits 25–24 are reserved for future use and always written as zero.
- **AF (bits 5–4):** fixed at `0x3` (`p`) by the meta-instruction.
- **disp (Immediate Value, bits 21–6):** signed 16-bit displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Unconditional branch forward
BRAN #20 ; Jump ahead 20 words

; Backward branch (loop)
:loop
    ; ... loop body ...
    BRAN|!= @loop ; Jump back if not zero
```

Example 15-1: Unconditional and Conditional Branching

BRSR

Branch to Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
BRSR #disp ; l = next PC; p = p + disp
BRSR @label ; l = next PC; p = p + label offset
BRSR|cond #disp ; Conditional PC-relative call
```

Operands: `#disp` is a signed 16-bit word displacement (`−32768` to `+32767`) added to the low word of `p` modulo 2^{16}; a call cannot change the program segment, so the displacement wraps within the current 64K segment, unless `SR.A` is set, in which case a wrap raises a segment-overflow fault. `@label` is resolved by the linker to a word displacement from the address of the call instruction itself.

Description:

Calls a PC-relative subroutine by saving the return address in `l` and branching to the computed target. The displacement base is the branching instruction's own address, so the **BRSR** link value is that address plus 2 (the address of the next instruction). Return with **RETS**, which assembles to `LDEA p, (#0, l)`.

Operation:

```
; Assembles to LDEL p, (#disp, p)
if condition:
    l = address_of_next_instruction
    p = p + displacement
```

Flags: Preserved.

Write-back: Inherits **LDEL** write-back to `l` and `p`.

Exceptions: Same as **LDEL**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no call occurs, `l` is not written, and status flags are preserved.

Timing: Same as **LDEL**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **LDEL**. Protected-mode calls must preserve the high words written to `l` and `p`. Because the link write sets `lh` to the current `ph`, a protected-mode call faults unless `lh` already equals `ph`; protected-mode code must keep `l` in the program segment.

Assembles to: **LDEL** with `p` as the source and destination. See Table 15.1 for the full family of legal forms.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x1C						00		p		disp																p		Cond			

Instruction Fields:

- **Register (bits 25–22):** bits 23–22 are fixed at `0x3` (`p`) by the meta-instruction. Bits 25–24 are reserved for future use and always written as zero.
- **AF (bits 5–4):** fixed at `0x3` (`p`) by the meta-instruction.
- **disp (Immediate Value, bits 21–6):** signed 16-bit displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; PC-relative call
BRSR @function

; Return from the subroutine
RETS
```

Example 15-2: Calling and Returning with BRSR/RETS

RETS

Return from Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
RETS ; p = l
RETS|cond ; Conditional return
```

Operands: None. The source address-register pair is implicitly `l`; the destination is implicitly `p`.

Description:

Returns from a subroutine by copying the link register to the program counter. Used after **LJSR** or **BRSR**.

Operation:

```
; Assembles to LDEA p, (#0, l)
if condition:
    p = l
```

Flags: Preserved.

Write-back: Inherits **LDEA** write-back to `p`.

Exceptions: Same as **LDEA**; may raise protected-mode privilege faults if the return would change `p`'s high word.

Condition field: If the condition is false, no return occurs and status flags are preserved.

Timing: Same as **LDEA**: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.

Assembles to: `LDEA p, (#0, l)`. See Table 15.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x18</td><td colspan="2">00</td><td colspan="2">p</td><td colspan="16">0x0000</td><td colspan="2">l</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** bits 23–22 are fixed at `0x3` (`p`) by the meta-instruction. Bits 25–24 are reserved for future use and always written as zero.
- **AF (bits 5–4):** fixed at `0x0` (`l`) by the meta-instruction.
- **Immediate (bits 21–6):** fixed at `0x0000` by the meta-instruction.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
:my_function
    ; ... function body ...
    ; Return to caller
    RETS
```

Example 15-3: Returning from a Subroutine

15.6 Load Effective Address

LDEA

Load Effective Address

Applicability: SIRC-1, all revisions

Syntax:

```
LDEA dest, (#disp, src) ; dest = src + offset
LDEA dest, (src) ; equivalent to LDEA dest, (#0, src)
LDEA dest, (r0, src) ; dest = src + r0
LDEA dest, -(#disp, src) ; src -= 1; dest = src + offset
LDEA dest, -(src) ; equivalent to LDEA dest, -(#0, src)
LDEA dest, -(r0, src) ; src -= 1; dest = src + r0
LDEA|cond dest, (#disp, src)
```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `r0` as a displacement. Pre-decrement forms also update `src`.

Description:

Computes an effective address and loads it into the destination address-register pair. This is useful for pointer arithmetic and for control flow when the destination is `p`. Pre-decrement forms also update the source address register pair.

Operation:

```
if condition:
    if pre_decrement:
        src.low = src.low - 1
    dest.high = src.high
    dest.low  = src.low + displacement
```

Flags: Preserved.

Write-back: If the condition is true, writes the computed address to `dest`. Pre-decrement forms also write the decremented source address back to `src`.

Exceptions: May raise a segment-overflow fault during effective-address calculation or source auto-update when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.

Condition field: If the condition is false, no destination or source address register is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.

Opcodes: `0x18` (Immediate), `0x19` (Register), `0x1A` (Pre-Dec Immediate), `0x1B` (Pre-Dec Register). See Table 15.1 for the full family of legal forms.

31–26	25–24	23–22	21–6	5–4	3–0
`0x18`	00	dest	disp	src	Cond

31–26	25–24	23–22	21–18	17–14	13	12–10	9–6	5–4	3–0
`0x19`	00	dest	0x0	rO	0	000	0x0	src	Cond

31–26	25–24	23–22	21–6	5–4	3–0
`0x1A`	00	dest	disp	src	Cond

31–26	25–24	23–22	21–18	17–14	13	12–10	9–6	5–4	3–0
`0x1B`	00	dest	0x0	rO	0	000	0x0	src	Cond

Instruction Fields:

- **dest (Reg or R1, bits 25–22):** bits 23–22 hold the destination address-register pair; `0x0` = `l`, `0x1` = `a`, `0x2` = `s`, `0x3` = `p`. Bits 25–24 are reserved for future use and always written as zero.
- **src (AF, bits 5–4):** source address-register pair; `0x0` = `l`, `0x1` = `a`, `0x2` = `s`, `0x3` = `p`. `p` must not be used with an auto-update form.
- **offset (Immediate Value, bits 21–6, immediate forms only):** 16-bit signed displacement.
- **rO (R3, bits 17–14, register forms only):** a general-purpose register supplying the displacement.
- **R2 and shift fields (bits 21–18 and 13–6, register forms only):** reserved and written as zero; a non-zero value is architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Pointer arithmetic
LDEA a, (#16, a) ; a = a + 16

; Computed jump
LDEA p, (r1, a) ; p = a + r1

; Pre-decrement source pointer and copy nearby address
LDEA a, -(#0, s) ; s -= 1; a = s
```

Example 15-4: Pointer Arithmetic and Computed Jumps with LDEA

LDEL

Load Effective Address and Link

Applicability: SIRC-1, all revisions

Syntax:

```
LDEL dest, (#disp, src) ; dest = src + offset; l = next PC
LDEL dest, (src) ; equivalent to LDEL dest, (#0, src)
LDEL dest, (r0, src) ; dest = src + r0; l = next PC
LDEL dest, (#disp, src)+ ; dest = src + offset; l = next PC; src += 1
LDEL dest, (src)+ ; equivalent to LDEL dest, (#0, src)+
LDEL dest, (r0, src)+ ; dest = src + r0; l = next PC; src += 1
LDEL|cond dest, (#disp, src)
```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `r0` as a displacement. Post-increment forms also update `src`.

Description:

Computes an effective address, saves the return address to `l`, and writes the computed address to the destination address-register pair. The source is read before the link is written, so `LDEL p, (#0, l)` jumps to the old value of `l`. **`LDEL`** is the real link-writing primitive behind **`BRSR`** and **`LJSR`**.

Operation:

```
if condition:
    target = src.high : (src.low + displacement)
    new_src_low = src.low + 1
    l = address_of_next_instruction
    dest = target
    if post_increment:
        src.low = new_src_low
```

Flags: Preserved.

Write-back: If the condition is true, writes the return address to `l` and the computed address to `dest`. Post-increment forms also write the incremented source address back to `src`.

Exceptions: May raise a segment-overflow fault during effective-address calculation or source auto-update when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.

Condition field: If the condition is false, no destination, link, or source address register is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions. Because the link write sets `lh` to the current `ph`, a protected-mode call faults unless `lh` already equals `ph`; protected-mode code must keep `l` in the program segment.

Opcodes: `0x1C` (Immediate), `0x1D` (Register), `0x1E` (Post-Inc Immediate), `0x1F` (Post-Inc Register). See Table 15.1 for the full family of legal forms.

31–26	25–24	23–22	21–6	5–4	3–0
`0x1C`	00	dest	disp	src	Cond

31–26	25–24	23–22	21–18	17–14	13	12–10	9–6	5–4	3–0
`0x1D`	00	dest	0x0	rO	0	000	0x0	src	Cond

31–26	25–24	23–22	21–6	5–4	3–0
`0x1E`	00	dest	disp	src	Cond

31–26	25–24	23–22	21–18	17–14	13	12–10	9–6	5–4	3–0
`0x1F`	00	dest	0x0	rO	0	000	0x0	src	Cond

Instruction Fields:

- **dest (Reg or R1, bits 25–22):** bits 23–22 hold the destination address-register pair; `0x1` = `a`, `0x2` = `s`, `0x3` = `p` (`0x0` = `l` is not a legal **`LDEL`** destination). Bits 25–24 are reserved for future use and always written as zero.
- **src (AF, bits 5–4):** source address-register pair; `0x0` = `l`, `0x1` = `a`, `0x2` = `s`, `0x3` = `p`. `p` must not be used with an auto-update form.
- **offset (Immediate Value, bits 21–6, immediate forms only):** 16-bit signed displacement.
- **rO (R3, bits 17–14, register forms only):** a general-purpose register supplying the displacement.
- **R2 and shift fields (bits 21–18 and 13–6, register forms only):** reserved and written as zero; a non-zero value is architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Absolute call through a
LDEL p, (#0, a) ; l = next PC; p = a

; Explicit destination form
LDEL a, (r1, s) ; l = next PC; a = s + r1

; Post-increment source pointer after call target calculation
LDEL p, (#0, a)+ ; l = next PC; p = old a; a += 1
```

Example 15-5: Calling Through a Register with LDEL

15.7 Long Jump Meta-Instructions

LJMP

Long Jump Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
LJMP src ; p = src
LJMP src, #disp ; p = src + disp
LJMP src, rO ; p = src + rO
LJMP|cond src ; Conditional long jump
```

Operands: `src` is an address-register pair. Optional `#disp` or `rO` operands select immediate or register displacement forms.

Description:

Performs an absolute jump through an address register. **`LJMP`** is an assembler meta-instruction, not a separate CPU opcode.

Operation:

```
; Assembles to LDEA p, ...
if condition:
    p = src + displacement
```

Flags: Preserved.

Write-back: Inherits **`LDEA`** write-back to `p`.

Exceptions: Same as **`LDEA`**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no jump occurs and status flags are preserved.

Timing: Same as **`LDEA`**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **`LDEA`**. Protected-mode jumps must preserve the high word of `p`.

Assembles to: **`LDEA`** with `p` as the destination. See Table 15.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x18</td><td colspan="2">00</td><td colspan="2">p</td><td colspan="16">disp</td><td colspan="2">src</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x19</td><td colspan="2">00</td><td colspan="2">p</td><td colspan="4">0x0</td><td colspan="4">rO</td><td>0</td><td colspan="3">000</td><td colspan="4">0x0</td><td colspan="2">src</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** bits 23–22 are fixed at `0x3` (`p`) by the meta-instruction. Bits 25–24 are reserved for future use and always written as zero.
- **src (AF, bits 5–4):** source address-register pair; `0x0` = `l`, `0x1` = `a`, `0x2` = `s`, `0x3` = `p`.
- **offset (Immediate Value, bits 21–6, `#disp` form only):** 16-bit signed displacement; fixed at `0x0000` for plain `LJMP src`.
- **rO (R3, bits 17–14, register form only):** a general-purpose register supplying the displacement.
- **R2 and shift fields (bits 21–18 and 13–6, register form only):** reserved and written as zero; a non-zero value is architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Jump to address in a
LJMP a ; p = a

; Jump with offset
LJMP a, #16 ; p = a + 16

; Jump with register displacement
MOVI r1, #4
LJMP a, r1 ; p = a + r1
```

Example 15-6: Jumping Through a Register with LJMP

LJSR

Long Jump to Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Syntax:

```
LJSR src ; p = src; l = next PC
LJSR src, #disp ; p = src + disp; l = next PC
LJSR src, rO ; p = src + rO; l = next PC
LJSR (#disp, src)+ ; p = src + offset; l = next PC; src += 1
LJSR (src)+ ; equivalent to LJSR (#0, src)+
LJSR (rO, src)+ ; p = src + rO; l = next PC; src += 1
LJSR|cond src ; Conditional long call
```

Operands: `src` is an address-register pair. Optional `#disp` or `rO` operands select immediate or register displacement forms. Post-increment forms also update `src`.

Description:

Calls a subroutine through an address register. The return address is saved in `l`; return with **`RETS`**. Post-increment forms advance `src` after computing the target. The source is read before the link is written, so `LJSR l` jumps to the old value of `l`. Because **`LJSR`** implicitly writes `l` and `p`, post-increment source registers must not be `l` or `p`; such forms are aliased address-register writes with architecturally undefined behavior and are rejected by the assembler (see Appendix C).

Operation:

```
; Assembles to LDEL p, ...
if condition:
    target = src.high : (src.low + displacement)
    new_src_low = src.low + 1
    l = address_of_next_instruction
    p = target
    if post_increment:
        src.low = new_src_low
```

Flags: Preserved.

Write-back: Inherits **`LDEL`** write-back to `l`, `p`, and any post-increment source register.

Exceptions: Same as **`LDEL`**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no call occurs, `l` is not written, any post-increment source register is preserved, and status flags are preserved.

Timing: Same as **`LDEL`**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **`LDEL`**. Protected-mode calls must preserve the high words written to `l` and `p`. Because the link write sets `lh` to the current `ph`, a protected-mode call faults unless `lh` already equals `ph`; protected-mode code must keep `l` in the program segment.

Assembles to: **`LDEL`** with `p` as the destination. See Table 15.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x1C</td><td colspan="2">00</td><td colspan="2">p</td><td colspan="16">disp</td><td colspan="2">src</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x1D</td><td colspan="2">00</td><td colspan="2">p</td><td colspan="4">0x0</td><td colspan="4">rO</td><td>0</td><td colspan="3">000</td><td colspan="4">0x0</td><td colspan="2">src</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** bits 23–22 are fixed at `0x3` (`p`) by the meta-instruction. Bits 25–24 are reserved for future use and always written as zero.

- **src (AF, bits 5–4):** source address-register pair; `0x0` = `l`, `0x1` = `a`, `0x2` = `s`, `0x3` = `p` for non-post-increment forms. Post-increment forms accept only `0x1` = `a` or `0x2` = `s`; `l` and `p` are aliased address-register writes with architecturally undefined behavior and are rejected by the assembler.
- **offset (Immediate Value, bits 21–6, `#disp` forms only):** 16-bit signed displacement; fixed at `0x0000` for plain `LJSR src`.
- **rO (R3, bits 17–14, register forms only):** a general-purpose register supplying the displacement.
- **R2 and shift fields (bits 21–18 and 13–6, register forms only):** reserved and written as zero; a non-zero value is architecturally undefined.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Call function at address in a
; supervisor mode only -- writing ah is privileged
MOVI ah, #0x0040
MOVI al, #0x0000 ; a = 0x00400000
LJSR a ; Call 0x00400000

; Call with offset
LJSR a, @function_offset ; p = a + function_offset

; Call with register displacement
MOVI r1, #8
LJSR a, r1 ; p = a + r1

; Call through a table entry and advance to the next entry
LJSR (#0, a)+ ; p = old a; a += 1
LJSR (r1, a)+ ; p = a + r1; a += 1
```

Example 15-7: Calling Through a Register with LJSR

15.8 Control Flow Patterns

15.8.1 If-Then-Else

```
; if (r1 > r2) { ... } else { ... }
CMPR r1, r2
BRAN|<= @else_part
; Then part
; ...
BRAN @end
:else_part
; Else part
; ...
:end
```

15.8.2 While Loop

```
; while (r1 > 0) { ... }
:while_start
    CMPI r1, #0
    BRAN|<= @while_end
    ; Loop body
    ; ...
    BRAN @while_start
:while_end
```

15.8.3 Switch/Jump Table

```
; Switch on r1 (0-3)
MOVR r2, r1 ; r2 = r1
; Scale the index to a 2-word BRAN entry and skip over this LDEA
ADDI r2, #2, LSL #1 ; r2 = (r2 << 1) + 2
LDEA p, (r2, p) ; Jump into the branch table below

:jump_table
    BRAN @case0
    BRAN @case1
    BRAN @case2
    BRAN @case3
```

CHAPTER 16

Coprocessor Instructions

16.1 Overview

The SIRC-1 CPU includes a coprocessor interface that allows delegation of certain operations to specialized coprocessors. The processing unit (coprocessor 0) executes ordinary instructions. The principal coprocessor reached through coprocessor calls is the Exception Unit (coprocessor 1), which handles exceptions, interrupts, and privileged operations.

Coprocessor calls are ordinary conditional instructions. If the instruction's condition is true, the processing unit computes a command operand and writes it to the 16-bit coprocessor select register (CSR, Section 2.3.2) during write-back. The selected coprocessor then interprets the command value. If the selected coprocessor is not present in the CPU model, the exception unit raises an invalid-opcode fault so the operation can be emulated or rejected by software. If the selected coprocessor is present but does not implement the requested operation, the exception unit raises an invalid-opcode fault in the same way (see the Model and Revision Policy, Section 16.4.1).

16.2 Legal Forms

Instruction	Syntax	Encoding	Notes
COPI	`COPI #imm16`	`0x0F`	16-bit coprocessor command operand
COPR	`COPR rS`	`0x3F`	Command operand is read from `rS`

Table 16.1: Legal Coprocessor Instruction Forms

The CPU encoding reserves the register field of the immediate format (opcode `0x0F`) and the R1, R2, and shift fields of the register format (opcode `0x3F`); the immediate format has no shift fields. The programmer-facing **`COPI`** and **`COPR`** syntax hides those fields; assemblers emit canonical values and coprocessors must not interpret them as destination-register writes. Opcode `0x2F` is undocumented (see Appendix C); it is not a coprocessor call.

16.3 Common Semantics

Conditional execution follows the rules in Chapter 11; a call whose condition is false does not write the CSR. The call's timing is given in Section B.3.1 of Appendix B.

`COPI` uses its 16-bit immediate value as the command operand; **`COPR`** uses the value of its source register. For standard coprocessor commands, bits 15–12 select the coprocessor ID, bits 11–8 select the coprocessor operation, and bits 7–0 are coprocessor-defined operand bits. The processing unit writes the command operand to the CSR. It does not write general-purpose registers, address registers, data memory, or status flags as part of the call, and hidden encoding fields are ignored. A coprocessor operation may define its own later CPU-state effects; exception-unit effects are documented in Chapter 6.

In protected mode, coprocessor-operation nibbles `0x0`–`0x7` are user-callable and `0x8`–`0xF` are supervisor-only. Privilege checks apply only when the instruction condition is true. For **`COPR`** the privilege decision is made on the runtime value of `rS` sampled during decode, not on any encoded field, so whether a given **`COPR`** instruction faults depends on register contents.

A fault raised by the call itself, such as a privilege violation or an invalid opcode, cancels the command, and software must not depend on the contents of the CSR after the fault. A command waiting in the CSR is never discarded by an interrupt or an instruction trace fault: those wait until the command has been dispatched and has completed (Section 6.3.3).
The processing-unit part of a coprocessor call performs no data-memory access and raises no data bus, data bus-protection, data alignment, or effective-address segment-overflow fault. A DMA command that the call starts can raise bus, bus-protection, and segment-overflow faults (Section 16.7). A coprocessor call in the last instruction slot of a segment still raises a segment-overflow fault when `SR.A` is set, because it advances the program counter; the call is canceled before its command is dispatched (Section 6.2.5). Instruction fetch can still raise the normal fetch faults before the call executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the processing-unit call commits and its command has been dispatched and has completed, unless an earlier fault prevents the instruction from completing. A traced **`EXCP`** is the exception: its trace fault is taken first and the trap is entered when the trace handler returns (Section 6.2.8). In protected mode, supervisor-only coprocessor operation nibbles are rejected before the command is dispatched. A missing coprocessor is detected after the command has been latched for dispatch and raises an invalid-opcode fault through the exception unit. A reserved operation on a coprocessor that is present is detected at the same point and raises the same fault (see the Model and Revision Policy, Section 16.4.1).

COPI/COPR

Coprocessor Call

Applicability: SIRC-1, all revisions

Syntax:

```
COPI #opcode ; Call coprocessor with 16-bit command
COPR rS ; Call coprocessor with command in rS
```

Operands: `COPI` takes a 16-bit immediate command operand. `COPR` takes any register name as its source; its 16-bit value is used as the command operand. A protected-mode `COPR sr` reads only the low byte of `sr` and therefore always builds a coprocessor-0 command. Hidden encoded register and shift fields are canonicalized by the assembler and are not programmer-visible operands.

Description:

Transfers a command to a coprocessor. For the standard command layout, the high nibble selects the coprocessor and the next nibble selects that coprocessor's operation. The remaining low byte is defined by the selected operation.

Operation:

```
if condition_true then
    CSR = command_operand
endif
```

Write-back: Writes the command operand to the CSR.

Flags: Preserved by the coprocessor-call instruction.

Exceptions: May raise privilege-violation faults for supervisor-only operations in protected mode. May raise invalid-opcode faults for missing coprocessors and for operations that a coprocessor which is present does not implement. Reserved operand values are architecturally undefined (see the Model and Revision Policy, Section 16.4.1).

Condition field: If the condition is false, no command is dispatched, the CSR is preserved, no coprocessor privilege or opcode checks are performed, and status flags are preserved.

Timing: 6 cycles for the processing-unit call, plus a further 6-cycle coprocessor dispatch slot (12 cycles in total), plus any bus wait states. A command naming coprocessor 0 has no dispatch slot and takes 6 cycles, as does a call whose condition is false. `RSET`, `WAIT`, DMA commands, and maths-unit commands differ; see their own entries.

Privilege: Available in protected mode and supervisor mode. In protected mode, coprocessor-operation nibbles `0x8`–`0xF` are supervisor-only and raise privilege-violation faults when executed. For `COPR` the privilege decision is made on the runtime value of `rS` sampled during decode, not on any encoded field, so whether a given `COPR` instruction faults depends on register contents.

Opcodes: `0x0F` (Immediate), `0x3F` (Register). Opcode `0x2F` is undocumented (Appendix C); it is not a coprocessor call. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">Command</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x3F</td><td colspan="4">0x0</td><td colspan="4">0x0</td><td colspan="4">rS</td><td>0</td><td colspan="3">000</td><td colspan="4">0x0</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22, Immediate format only):** reserved; the assembler emits `0x0` and coprocessors must not interpret it as a destination register.

- **Register 1, Register 2 (Register format, bits 25–18):** reserved; the assembler emits `0x0` for both.
- **rS (R3, Register format, bits 17–14):** any of the 16 registers, `0x0`–`0xF`; its value becomes the 16-bit command operand.
- **Command (Immediate Value, bits 21–6, Immediate format only):** 16-bit command operand for `COPI`.
- **Shift fields (Register format, bits 13–6):** reserved; the assembler emits `0` and coprocessors must not interpret them.
- **AF (bits 5–4):** reserved; the assembler emits `00` and coprocessors must not interpret it as a status-update selector.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Wait for interrupt (Exception Unit)
COPI #0x1900 ; Coprocessor 1, operation 0x9
; Prefer the WAIT meta-instruction for normal assembly code.

; Return from exception
COPI #0x1A00 ; Coprocessor 1, operation 0xA
; Prefer the RETE meta-instruction for normal assembly code.

; Software exception (system call)
COPI #0x1180 ; Coprocessor 1, operation 0x1, vector 0x80
; Prefer EXCP #0x80 for normal assembly code.
```

Example 16-1: Direct Coprocessor Calls

16.4 Coprocessor Compatibility

Standard coprocessor IDs are stable across SIRC-1 models. A CPU model may omit an optional coprocessor, but it must not assign that coprocessor ID to an incompatible unit. If software calls a coprocessor ID absent from the selected CPU model, the CPU raises an invalid-opcode fault. Calling an operation that an implemented coprocessor does not support raises the same fault; see the Model and Revision Policy below.

ID	Name	Required	Operations	Compatibility rule
`0x0`	Processing Unit	Yes	Internal	Executes SIRCIS processing-unit instructions
`0x1`	Exception Unit	Yes	`0x1`, `0x9`–`0xD` (software); `0x0` (no-op); `0xE`–`0xF` (internal)	Handles reset, faults, interrupts, traps, and links
`0x2`	DMA Unit	No	`0x8`–`0xA`	Optional standard DMA transfer unit
`0x3`	Integer Maths Unit	No	`0x0`–`0x3`	Optional standard integer multiply/divide unit
`0x4`–`0xF`	Reserved/model space	No	Model-defined	Reserved unless a CPU model documents the assignment

Table 16.2: Standard Coprocessor IDs

A command with coprocessor ID `0x0` selects the processing unit, which is already running, so it is a no-op for every operation nibble and operand. The supervisor-only check still applies: in protected mode, operation nibbles `0x8`–`0xF` raise a privilege-violation fault as for any other coprocessor. The CSR is `0x0000` again by the next instruction boundary, so the call never delays exception selection. Exception-unit operation nibble `0x0` is the internal no-operation encoding, and is also a no-op when software issues it. Operation nibbles `0x2`–`0x8` are reserved and raise an invalid-opcode fault. Operations `0xE` and `0xF` are internal encodings the exception unit uses to dispatch a fault or a hardware exception; see Chapter 6 for the internal-instruction mechanism.

16.4.1 Model and Revision Policy

- Required coprocessors and documented operations must remain backward compatible within the SIRC-1 family.
- Optional standard coprocessors keep their assigned IDs even on models that omit them.
- Reserved coprocessor IDs must raise invalid-opcode faults when they are not implemented by the selected CPU model. Reserved operation nibbles within an implemented coprocessor other than the processing unit (ID `0x0`) raise invalid-opcode faults, so a later model can add an operation and older models can emulate it. Exception-unit operation `0x0` is a defined no-op. Reserved operand values within an implemented operation are architecturally undefined.
- Software may probe optional coprocessors by executing the documented operation and handling an invalid-opcode fault, or by using a model-specific capability table if one is provided by system firmware. The invalid-opcode fault records the offending 16-bit coprocessor command word in the `return_address` field of the fault-metadata link register (link register `#7`), which an emulation handler reads with **`ETFR`**. The CSR is cleared before the handler runs.
- Processing-unit undocumented opcodes are a separate case: they are accepted by the processing unit and have architecturally undefined effects unless a future CPU revision documents them.

16.5 Exception Unit (Coprocessor 1)

The Exception Unit handles:

- Faults (highest priority)
- Hardware exceptions (interrupts, multiple priority levels)
- Software exceptions (traps)
- Privileged operations

16.5.1 Common Operations

Opcode	Mnemonic	Operation
`0x1100–0x11FF`	EXCP	Software exception (trap) at the vector
`0x1900`	WAIT	Wait for interrupt
`0x1A00`	RETE	Return from exception
`0x1B00`	RSET	System reset
`0x1C00 | sel«4 | n`	ETFR	Transfer from exception link register
`0x1D00 | sel«4 | n`	ETTR	Transfer to exception link register

Table 16.3: Exception Unit Operations

For **`ETFR`** and **`ETTR`**, `sel` is the register selector and `n` is the link-register index `0`–`7`. Command bits 7–6 are not decoded, bits 5–4 select the register (`00` none, `01` saved address, `10` saved status, `11` both), and bits 3–0 hold the link-register index, `0`–`7`; values with bit 6 or bit 7 set are not distinct operations. Coprocessor-1 operations `0xE` and `0xF` are the exception unit's internal encodings for dispatching a fault or a hardware exception; a software-issued **`COPI`** or **`COPR`** naming operation `0xE` or `0xF` is architecturally undefined.

16.6 Exception Unit Meta-Instructions

EXCP

Software Exception Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Syntax: `EXCP #vector`

Operands: `#vector` is the low 8-bit software exception vector selector.

Description: Triggers a software exception at the specified vector. Used by programs running in protected mode to invoke system calls or request supervisor-mode services through the user trap vectors, `0x60`–`0xFF`; a vector below `0x60` raises a privilege-violation fault in protected mode.

In supervisor mode, a vector below `0x60` is accepted and runs the handler that vector points at, but as an ordinary trap: priority 1, state saved in link register 0, exception level 1. Nothing else about the exception that normally uses that vector happens. **Warning:** a handler entered this way that reads its own link register with **`ETFR`** sees stale contents. A fault handler, for example, reads link register 7 and gets the metadata and return address of whichever fault last ran, not this call. A plain **`RETE`** still returns correctly, because it uses link register 0 at exception level 1.

Operation:

```
if condition_true then
    if protected_mode and vector < 0x60 then
        raise privilege_violation_fault
    else if current_exception_level != 0 then
        CSR = 0x0000                ; discard the trap
    else
        link[0] = { address of next instruction, sr,
                    current_exception_level }
        sr.P = 0
        sr.T = 0
        sr.EA = 1
        current_exception_level = 1
        pc = vector_table[vector]
    endif
endif
```

Write-back: Inherits **`COPI`** command dispatch to the exception unit.

Flags: Preserved by the coprocessor-call instruction; exception entry effects are defined by the exception unit.

Exceptions: Dispatches a software exception through the exception unit. Raises a privilege-violation fault in protected mode if the selected vector is below `0x60`. Software exceptions are accepted only at exception level 0; an otherwise legal **`EXCP`** executed inside any exception or fault handler is discarded – the CSR is cleared, no vector is fetched, and no fault is raised. The privilege check comes first, so a protected-mode **`EXCP`** with a vector below `0x60` inside a handler still raises a privilege-violation fault. Nested system calls from a handler are therefore not possible.

Condition field: If the condition is false, no trap is dispatched and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot. A discarded **`EXCP`** (inside a handler) also takes 12 cycles: its dispatch slot runs with no-op phases.

Privilege: User-callable in protected mode and supervisor mode.

Assembles to: `COPI #0x1100 | vector`. See Table 16.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x0F`	0x0	0x1100 | #vector	00	Cond

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x1100 | #vector`; bits 15–12 select coprocessor 1, bits 11–8 select operation `0x1`, and bits 7–0 hold `#vector`, `0x00`–`0xFF`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
MOVI r1, #0x41 ; Character to print ('A')
EXCP #0x80 ; Trap to OS print routine
```

Example 16-2: Triggering a Software Exception

WAIT

Wait for Interrupt Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Syntax: `WAIT`

Operands: None.

Description: Stops instruction fetch and places the CPU in a low-power wait state. The CPU issues no bus cycles while waiting. It leaves the wait state only when a hardware interrupt can be taken – one that is enabled (Level 5 is always enabled) and whose priority is higher than the current exception level – or when the CPU is reset. A request of that kind that is already pending when **`WAIT`** starts ends the wait at once, so a request that arrives just before the **`WAIT`** is not missed. Interrupts masked by the hardware interrupt enable bits (`SR.E1`–`SR.E4`), or outranked by the current exception level, do not wake the CPU; they stay pending under the normal rules. Faults and software exceptions cannot occur while waiting.

Operation:

```
if condition_true then
    if protected_mode then
        raise privilege_violation_fault
    else
        stop instruction fetch
        wait until a hardware interrupt can be taken
   or the CPU is reset
    endif
endif
```

Write-back: Inherits **`COPI`** command dispatch to the exception unit.

Flags: Preserved by the coprocessor-call instruction.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, the CPU does not enter wait state and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot, then idle until an interrupt that can be taken arrives or the CPU is reset.

Privilege: Supervisor mode only.

Assembles to: `COPI #0x1900`. See Table 16.1 for the full family of legal forms.

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0x0F	0x0	0x1900	00	Cond

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1900` (coprocessor 1, operation `0x9`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
:idle_loop
    WAIT ; Wait until an interrupt or reset occurs
    BRAN @idle_loop ; After the interrupt handler returns, wait again
```

Example 16-3: Idling Until an Interrupt

RETE

Return from Exception Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Syntax: `RETE`

Operands: None.

Description: Returns from an exception handler by restoring the saved program counter and status register. `SR.EA` is not taken from the saved status register: it is set if the restored exception level is non-zero and cleared if it is zero, so it always agrees with the exception level even if the saved value was edited with **`ETTR`**. Executing **`RETE`** at exception level 0, that is, outside any exception or fault handler, is architecturally undefined.

Operation:

```
if condition_true then
    if protected_mode then
        raise privilege_violation_fault
    else
        n = current_exception_level - 1
        pc = link[n].return_address
        sr = link[n].return_status
        current_exception_level = link[n].saved_level
        sr.EA = (current_exception_level != 0)
    endif
endif
```

Write-back: Inherits **`COPI`** command dispatch to the exception unit.

Flags: Restored by the exception unit as part of exception return.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, no exception return occurs and current status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: Supervisor mode only.

Assembles to: `COPI #0x1A00`. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x1A00</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1A00` (coprocessor 1, operation `0xA`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
:timer_handler
    ; ... service the timer interrupt ...
    RETE ; Return to the interrupted program
```

Example 16-4: Returning from an Interrupt Handler

RSET

System Reset Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Syntax: `RSET`

Operands: None.

Description: Requests a full processor reset. The status register, the exception level, and the pending hardware exceptions are cleared when reset is requested. The reset output is then asserted and held for 6 cycles, after which the exception unit fetches the reset vector and loads the program counter from it (Section 6.9.2). General-purpose registers, the `a`, `l`, and `s` pairs, and the exception link registers are not cleared by reset; software must treat their contents as architecturally undefined.

Operation:

```
if condition_true then
    if protected_mode then
        raise privilege_violation_fault
    else
        sr = 0x0000
        current_exception_level = 0
        clear pending hardware exceptions (and the L5 overrun flag)
        assert reset output for 6 cycles
        pc = vector_table[reset_vector]
    endif
endif
```

Write-back: Inherits **`COPI`** command dispatch to the exception unit, except that the reset request replaces the 6-cycle dispatch slot.

Flags: Reset by the exception unit as part of reset processing.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, no reset occurs and current status flags are preserved.

Timing: 6 cycles for the call, plus a 6-cycle $\overline{\text{RSTO}}$ hold, plus 6 cycles for the exception unit to fetch the reset vector – 18 cycles from the **`RSET`** fetch to the first instruction of the reset handler (no dispatch slot).

Privilege: Supervisor mode only.

Assembles to: `COPI #0x1B00`. See Table 16.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x0F`	0x0	0x1B00	00	Cond

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1B00` (coprocessor 1, operation `0xB`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Request a full processor reset from supervisor code
RSET ; Reboots through the reset vector
```

Example 16-5: Requesting a Software Reset

ETFR/ETTR

Exception Link Transfer Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Syntax:

```
ETFR #n ; Transfer saved address and status to a and r7
ETFR a, #n ; Transfer saved address to a only
ETFR r7, #n ; Transfer saved status to r7 only
ETTR #n ; Transfer a and r7 to saved address and status
ETTR #n, a ; Transfer a to saved address only
ETTR #n, r7 ; Transfer r7 to saved status only
```

Operands: `#n` selects one of the eight exception link registers and must be in the range `#0`–`#7`. Link register `#7` holds fault metadata; link register $L-1$ holds the state saved on entry to an exception at level L. Values above `#7` are architecturally undefined. Optional operands select whether to transfer the saved return address, saved status register, or both.

Description: **`ETFR`** copies saved exception state from an exception link register to CPU registers. **`ETTR`** copies CPU register state back to the selected exception link register before returning.

Operation:

```
if condition_true then
    if protected_mode then
        raise privilege_violation_fault
    else if operation == ETFR then
        if x includes saved address then a = link[n].return_address
        if x includes saved status then r7 = link[n].return_status
    else
        if x includes saved address then link[n].return_address = a
        if x includes saved status then link[n].return_status = r7
    endif
endif
```

Write-back: **`ETFR`** writes selected saved state to `a`, `r7`, or both. **`ETTR`** writes `a`, `r7`, or both to selected saved exception state.

Flags: Preserved by the coprocessor-call instruction.

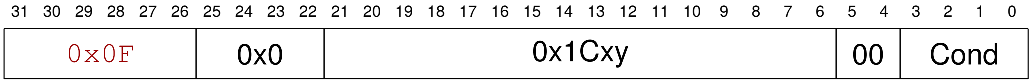
Exceptions: Raises a privilege-violation fault if executed in protected mode.

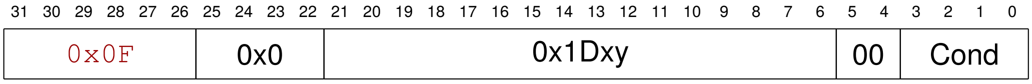
Condition field: If the condition is false, no transfer occurs and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: Supervisor mode only.

Assembles to: `COPI #0x1Cxy` or `COPI #0x1Dxy`. See Table 16.1 for the full family of legal forms.





Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.

- **Immediate (bits 21–6):** `0x1Cxy` (**ETFR**) or `0x1Dxy` (**ETTR**); bits 15–12 select coprocessor 1, bits 11–8 select operation `0xC` or `0xD`. Within the low operand byte `xy`, the high hexadecimal digit `x` holds bits 7–4: bits 7–6 are not decoded and are written as zero, and bits 5–4 select the register (`00` none, `01` saved address only, `10` saved status only, `11` both). The low hexadecimal digit `y` holds bits 3–0, which hold `#n`, the link-register index `0`–`7`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
:alignment_fault_handler
    ETFR a, #6 ; Get faulting return address
    MOVI al, #0x0100 ; Correct the address
    ETTR #6, a ; Write corrected address back
    RETE
```

Example 16-6: Correcting a Faulting Return Address

See Chapter 6 for detailed documentation of exception handling and all exception coprocessor instructions.

16.7 DMA Unit (Coprocessor 2)

The optional DMA unit provides bulk word transfers using the existing address register pairs and the general-purpose register file. It is intentionally simple: it has no descriptor table, hidden parameter registers, or autonomous background execution. When a DMA command is accepted, the processing unit stops fetching further instructions until the DMA command completes or faults. CPU models without the hardware raise an invalid-opcode fault for these commands, allowing supervisor software to fall back to ordinary **LOAD**/**STOR** loops.

A DMA transfer cannot be interrupted. Hardware interrupts, including Level 5, wait until the transfer has completed (Section 6.3.3); only a fault raised by the transfer itself, or a reset, ends it early. Masking the maskable interrupt lines does not shorten the wait. The count of a single command is limited (Table 16.4), so a program that needs a bounded interrupt latency can split a large copy into several commands, which leaves an instruction boundary between them where interrupts are taken.

DMA operations are supervisor-only because they perform memory access using supervisor-visible address state. In protected mode, they raise a privilege-violation fault before dispatch.

16.7.1 DMA Operations

Mnemonic	Command	Count	Registers Used	Operation
DMAR	`0x2800 | operand`	`#-7`–`#7`	address `a`, `l` or `s`; writes `r1` to `rn`, where n is the count magnitude	Read memory into registers
DMAW	`0x2900 | operand`	`#-7`–`#7`	address `a`, `l` or `s`; reads `r1` to `rn`, where n is the count magnitude	Write registers to memory
DMAT	`0x2A00 | n`	0–255	source `a`, destination `l`; clobbers `r1`–`r7` as its transfer window	Copy memory at `a` to memory at `l`

Table 16.4: DMA Unit Operations

For **DMAR** and **DMAW**, the command operand byte is encoded as follows: bits 7–6 select the address register (`00` = `a`, `01` = `l`, `10` = `s`, `11` reserved), bit 5 selects direction (`0` = forward, `1` = backward), bits 2–0 hold the count magnitude, and bits 4–3 are reserved and written as zero. Negative counts are encoded as the backward direction bit plus the positive magnitude, not as a two's-complement count field. A zero count is encoded with the direction bit clear (forward). A backward count of zero (direction bit set, magnitude zero) is a no-op, identical to a forward count of zero. Note: this DMA operand field is not the AF address-register field; its `00`/`01` assignment differs from Table 8.6 in Chapter 8.

For all DMA operations, a count of zero is a no-op: no memory access occurs, registers are preserved, and address registers are not advanced. CPU status flags are preserved. If the condition is false, no DMA command is dispatched, no coprocessor privilege or opcode checks are performed, and the CSR, registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

DMA timing assumes the command condition is true and every bus access is acknowledged without wait states. The **COPI** dispatch takes the normal 6 cycles; dispatch to the DMA unit then occupies one further complete 6-cycle slot beginning at the next phase 0, matching the coprocessor-1 dispatch model (see Section B.3.1 of Appendix B). Phase 0 of that slot is a setup cycle with no bus access: the DMA unit accepts the command, decodes the count, and latches the address registers. From phase 1, the DMA unit performs one bus access per cycle in the no-wait case. **DMAT** reads a chunk of up to seven words from `a` and then writes it to `l`, so each word takes two cycles. The transfer runs on into further complete 6-cycle slots as needed, and instruction fetch resumes at the phase 0 after the slot in which the last access completes. With k bus accesses ($|n|$ for **DMAR** and **DMAW**, $2n$ for **DMAT**), the instruction takes $6 + 6\lceil(k + 1)/6\rceil$ cycles: 12 cycles for up to five accesses, 18 for six to eleven, and so on. A zero count needs only the setup cycle and takes 12 cycles. A false condition skips the DMA command entirely and takes only the normal 6-cycle **COPI** instruction time. A wait state on a DMA access holds that phase until the access is acknowledged, as for any other bus access. The rest of the transfer, and the slot boundaries after it, move one cycle later for each wait state, so each wait state adds exactly one cycle to the totals above.

Missing DMA hardware and reserved DMA operations raise an invalid-opcode fault. Reserved operand encodings are architecturally undefined (see the Model and Revision Policy above). DMA memory reads and writes can raise data bus and bus-protection faults using DMA read/write bus access types. They can also raise segment-overflow faults, as described in Section 16.7.2. Partial side effects are possible if a fault occurs mid-transfer, and software must not assume that re-executing the same DMA instruction restarts the original operation; see each operation's **Fault side effects** entry below for what state a handler can rely on, and see Section 16.7.3 for what this means for a fault handler's recovery strategy.

16.7.2 Segment Overflow

DMA address arithmetic works on the low word of the selected address register only; the high word is never changed. The segment-overflow check is made at access time, one access at a time. It is never made up front for the whole transfer.

- **Each word access.** The address of every access is the register's low word plus the word's offset. If `SR.A` is set and forming that address would carry out of the low word (wrap past `0xFFFF`), a segment-overflow fault is raised at that access, before it is performed. Words already transferred stay transferred. If `SR.A` is clear, the low word wraps modulo 2^{16} and the transfer continues.

- **Negative counts.** The decrement of the address register is the address calculation for the first access. If it underflows with `SR.A` set, the fault is raised before the register is modified and before any word is transferred, so the register still holds its original value.

- **Address-register advance.** The advance at the end of a forward **DMAR** or **DMAW**, and after each completed **DMAT** chunk, is checked the same way. If it carries out of the low word with `SR.A` set, the fault is raised at that point, after the words have been transferred and with the register unchanged. A transfer that ends exactly at the top of the segment therefore faults on the advance even though every access was in range, as a post-increment **`LOAD`** does. For **`DMAT`**, both chunk advances are checked before either is applied: if either would carry out of the low word with `SR.A` set, neither `a` nor `l` changes, and the fault records the `a` advance (DMA read type) if it wraps, otherwise the `l` advance (DMA write type).

- **`DMAT`.** Reads use `a` and writes use `l`; an overflowing read access faults at the read and an overflowing write access faults at the write.

The fault metadata is captured as for any other segment-overflow fault, with the bus access type of the DMA access whose address overflowed. For a forward **`DMAR`** or **`DMAW`** advance it is the type of that command's accesses; for a **`DMAT`** advance it is the type given in the list above.

16.7.3 Retrying After a Fault

Bus Fault, Bus Protection Fault, and Segment Overflow Fault are all classified *Retryable* (Chapter 6), and for an ordinary single-access instruction that classification means **`RETE`** safely re-executes the whole instruction from a clean, unstarted state. **`DMAR`**, **`DMAW`**, and **`DMAT`** are the only instructions in the SIRCIS instruction set whose execution spans more than one data-memory bus access, and for them this guarantee does not hold uniformly: simply returning with **`RETE`** after fixing the fault condition can re-run the instruction from the wrong starting point instead of from the beginning.

Forward transfers (`DMAR`/`DMAW` with a positive count) are the one case where a bare `RETE` is safe. The selected address register's advance is the last step of the operation and only runs once the whole transfer has completed, so a fault partway through leaves the address register exactly where it started. Retrying with **`RETE`** re-reads that unchanged value and redoes the entire transfer from the true beginning; some already-transferred words are transferred a second time, but the result is correct once the fault condition is fixed.

Backward transfers (`DMAR`/`DMAW` with a negative count) are not safe to retry directly. The address register is decremented to its final target address *before* the transfer loop begins. This is unlike pre-decrement addressing (Chapter 8), whose address-register update is only committed at write-back and so is undone by a fault; the DMA decrement is committed before the transfer loop begins, so by the time a fault can occur, the address register already holds the fully-decremented address, regardless of how many words actually transferred. A bare **`RETE`** re-reads that already-decremented value and decrements it a second time, landing the retried transfer on the wrong memory range entirely. The one exception is a decrement that underflows with `SR.A` set: that fault is raised before the register is modified (Section 16.7.2), so the register keeps its original value and a bare **`RETE`** raises the same fault again.

`DMAT` is not safe to retry directly either, for a related reason. `a` and `l` only advance once a complete chunk (up to seven words) has been both read and written, so a fault leaves them reflecting every fully-completed prior chunk – but the instruction's `#n` operand stays fixed at its original encoded value, not the remaining count. A bare **`RETE`** restarts from the already-advanced addresses and transfers the full original count again, overrunning the intended end of the transfer by the number of words in the chunks the first attempt fully completed.

What a handler can and cannot recover. Except for **`DMAT`**'s chunk boundary, the CPU exposes no transfer-progress counter: for **`DMAR`**/**`DMAW`**, the address register's fault-time value carries no information about how many of the `#n` words actually transferred, since it is either left untouched or already at its final position – never in between – and a partially-filled register range from **`DMAR`** cannot be distinguished from stale values left over from before the instruction ran. A fault handler cannot reconstruct a precise resume point for a backward **`DMAR`**/**`DMAW`**, or for the in-flight chunk of a faulted **`DMAT`**, from architectural state alone. In practice:

- For a positive-count **`DMAR`**/**`DMAW`**, correct the fault condition and return with **`RETE`** as for any other retryable fault – the address register is already untouched, so this naturally redoes the whole transfer.

- For a negative-count **DMAR**/**DMAW**, or for **DMAT**, do not return directly into the faulted instruction without first repairing its inputs. The instruction word itself is unchanged, so the simplest safe strategy is to restore the address register(s) to the values they held before the original attempt – values software must have saved independently, since the CPU will not retain them – and only then return with **RETE**. This reruns the identical instruction from a clean state instead of resuming mid-transfer. Use **ETFR**/**ETTR** instead, to redirect the return address elsewhere, only once a retry budget is exhausted and the caller needs to be told the transfer failed.
- Treat a mid-transfer fault on a backward **DMAR**/**DMAW** as destructive to the original operand values: software that needs to recover precisely should keep its own record of the transfer's starting address and count before issuing the instruction, rather than relying on reconstructing it afterward.

Example 16-7 below wraps this pattern into a small, retryable block-copy routine. `l` is both **DMAT**'s destination register and the subroutine link register (Chapter 3; see also **DMAT**'s own Write-back field below), so the routine cannot take its destination address as an incoming `l` value the way an ordinary subroutine would. Instead, the caller passes the source address in `a`, unchanged, and the destination address in `r1` (high word) and `r2` (low word); on entry, the routine saves the return address that `l` holds to the stack before anything else touches `l`, loads `l` from the destination parameter, and restores the saved return address into `l` immediately before every **RETS**. It also saves the original transfer parameters on the stack before issuing **DMAT** (registers `r1`–`r7` are not a safe place to keep them, since **DMAT** clobbers them as its own transfer window), and the fault handler installed at the Bus Fault, Bus Protection Fault, and Segment Overflow Fault vectors restores those parameters and retries in place, up to a fixed retry budget, before giving up. This handler assumes `dma_copy_safe` is the only DMA user in the system; a handler shared with other DMA callers would also need to check the faulting return address (**ETFR** `a, #6`) against the expected range before treating a fault as its own.

Part 1: the copy routine.

```
:dma_copy_safe
    STOR -(#0, s), lh ; save the caller's return address first,
    STOR -(#0, s), ll ; before l is repurposed as the destination
    MOVR lh, r1 ; destination high word (parameter)
    MOVR ll, r2 ; destination low word (parameter)

    STOR -(#0, s), ah ; save the original source/dest addresses
    STOR -(#0, s), al ; and the retry budget where DMAT can't
    STOR -(#0, s), lh ; touch them
    STOR -(#0, s), ll
    MOVI r1, #3 ; retry budget (3 attempts in total)
    STOR -(#0, s), r1

:dma_copy_attempt
    DMAT a, l, #16 ; may fault partway through
    ADDI sl, #5 ; success: discard the retry-loop state
    LOAD ll, (#0, s) ; restore the caller's return address
    LOAD lh, (#1, s)
    ADDI sl, #2
    RETS
```

Part 2: the fault handler, installed at the Bus Fault, Bus Protection Fault, and Segment Overflow Fault vectors.

```
:dma_fault_handler
    ETFR r7, #7 ; r7 = fault metadata
    MOVR r6, r7 ; copy it -- ANDI below would destroy r7
    ANDI r6, #0x0007 ; isolate the bus-access-type field
    CMPI r6, #0x0005 ; BAT = 0x5 (DMA read burst)?
    BRAN|== @dma_fault_is_dma
    CMPI r6, #0x0006 ; BAT = 0x6 (DMA write burst)?
    BRAN|!= @dma_fault_not_dma ; neither -- not this handler's fault

:dma_fault_is_dma
    LOAD r1, (#0, s) ; retry budget
    SUBI r1, #1
    STOR (#0, s), r1
    BRAN|== @dma_copy_give_up ; out of retries

    LOAD ll, (#1, s) ; restore the original addresses -- a/l
    LOAD lh, (#2, s) ; may already be wrong, for example
    LOAD al, (#3, s) ; double-decremented by a backward transfer
    LOAD ah, (#4, s)
    RETE ; retry dma_copy_attempt from a clean state

:dma_copy_give_up
    ADDI sl, #5 ; discard the retry-loop state
    ETFR a, #6 ; get the faulting return address
    MOVI al, @dma_copy_failed ; redirect past the retry loop
    ETTR #6, a
    RETE

:dma_fault_not_dma
    ; ... handling for faults unrelated to this transfer ...

:dma_copy_failed
    ; ... signal failure to the caller; left to the calling convention
    ; in use ...
    LOAD ll, (#0, s) ; restore the caller's return address
    LOAD lh, (#1, s)
    ADDI sl, #2
    RETS
```

Example 16-7: A Retry-Safe DMA Copy Routine

DMAR

DMA Read to Registers Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), an optional coprocessor absent on models that do not implement it.

Syntax: `DMAR addr, #n`

Operands: `addr` must be `a`, `l`, or `s`. `#n` is a literal count operand in the range `#-7`–`#7`; the assembler rejects a literal outside that range.

Description: Reads sequential words from the selected address register into `r1` through `rn`. Positive counts read from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then read the ascending memory block from that new address.

Operation:

```
count = abs(n)
if n < 0:
    addr.low = addr.low - count
transfer = addr
for i in 1 to count (inclusive):
    r[i] = memory[transfer]
    transfer.low = transfer.low + 1
if n > 0:
    addr.low = addr.low + count
```

Write-back: A successful positive-count **`DMAR`** advances the selected address register by the count magnitude. A successful negative-count **`DMAR`** leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements. Writes `r1` through `rn`.

Flags: Preserved.

Exceptions: Missing DMA hardware raises an invalid-opcode fault. The count field is 3 bits wide and every bit pattern (`0`–`7`) is a legal magnitude, so there is no invalid encoding for it to reject; the assembler's own `#-7`–`#7` range check applies to source syntax only and does not guard against an illegal encoding. Reserved operand encodings are architecturally undefined (see the Model and Revision Policy in Section 16.4.1). In protected mode, raises a privilege-violation fault. DMA reads can raise data bus, bus-protection, and segment-overflow faults (see Section 16.7.2).

Fault side effects: If a fault occurs mid-transfer, only the registers among `r1` through `rn` read before the fault contain memory data; the rest are unmodified. The selected address register's state depends on direction, per the operation above: for a positive count, the address-register advance is the last step of the operation and does not run until the whole transfer completes, so a fault leaves it at its original, pre-transfer value no matter how many words were read. For a negative count, the address register is decremented to its final position *before* the transfer loop begins, so it already reflects the full decrement even if the fault occurs on the very first word (unless the decrement itself underflowed, which faults before the register is modified). Software must not assume the full requested count completed in either case. See Section 16.7.3 for what a handler can safely do about it.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $6 + 6\lceil(|n| + 1)/6\rceil$ cycles when the command condition is true and the DMA read cycles are acknowledged without wait states. The $n = 0$ no-op form and transfers of up to five words take 12 cycles. Wait states add to the relevant DMA read bus cycle. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.
Assembles to: `COPI #0x2800 | operand`. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x2800 | operand</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x2800 |` operand; bits 15–12 select coprocessor 2 and bits 11–8 select operation `0x8`. Within the low operand byte: bits 7–6 select the address register (`00` = `a`, `01` = `l`, `10` = `s`, `11` reserved – this is not the AF address-register encoding), bit 5 selects direction (`0` forward, `1` backward), bits 4–3 are reserved and written as zero, and bits 2–0 hold the count magnitude, `0`–`7`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
MOVI ah, #0x0040
MOVI al, #0x0000 ; a = 0x00400000, start of source block
DMAR a, #3 ; r1, r2, r3 = memory[a], memory[a+1], memory[a+2]; a += 3
```

Example 16-8: Reading a Memory Block into Registers

DMAW

DMA Write from Registers Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), an optional coprocessor absent on models that do not implement it.

Syntax: `DMAW addr, #n`

Operands: `addr` must be `a`, `l`, or `s`. `#n` is a literal count operand in the range `#-7`–`#7`; the assembler rejects a literal outside that range.

Description: Writes `r1` through `rn` to sequential words at the selected address register. Positive counts write from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then write the ascending memory block from that new address. Register order always corresponds to ascending memory addresses, so `DMAW s, #-n` followed by `DMAR s, #n` restores `r1` through `rn`.

Operation:

```
count = abs(n)
if n < 0:
    addr.low = addr.low - count
transfer = addr
for i in 1 to count (inclusive):
    memory[transfer] = r[i]
    transfer.low = transfer.low + 1
if n > 0:
    addr.low = addr.low + count
```

Write-back: A successful positive-count **`DMAW`** advances the selected address register by the count magnitude. A successful negative-count **`DMAW`** leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements. General-purpose registers are preserved; **`DMAW`** only reads them.

Flags: Preserved.

Exceptions: Missing DMA hardware raises an invalid-opcode fault. The count field is 3 bits wide and every bit pattern (`0`–`7`) is a legal magnitude, so there is no invalid encoding for it to reject; the assembler's own `#-7`–`#7` range check applies to source syntax only and does not guard against an illegal encoding. Reserved operand encodings are architecturally undefined (see the Model and Revision Policy in Section 16.4.1). In protected mode, raises a privilege-violation fault. DMA writes can raise data bus, bus-protection, and segment-overflow faults (see Section 16.7.2).

Fault side effects: If a fault occurs mid-transfer, only the memory words written before the fault reflect the source registers; the rest of the destination range is unchanged. The selected address register's state depends on direction, per the operation above: for a positive count, the address-register advance is the last step of the operation and does not run until the whole transfer completes, so a fault leaves it at its original, pre-transfer value no matter how many words were written. For a negative count, the address register is decremented to its final position *before* the transfer loop begins, so it already reflects the full decrement even if the fault occurs on the very first word (unless the decrement itself underflowed, which faults before the register is modified). Software must not assume the full requested count completed in either case. See Section 16.7.3 for what a handler can safely do about it.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $6+6\lceil(|n|+1)/6\rceil$ cycles when the command condition is true and the DMA write cycles are acknowledged without wait states. The $n=0$ no-op form and transfers of up to five words take 12 cycles. Wait states add to the relevant DMA write bus cycle. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.
Assembles to: `COPI #0x2900 | operand`. See Table 16.1 for the full family of legal forms.

31 30 29 28 27 26	25 24 23 22	21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6	5 4	3 2 1 0
`0x0F`	0x0	0x2900 | operand	00	Cond

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x2900 |` operand; bits 15–12 select coprocessor 2 and bits 11–8 select operation `0x9`. Within the low operand byte: bits 7–6 select the address register (`00` = `a`, `01` = `l`, `10` = `s`, `11` reserved – this is not the AF address-register encoding), bit 5 selects direction (`0` forward, `1` backward), bits 4–3 are reserved and written as zero, and bits 2–0 hold the count magnitude, `0`–`7`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
MOVI ah, #0x0040
MOVI al, #0x0000 ; a = 0x00400000, start of destination block
DMAW a, #3 ; memory[a], memory[a+1], memory[a+2] = r1, r2, r3; a += 3
```

Example 16-9: Writing Registers to a Memory Block

DMAT

DMA Memory Transfer Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), an optional coprocessor absent on models that do not implement it.

Syntax: `DMAT a, l, #n`

Operands: The source and destination address registers must be written explicitly as `a` and `l`. `#n` is an 8-bit literal count operand in the range 0–255. A count of zero is a no-op.

Description: Copies sequential words from the address in `a` to the address in `l`, using `r1`–`r7` as an internal transfer window in chunks of up to seven words.

Operation:

```
remaining = n
while remaining > 0:
    chunk = min(remaining, 7)
    read chunk words from memory[a] into r1..r[chunk]
    write chunk words from r1..r[chunk] to memory[l]
    a.low = a.low + chunk
    l.low = l.low + chunk
    remaining = remaining - chunk
```

Write-back: A successful non-zero **`DMAT`** advances `a` and `l` by the number of words transferred. Address-register high words are not changed by the transfer increments. Each chunk writes `r1`–`r`c, where c is the chunk size. After completion, `r1`–`r`c hold the last chunk and the higher registers hold the words of the previous chunk, or their original values if there was none. Software should treat `r1`–`r7` as clobbered. Because `l` is also the subroutine link register (Chapter 3), a subroutine that issues **`DMAT`** must save and restore its return address around the transfer.

Flags: Preserved.

Exceptions: Missing DMA hardware and reserved DMA operations raise an invalid-opcode fault. Reserved operand encodings are architecturally undefined (see the Model and Revision Policy in Section 16.4.1). In protected mode, raises a privilege-violation fault. DMA reads and writes can raise data bus, bus-protection, and segment-overflow faults (see Section 16.7.2).

Fault side effects: `a` and `l` only advance once a full chunk (up to seven words) has both been read and written successfully, so a fault leaves both address registers at the last fully completed chunk boundary, not at the exact word. Within the chunk where the fault occurs, the state of `r1`–`r7` depends on which half of the chunk failed: a fault during the chunk's memory reads can leave them holding a mix of this chunk's already-read words and the previous chunk's stale contents for the words not yet read; a fault during the chunk's memory writes (after its reads all succeeded) means `r1`–`r7` fully reflect the current chunk, but the destination range may be only partially written for it. Software must not assume the full requested count completed, and cannot use `r1`–`r7` alone to identify which words within the current chunk were actually transferred. See Section 16.7.3 for what a handler can safely do about it.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $6 + 6\lceil(2n+1)/6\rceil$ cycles when the command condition is true and all DMA read and write cycles are acknowledged without wait states. The $n = 0$ no-op form and transfers of up to two words take 12 cycles. Wait states add to the relevant DMA read or write bus cycle. Instruction fetch resumes only after the DMA operation completes or faults. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.

Assembles to: `COPI #0x2A00 | n`. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x2A00 | #n</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x2A00 | #n`; bits 15–12 select coprocessor 2, bits 11–8 select operation `0xA`, and bits 7–0 hold `#n`, `0`–`255`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

Overlap: Overlapping source and destination ranges are architecturally undefined unless the source and destination addresses are identical.

Notes:

- **`DMAT`**'s `#n` is fixed at assembly time; it has no register-operand form. SIRC-1 meta-instructions always assemble to exactly one real instruction, so a runtime-variable count cannot be a **`DMAT`** form without turning it into a multi-instruction expansion, which would make its instruction count vary with its operand, unlike every other meta-instruction.
- A transfer whose length is only known at runtime is still possible, just not as a single mnemonic: build the command word in a register and dispatch it directly with the general **`COPR`** instruction instead of **`DMAT`**, for example:

```
ANDI[N] r1, #0x00FF ; keep only the low byte (the count)
ORRI[N] r1, #0x2A00 ; combine with DMAT's own fixed header
COPR r1 ; dispatch -- identical to DMAT from here
```

  Any register may hold the command; **`COPR`** reads it at decode, before **`DMAT`** overwrites `r1`. The `[N]` suffix keeps these two setup steps from disturbing the status flags, matching **`DMAT`**'s own **Flags:** Preserved entry above. This is ordinary use of the already-documented **`COPR`** instruction (see the **`COPI`**/**`COPR`** entry above), not a distinct form of **`DMAT`**. Once dispatched, it behaves exactly like **`DMAT`** – including this entry's Fault side effects above – since the DMA unit cannot tell whether a command arrived via **`COPI`** or **`COPR`**.

```
MOVI ah, #0x0040
MOVI al, #0x0000 ; a = 0x00400000, source block
MOVI lh, #0x0050
MOVI ll, #0x0000 ; l = 0x00500000, destination block
DMAT a, l, #16 ; Copy 16 words from memory[a] to memory[l]
```

Example 16-10: Copying a Memory Block with DMAT

16.8 Integer Maths Unit (Coprocessor 3)

The optional integer maths unit provides a stable multiply/divide convention. CPU models without the hardware raise an invalid-opcode fault for these commands, allowing supervisor software to emulate the operation and return as if the coprocessor call had completed. Because the fault is retryable, the handler must advance the saved return address in link register 6 by 2 with **`ETFR`**/**`ETTR`** before **`RETE`** (see the Invalid Opcode Fault section of Chapter 6).

Reference source listings for software emulation routines are supplied with the SIRC-1 distribution media. Those listings are examples of a supervisor compatibility handler, not additional instruction encodings.

16.8.1 Maths Operations

Mnemonic	Command	Inputs	Outputs	Operation
MULU	`0x3000`	`r1`, `r2`	`r1:r2`, `r3`	Unsigned 16 x 16 -> 32 multiply
MULS	`0x3100`	`r1`, `r2`	`r1:r2`, `r3`	Signed 16 x 16 -> 32 multiply
DIVU	`0x3200`	`r1:r2`, `r3`	`r1`, `r2`, `r3`	Unsigned divide
DIVS	`0x3300`	`r1:r2`, `r3`	`r1`, `r2`, `r3`	Signed divide

Table 16.5: Integer Maths Unit Operations

Bit	Name	Meaning
0	E	Any maths error
1	DZ	Divide by zero
2	OV	Quotient overflow
3–15	Reserved	Written as zero, ignore read

Table 16.6: Maths Status Word in `r3`

Integer maths operations preserve CPU status flags. Missing maths hardware raises an invalid-opcode fault. Reserved maths operations raise an invalid-opcode fault (see the Model and Revision Policy, Section 16.4.1). Divide by zero and quotient overflow are reported in `r3`, not as arithmetic exceptions. If the condition is false, no maths command is dispatched, no coprocessor opcode check is performed, and registers and status flags are preserved.

MULU/MULS

Integer Multiply Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 3 (Integer Maths Unit), an optional coprocessor absent on models that do not implement it.

Syntax:

```
MULU ; Unsigned multiply
MULS ; Signed multiply
```

Operands: None in assembly syntax. The operations multiply `r1` by `r2`.

Description: Multiplies two 16-bit operands and writes the 32-bit product to `r1`:`r2`, high word first. **`MULU`** treats both inputs as unsigned. **`MULS`** treats both inputs as two's-complement signed values.

Operation:

```
MULU:
    product = unsigned(r1) * unsigned(r2)
    r1 = product[31:16]
    r2 = product[15:0]
    r3 = 0

MULS:
    product = signed(r1) * signed(r2)
    r1 = product[31:16]
    r2 = product[15:0]
    r3 = 0
```

Write-back: Writes `r1`, `r2`, and `r3`.

Flags: CPU status flags are preserved. `r3` is written as zero.

Exceptions: Missing maths hardware raises an invalid-opcode fault (see the Model and Revision Policy, Section 16.4.1). These operations perform no data-memory bus access.

Condition field: If the condition is false, no maths command is dispatched and registers and status flags are preserved.

Timing: At least 12 cycles: 6 for the **`COPI`** call and 6 for the dispatch slot that begins at the next phase 0. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes, and a pending interrupt waits for them (Section 6.3.3).

Privilege: User-callable in protected mode and supervisor mode.

Assembles to: `COPI #0x3000` or `COPI #0x3100`. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x3000</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x3100</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x3000` (**`MULU`**) or `0x3100` (**`MULS`**); bits 15–12 select coprocessor 3, bits 11–8 select the operation, and bits 7–0 are unused and written as zero.
- **AF (bits 5–4):** fixed at `00`.

- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
MOVI r1, #1000 ; Multiplicand
MOVI r2, #2000 ; Multiplier
MULU ; r1:r2 = 1000 * 2000, r3 = 0
```

Example 16-11: Unsigned 16x16 Multiply

DIVU/DIVS

Integer Divide Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 3 (Integer Maths Unit), an optional coprocessor absent on models that do not implement it.

Syntax:

```
DIVU ; Unsigned divide
DIVS ; Signed divide
```

Operands: None in assembly syntax. The dividend is `r1`:`r2`, high word first. The divisor is `r3`.

Description: Divides a 32-bit dividend by a 16-bit divisor. Successful division writes the remainder to `r1`, the quotient to `r2`, and zero to `r3`.

Operation:

```
if divisor is zero:
    r3 = E | DZ
else if quotient is outside 0..0xFFFF (DIVU) or -32768..32767 (DIVS):
    r3 = E | OV
else:
    r1 = remainder
    r2 = quotient
    r3 = 0
```

Write-back: Successful division writes `r1`, `r2`, and `r3`. Divide-by-zero and quotient-overflow cases leave `r1`:`r2` unchanged and write the maths status word to `r3`.

Flags: CPU status flags are preserved. Maths errors are reported through `r3`, not through `sr`.

Exceptions: Missing maths hardware raises an invalid-opcode fault (see the Model and Revision Policy, Section 16.4.1). Divide by zero and quotient overflow do not raise arithmetic exceptions. These operations perform no data-memory bus access.

Condition field: If the condition is false, no maths command is dispatched and registers and status flags are preserved.

Timing: At least 12 cycles: 6 for the **COPI** call and 6 for the dispatch slot that begins at the next phase 0. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes, and a pending interrupt waits for them (Section 6.3.3). Software emulation timing is defined by the invalid-opcode handler.

Privilege: User-callable in protected mode and supervisor mode.

Assembles to: `COPI #0x3200` or `COPI #0x3300`. See Table 16.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x3200</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x0F</td><td colspan="4">0x0</td><td colspan="16">0x3300</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x3200` (**DIVU**) or `0x3300` (**DIVS**); bits 15–12 select coprocessor 3, bits 11–8 select the operation, and bits 7–0 are unused and written as zero.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

Signed division: **`DIVS`** treats `r1`:`r2` as a 32-bit two's-complement dividend and `r3` as a 16-bit two's-complement divisor. The quotient is truncated toward zero and the remainder has the sign of the dividend. The quotient must lie in `-32768`–`32767`; any other quotient, including `-2147483648 / -1`, is quotient overflow. The remainder lies in `-32768`–`32767` and is written to `r1` as a 16-bit two's-complement value.

```
MOVI r1, #0 ; High word of dividend
MOVI r2, #100 ; Low word of dividend
MOVI r3, #7 ; Divisor
DIVU ; r1 = remainder, r2 = quotient, r3 = 0 on success
CMPI r3, #0 ; r3 is the maths status word, zero on success
BRAN|!= @divide_error ; DZ or OV is set; r1:r2 still hold the dividend
```

Example 16-12: Unsigned Divide with Error Check

CHAPTER 17

Meta-Instructions

17.1 Overview

Meta-instructions are assembly mnemonics that lower to ordinary SIRCIS instruction encodings. They exist to make common operations easier to read and write; they do not introduce additional CPU opcodes. Every meta-instruction is an alias for exactly one real instruction encoding, never a sequence of several: this keeps a program's instruction count, and therefore its timing, predictable directly from its source, without needing to know which mnemonics are "real" and which secretly expand to more than one. An operation that cannot be expressed as a single instruction (for example, a runtime-variable operand where the underlying encoding only has room for a compile-time immediate) is documented as an explicit multi-instruction technique where it comes up, not given its own meta-instruction mnemonic.

This table is the canonical list of meta-instructions; Section 12.4 summarizes it.

Meta-Instruction	Primary Documentation	Lowers To	Purpose
`NOOP`	Chapter 17	`ADDI[N] r1, #0`	No operation
`SHFT`	Chapter 13	`ORRI[S] rD, #0, shift`	Shift with status update
`BRAN`	Chapter 15	`LDEA p, ...`	PC-relative branch
`BRSR`	Chapter 15	`LDEL p, ...`	PC-relative call
`LJMP`	Chapter 15	`LDEA p, ...`	Long jump
`LJSR`	Chapter 15	`LDEL p, ...`	Long call
`RETS`	Chapter 15	`LDEA p, (#0, l)`	Return from subroutine
`EXCP`	Chapter 16	`COPI #0x11vv`	Software exception
`WAIT`	Chapter 16	`COPI #0x1900`	Wait for interrupt
`RETE`	Chapter 16	`COPI #0x1A00`	Return from exception
`RSET`	Chapter 16	`COPI #0x1B00`	System reset
`ETFR`	Chapter 16	`COPI #0x1Cxy`	Read exception state
`ETTR`	Chapter 16	`COPI #0x1Dxy`	Write exception state
`DMAR`	Chapter 16	`COPI #0x28xx`	DMA read to registers
`DMAW`	Chapter 16	`COPI #0x29xx`	DMA write from registers
`DMAT`	Chapter 16	`COPI #0x2Ann`	DMA memory transfer
`MULU`	Chapter 16	`COPI #0x3000`	Unsigned multiply
`MULS`	Chapter 16	`COPI #0x3100`	Signed multiply
`DIVU`	Chapter 16	`COPI #0x3200`	Unsigned divide
`DIVS`	Chapter 16	`COPI #0x3300`	Signed divide

Table 17.1: Meta-Instruction Cross-Reference

NOOP

No Operation

Applicability: SIRC-1, all revisions

Syntax: `NOOP`

Operands: None.

Description: Does nothing for 6 cycles. Useful for timing delays, instruction padding, or placeholders during development.

Operation: Executes an add-immediate-zero instruction whose destination is `r1`: `r1` is rewritten with its own value plus zero, so no architectural state changes occur. Because the status-register update source is `None`, no condition flags are written. Register field `0x0` is deliberately assigned to `r1`, an ordinary unprivileged register (see Chapter 3), so this instruction is unprivileged and an all-zero instruction word (as read from erased or uninitialized memory) always executes safely as a no-op, in both protected and supervisor mode.

Write-back: No architectural state changes.

Flags: Preserved.

Exceptions: Inherits **`ADDI[N]`** exception behavior. It does not access data memory and raises no instruction-specific faults; instruction-fetch faults can still occur before it executes, and trace faults can still occur after it commits. Like any instruction that does not branch, it raises a segment-overflow fault if it occupies the last instruction slot of a segment while `SR.A` is set (Section 6.2.5).

Condition field: If the condition is false, no state changes occur. Conditional **`NOOP`** is allowed but normally not useful.

Timing: Same as **`ADDI`**: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode.

Assembles to: `ADDI[N] r1, #0`. See Table 13.1 for the full family of legal forms.

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="6">0x00</td><td colspan="4">0x0</td><td colspan="16">0x0000</td><td colspan="2">00</td><td colspan="4">Cond</td></tr>
</table>

Instruction Fields:

- **Register (bits 25–22):** fixed at `0x0` (`r1`).
- **Immediate (bits 21–6):** fixed at `#0`.
- **AF (bits 5–4):** fixed at `00` (status register not updated).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 10.1.

```
; Wait 18 cycles (6 cycles per NOOP)
NOOP
NOOP
NOOP
```

Example 17-1: Timing Delay with NOOP

CHAPTER A

Opcode Map

A.1 Complete Opcode Reference

This appendix provides a complete map of all 64 possible opcodes in the SIRCIS instruction set.

Table A.1: Complete SIRCIS Opcode Map

Hex	Binary	Mnemonic	Description
ALU Immediate (`0x00`–`0x0F`)			
`0x00`	000000	ADDI	Add Immediate
`0x01`	000001	ADCI	Add with Carry Immediate
`0x02`	000010	SUBI	Subtract Immediate
`0x03`	000011	SBCI	Subtract with Carry Immediate
`0x04`	000100	ANDI	AND Immediate
`0x05`	000101	ORRI	OR Immediate
`0x06`	000110	XORI	XOR Immediate
`0x07`	000111	MOVI	Move Immediate
`0x08`	001000	CMNI	Compare Negative Immediate
`0x09`	001001	CNCI	Compare Negative with Carry Immediate
`0x0A`	001010	CMPI	Compare Immediate
`0x0B`	001011	CPCI	Compare with Carry Immediate
`0x0C`	001100	TSAI	Test AND Immediate
`0x0D`	001101	TSOI	Test OR Immediate
`0x0E`	001110	TSXI	Test XOR Immediate
`0x0F`	001111	COPI	Coprocessor Immediate
Memory and Control (`0x10`–`0x1F`)			
`0x10`	010000	STOR	Store Indirect Immediate
`0x11`	010001	STOR	Store Indirect Register
`0x12`	010010	STOR	Store Pre-Decrement Immediate
`0x13`	010011	STOR	Store Pre-Decrement Register
`0x14`	010100	LOAD	Load Indirect Immediate
`0x15`	010101	LOAD	Load Indirect Register
`0x16`	010110	LOAD	Load Post-Increment Immediate
`0x17`	010111	LOAD	Load Post-Increment Register
`0x18`	011000	LDEA	Load Effective Address Immediate
`0x19`	011001	LDEA	Load Effective Address Register
`0x1A`	011010	LDEA	Load Effective Address Pre-Decrement Immediate
`0x1B`	011011	LDEA	Load Effective Address Pre-Decrement Register
`0x1C`	011100	LDEL	Load Effective Address and Link Immediate
`0x1D`	011101	LDEL	Load Effective Address and Link Register
`0x1E`	011110	LDEL	Load Effective Address and Link Post-Increment Immediate
`0x1F`	011111	LDEL	Load Effective Address and Link Post-Increment Register
ALU Short Immediate (`0x20`–`0x2F`)			
`0x20`	100000	ADDI	Add Short Immediate

continued on next page

Table A.1 (continued)

Hex	Binary	Mnemonic	Description
`0x21`	100001	ADCI	Add with Carry Short Immediate
`0x22`	100010	SUBI	Subtract Short Immediate
`0x23`	100011	SBCI	Subtract with Carry Short Immediate
`0x24`	100100	ANDI	AND Short Immediate
`0x25`	100101	ORRI	OR Short Immediate
`0x26`	100110	XORI	XOR Short Immediate
`0x27`	100111	–	Undocumented
`0x28`	101000	CMNI	Compare Negative Short Immediate
`0x29`	101001	CNCI	Compare Negative with Carry Short Immediate
`0x2A`	101010	CMPI	Compare Short Immediate
`0x2B`	101011	CPCI	Compare with Carry Short Immediate
`0x2C`	101100	TSAI	Test AND Short Immediate
`0x2D`	101101	TSOI	Test OR Short Immediate
`0x2E`	101110	TSXI	Test XOR Short Immediate
`0x2F`	101111	–	Undocumented
		ALU Register (`0x30`–`0x3F`)	
`0x30`	110000	ADDR	Add Register
`0x31`	110001	ADCR	Add with Carry Register
`0x32`	110010	SUBR	Subtract Register
`0x33`	110011	SBCR	Subtract with Carry Register
`0x34`	110100	ANDR	AND Register
`0x35`	110101	ORRR	OR Register
`0x36`	110110	XORR	XOR Register
`0x37`	110111	MOVR	Move Register
`0x38`	111000	CMNR	Compare Negative Register
`0x39`	111001	CNCR	Compare Negative with Carry Register
`0x3A`	111010	CMPR	Compare Register
`0x3B`	111011	CPCR	Compare with Carry Register
`0x3C`	111100	TSAR	Test AND Register
`0x3D`	111101	TSOR	Test OR Register
`0x3E`	111110	TSXR	Test XOR Register
`0x3F`	111111	COPR	Coprocessor Register

A.2 Opcode Patterns

This section summarizes the opcode decoding rules for quick reference. Chapter 7 defines them, and Section 7.9 shows them as decode trees, one figure per opcode group.

A.2.1 Format Identification

Bit numbers in this section refer to the 6-bit opcode value (instruction bits 31–26), not to the instruction word.

- Opcode bits 5–4 = 00: Immediate format (`0x00`–`0x0F`)
- Opcode bits 5–4 = 01: Memory and control operations (`0x10`–`0x1F`). Format is selected by opcode bit 0: even opcodes use Immediate format, odd opcodes use Register format.
- Opcode bits 5–4 = 10: Short Immediate with Shift (`0x20`–`0x2F`)
- Opcode bits 5–4 = 11: Register format (`0x30`–`0x3F`)

A.2.2 Save vs. Test

- Bit 3 = 0 and low nibble `0`–`6`: save result (`0x00`–`0x06`, `0x20`–`0x26`, `0x30`–`0x36`)
- Bit 3 = 1 and low nibble `8`–`E`: test only, discard result (`0x08`–`0x0E`, `0x28`–`0x2E`, `0x38`–`0x3E`)
- Low nibble `F` is the coprocessor call (`0x0F`, `0x3F`), not a test form

- Low nibble `7` is **`MOVI`**/**`MOVR`** (`0x07`, `0x37`)
- `0x27` and `0x2F` are undocumented

A.2.3 Memory Operations Decoding

For opcodes `0x10`–`0x1F`, bits encode memory operation type:

- Bit 4 = 1: Always set for memory/control section
- Bits 3–2: Operation type
  - 00 = Store
  - 01 = Load
  - 10 = Load Effective Address
  - 11 = Load EA with Link (jump to subroutine)
- Bit 1: Auto-update of the source address register (0 = no update, 1 = pre-decrement for STOR/LDEA, post-increment for LOAD/LDEL). Only the low word of the pair is adjusted, and always by one word regardless of the displacement (Chapter 8).
- Bit 0: immediate displacement (0) or register offset (1)

A.3 Undocumented Instructions

Two opcodes are undocumented and have no public assembly syntax: `0x27` and `0x2F`, both in the Short Immediate group. Software must not use them. Their behavior is architecturally undefined and reserved for future revisions (see Appendix C).

A.4 Quick Lookup Table

Operation	Immediate	Short+Shift	Register
ADD	`0x00`	`0x20`	`0x30`
ADC	`0x01`	`0x21`	`0x31`
SUB	`0x02`	`0x22`	`0x32`
SBC	`0x03`	`0x23`	`0x33`
AND	`0x04`	`0x24`	`0x34`
ORR	`0x05`	`0x25`	`0x35`
XOR	`0x06`	`0x26`	`0x36`
MOV	`0x07`	–	`0x37`
CMN	`0x08`	`0x28`	`0x38`
CNC	`0x09`	`0x29`	`0x39`
CMP	`0x0A`	`0x2A`	`0x3A`
CPC	`0x0B`	`0x2B`	`0x3B`
TSA	`0x0C`	`0x2C`	`0x3C`
TSO	`0x0D`	`0x2D`	`0x3D`
TSX	`0x0E`	`0x2E`	`0x3E`
COP	`0x0F`	–	`0x3F`
Undocumented	–	`0x27`, `0x2F`	–

Table A.2: Instruction Variant Quick Reference

Opcodes `0x27` and `0x2F` are undocumented (Appendix C). They are not short-immediate **`MOVI`** or **`COPI`** forms.

CHAPTER B

Instruction Timing

B.1 Overview

Most SIRCIS instructions follow the six-stage execution phase model and complete in 6 clock cycles when no bus wait states are required. A bus operation may hold the current phase until it is acknowledged, adding wait cycles without adding execution phases. Coprocessor calls are the exception: the dispatched coprocessor occupies at least one further slot after the six-phase call (Section B.3.1). This fixed execution sequence greatly simplifies software development and hardware design. See the *SIRC-1 Hardware Reference Manual* for the bus signal timing, wait-state, and response-priority rules.

B.2 Six-Stage Execution Phases

For a detailed description of each phase, see Chapter 2.

1. **Instruction Fetch (High Word)** – 1 cycle
2. **Instruction Fetch (Low Word)** – 1 cycle
3. **Decode and Register Fetch** – 1 cycle
4. **Execute and Address Calculation** – 1 cycle
5. **Memory Access** – 1 cycle
6. **Write Back** – 1 cycle

B.3 Timing Characteristics

Instruction Type	Cycles
ALU (Register)	6
ALU (Immediate)	6
Load from Memory	6
Store to Memory	6
Branch (taken)	6
Branch (not taken)	6
Conditional processing-unit instruction (executed)	6
Conditional processing-unit instruction (not executed)	6
NOOP	6
DMAR/**DMAW**	$6 + 6\lceil(\lvert n\rvert + 1)/6\rceil$
DMAT	$6 + 6\lceil(2n + 1)/6\rceil$
MULU/**MULS**/**DIVU**/**DIVS**	At least 12; implementation-defined
COPI/**COPR**	12 (6 + 6 coprocessor dispatch); 6 for coprocessor `0x0`, which has no dispatch slot
Exception-unit meta-instructions (**EXCP**, **RETE**, **ETFR**, **ETTR**)	12 (6 + 6 dispatch); a discarded **EXCP** also takes 12, as its dispatch slot runs with no-op phases
RSET	18 (6 call + 6 $\overline{\text{RSTO}}$ hold + 6 reset-vector fetch; no dispatch slot)
WAIT	12 + idle (0 if an interrupt that can be taken is already pending) + 6 for the interrupt dispatch, or until reset

Table B.1: Instruction Timing

The cycle counts in this appendix assume that every bus operation is acknowledged without a wait state. For **DMAR**, **DMAW**, and **DMAT**, phase 0 of the DMA slot is a setup cycle with no bus access, and the DMA unit then performs one bus access per cycle from phase 1, running on into further whole 6-cycle slots as needed (Section 16.7). Any coprocessor call or meta-instruction whose condition is false writes nothing to the CSR, dispatches nothing, and takes 6 cycles; the coprocessor rows give the condition-true figure.

B.3.1 Coprocessor Call Timing

This subsection is the canonical statement of the coprocessor-call timing rule; Chapters 1, 12 and 16 refer here. A coprocessor call whose condition is true (**COPI**, **COPR** and every meta-instruction that assembles to one) takes its own 6 cycles and then a second 6-cycle slot for the coprocessor dispatch, which begins at the next phase 0, so most calls take 12 cycles before the next instruction is fetched. The exceptions are:

- A call whose condition is false skips its dispatch slot and takes 6 cycles.
- A call naming coprocessor `0x0` has no dispatch slot and takes 6 cycles.
- **RSET** has no dispatch slot, because its reset request replaces the slot, and takes 18 cycles.
- **WAIT**, DMA transfers and maths-unit operations can take longer than 12 cycles: a **WAIT** idles until woken, a DMA transfer of more than five accesses runs into further slots, and a maths operation may add implementation-defined cycles.

B.3.2 Exception and Fault Timing

A hardware-exception or fault dispatch occupies one 6-cycle slot: the vector's high word is fetched at phase 0, the low word at phase 1, and entry to the handler is committed at phase 3. A latched interrupt is dispatched at the next phase 0 at which the CSR is empty and no coprocessor is active (Section 6.3.3). The worst-case latency from the interrupt request to the handler's first instruction fetch is the time the current instruction still needs plus 6 cycles to dispatch, with the current instruction being as long as its entry in Table B.1 allows:

- Most instructions: 6 + 6 = 12 cycles.
- A coprocessor call: 12 + 6 = 18 cycles, because its dispatch slot completes first.
- **`DMAR`** or **`DMAW`** with a count of 7: 18 + 6 = 24 cycles.
- **`DMAT`** with a count of 255: 522 + 6 = 528 cycles (22 microseconds at an example 24 MHz clock), because a DMA transfer cannot be interrupted.
- A maths-unit command: 12 + its implementation-defined extra cycles + 6. The architecture sets no upper bound, so an implementation must document it.

These figures assume no bus wait states. The **`DMAT`** figure is the worst case only for parts without a maths unit, or whose maths operations finish in fewer cycles. Reset holds $\overline{\text{RSTO}}$ for the cycles $\overline{\text{RSTI}}$ is asserted plus five more (6 cycles for a one-cycle $\overline{\text{RSTI}}$ or for **`RSET`**) before the reset-vector fetch; a failed reset-vector fetch repeats the hold.

A retryable fault cancels the faulting instruction at the phase that detects it. Once an instruction has started, its six-phase sequence is never cut short: the remaining phases still run, but as no-ops that perform no bus access and commit no results, so the canceled instruction occupies its full six cycles. Alignment is detected at phase 0, before the instruction starts, and issues no bus cycle; its dispatch slot takes the place of the instruction, so it costs no extra cycles. Privilege Violation is detected at phase 2 for a write to a privileged register field and for a supervisor-only coprocessor operation, at phase 3 for an **`EXCP`** vector below `0x60`, and at phase 5 for an address-pair write that would change a high word. Segment Overflow is detected at phase 3. Bus and Bus Protection faults are detected at the phase that acknowledges the failing bus access. The first fault detected cancels the instruction, and the checks belonging to its remaining phases are not made, so an instruction that meets two fault conditions raises the one detected in the earlier phase. In phase 3, a privilege violation takes precedence over a segment overflow, and an instruction that wraps more than one address (the effective address, the auto-update or the program counter) raises a single segment-overflow fault. In every other case the exception-unit dispatch slot begins at the next phase 0, after the remaining phases have run. Alignment, Bus, Bus Protection, Invalid Opcode, Privilege Violation, and Segment Overflow all save the address of the faulting instruction and are retryable. The instruction trace fault is raised after its instruction has completed, so nothing is canceled.

A bus or protection fault during an exception-vector fetch is handled the same way as one during an instruction fetch. The exceptions are a failed fault-vector fetch, which raises a double fault; a failed double-fault vector fetch, which resets the CPU; and a failed reset-vector fetch, which raises no fault and repeats the reset-output hold (Chapter 6). The dispatch slot in progress is canceled and its remaining phases run as no-ops, and the dispatch for the new fault begins at the next phase 0. Only reset restarts the phase sequence immediately: it discards all state and returns to phase 0.

B.4 Conditional Execution

When a conditional instruction's condition is not met:

- All six phases still occur and consume one cycle each
- The execute, memory-access, and write-back phases perform no operation: no effective address is computed, no data-address `SR.A` overflow check is made, and no data bus cycle is driven
- The program-counter advance is still checked: with `SR.A` set, a skipped instruction in the last instruction slot of a segment raises a segment-overflow fault in phase 3 (Section 6.2.5)
- No register, memory, or address-register update occurs
- Status flags are not updated

This means conditional instructions do not branch and avoid control-flow stalls, but failed conditions still consume execution time.

B.5 Program Timing Examples

B.5.1 Simple Loop

```
; Process 10 elements (assuming no bus wait states)
MOVI r7, #10 ; 6 cycles
:loop
    LOAD r1, (#0, a)+ ; 6 cycles
    ADDI r1, #1 ; 6 cycles
    STOR (#-1, a), r1 ; 6 cycles
    SUBI r7, #1 ; 6 cycles
    BRAN|!= @loop ; 6 cycles
; Total per iteration: 30 cycles
; Total for 10 iterations: 6 + (10 * 30) = 306 cycles
```

B.5.2 Function Call Overhead

```
; Call function
LJSR a ; 6 cycles

; Function body executes...

; Return
RETS ; 6 cycles
; Total overhead: 12 cycles
```

B.5.3 Conditional Block

```
CMPR r1, r2 ; 6 cycles
ADDR[N]|HI r3, r4, r5 ; 6 cycles (exec if r1 > r2, unsigned)
ADDR|LO r3, r6, r7 ; 6 cycles (exec if r1 <= r2, unsigned)
; Total: 18 cycles (one conditional executes, one doesn't)
```

B.6 Performance Considerations

B.6.1 No Pipeline Stalls

The SIRC-1 does not implement pipelining in the traditional sense where multiple instructions are in flight simultaneously. Instead:

- Each instruction completes fully before the next begins
- No data hazards or phase bubbles
- No branch prediction penalties
- Base execution time is predictable before bus wait states are considered

B.6.2 Memory Access

- Instruction fetch, data memory access, and exception vector fetch each occupy one execution phase per 16-bit word transferred.
- If the external memory system has not acknowledged the transfer ($\overline{\text{BACK}}$) and has not aborted it with a bus error ($\overline{\text{BERR}}$) or bus protection error ($\overline{\text{BPER}}$), the CPU holds that phase until the transfer is acknowledged.
- Each wait state adds one clock cycle to the instruction, including a wait on a DMA access: the held phase delays the rest of the transfer and the slot boundaries after it, so the DMA total is not rounded up again
- Every instruction performs two instruction-fetch bus accesses; a load or store adds one data access; an exception-unit dispatch that enters a handler (a hardware interrupt, a fault, an accepted **`EXCP`**, or reset) adds two exception-vector-fetch accesses; any other coprocessor dispatch (**`RETE`**, **`ETFR`**, **`ETTR`**, **`WAIT`**, or a maths command) performs no bus access of its own, although a DMA command then adds one access per word read or written. A system that inserts w wait states per access therefore adds $2w$ cycles to an ALU instruction, $3w$ cycles to a load or store, $(k+2)w$ cycles to a DMA command with k accesses, $4w$ cycles to an accepted **`EXCP`** or to **`RSET`**, and $2w$ cycles to the other exception-unit meta-instructions.

B.6.3 Optimization Guidelines

1. **Use conditional execution for short sequences**

```
; Instead of:
CMPR r1, r2
BRAN|LO @skip
ADDI r3, #1
:skip

; Use:
CMPR r1, r2
ADDI|HI r3, #1 ; Saves branch overhead
```

2. **Combine operations with shifts**

```
; Instead of:
ADDR r1, r2, r2 ; r1 = 2 * r2
ADDR r1, r1, r2 ; r1 = 3 * r2

; Use:
ADDR r1, r2, r2, LSL #1 ; Multiply by 3 in one instruction
```

3. **Use post-increment/pre-decrement for iteration**

```
; Instead of:
LOAD r1, (#0, a)
ADDI al, #1

; Use:
LOAD r1, (#0, a)+ ; Same register result in one instruction (6 cycles saved); flags are not changed
```

4. **Loop unrolling reduces overhead**

```
; Instead of processing one item per iteration,
; process multiple items to reduce branch overhead
:loop
    LOAD r1, (#0, a)+
    LOAD r2, (#0, a)+
    LOAD r3, (#0, a)+
    LOAD r4, (#0, a)+
    ; Process 4 items...
    SUBI r7, #4
    BRAN|HI @loop
```

B.7 Theoretical Performance

At a given clock frequency:

- **Instructions per second:** $f_{clock}/6$
- **At 10 MHz:** $\approx$ 1.67 million instructions/second
- **At an example 24 MHz clock:** $\approx$ 4.0 million instructions/second

These are theoretical maximums assuming perfect code with no inefficiencies.

The SIRC-1's fixed 6-cycle execution trades peak throughput for predictability: every ordinary instruction takes the same number of cycles regardless of the operation or operands, which is simpler to implement in hardware and easier to reason about than a variable-cycle-count design.

CHAPTER C

Undocumented Instructions

C.1 Overview

The SIRCIS instruction set includes two undocumented instruction opcodes, `0x27` and `0x2F`. They have no public assembly syntax and may be assigned in a future CPU revision. Their behavior is architecturally undefined: an implementation decodes them without raising an invalid-opcode fault, but what they do is not part of the architectural contract and may change in any later revision. Software must not use them.

Undefined and undocumented encodings remain bound by the privilege rules. In protected mode they cannot change the privileged byte of the status register, a high word of an address register, or the state of the exception unit, and they may write only the registers named in their fields.

Warning: Undocumented instructions must not be used in production code.

C.2 Undefined Operand Combinations

Undefined operand combinations differ from undocumented opcodes: the opcode is documented, but a particular choice of operands writes the same address-register pair more than once during one instruction. The privilege rules in the Overview apply to these combinations.

C.2.1 Aliased Address-Register Writes

The auto-update effective-address forms can write both a destination address-register pair and an updated source address-register pair. **`LDEL`** also writes the link-register pair. Any of the following is architecturally undefined:

- two write-back targets in the same address-register pair, including the link write and the program-counter advance
- a **`LOAD`** post-increment whose destination is a half of the updated pair
- a **`STOR`** pre-decrement whose source is a half of the updated pair

```
; Undefined: a is both destination and auto-updated source
LDEA a, -(#0, a)

; Undefined: l is both link register and auto-updated source
LDEL p, (#0, l)+

; Undefined: l is both link register and destination
LDEL l, (#0, a)

; Undefined: the auto-update of p competes with the program-counter advance
LOAD r1, (#0, p)+
```

Programs must not use aliased register writes. The assembler rejects the known hazardous forms so that programs do not accidentally depend on undefined write-back ordering.

C.3 Undocumented Opcode List

Opcode	Format	Notes
`0x27`	Short Imm	No public assembly syntax
`0x2F`	Short Imm	No public assembly syntax

Table C.1: Undocumented Instruction Opcodes

C.4 Implementation Notes

C.4.1 Implementation Behavior

An implementation – whether a simulator or a hardware implementation – decodes undocumented instructions normally rather than rejecting them. Their exact behavior depends on implementation details and is not part of the architectural contract. It is not required to be preserved in future CPU revisions.

C.5 Why Undocumented?

These instructions were left undocumented for two reasons:

1. **Opcode space for expansion**: the opcodes may be used for future features.
2. **Implementation flexibility**: the opcodes allow hardware optimizations.

C.6 Recommendations

C.6.1 For Application Developers

Applications must not use undocumented instructions in production code.

- Behavior may change between CPU revisions.
- Assemblers may reject them or emit warnings.
- Code may not be portable.
- Debugging tools may not recognize them.

C.6.2 For Compiler Writers

- Compilers must not generate undocumented instructions.
- Use only documented instructions for all operations.
- If optimization requires new operations, request formal documentation.

C.6.3 For Hardware Implementers

- Do not promise any behavior for these opcodes; it is architecturally undefined.
- Consider leaving them unassigned for future standardization.

C.7 Future Standardization

Either undocumented opcode may be formally specified in a future SIRCIS revision, or repurposed. Until then, software must not depend on either.

C.8 Opcode Space for Expansion

The SIRCIS instruction set has limited expansion space:

- Two undocumented opcodes, `0x27` and `0x2F`, which may be assigned in a future revision
- The reserved shift type (111) could enable additional shift modes.
- The reserved status-update source encoding (AF = 11) could select a new flag-update source. Until then, an ALU instruction with AF = 11 completes and writes its result, but N, Z, C, and V are architecturally undefined (see Chapter 7), and assemblers never emit it.
- Standard optional coprocessors and model-specific coprocessors can provide extended functionality through the documented coprocessor compatibility rules.

Any future expansion must maintain backward compatibility with existing code.

CHAPTER D

Example Programs

This appendix contains sample programs that demonstrate several features of the CPU.

D.1 Program Conventions

The assembly syntax used in these programs is summarized in the Assembly Language Summary in the Notation and Glossary chapter, and described in full in the *SIRC-1 Assembly Programmer's Manual*.

Every program in this appendix that is loaded at segment zero begins with a `.ORG 0x0000` / `.DQ @label` pair, which places the reset vector (Chapter 6): the CPU begins execution by reading a stored pointer from word addresses `0x000000`–`0x000001`, not by executing whatever instruction happens to be placed there. The separately mapped high segment and the standalone data segment shown later do not need one.

D.2 Byte Sieve

This program computes primes below 1000 using the Sieve of Eratosthenes, storing one word per candidate number in a memory-mapped array rather than in a register or the stack. It is a compact illustration of addressing-register setup (`ah`/`al` and `sh`/`sl`) and of using **`STOR`** and **`LOAD`** with an indexed effective address to walk an array. The two nested loops starting at labels `:2` and `:3` (lines 35–50) are the core of the sieve: the outer loop advances the candidate, and the inner loop marks its multiples as composite.

Listing D.1: Example D-1: Byte sieve

```
1  ; https://rosettacode.org/wiki/Sieve_of_Eratosthenes
2
3  .EQU $MAX_PRIME #1000
4
5  ; Reserved space for 256x32 bit exception vectors
6  .ORG 0x0000
7  .DQ @main
8
9  .ORG 0x0200
10 :main
11
12 ; File I/O segment
13 MOVI ah, #0x00F0
14 ; Array Start = 0x00F0_0000
15 MOVI al, #0x0000
16
17 ; Stack segment (other end of mapped file)
18 MOVI sh, #0x00F0
19 ; Stack top = 0x00F0_FFFF
20 MOVI sl, #0xFFFF
21
22 MOVI r7, #2
23 MOVI r1, #2
24 MOVI r2, #1
25
26 :1
27
28 STOR (r1, a), r2
29 ADDI r1, #2
30 CMPI r1, $MAX_PRIME
31 BRAN|<= @1
32 MOVI r1, #3
```

```
MOVI r3, #1

:2

LOAD r2, (r1, a)
CMPI r2, #1
BRAN|== @4
; THIS NUMBER IS A PRIME!
; TODO: Log this somehow
STOR -(s), r1
MOVR r7, r1
MOVR r2, r1

:3
STOR (r2, a), r3
ADDR r2, r1
CMPI r2, $MAX_PRIME
BRAN|<= @3

:4
ADDI r1, #2
CMPI r1, $MAX_PRIME
BRAN|<= @2

:DONE

; Halt CPU
COPI #0x14FF
```

D.3 Faults

This pair of programs exercises every fault type the CPU can raise and the vector table that routes them to handlers. `faults.sasm` deliberately triggers, in sequence, a bus fault, an alignment fault, two segment-overflow faults (one from an effective-address calculation, one from program-counter overflow), an instruction-trace fault, a bus-protection fault, a privilege violation, a level-five interrupt whose handler raises a second level-five request and so a level-five conflict fault, and finally an invalid-opcode fault (which itself triggers a nested double fault). Each handler, defined from line 158 onward, adds a handler-specific amount to one of `r1`–`r6`, so the final register dump shows which faults fired. The segment-overflow handler (lines 197–242) is worth particular attention: it reads exception metadata with **`ETFR`** to distinguish a program-counter overflow, which records no bus access, from a **`LOAD`** or **`STOR`** effective-address overflow, which records a data read or write, before deciding how to fix up the return address.

Listing D.2: Example D-2: Fault handling

```
1  ;; Segments
2  .EQU $PROGRAM_SEGMENT #0x0000
3  .EQU $SCRATCH_SEGMENT #0x0001
4  .EQU $PROGRAM_SEGMENT_HIGH #0x0002
5
6  ;; Devices
7  ; Debug
8  .EQU $DEBUG_DEVICE_SEGMENT #0x000B
9  .EQU $DEBUG_DEVICE_BUS_ERROR #0x0000
10 .EQU $DEBUG_DEVICE_EXCEPTION_L5 #0x0005
11 .EQU $DEBUG_DEVICE_BUS_PROTECTION_ERROR #0x0006
12
13
14 ;; Bus access types (fault metadata register, bits 0-2)
15 .EQU $BAT_NONE #0x0000
16 .EQU $BAT_DATA_WRITE #0x0003
17
```

```

; Reserved space for 256x32 bit exception vectors
.ORG 0x0000
.DQ @init
.ORG 0x0002
.DQ @bus_fault_handler
.ORG 0x0004
.DQ @alignment_fault_handler
.ORG 0x0006
.DQ @segment_overflow_fault_handler
.ORG 0x0008
.DQ @invalid_opcode_fault_handler
.ORG 0x000A
.DQ @privilege_violation_fault_handler
.ORG 0x000C
.DQ @instruction_trace_fault_handler
.ORG 0x000E
.DQ @level_five_interrupt_conflict_handler
.ORG 0x0010
.DQ @double_fault_handler
.ORG 0x0012
.DQ @bus_protection_fault_handler

.ORG 0x0020
.DQ @level_five_interrupt_handler

.ORG 0x0200
:init
MOVI r1, #1
MOVI r2, #1
MOVI r3, #1
MOVI r4, #1
MOVI r5, #1
MOVI r6, #1
MOVI r7, #1

; Get the debug device to raise a fault on the bus
MOVI ah, $DEBUG_DEVICE_SEGMENT
MOVI al, $DEBUG_DEVICE_BUS_ERROR
STOR (a), r1

:after_bus_error_store

; Jumping to an odd address triggers an alignment fault
MOVI pl, #0x0201

:after_odd_address

; Loading/storing from an odd address should not trigger an alignment fault
MOVI ah, $SCRATCH_SEGMENT
MOVI al, #0x0
MOVI r7, #0xCAFE
STOR (#1, a), r7
LOAD r7, (#1, a)


; Calculating an effective address that overflows triggers a segment overflow fault
; (but only if the SR is set up the right way)
```

```
MOVI ah, $PROGRAM_SEGMENT
MOVI al, #0xFFFE

; This should not trigger fault
LOAD r7, (#2, a)
; Switch on trap on overflow bit
ORRI sr, #0b0100_0000_0000_0000
; This _should_ trigger a fault
LOAD r7, (#2, a)

:after_segment_overflow_load

; Switch off trap on overflow bit
ANDI sr, #0b1011_1111_1111_1111

; Segment overflow again, but this time the PC overflow causes it

MOVI ah, $PROGRAM_SEGMENT_HIGH
MOVI al, #0xFFFE

; This should not trigger fault
LJMP a

:return_from_testing_pc_overflow_no_fault

; Switch on trap on overflow bit
ORRI sr, #0b0100_0000_0000_0000

MOVI ah, $PROGRAM_SEGMENT_HIGH
MOVI al, #0xFFFE

; This _should_ trigger a fault
LJMP a

:return_from_testing_pc_overflow_with_fault

; Switch off trap on overflow bit
ANDI sr, #0b1011_1111_1111_1111

; Instruction trace fault test
; Set T bit - the next instruction will be traced and the handler will fire
ORRI sr, #0b1000_0000_0000_0000
; This instruction is traced
NOOP
; T bit was cleared by the trace handler, so no further instructions are traced

; Bus protection fault test
; Get the debug device to raise a protection error on the bus
MOVI ah, $DEBUG_DEVICE_SEGMENT
MOVI al, $DEBUG_DEVICE_BUS_PROTECTION_ERROR
MOVI r7, #0x1
STOR (a), r7

:after_bus_protection_error_store

; Switch to non privileged mode
ORRI sr, #0b0000_0001_0000_0000
; Try to escape the current segment (this should be blocked -- the fault
    handler must not
; let this write actually take effect)
MOVI ph, #0xFEFE

:after_privilege_violation_load

; Get the debug device to raise an interrupt (this one should be fine)
MOVI r7, #0x1
MOVI ah, $DEBUG_DEVICE_SEGMENT
MOVI al, $DEBUG_DEVICE_EXCEPTION_L5
STOR (a), r7

; Invalid opcode fault test (kept last: its handler deliberately triggers a nested double
; fault, which -- since faults share a single link register with no stack -- leaves
; current_exception_level stuck at the fault level rather than correctly unwinding back to
; 0. Running it after every other fault test, rather than before, keeps that lingering state
; from affecting them.)
; There is no coprocessor at ID 4, so should trigger an invalid opcode fault
COPI #0x4000

:after_invalid_opcode_copi

; Halt CPU
COPI #0b0001_0100_1111_1111

.ORG 0x0300

:bus_fault_handler
ADDI r1, #1

; Bus Fault is retryable: fix up the return address so RETE doesn't retry the same
; faulting STOR forever.
ETFR a, #6
MOVI al, @after_bus_error_store
ETTR #6, a

RETE

:bus_protection_fault_handler
ADDI r5, #1

; Bus Protection Fault is retryable: fix up the return address so RETE doesn't retry the
; same faulting STOR forever.
ETFR a, #6
MOVI al, @after_bus_protection_error_store
ETTR #6, a

RETE

:alignment_fault_handler
ADDI r2, #1

```

```
; Fix up the return address because the original PC causes an alignment fault
; Returning without fixing up the return address would cause an endless loop

; Transfer current ELR into 'a' register
ETFR a, #6
; Correct the lower word
MOVI al, @after_odd_address
; Transfer corrected address back to ELR
ETTR #6, a

RETE

:segment_overflow_fault_handler
; Check which kind of address computation caused the overflow, using the
; bus access type captured in the fault metadata register

; Transfer fault metadata to r7
ETFR r7, #7
; Mask off the bus access type (bits 0-2) from the fault metadata register
ANDI r7, #0x7

CMPI r7, $BAT_NONE
BRAN|== @segment_overflow_fault_handler_pc
; Mask off the bus access type (bits 0-2) from the fault metadata register
ANDI r7, #0x7

CMPI r7, $BAT_DATA_WRITE
BRAN|== @segment_overflow_fault_handler_address

BRAN @segment_overflow_fault_handler_end

:segment_overflow_fault_handler_pc

ADDI r3, #0x0F00

; Fix up the return address because it isn't valid
; Load address after fault
MOVI ah, $PROGRAM_SEGMENT
MOVI al, @return_from_testing_pc_overflow_with_fault
; Transfer corrected address back to ELR
ETTR #6, a

BRAN @segment_overflow_fault_handler_end

:segment_overflow_fault_handler_address

ADDI r3, #0x000F

; Segment Overflow is retryable: fix up the return address so RETE doesn't retry the same
; faulting access forever. (This branch is not reached by anything in this example today,
; but is fixed defensively so it isn't a landmine if that changes.)
ETFR a, #6
MOVI al, @after_segment_overflow_load
ETTR #6, a

```

```
:segment_overflow_fault_handler_end

RETE

:invalid_opcode_fault_handler
; A double fault will blat the EU link registers so we need to save them before we trigger it
ETFR #6

ADDI r4, #1
; Trigger another invalid opcode fault to test double fault handling
; This will cause a fault while already handling a fault
COPI #0xF000

:after_nested_invalid_opcode_copi

; Restore link register
ETTR #6

; Invalid Opcode Fault is retryable: fix up the return address so RETE doesn’t retry the
; trapped COPI forever.
ETFR a, #6
MOVI al, @after_invalid_opcode_copi
ETTR #6, a

RETE

:privilege_violation_fault_handler
ADDI r5, #1

; Make sure that the CPU is back in privileged mode again after handling this fault

; Transfer current ELR into ’r7’ register
ETFR r7, #6
; Correct the lower word
ANDI r7, #0xFEFF
; Transfer corrected SR back to ELR
ETTR #6, r7

; Privilege Violation is retryable: the link register holds the blocked instruction’s own
; address. Retrying it now that we’re back in privileged mode would let it actually take
; effect -- but this instruction is deliberately testing that an escape attempt is blocked,
; so redirect past it instead of retrying it.
ETFR a, #6
MOVI al, @after_privilege_violation_load
ETTR #6, a

RETE

:double_fault_handler
; A double fault occurred - this is typically a last-chance handler
; In this example, we’ll increment r4 to show we were here
; and then return (though in production you’d probably reset)
ADDI r4, #1
```

```

; The link register holds the address of the instruction that triggered the double fault
; (the nested COPI above) -- fix up the return address so RETE doesn't retrigger it forever.
ETFR a, #6
MOVI al, @after_nested_invalid_opcode_copi
ETTR #6, a

RETE

:level_five_interrupt_conflict_handler
ADDI r6, #0x1
RETE

:level_five_interrupt_handler
ADDI r6, #0x1000

; Trigger another L5 interupt while it is already being serviced
; which will trigger a fault
MOVI r7, #0x1
MOVI ah, $DEBUG_DEVICE_SEGMENT
MOVI al, $DEBUG_DEVICE_EXCEPTION_L5
STOR (a), r7

RETE

:instruction_trace_fault_handler
ADDI r6, #0x100
; Clear T bit from the saved SR so RETE does not re-enable tracing
ETFR r7, #6
ANDI r7, #0x7FFF
ETTR #6, r7
RETE

; Trampoline for faults-high to come back
.ORG 0xFFF0
MOVI pl, @return_from_testing_pc_overflow_no_fault
```

`faults-high.sasm` is a second, separately assembled segment that the program-counter overflow test in Example D-2 jumps into; it exists only to let that fault trigger without the CPU executing the vector table sitting at address `0x0000` in the main segment. It jumps back into `faults.sasm` through the trampoline at `0xFFF0`.

Listing D.3: Example D-3: High-address fault handling

```
;; Segments
.EQU $PROGRAM_SEGMENT #0x0000
.EQU $SCRATCH_SEGMENT #0x0001
.EQU $PROGRAM_SEGMENT_HIGH #0x0002

; Section to test program counter overflow
; without trying to execute the vector table
; that lives at 0x0000 in the main segment

; Note: This is not linked together with the main program
; This is compiled as a separate program and mapped as a separate segment

.ORG 0x0000

MOVI ah, $PROGRAM_SEGMENT
MOVI al, #0xFFF0
LJMP a


; TODO: Work out how to avoid padding this out
; I had to pad it out because even though I said this segment has a length of 0xFFFF
; I think the way files are mapped means that length is ignored
.ORG 0xFFFE
MOVR r1, r1
```

D.4 Hardware Exception

This example shows a device-driven interrupt handled through the serial device: the program prints a help message, waits on **`WAIT`** for a hardware exception, and exits once the handler reports that the expected character has arrived in `r7`. The exception handler itself is installed at vector `0x40` (word address `0x0080`) and defined from line 137 onward; it saves the link register, calls `read_pending_byte` and `write_pending_byte` in turn, and restores the link register before returning with **`RETE`**. Those two subroutines show the poll-and-acknowledge pattern used to drain and refill the serial device's receive and send buffers one byte at a time.

Listing D.4: Example D-4: Hardware exception

```
1  ; Reserved space for 256x32 bit exception vectors
2
3
4  .EQU $FALSE #0x0
5  .EQU $TRUE #0x1
6
7  ; When this character is typed, the program should exit
8  .EQU $EXPECTED_CHAR #0x61
9
10 ;; Segments
11 .EQU $PROGRAM_SEGMENT #0x0000
12 .EQU $SCRATCH_SEGMENT #0x0001
13
14 ;; Devices
15 ; Serial
16 .EQU $SERIAL_DEVICE_SEGMENT #0x000A
17 .EQU $SERIAL_DEVICE_BAUD #0x0000
18 .EQU $SERIAL_DEVICE_RECV_ENABLED #0x0001
19 .EQU $SERIAL_DEVICE_RECV_PENDING #0x0002
20 .EQU $SERIAL_DEVICE_RECV_DATA #0x0003
21 .EQU $SERIAL_DEVICE_SEND_ENABLED #0x0004
22 .EQU $SERIAL_DEVICE_SEND_PENDING #0x0005
23 .EQU $SERIAL_DEVICE_SEND_DATA #0x0006
24
25 ;; Scratch Variables
26 .EQU $MESSAGE_SEND_BASE #0x0000
27 .EQU $MESSAGE_SEND_POINTER #0x0001
28
29
30 ;; Exception Vectors
31 ; First vector is the reset vector
32 .ORG 0x0000
33 .DQ @start
34
35 .ORG 0x0080
36 .DQ @exception_handler_p3
37
38 ;;;;;;; MAIN CODE SECTION
```

```

.ORG 0x0200
:start

; Enable all four maskable hardware interrupt lines (bits 9-12 of SR)
ORRI sr, #0b0001_1110_0000_0000

; Setup routines
BRSR @setup_serial
BRSR @print_help
; Make sure this is after the initial help message print (see the subroutine for more info)
BRSR @enable_serial_recv

:wait_for_char
; Wait for exception (will spin until interrupted)
WAIT

; Pending byte should be in r7 after exeception handler runs
CMPI r7, $EXPECTED_CHAR
BRAN|== @finish

BRAN @wait_for_char

:setup_serial

MOVI r1, #9600
MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_BAUD
STOR (a), r1

RETS

:enable_serial_recv

; When piping data to stdin, an EOF character gets sent which closes stdin
; This means that we can't trigger any more interrupts manually after the data is piped
; (stdin is currently the only way to externally trigger an interrupt)
; If the data is coming in at the same type we are sending data out,
; it ends up with the program gets stuck waiting for an interrupt and never finishes
; TODO: Actually that shouldn't be a problem - better investigate this further (
; maybe the exception handler routines are conflicting with each other)

MOVI r1, #0x1
MOVI al, $SERIAL_DEVICE_RECV_ENABLED
STOR (a), r1

RETS

:print_help

MOVI r1, @help_message
BRAN @print

:print_exit

```

```
MOVI r1, @exit_message
BRAN @print

:print

MOVI ah, $SCRATCH_SEGMENT
MOVI al, $MESSAGE_SEND_BASE
STOR (a), r1

; Align pointer
MOVI r1, #1

MOVI ah, $SCRATCH_SEGMENT
MOVI al, $MESSAGE_SEND_POINTER
STOR (a), r1

MOVI r1, $TRUE
MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_ENABLED
STOR (a), r1

:wait_for_print_finish

MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_ENABLED
LOAD r1, (a)

CMPI r1, $TRUE
BRAN|== @wait_for_print_finish

RETS

:finish

BRSR @print_exit

; Halt CPU
COPI #0b0001_0100_1111_1111

;;;;;;; EXCEPTION HANDLERS

.ORG 0x0400
:exception_handler_p3

; Save the link register
LDEA s, (l)

BRSR @read_pending_byte
BRSR @write_pending_byte

; Restore the link register
LDEA l, (s)

RETE

:read_pending_byte
; Check if there is something we need to read
MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_RECV_PENDING
```

```
LOAD r1, (a)

; Return early if not
CMPI r1, $FALSE
RETS|==

; Read pending byte
MOVI al, $SERIAL_DEVICE_RECV_DATA
LOAD r7, (a)

MOVI al, $SERIAL_DEVICE_RECV_PENDING
MOVI r1, $FALSE
STOR (a), r1
RETS

:write_pending_byte
; Check if the device is ready to write
MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_PENDING
LOAD r1, (a)

; Return early if not (there is already a send pending)
CMPI r1, $TRUE
RETS|==

MOVI ah, $SCRATCH_SEGMENT
MOVI al, $MESSAGE_SEND_POINTER
LOAD r2, (a)
MOVI al, $MESSAGE_SEND_BASE
LOAD al, (a)

MOVI ah, $PROGRAM_SEGMENT
LOAD r3, (r2, a)

; If the message buffer (zero terminated) has been sent. Stop sending
CMPI r3, #0
BRAN|== @stop_send

MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_DATA
STOR (a), r3

; Increment by two because of lack of packing (a word actually is padded to
    a DW)
ADDI r2, #2
MOVI ah, $SCRATCH_SEGMENT
MOVI al, $MESSAGE_SEND_POINTER
STOR (a), r2

MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_PENDING
MOVI r1, $TRUE
STOR (a), r1

RETS

:stop_send
MOVI ah, $SERIAL_DEVICE_SEGMENT
MOVI al, $SERIAL_DEVICE_SEND_ENABLED
MOVI r1, $FALSE
STOR (a), r1

RETS
```

The data segment below supplies the two zero-terminated, word-per-character messages (`help_message` and `exit_message`) that Example D-4 prints at startup and on exit.

Listing D.5: Example D-5: Hardware exception data

```
;;;;;;; DATA

; TODO: A string data type that does this for us
:help_message
.DW #84
.DW #121
.DW #112
.DW #101
.DW #32
.DW #39
.DW #97
.DW #39
.DW #32
.DW #116
.DW #111
.DW #32
.DW #101
.DW #120
.DW #105
.DW #116
.DW #10
.DW #0

:exit_message
.DW #71
.DW #111
.DW #111
.DW #100
.DW #98
.DW #121
.DW #101
.DW #33
.DW #10
.DW #0
```

D.5 Software Exception

This short program demonstrates a software-raised exception with **`EXCP`** and confirms that nesting is blocked while a handler is running. It sets `r1` to 1, raises exception `0x60`, and expects `r1` to hold `0xFF` once the handler at line 26 has run. That handler itself raises a second exception, `0x61`, from inside the handler; because the handler at address `0x0500` (line 37) sets `r1` to `0xBB` only if it runs, the final value of `r1` shows whether the nested exception was correctly ignored.

Listing D.6: Example D-6: Software exception

```
1 ; Reserved space for 256x32 bit exception vectors
2
3 ; First vector is the reset vector
4 .ORG 0x0000
5 .DQ @start
```

```

; 96th vector (*2 words per vector)
.ORG 0x00C0
.DQ @exception_handler
.DQ @exception_handler_that_should_be_ignored

.ORG 0x0200
:start

; Reset register
MOVI r1, #1
; Trigger Software Exception
EXCP #0x60

; r1 should be 0x0FF here after exception handler runs

; Halt CPU
COPI #0x14FF

.ORG 0x0300
:exception_handler

; Load a value to detect if this code runs
; Final value of r1 should be this value
MOVI r1, #0xFF
; Try and trigger nested software exception (should be ignored)
EXCP #0x61
; Return from exception
RETE

.ORG 0x0500
:exception_handler_that_should_be_ignored

; Load a value to detect if this code runs
; Final value of r1 should _not_ be BB because this vector should never be jumped to
MOVI r1, #0xBB
; Return from exception
RETE
```

D.6 Store Load

This program is a self-checking test of indexed and immediate-offset memory access, followed by an exercise of the register-offset and immediate-offset forms of **LJMP** and **LJSR**. The store and load block (lines 8–19) round-trips values through memory using an immediate offset and a register offset from an address-register pair. The remainder of the listing chains together register-offset, immediate-offset, and plain register forms of **LJMP** and **LJSR**; every path not on the intended route lands on an `LJMP s`/`LJSR s` instruction commented `FAIL!`, so a working implementation reaches the `:pass` label at the end while a faulty one jumps into the "forbidden zone" and fails at `:fail`.

Listing D.7: Example D-7: Store/load

```
1  ; Reserved space for 256x32 bit exception vectors
2
3  .ORG 0x0000
4  .DQ @init
5
6  .ORG 0x0200
7
8  :init
9  MOVI r1, #0xCAFE
```

```
MOVI r3, #0xBEEF
MOVI r5, #0x000A

MOVI ah, #0x0008
MOVI al, #0xAAF0

STOR (#0x000F, a), r1
LOAD r2, (#0x000F, a)
STOR (r5, a), r3
LOAD r4, (r5, a)


; Lets test jumping!

; TODO: Add some tests that:
; LJMP to another segment
; LJMP with offset
; LJMP with shifted offset
; Same three things but with LJSR
; Make a maze of jumps so if the jump goes to the wrong place it will land in the forbidden zone where every instruction jumps to the fail label

; r7 will store the test result
MOVI r7, #0
; Use the stack address register pair to store fail address
MOVI sh, #0x0
MOVI sl, @fail
; Address register pair used as part of test
MOVI ah, #0x0
MOVI al, #0x0

BRAN #12 ; Step 1
LJMP s ; FAIL!
LJMP s ; FAIL!
BRAN #10 ; Step 3
LJMP s ; FAIL!
LJMP s ; FAIL!
BRAN #-6 ; Step 2
LJMP s ; FAIL!
BRAN @ljmp_register_offset


:ljmp_register_offset
MOVI al, @ljmp_register_offset_anchor
MOVI r1, #-6
MOVI r2, #4
BRAN @ljmp_register_offset_anchor
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP a, r2
LJMP s ; FAIL!
LJMP s ; FAIL!
:ljmp_register_offset_anchor
LJMP a, r1
LJMP s ; FAIL!
BRAN @ljmp_immediate_offset
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!


:ljmp_immediate_offset
MOVI al, @ljmp_immediate_offset_anchor
BRAN @ljmp_immediate_offset_anchor
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP a, #4
LJMP s ; FAIL!
LJMP s ; FAIL!
:ljmp_immediate_offset_anchor
LJMP a, #-6
LJMP s ; FAIL!
BRAN @ljsr_register_offset
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!
LJMP s ; FAIL!


:ljsr_register_offset
MOVI al, @ljsr_register_offset_anchor
MOVI r1, #-6
MOVI r2, #4
BRAN @ljsr_register_offset_anchor
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR a, r2
LJSR s ; FAIL!
LJSR s ; FAIL!
:ljsr_register_offset_anchor
LJSR a, r1
LJSR s ; FAIL!
BRAN @ljsr_immediate_offset
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!


:ljsr_immediate_offset
MOVI al, @ljsr_immediate_offset_anchor
BRAN @ljsr_immediate_offset_anchor
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR a, #4
LJSR s ; FAIL!
LJSR s ; FAIL!
:ljsr_immediate_offset_anchor
LJSR a, #-6
LJSR s ; FAIL!
BRAN @finish
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!
LJSR s ; FAIL!


:finish
MOVI r7, #0x0FAB

MOVI sh, #0x0
MOVI sl, @pass
LJMP s

.ORG 0x0400

:fail
MOVI r7, #0xFA11

:pass

; Halt CPU
COPI #0x14FF
```

CHAPTER E

Quick Reference

This appendix collects the most frequently consulted tables in a single location for use during development. Section E.4 lists where to find additional reference tables defined in the main chapters.

E.1 Instruction Mnemonic Index

All instruction mnemonics in alphabetical order. *Type*: ALU, Memory (Mem), Control-flow (CF), Coprocessor (COP), or Meta. *N*, *Z*, *C*, *V*: use the symbols defined in Table 11.2 (Chapter 11): `*` (updated), `0`, `1`, `-` (preserved), `S` (from the shifter), and `U` (undefined). Rows marked † are preserved by the call instruction itself, but the selected coprocessor may define further effects once dispatched. Rows marked ‡ are updated from a named source rather than from the instruction's own result. Meta-instructions inherit the flag behavior of the underlying CPU instruction. For full encoding details, see the chapter cited in the header of each group.

Table E.1: Alphabetical Mnemonic Index

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
A – Ch.13							
ADCI	ALU	0x01/0x21	*	*	*	*	Add with carry, immediate or short immediate
ADCR	ALU	0x31	*	*	*	*	Add with carry, register
ADDI	ALU	0x00/0x20	*	*	*	*	Add immediate or short immediate
ADDR	ALU	0x30	*	*	*	*	Add register
ANDI	ALU	0x04/0x24	*	*	0	0	Bitwise AND immediate or short immediate
ANDR	ALU	0x34	*	*	0	0	Bitwise AND register
B – Ch.15 / Ch.17							
BRAN	Meta	LDEA p,...	-	-	-	-	PC-relative conditional branch
BRSR	Meta	LDEL p,...	-	-	-	-	PC-relative call (saves return address to I)
C – Ch.13 / Ch.16							
CMNI	ALU	0x08/0x28	*	*	*	*	Compare negative immediate or short immediate (no write-back)
CMNR	ALU	0x38	*	*	*	*	Compare negative register (no write-back)
CMPI	ALU	0x0A/0x2A	*	*	*	*	Compare immediate or short immediate (no write-back)
CMPR	ALU	0x3A	*	*	*	*	Compare register (no write-back)
CNCI	ALU	0x09/0x29	*	*	*	*	Compare negative with carry, immediate or short immediate (no write-back)
CNCR	ALU	0x39	*	*	*	*	Compare negative with carry, register (no write-back)
COPI	COP	0x0F	-	-	-	-†	Coprocessor call immediate
COPR	COP	0x3F	-	-	-	-†	Coprocessor call register
CPCI	ALU	0x0B/0x2B	*	*	*	*	Compare with carry, immediate or short immediate (no write-back)
CPCR	ALU	0x3B	*	*	*	*	Compare with carry, register (no write-back)
D – Ch.16 / Ch.17							

continued on next page

Table E.1 (continued)

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
DIVS	Meta	COPI #0x3300	-	-	-	-†	Signed divide (Integer Maths Unit, coprocessor 0x3)
DIVU	Meta	COPI #0x3200	-	-	-	-†	Unsigned divide (Integer Maths Unit, coprocessor 0x3)
DMAR	Meta	COPI #0x28xx	-	-	-	-	DMA read into registers (DMA unit, coprocessor 0x2)
DMAT	Meta	COPI #0x2Ann	-	-	-	-	DMA memory-to-memory transfer
DMAW	Meta	COPI #0x29xx	-	-	-	-	DMA write from registers
E – Ch.17 / Ch.16							
ETFR	Meta	COPI #0x1Cxy	-	-	-	-	Read exception-unit state
ETTR	Meta	COPI #0x1Dxy	-	-	-	-	Write exception-unit state
EXCP	Meta	COPI #0x11vv	-	-	-	-	Raise software exception (trap) with vector `vv`
L – Ch.14 / Ch.15 / Ch.17							
LDEA	CF	`0x18`–`0x1B`	-	-	-	-	Load effective address
LDEL	CF	`0x1C`–`0x1F`	-	-	-	-	Load effective address and link (saves PC to l)
LJMP	Meta	LDEA p,...	-	-	-	-	Long (absolute) jump
LJSR	Meta	LDEL p,...	-	-	-	-	Long (absolute) call
LOAD	Mem	`0x14`–`0x17`	-	-	-	-	Load from memory
M – Ch.13 / Ch.17 / Ch.16							
MOVI	ALU	`0x07`	*	*	0	0	Move immediate
MOVR	ALU	`0x37`	*	*	0	0	Move register
MULS	Meta	COPI #0x3100	-	-	-	-†	Signed multiply (Integer Maths Unit, coprocessor 0x3)
MULU	Meta	COPI #0x3000	-	-	-	-†	Unsigned multiply (Integer Maths Unit, coprocessor 0x3)
N – Ch.17							
NOOP	Meta	ADDI[N] r1,#0	-	-	-	-	No operation
O – Ch.13							
ORRI	ALU	`0x05`/`0x25`	*	*	0	0	Bitwise OR immediate or short immediate
ORRR	ALU	`0x35`	*	*	0	0	Bitwise OR register
R – Ch.15 / Ch.17 / Ch.16							
RETE	Meta	COPI #0x1A00	*	*	*	*‡	Return from exception handler (SR restored from the link register)
RETS	Meta	LDEA p,(#0,l)	-	-	-	-	Return from subroutine
RSET	Meta	COPI #0x1B00	0	0	0	0	System reset (SR cleared to 0)
S – Ch.13 / Ch.14 / Ch.17							
SBCI	ALU	`0x03`/`0x23`	*	*	*	*	Subtract with carry, immediate or short immediate
SBCR	ALU	`0x33`	*	*	*	*	Subtract with carry, register
SHFT	Meta	ORRI[S] rD,#0,shift	S	S	S	S	Shift with status-register update
STOR	Mem	`0x10`–`0x13`	-	-	-	-	Store to memory
SUBI	ALU	`0x02`/`0x22`	*	*	*	*	Subtract immediate or short immediate

continued on next page

Table E.1 (continued)

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
SUBR	ALU	`0x32`	*	*	*	*	Subtract register
T – Ch.13							
TSAI	ALU	`0x0C`/`0x2C`	*	*	0	0	Test AND immediate or short (no write-back)
TSAR	ALU	`0x3C`	*	*	0	0	Test AND register (no write-back)
TSOI	ALU	`0x0D`/`0x2D`	*	*	0	0	Test OR immediate or short (no write-back)
TSOR	ALU	`0x3D`	*	*	0	0	Test OR register (no write-back)
TSXI	ALU	`0x0E`/`0x2E`	*	*	0	0	Test XOR immediate or short (no write-back)
TSXR	ALU	`0x3E`	*	*	0	0	Test XOR register (no write-back)
W – Ch.16 / Ch.17							
WAIT	Meta	COPI #0x1900	-	-	-	-	Idle until a hardware interrupt can be taken or a reset occurs
X – Ch.13							
XORI	ALU	`0x06`/`0x26`	*	*	0	0	Bitwise XOR immediate or short immediate
XORR	ALU	`0x36`	*	*	0	0	Bitwise XOR register

† Preserved by the coprocessor-call instruction itself; the selected coprocessor may define further effects once the call is dispatched. ‡ **RETE** restores all four flags from the link register.

E.2 Status Register Bit Reference

The status register (`sr`) is 16 bits wide. The high byte (bits 15–8) is privileged: in protected mode, reads return zero for those bits and writes to them are silently ignored, with no fault. The low byte (bits 7–0) can be read and written in both modes.

Bit	Name	Access	Reset	Function
15	T	Privileged	0	Trace mode: fault after every instruction (used by debuggers)
14	A	Privileged	0	Address overflow trap: fault when an address calculation overflows
13	EA	Privileged	0	Exception Active: set automatically while inside an exception handler (read-only)
12	E4	Privileged	0	Enable hardware interrupt line 4 (level 4, highest maskable)
11	E3	Privileged	0	Enable hardware interrupt line 3
10	E2	Privileged	0	Enable hardware interrupt line 2
9	E1	Privileged	0	Enable hardware interrupt line 1 (level 1, lowest maskable)
8	P	Privileged	0	Protected mode: 0 = supervisor, 1 = protected
7–4	–	Either	0	Reserved for future use; software must not rely on the value read
3	V	Either	0	Overflow: signed arithmetic overflow occurred
2	C	Either	0	Carry: unsigned carry out or borrow
1	N	Either	0	Negative: result bit 15 = 1 (two's complement sign bit)
0	Z	Either	0	Zero: result was zero

Table E.2: Status Register Bit Summary

Hardware interrupt line 5 (level 5) is a Non-Maskable Interrupt (NMI) and is always enabled; it has no enable bit in `sr`. See Chapter 5 for full details and examples.

E.3 Bus Access Type (BAT) Reference

The 3-bit Bus Access Type (BAT) field accompanies every bus cycle and is also captured in fault metadata. External devices (memory controllers, coprocessors) can use it to determine the nature of the access.

BAT[2:0]	Value	Meaning
000	0	No I/O (internal cycle, no bus cycle)
001	1	Instruction Fetch
010	2	Data Read
011	3	Data Write
100	4	Exception Vector Fetch
101	5	DMA Coprocessor Read Burst
110	6	DMA Coprocessor Write Burst
111	7	Reserved

Table E.3: Bus Access Type (BAT) Encoding

See the *SIRC-1 Hardware Reference Manual* for how the BAT and PROT (protected-mode) signals reach external devices, including usage examples that combine them.

E.4 Where to Find Other Tables

Table	**Chapter / Appendix**	**Label**
Complete opcode map (all 64 opcodes)	Appendix A	A.1
Register encoding (4-bit IDs)	Chapter 3	3.1
Condition code encoding	Chapter 7	10.1
Shift type encoding	Chapter 7	9.1
Addressing mode summary	Chapter 8	8.1
Exception vector table	Chapter 6	6.1
Instruction timing (cycle counts)	Appendix B	B.1
Complete instruction list by opcode	Chapter 12	12.1
Meta-instructions cross-reference	Chapter 17	17.1
Flag effect symbols	Chapter 11	11.2

Table E.4: Reference Table Locations

SIRC-1 Architecture Reference Manual

The SIRC-1 brings together a clean 16-bit load/store architecture, a fully specified six-stage execution model, and an extensible coprocessor interface, without sacrificing the predictability system programmers and hardware designers depend on. This manual documents every instruction, register, exception, and bus signal to an implementation-ready level of detail, so that assemblers, compilers, operating systems, and hardware implementations can all be built directly from its pages, without consulting any other source.

This manual contains:

- The complete SIRCIS instruction set reference, arranged for quick lookup by mnemonic
- Full exception, interrupt, and reset specifications
- Cross-references to the companion SIRC-1 Hardware Reference Manual for pinout and bus timing
- Coprocessor and future-model compatibility guidelines

ISBN 0-000000-00-0

SILICON INTEGRATED RESEARCH CORPORATION
1400 Fabrication Way, Milpitas Flat, CA
Printed in the United States of America

First Printing, 1989 • Version 1.0 • $49.95